

# STUDENT GUIDE

## FACULTY OF INFORMATION TECHNOLOGY

INFORMATION SYSTEMS 622



**FACULTY OF INFORMATION TECHNOLOGY**

**STUDY GUIDE**

**MODULE: INFORMATION SYSTEM 622**

Copyright © 2026

Richfield Graduate Institute of Technology (Pty) Ltd Registration Number: 2000/000757/07

All rights reserved; no part of this publication may be reproduced in any form or by any means, including photocopying machines, without the written permission of the Institution

## Contents

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>CHAPTER 1: Development Strategies.....</b>                     | <b>8</b>  |
| <b>1.1 TRADITIONAL VERSUS WEB-BASED SYSTEMS DEVELOPMENT .....</b> | <b>8</b>  |
| <b>1.2 EVOLVING TRENDS.....</b>                                   | <b>10</b> |
| <b>1.3 IN-HOUSE SOFTWARE DEVELOPMENT OPTIONS.....</b>             | <b>11</b> |
| <b>1.10 SUMMARY .....</b>                                         | <b>31</b> |
| <b>Key Points:.....</b>                                           | <b>31</b> |
| <b>Conclusion .....</b>                                           | <b>33</b> |
| <b>CHAPTER 2: User Interface Design .....</b>                     | <b>37</b> |
| <b>2.1 USER INTERFACES.....</b>                                   | <b>37</b> |
| <b>2.2 HUMAN-COMPUTER INTERACTION.....</b>                        | <b>39</b> |
| <b>2.3 GUIDELINES FOR USER INTERFACE DESIGN .....</b>             | <b>41</b> |
| 2.3.1 Understand the Business.....                                | 41        |
| 2.3.2 Maximize Graphical Effectiveness .....                      | 41        |
| 2.3.3 Think Like a User.....                                      | 41        |
| 2.3.4 Use Models and Prototypes .....                             | 42        |
| 2.3.5 Focus on Usability.....                                     | 42        |
| 2.3.6 Invite Feedback .....                                       | 42        |
| 2.3.7 Document Everything.....                                    | 42        |
| <b>2.4 GUIDELINES FOR USER INTERFACE DESIGN .....</b>             | <b>43</b> |
| 2.4.1 Create an Interface That Is Easy to Learn and Use.....      | 43        |
| 2.4.2 Enhance User Productivity .....                             | 44        |
| 2.4.3 Provide Flexibility .....                                   | 45        |
| 2.4.4 Provide Users with Help and Feedback.....                   | 45        |
| 2.4.5 Create an Attractive Layout and Design.....                 | 47        |
| 2.4.6 Enhance the Interface .....                                 | 48        |
| 2.4.7 Focus on Data Entry Screens.....                            | 50        |
| 2.4.8 Use Validation Rules .....                                  | 52        |
| 2.4.9 Manage Data Effectively .....                               | 55        |
| 2.4.10 Reduce Input Volume.....                                   | 55        |
| <b>2.5 SOURCE DOCUMENT AND FORM DESIGN .....</b>                  | <b>55</b> |
| <b>2.6 PRINTED OUTPUT .....</b>                                   | <b>57</b> |
| 2.6.1 Report Design .....                                         | 58        |
| 2.6.2 Report Design Principles.....                               | 58        |
| 2.6.3 Types of Reports.....                                       | 60        |
| <b>2.7 TECHNOLOGY ISSUES .....</b>                                | <b>60</b> |
| <b>2.8 SECURITY AND CONTROL ISSUES .....</b>                      | <b>64</b> |
| 2.8.1 Output Security and Control.....                            | 64        |
| 2.8.2 Input Security and Control .....                            | 65        |

|                                                                                |            |
|--------------------------------------------------------------------------------|------------|
| <b>2.9 EMERGING TRENDS .....</b>                                               | <b>66</b>  |
| 2.9.2 Responsive Web Design.....                                               | 67         |
| 2.9.3 Prototyping.....                                                         | 67         |
| <b>2.10 SUMMARY .....</b>                                                      | <b>69</b>  |
| <b>CHAPTER 3: Data Design .....</b>                                            | <b>77</b>  |
| <b>3.1 DATA DESIGN CONCEPTS.....</b>                                           | <b>77</b>  |
| 3.1.1 Data Structures .....                                                    | 77         |
| 3.1.2 Mario and Danica: A Data Design Example .....                            | 78         |
| 3.1.3 Database Management Systems .....                                        | 80         |
| <b>3.2 DBMS COMPONENTS.....</b>                                                | <b>81</b>  |
| 3.2.1 Interfaces for Users, Database Administrators, and Related Systems ..... | 82         |
| 3.2.2 Schema .....                                                             | 83         |
| 3.2.3 Physical Data Repository.....                                            | 83         |
| <b>3.3 WEB-BASED DESIGN.....</b>                                               | <b>83</b>  |
| <b>3.4 DATA DESIGN TERMS .....</b>                                             | <b>85</b>  |
| 3.4.1 Definitions .....                                                        | 85         |
| 3.4.2 Referential Integrity.....                                               | 88         |
| <b>3.5 ENTITY RELATIONSHIP DIAGRAMS .....</b>                                  | <b>89</b>  |
| 3.5.1 Drawing an ERD .....                                                     | 89         |
| 3.5.2 Types of Relationships .....                                             | 90         |
| <b>3.6 Normalization Stages .....</b>                                          | <b>93</b>  |
| 3.6.1 Standard Notation Format .....                                           | 93         |
| 3.6.2 First Normal Form.....                                                   | 95         |
| 3.6.3 Second Normal Form .....                                                 | 96         |
| 3.6.4 Third Normal Form .....                                                  | 99         |
| 3.6.5 Two Real-World Examples .....                                            | 101        |
| <b>3.7 CODES.....</b>                                                          | <b>109</b> |
| 3.7.1 Overview of Codes.....                                                   | 109        |
| 3.7.2 Types of Codes.....                                                      | 110        |
| 3.7.3 Designing Codes.....                                                     | 113        |
| <b>3.8 DATA STORAGE AND ACCESS.....</b>                                        | <b>114</b> |
| 3.8.1 Tools and Techniques.....                                                | 114        |
| <b>3.9 DATA CONTROL .....</b>                                                  | <b>118</b> |
| <b>3.10 SUMMARY .....</b>                                                      | <b>119</b> |
| <b>CHAPTER 4: System Architecture .....</b>                                    | <b>123</b> |
| <b>4.1 ARCHITECTURE CHECKLIST.....</b>                                         | <b>123</b> |
| 4.1.3 Initial Cost and TCO .....                                               | 125        |
| 4.1.4 Scalability.....                                                         | 126        |
| 4.1.5 Web Integration.....                                                     | 126        |
| 4.1.6 Legacy Systems.....                                                      | 126        |
| 4.1.7 Processing Options .....                                                 | 126        |

|                                                           |            |
|-----------------------------------------------------------|------------|
| 4.1.8 Security Issues.....                                | 127        |
| 4.1.9 Corporate Portals.....                              | 127        |
| <b>4.2 THE EVOLUTION OF SYSTEM ARCHITECTURE.....</b>      | <b>128</b> |
| 4.2.1 Mainframe Architecture .....                        | 128        |
| 4.2.2 Impact of the Personal Computer .....               | 129        |
| 4.2.3 Network Evolution .....                             | 129        |
| <b>4.3 CLIENT/SERVER ARCHITECTURE.....</b>                | <b>131</b> |
| 4.3 CLIENT/SERVER ARCHITECTURE .....                      | 132        |
| 4.3.2 Client/Server Tiers: .....                          | 133        |
| 4.3.3 Middleware.....                                     | 134        |
| 4.3.4 Cost-Benefit Issues.....                            | 134        |
| 4.3.5 Performance Issues.....                             | 135        |
| <b>4.4 THE IMPACT OF THE INTERNET.....</b>                | <b>135</b> |
| 4.4.1 Web 2.0 .....                                       | 137        |
| <b>4.5 E-COMMERCE ARCHITECTURE.....</b>                   | <b>138</b> |
| 4.5.1 In-House Solutions .....                            | 138        |
| 4.5.2 Packaged Solutions.....                             | 139        |
| 4.5.3 Service Providers.....                              | 139        |
| <b>4.6 PROCESSING METHODS.....</b>                        | <b>140</b> |
| 4.6.1 Online Processing.....                              | 140        |
| 4.6.2 Batch Processing .....                              | 142        |
| 4.6.3 Example.....                                        | 142        |
| <b>4.7 NETWORK MODELS .....</b>                           | <b>144</b> |
| 4.7.1 The OSI Model.....                                  | 144        |
| 4.7.2 Network Topology .....                              | 144        |
| 4.7.3 Network Devices .....                               | 149        |
| <b>4.8 WIRELESS NETWORKS .....</b>                        | <b>150</b> |
| 4.8.1 Standards .....                                     | 150        |
| 4.8.2 Topologies.....                                     | 151        |
| 4.8.3 Trends.....                                         | 151        |
| <b>4.9 SYSTEMS DESIGN COMPLETION .....</b>                | <b>152</b> |
| 4.9.1 System Design Specification .....                   | 153        |
| 4.9.3 Presentations.....                                  | 154        |
| <b>4.10 SUMMARY .....</b>                                 | <b>155</b> |
| <b>EXERCISE .....</b>                                     | <b>157</b> |
| <b>CHAPTER 5: Managing Systems Implementation.....</b>    | <b>158</b> |
| <b>5.1 QUALITY ASSURANCE.....</b>                         | <b>158</b> |
| <b>Software Engineering .....</b>                         | <b>159</b> |
| <b>Systems Engineering .....</b>                          | <b>159</b> |
| 5.1.3 International Organization for Standardization..... | 161        |
| <b>5.2 APPLICATION DEVELOPMENT .....</b>                  | <b>162</b> |

|                                                               |            |
|---------------------------------------------------------------|------------|
| 5.2.1 Review the System Design.....                           | 163        |
| 5.2.2 Application Development Tasks .....                     | 163        |
| 5.2.3 Systems Development Tools .....                         | 164        |
| <b>5.3 STRUCTURED DEVELOPMENT .....</b>                       | <b>166</b> |
| 5.3.1 Structure Charts.....                                   | 167        |
| 5.3.2 Cohesion and Coupling .....                             | 169        |
| 5.3.2 Coupling.....                                           | 170        |
| 5.3.3 Drawing a Structure Chart.....                          | 170        |
| <b>5.4 OBJECT-ORIENTED DEVELOPMENT .....</b>                  | <b>173</b> |
| 5.4.2 Implementation of Object-Oriented Designs .....         | 175        |
| 5.4.3 Object-Oriented Cohesion and Coupling.....              | 175        |
| <b>5.5 AGILE DEVELOPMENT.....</b>                             | <b>176</b> |
| 5.5.1 Extreme Programming (XP).....                           | 177        |
| 5.5.2 User Stories .....                                      | 178        |
| 5.5.3 Iterations and Releases .....                           | 178        |
| <b>5.6 CODING .....</b>                                       | <b>179</b> |
| <b>5.7 TESTING.....</b>                                       | <b>179</b> |
| <b>Testing Process After Coding .....</b>                     | <b>179</b> |
| 5.7.1 Unit Testing.....                                       | 180        |
| <b>5.8 DOCUMENTATION .....</b>                                | <b>183</b> |
| 5.8.1 Program Documentation.....                              | 184        |
| 5.8.2 System Documentation.....                               | 184        |
| 5.8.3 Operations Documentation .....                          | 184        |
| 5.8.4 User Documentation.....                                 | 185        |
| 5.8.5 Online Documentation.....                               | 186        |
| <b>5.9 INSTALLATION .....</b>                                 | <b>187</b> |
| 5.9.1 Operational and Test Environments .....                 | 188        |
| 5.9.2 System Changeover.....                                  | 189        |
| 5.9.3 Data Conversion .....                                   | 192        |
| 5.9.4 Training .....                                          | 193        |
| 5.9.5 Post-Implementation Tasks .....                         | 197        |
| <b>5.10 SUMMARY .....</b>                                     | <b>201</b> |
| <b>EXERCISE .....</b>                                         | <b>204</b> |
| <b>CHAPTER 6: Managing Systems Support and Security .....</b> | <b>206</b> |
| <b>6.1 USER SUPPORT.....</b>                                  | <b>206</b> |
| 6.1.1 User Training.....                                      | 207        |
| 6.1.2 Help Desks .....                                        | 207        |
| 6.1.3 Outsourcing Issues .....                                | 208        |
| <b>6.2 MAINTENANCE TASKS .....</b>                            | <b>208</b> |
| 6.2.1 Types of Maintenance .....                              | 209        |
| 6.2.2 Corrective Maintenance.....                             | 210        |

|                                                      |            |
|------------------------------------------------------|------------|
| 6.2.3 Adaptive Maintenance .....                     | 212        |
| 6.2.4 Perfective Maintenance .....                   | 212        |
| 6.2.5 Preventive Maintenance .....                   | 212        |
| <b>6.3 MAINTENANCE MANAGEMENT .....</b>              | <b>213</b> |
| 6.3.1 The Maintenance Team .....                     | 213        |
| 6.3.2 Maintenance Requests .....                     | 214        |
| 6.3.3 Establishing Priorities .....                  | 215        |
| 6.3.4 Configuration Management .....                 | 216        |
| 6.3.5 Maintenance Releases .....                     | 216        |
| <b>6.3.6 Version Control .....</b>                   | <b>217</b> |
| 6.3.7 Baselines .....                                | 218        |
| <b>6.4 SYSTEM PERFORMANCE MANAGEMENT .....</b>       | <b>219</b> |
| 6.4.1 Fault Management .....                         | 219        |
| 6.4.2 Performance and Workload Measurement .....     | 220        |
| 6.4.3 Capacity Planning .....                        | 221        |
| <b>6.5 SYSTEM SECURITY .....</b>                     | <b>223</b> |
| 6.5.1 System Security Concepts .....                 | 223        |
| <b>6.6 SECURITY LEVELS .....</b>                     | <b>227</b> |
| 6.6.1 Physical Security .....                        | 228        |
| 6.6.2 Network Security .....                         | 231        |
| <b>6. 7 BACKUP AND RECOVERY .....</b>                | <b>236</b> |
| 6.7.1 Global Terrorism .....                         | 236        |
| 6.7.2 Backup Policies .....                          | 236        |
| 6.7.4 Retention Periods .....                        | 238        |
| 6.7.5 Business Continuity Issues .....               | 238        |
| <b>6.8 SYSTEM RETIREMENT .....</b>                   | <b>239</b> |
| <b>6.9 FUTURE CHALLENGES AND OPPORTUNITIES .....</b> | <b>240</b> |
| 6.9.1 Trends and Predictions .....                   | 240        |
| 6.9.2 Strategic Planning for IT Professionals .....  | 242        |
| 6.9.3 IT Credentials and Certification .....         | 242        |
| 6.9.4 Critical Thinking Skills .....                 | 243        |
| 6.9.5 Cyberethics .....                              | 243        |
| <b>SUMMARY .....</b>                                 | <b>244</b> |
| <b>EXERCISES .....</b>                               | <b>246</b> |
| <b>REFERENCES .....</b>                              | <b>247</b> |

## CHAPTER 1: Development Strategies



### LEARNING OUTCOMES

After reading this Section of the guide, the learner should be able to:

1. **Compare traditional and web-based systems development**
2. **Discuss how Web 2.0, cloud computing, and mobile devices influence systems development**
3. **Explain the process of choosing one of the four in-house software development options**
4. **Define outsourcing in the context of software development**
5. **Identify the main benefits and concerns related to offshoring**
6. **Explain the concept of Software as a Service (SaaS)**
7. **Describe the role of a systems analyst in choosing a development strategy**
8. **Walk through the five steps of the software acquisition process**
9. **Explain the differences between a Request for Proposal (RFP) and a Request for Quotation (RFQ)**

### 1.1 TRADITIONAL VERSUS WEB-BASED SYSTEMS DEVELOPMENT

A few years ago, companies typically either developed software in-house, purchased a software package (which might require some adjustments), or hired consultants and external resources to do the work. Today, companies have many more options for acquiring software, such as application service providers, web-hosted software, and businesses offering a variety of enterprise-wide software solutions. This increase in choices is largely due to the significant changes in business operations made possible by the Internet.

A systems analyst must decide whether development will occur in a traditional environment or within a web-based framework. While there are similarities, there are also distinct differences between the two. For example, in web-based systems, the web itself becomes a key part of the application rather than just a communication channel. This shift requires new application development tools and solutions to handle web-based systems.

Two popular web-based development environments are Microsoft's .NET and the open-source MERN stack. Microsoft's .NET platform supports the development and operation of various applications written in C# and Visual Basic, including web, mobile, and desktop applications. The MERN stack (MongoDB, Express, React, Node) is used to build web applications entirely in JavaScript, with MongoDB serving as the database and the other tools (Express, React, and Node) functioning as frameworks for full-stack web development.

Despite the growing trend toward web-based systems, many organizations still rely on traditional systems. This is often because they use legacy applications that are difficult to replace or because they don't require web components to meet user needs. When deciding between traditional and web-based development, it's important to consider the key differences between the two approaches. Building an application in a web-based environment can offer substantial benefits but also comes with potential risks compared to traditional development. Below are some key characteristics of both traditional and web-based development:

### **1. 1. 1 Traditional Development In a traditional systems development. environment.**

- Compatibility issues, such as integrating with existing hardware, software platforms, and legacy systems, influence design decisions.
- Systems are typically designed to run on local or wide-area networks within the company.
- While systems may use the Internet for certain features, web-based components are usually supplementary rather than central to the design.
- Development usually follows one of three main approaches: in-house development, purchasing a software package with potential modifications, or using external consultants.
- Scalability can be limited by network constraints.
- Many applications require significant desktop computing power.
- Security concerns are generally less complex since the system operates within a private network, rather than over the Internet.

### **1.1.2 Web-Based Development: In a web-based systems development environment**

- Systems are developed within an Internet-based framework like .NET.
- In web-based development, the web itself is the platform, not just a means of communication.
- Web-based systems are highly scalable and can operate across different hardware environments.
- Larger companies often use web-based systems as enterprise-wide solutions for tasks like customer relationship management, order processing, and materials management.
- Web-based software is seen as a service, meaning it's less reliant on desktop computing resources.
- When acquiring web-based software as a service, companies can limit their in-house involvement, allowing the vendor to handle installation, configuration, and maintenance for agreed-upon fees.

- Web-based systems often require additional software layers, known as middleware, to interact with existing software and legacy systems.
- Web-based solutions present more complex security issues, which must be properly addressed.

## 1.2 EVOLVING TRENDS

In the rapidly evolving world of IT, Internet technology stands out as one of the most dynamic areas. Key trends driving this change include Web 2.0, cloud computing, and mobile devices. Systems analysts need to stay informed about these trends and consider them when planning large-scale systems.

### Web 2.0

The term "Web 2.0" refers to a second generation of the web that enables greater collaboration, interaction, and information sharing. Unlike static HTML webpages, Web 2.0 relies on continuously available user applications that allow unlimited users to access, modify, and exchange data. This new web environment enhances interactive experiences, such as wikis, blogs, and social networking platforms like Facebook, LinkedIn, and Twitter.

### Cloud Computing

Cloud computing, as defined by the National Institute of Standards and Technology (NIST), is "a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction." Cloud computing is often symbolized by a cloud icon, representing a network or the Internet. It can be seen as an online Software as a Service (SaaS) environment supported by powerful computers, making Web 2.0 technologies and applications possible.

#### NIST Cloud Computing Program - NCCP



**FIGURE 7-1** NIST definition of cloud computing.

Sources: [1] NIST

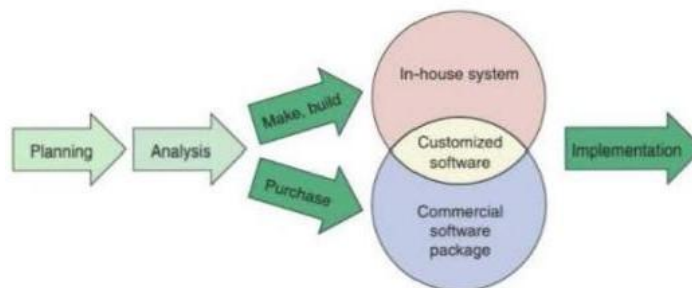
Mobile devices (Tilley and Rosenblatt, 2024)

Mobile devices have become widespread, with smartphones and tablets now commonly used

both personally and within businesses. These devices now have sufficient computing power to perform processing "at the edge," meaning at the user's location, rather than relying solely on centralized servers. Developing applications for mobile devices requires new platforms, but many modern development tools are designed to support both web-based and mobile app development simultaneously.

### 1.3 IN-HOUSE SOFTWARE DEVELOPMENT OPTIONS

A company has the option to either develop its own systems or purchase a software package that may require customization. These development alternatives are illustrated in Figure 7-2. Several factors influence this decision, but the most significant consideration is the total cost of ownership (TCO), which was discussed in Chapter 4. In addition to these choices, companies may also create user applications based on commercial software packages, such as Microsoft Office, to enhance user productivity and efficiency.



**FIGURE 7-2** Instead of outsourcing, a company can choose to develop a system in-house or purchase and possibly customize a commercial package.

(Tilley and Rosenblatt, 2024)

#### 1 .3. 1 Make or Buy Decision

The decision to develop software in-house versus purchasing it is commonly referred to as a "make or buy" or "build or buy" decision. In this case, the company's IT department handles the development of custom software, while a software package is obtained from a vendor or application service provider.

A software package could either be a standard commercial application or one customized specifically for the purchaser. Companies that create software for sale are called software vendors. On the other hand, a firm that enhances a commercial package by adding custom features and tailoring it for a particular industry is called a value-added reseller (VAR).

Software packages are available for virtually every business activity. A "horizontal application" is a software package that can be used across various industries. For example, an accounting package is considered a horizontal application because it can be utilized by many different businesses or divisions within large, diverse companies.

In contrast, a "vertical application" is developed to meet the specific information requirements of a particular industry. Examples of businesses with unique system needs

include colleges, banks, hospitals, insurance companies, construction companies, real estate firms, and airlines. Figure 7-3 shows a typical restaurant point-of-sale (POS) system operating on various devices. While these organizations may require vertical applications for specialized needs, they often use horizontal applications for more general business functions, like payroll processing and accounts payable.



**FIGURE 7-3** Restaurants use vertical applications like point of sale (POS) systems to support their unique business requirements.  
Source: rmpos

(Tilley and Rosenblatt, 2024)

When choosing among in-house software development options—whether developing a system, purchasing a software package, or customizing an existing package—each option has its advantages, disadvantages, and cost considerations, as outlined in Figure 7-4. These software acquisition alternatives are explained in greater detail in the following sections.

| REASONS FOR IN-HOUSE DEVELOPMENT                     | REASONS FOR PURCHASING A SOFTWARE PACKAGE     |
|------------------------------------------------------|-----------------------------------------------|
| Satisfy unique business requirements                 | Lower costs                                   |
| Minimize changes in business procedures and policies | Requires less time to implement               |
| Meet constraints of existing systems                 | Proven reliability and performance benchmarks |
| Meet constraints of existing technology              | Requires less technical development staff     |
| Develop internal resources and capabilities          | Future upgrades provided by the vendor        |
| Satisfy unique security requirements                 | Obtain input from other companies             |

**FIGURE 7-4** Companies consider various factors when comparing in-house development with the purchase of a software package.

(Tilley and Rosenblatt, 2024)

### 1.3.2 Developing Software In-House

With a wide range of software packages available to address both horizontal and vertical business operations, a company may still choose to develop its own software. Typically, firms opt for in-house development to meet unique business needs, minimize disruptions to current processes, accommodate existing systems and technology, and build internal resources and capabilities.

**Satisfy Unique Business Requirements:**

Companies often decide to develop software in-house because no existing commercial software can meet their specific business requirements. For example, a college may need a custom course scheduling system that considers curriculum needs, student demand, classroom availability, and instructors. Similarly, a package delivery company might require a system to optimize delivery routes and loading patterns for its fleet. When off-the-shelf software cannot fulfill these specialized needs, custom development becomes the best option.

#### **Minimize Changes in Business Procedures and Policies:**

Another reason a company may choose to build its own software is to avoid disrupting existing business operations. Installing new software usually requires adjustments to current procedures, but if an off-the-shelf package would cause too much disruption, developing custom software might be a preferable choice.

#### **Meet Constraints of Existing Systems:**

New software must often integrate with existing systems. For example, if a company installs a new budgeting system, it may need to work with its current accounting software. If finding a compatible software package proves difficult, the company might opt for in-house development to ensure seamless integration with their existing systems.

#### **Meet Constraints of Existing Technology:**

Developing software in-house may be necessary when a new system must work within the limitations of existing hardware or legacy systems. A custom design or an upgrade to the technology environment might be required to ensure the new system fits within these constraints. Systems analysts assess technical feasibility during the preliminary investigation phase, and later, during systems analysis, they evaluate whether in-house development is the best solution.

#### **Develop Internal Resources and Capabilities:**

By developing software in-house, companies can build and train an IT team that understands the organization's specific business needs. This internal team can provide a competitive advantage by responding quickly to business challenges or opportunities. Outsourcing, though attractive for short-term needs, may not result in a lower total cost of ownership (TCO) over time. Many top managers feel more comfortable with an internal IT team that offers long-term stability and can provide ongoing support. Additionally, in-house development allows a company to leverage its existing IT staff's skills, who are already part of the organization and compensated accordingly.

### **1.3.3 Purchasing a Software Package**

If a company decides against outsourcing, purchasing a commercially available software package can be an attractive alternative to developing its own software. There are several advantages to buying a software package instead of building custom software in-house:

#### **Lower Costs:**

Software packages are typically less expensive than custom-developed software, particularly in terms of initial investment. This is because the development costs are spread across many customers. However, while the upfront cost is lower, there may still be additional costs related to business disruptions, changes in business processes, and employee retraining.

#### **Requires Less Time to Implement:**

Commercial software packages are already designed, programmed, tested, and documented, which saves significant time compared to developing software from scratch. While installation and integration into the existing systems environment will still take time, it's generally quicker than building a system in-house. That said, the total cost of ownership (TCO) might still be higher due to training expenses and software customizations.

#### **Proven Reliability and Performance Benchmarks:**

If the software package has been on the market for some time, most major issues are likely already identified and fixed by the vendor. Popular software products are also often reviewed and rated by independent organizations, which can provide insights into their reliability and performance.

#### **Requires Less Technical Development Staff:**

By purchasing a software package, companies can reduce the number of programmers and systems analysts needed on the IT staff. The IT team can then focus on systems and projects that require custom solutions, rather than maintaining or developing software that's available off-the-shelf.

#### **Future Upgrades Provided by the Vendor:**

Vendors regularly upgrade their software packages, adding new features, enhancements, and improvements. These upgrades can include things like support for new hardware or storage technologies. Often, these updates are based on feedback from users, ensuring the software evolves to meet new needs.

#### **Input from Other Companies:**

Using a commercial software package allows companies to tap into the experiences of other organizations that have already implemented the software. This can include reaching out to other users for feedback, visiting their sites, or participating in user groups to share insights. These resources can help guide decision-making and provide valuable real-world perspectives before committing to the software.

### **1.3.4 Customizing a Software Package**

If a standard software package doesn't fully meet a company's needs, there are several ways to customize it. Here are three common approaches:

#### **1. Purchase a Basic Package with Vendor Customization:**

Many vendors offer a basic version of a software package with optional add-on components that can be customized to fit specific requirements. This option is useful when the standard application doesn't satisfy all the needs of every customer. For example, a human resources information system (HRIS) is often customizable because each company has its own way of handling employee compensation and benefits.

#### **2. Negotiate Directly with the Vendor for Enhancements:**

Companies can negotiate with the software vendor to make specific enhancements or changes to the software. This usually involves paying the vendor to make the adjustments needed to meet the company's requirements. This option can be effective but may come with additional costs.

#### **3. Purchase the Package and Modify It Internally:**

Some companies may choose to purchase the software and make their own modifications, provided this is allowed under the software's license terms. The downside of this approach is that the company's systems analysts and programmers may not be familiar with the software, which means they will need time to learn it and make the necessary changes correctly. Furthermore, if the software is customized, some of the benefits of purchasing a standard package—such as reduced implementation time and lower costs—may be lost.

#### **Additional Considerations:**

- **Cost and Time:** If the vendor customizes the software, the modified package may be more expensive and take longer to deliver.
- **Future Support:** One significant issue with customization is future support. While vendors regularly upgrade their standard packages, they may not support customized versions. If a company makes its own modifications, future

updates to the software may require the company to reapply their changes, as the vendor will not support custom modifications.

### 1.3.5 Creating User Applications

Business requirements can sometimes be met through user applications instead of formal information systems or commercial software packages. User applications, often part of user productivity systems, make use of standard business software like Microsoft Word or Excel, which is configured to enhance user productivity.

For example, an IT support person might configure a form letter in Microsoft Word, linking it to a spreadsheet in Excel that calculates incentives and discounts for a sales representative responding to customer price requests. The IT staff may also design a user interface with screens, commands, and features to improve user interaction with the application.

In certain situations, user applications can offer a simple, cost-effective solution. IT departments often have a backlog of projects, so smaller solutions for individual users or small groups may not be prioritized. However, application software is now more powerful, flexible, and user-friendly, with software suites from companies like Apple and Microsoft that allow for easier data exchange between programs. These suites often include tutorials, wizards, and help features that guide users, especially those who know what they need to do but aren't familiar with how to execute it.

Many companies empower lower-level employees by providing them with greater access to data and more powerful data management tools. This allows employees to perform their jobs more independently, without needing constant IT department intervention. This can be achieved by creating effective user interfaces for enterprise-wide applications like accounting, inventory, and sales systems, or by customizing standard productivity software like Microsoft Word or Excel to create user applications.

This empowerment strategy improves IT department productivity by reducing the time spent on daily user concerns and data requests, allowing the team to focus on higher-priority systems that support strategic business goals. While empowerment offers cost-saving benefits, it does require the company to provide technical support, such as hotline assistance, training, and guidance for users who need help. Additionally, if user applications access corporate data, proper controls must be in place to ensure data security and integrity. For example, some files should be hidden from view, while others can be read-only to prevent unauthorized changes.

#### CASE IN POINT 7.1: DOUG'S SPORTING GOODS

Doug's Sporting Goods sells hiking and camping supplies. The company has grown considerably in the past two years. They want to develop a customer order entry system and hired your IT consulting firm to advise them about development strategies. They are leaning toward in-house development because they do not want to depend on outside vendors and suppliers for technical support and upgrades. They also say they are not interested in selling on the Web, but that could change in the future. They want to meet with you tomorrow to make a decision. What will you say to them at the meeting?

(Tilley and Rosenblatt, 2024)

## 1.4 OUTSOURCING

Outsourcing refers to the practice of transferring the development, operation, or maintenance of information systems to an external firm that provides these services for a fee, either on a temporary or long-term basis. Outsourcing can cover a range of activities, from small programming tasks to more comprehensive services. It may include renting software from a service provider, outsourcing basic business processes (referred to as business process outsourcing, or BPO), or even managing a company's entire IT function.

### 7.4. 1 The Growth of Outsourcing

Traditionally, companies outsourced IT tasks to control costs and handle rapid technological changes. According to Oracle, businesses spend up to 80% of their IT budgets maintaining existing software and systems, which often forces IT managers to focus on managing upgrades rather than pursuing revenue-generating projects. While these reasons remain valid, outsourcing has evolved into a broader part of many organizations' IT strategies. This trend has also influenced software vendors, prompting them to adjust their marketing approaches.

A company that offers outsourcing solutions is called a service provider. Some service providers specialize in specific software applications, while others offer business services like order processing and customer billing. Additionally, there are service providers that offer enterprise-wide software solutions that integrate and manage functions such as accounting, manufacturing, and inventory control.

Two popular outsourcing options are **Application Service Providers (ASPs)** and **Internet Business Services (IBS)**:

- **Application Service Providers (ASPs):** An ASP is a firm that delivers a software application or access to an application, usually for a usage or subscription fee. ASPs provide more than just software licenses—they rent out an operational package to customers. ASPs typically offer commercially available software, such as databases and accounting packages. For example, if a company uses an ASP for a data management package, the company avoids the need to design, develop, implement, or maintain the software itself.
- **Internet Business Services (IBS):** IBS firms offer powerful web-based support for business transactions like order processing, billing, and customer relationship management (CRM). This service is also known as **managed hosting**, where an outside firm (the host) manages system operations. IBS solutions are attractive because they provide online data center support, mainframe computing power for mission-critical functions, and universal access via the Internet. Many companies, such as Rackspace, compete in the managed cloud services market.



**FIGURE 7-5** Rackspace managed cloud services.  
Source: Rackspace.

(Tilley and Rosenblatt, 2024)

### 1.4.2 Outsourcing Fees

Companies that offer **Software as a Service (SaaS)**, rather than selling a product outright, typically use fee structures based on how the application is utilized by customers during a specific time period. There are several common pricing models:

1. **Fixed Fee Model:** This model charges a set fee based on a specified level of service and user support, regardless of how much the application is used.
2. **Subscription Model:** Here, the fee is variable and depends on the number of users or workstations that have access to the application. As the number of users increases, the cost also increases.
3. **Usage or Transaction Model:** This model charges a fee based on the volume of transactions or operations performed by the application. The more the application is used, the higher the cost.

When a company is considering outsourcing, it should estimate the expected usage characteristics to determine which fee structure will be most suitable. Once the appropriate model is identified, the company can then negotiate a service provider contract that aligns with that model.

### 1.4.3 Outsourcing Issues and Concerns

When a company decides to outsource its IT functions, it makes a significant decision that can impact its resources, operations, and overall profitability. Outsourcing **mission-critical IT systems** should only be considered if it results in a cost-effective, reliable solution that aligns with the company's long-term strategy and acceptable levels of risk. Outsourcing IT work

overseas adds additional complexities, including concerns related to control, culture, communication, and security.

Besides the strategic long-term consequences, outsourcing also raises several concerns. For example, companies must entrust sensitive data to an external service provider, relying on them to ensure security, confidentiality, and quality. Before outsourcing, businesses need to thoroughly evaluate issues such as insurance, liability, licensing, information ownership, warranties, and disaster recovery plans.

Most importantly, a company must realize that the quality of the outsourcing solution depends on the capabilities of the outsourcing firm. Given the dynamic nature of the economy, where business failures and future uncertainties can arise, it is crucial to review the history and financial health of the outsourcing provider before committing.

Mergers and acquisitions can also affect outsourcing clients. Even if the outsourcing firm is large and financially stable, a merger or acquisition could disrupt services or impact the relationship with clients. If stability is crucial, these potential issues should be carefully considered.

Outsourcing can be particularly appealing for companies with fluctuating workloads, such as defense contractors, or for those lacking the time or expertise to manage tasks like application development. Outsourcing allows companies to avoid the burden of hiring additional IT staff during busy times and downsizing when work slows down. However, a significant downside of outsourcing is the potential impact on employee job security. Talented IT professionals often prefer positions where the company is committed to in-house IT development. If they feel insecure about their roles, they may opt to work directly for the service provider instead.

## 1.5 OFFSHORING

Offshoring, also known as offshore outsourcing or global outsourcing, is the practice of shifting IT development, support, and operations to other countries. Similar to the outflow of manufacturing jobs over several decades, many companies have moved their IT operations overseas.

IT work can be offshored more quickly than manufacturing because it's easier to transfer work via networks and send consultants around the globe than it is to ship raw materials, build factories, or deal with tariffs and transportation challenges. A few years ago, the consulting firm Gartner predicted the steady growth of offshore outsourcing, with the trend evolving from basic, labor-intensive tasks like maintenance and support to higher-level systems development and software design.

The primary motivation for offshoring is the same as for domestic outsourcing: **cost savings**. However, offshore outsourcing presents its own set of unique risks and concerns. For example, workers, customers, and shareholders in some companies have protested offshoring, raising awareness about its potential negative economic impact. Additionally, offshore outsourcing involves specific challenges, including:

- **Project control:** Managing projects remotely, especially when teams are spread out globally, can lead to difficulties in tracking progress and ensuring quality.
- **Security issues:** Outsourcing sensitive data or business functions abroad can expose companies to potential security risks, especially when different countries have varying data protection laws.
- **Cultural differences:** Misunderstandings or miscommunications may arise from differences in language, work habits, or business customs, which can affect productivity and relationships.
- **Effective communication:** Time zone differences and physical distance can make it difficult to maintain real-time communication, leading to delays or misalignment between teams.

These factors make offshore outsourcing a decision that requires careful consideration, weighing both the potential cost savings and the possible risks involved.

#### CASE IN POINT 7.2: TURNKEY SERVICES

Turnkey Services is an ASP that offers payroll and tax preparation services for hundreds of businesses in the Midwest. The firm is considering a major expansion into accounting and financial services and is looking into the possibility of supporting this move by hiring IT subcontractors in several foreign countries. Turnkey's president has asked you to help him reach a decision. Specifically, he wants you to cite the pros and cons of offshoring. He wants you to present your views at a meeting of Turnkey managers next week. How will you proceed?

(Tilley and Rosenblatt, 2024)

### 1.6 SOFTWARE AS A SERVICE

In the traditional software model, vendors develop and sell application packages to customers. Customers purchase licenses that grant them the right to use the software under the terms of the license agreement. While this model still represents a large portion of software acquisition, a newer model called **Software as a Service (SaaS)** is dramatically changing the landscape.

**SaaS** is a software deployment model where an application is hosted as a service and provided to customers over the internet. The key benefit of SaaS is that it reduces the customer's need for maintaining, operating, and supporting software. Essentially, SaaS offers the functionality customers need, but without the associated costs of development, infrastructure, and maintenance.

SaaS allows businesses to access software through a subscription or usage-based fee rather than a one-time license purchase, making it easier to scale and more cost-effective in the long term. Additionally, it ensures that customers always have access to the latest features, as the service provider handles updates and maintenance.

In a highly competitive market, major vendors are continuously working to deliver improved solutions. One prominent example of SaaS is **Office 365** by Microsoft. Office 365 is a full-fledged version of Microsoft Office that runs directly in a browser window, making it accessible from anywhere with an internet connection. This shift to SaaS has been widely adopted by both consumers and businesses due to its flexibility, ease of use, and reduced infrastructure requirements.



**FIGURE 7-6** Microsoft Office 365 provides web-based access to the complete Office suite.  
Source: Microsoft Corporation.

(Tilley and Rosenblatt, 2024)

## 1.7 SELECTING A DEVELOPMENT STRATEGY

Choosing the right development strategy is a critical decision that requires companies to evaluate several factors. The systems analyst plays a vital role in this process, especially when it comes to analyzing the costs and benefits of each development option. This analysis is crucial for providing management with objective data to make an informed decision. A cost-benefit checklist can be a helpful tool in guiding this evaluation.

### 1.7. 1 The Systems Analyst's Role

At some point during the systems development process, a company must decide whether to outsource, develop software in-house, acquire a software package, create user applications, or use a combination of these approaches. This decision depends on the company's current and projected future needs and will impact the subsequent phases of the SDLC and the systems analyst's involvement in the project. For example, choosing to develop software in-house typically requires more input from the systems analyst compared to outsourcing or opting for a commercial package. Management typically makes this decision after receiving written recommendations from the IT staff and a formal presentation, as discussed later in the chapter.

In some cases, a single system might incorporate a mix of software solutions. For instance, a company might purchase a standard payroll software package and develop custom software to manage the interface between that package and its in-house manufacturing cost analysis system.

The process of evaluating and selecting alternatives is complex. The goal is to choose the option with the lowest TCO, though actual costs and performance can be difficult to predict. When selecting hardware and software, systems analysts often work as part of a team to ensure all critical factors are considered, making the decision more comprehensive. Users should also be part of the team to contribute to the selection process and feel a sense of ownership in the new system.

The main goal of the evaluation and selection team is to rule out alternatives that don't meet requirements, rank feasible options, and present the viable alternatives to management for final approval. The process starts with a thorough analysis of the costs and benefits of each alternative.

### 1.7.2 Analyzing Cost and Benefits

Financial analysis tools have long been essential in helping individuals and organizations work with numbers, from the abacus to modern calculators. This section focuses on cost and benefit analysis and highlights popular tools that can assist a systems analyst in evaluating an IT project.

As discussed in Chapter 2, economic feasibility is one of the four feasibility measures conducted during the preliminary investigation of a systems request. Now, at the end of the systems analysis phase of the SDLC, financial analysis tools and techniques come into play to assess development strategies and determine the project's next steps. Part C of the Systems Analyst's Toolkit outlines three widely used tools: payback analysis, return on investment (ROI), and net present value (NPV). Payback analysis helps determine how long it will take for an information system to pay for itself through cost savings and increased benefits. ROI calculates a percentage rate that compares the total net benefits (the return) from a project to the total costs (the investment) of the project. NPV measures the total value of benefits minus the total costs, adjusting both for the point in time at which they occur.

These tools, along with others, can also be used to calculate Total Cost of Ownership (TCO), which was explained in Chapter 4. At this stage, the analyst will identify various systems development strategies and select a course of action. For instance, a company may find that the TCO of developing a system in-house is higher compared to outsourcing the project or using an Application Service Provider (ASP). Accurate forecasting of TCO is crucial, as nearly 80% of total costs occur after the hardware and software are purchased, according to Gartner, Inc. An IT department can either develop its own TCO estimates or use TCO calculators provided by vendors. For example, HP Enterprise offers several free TCO calculators to help determine the ROI of different development strategies and migration options, as shown in Figure 7-7.



**FIGURE 7-7** HP Enterprise provides several free TCO calculators.

Source: Hewlett Packard (HP)

(Tilley and Rosenblatt, 2024)

### 1. 7 .3 Cost-Benefit Analysis Checklist

In Chapter 2, we learned how to use the payback analysis tool during the preliminary investigation phase to assess a project's economic feasibility. Now, all the financial analysis tools will be employed to evaluate different development strategies. The most effective way to apply these tools is by creating a cost-benefit checklist, which includes the following steps:

- **List each development strategy being considered.**
- **Identify all costs and benefits for each alternative.** Be sure to specify when costs will be incurred and when benefits will be realized.
- **Consider future growth and the need for scalability.**
- **Include support costs for hardware and software.**
- **Analyze different software licensing options,** such as fixed fees and models based on the number of users or transactions.
- **Apply the financial analysis tools** (payback analysis, ROI, NPV) to each alternative.
- **Study the results** and prepare a report for management.

#### CASE IN POINT 7.3: STERLING ASSOCIATES

Sterling Associates specializes in advising clients on IT projects and information systems development. Marketing is creating a brochure for prospective new clients, and they want you to develop a section that describes payback analysis, ROI, and NPV in simple terms and mentions the pros and cons of each financial analysis tool. How do you proceed?

(Tilley and Rosenblatt, 2024)

## 1.8 THE SOFTWARE ACQUISITION PROCESS

Although each situation can vary, the following section outlines a typical example of the issues and tasks involved in software acquisition.

### Step 1: Evaluate the Information System Requirements

Based on the analysis of the system requirements, key features of the system must be identified, network and web-related issues must be considered, future growth and volume should be estimated, and any hardware, software, or personnel constraints should be specified. Additionally, an RFP (Request for Proposal) or a quotation should be prepared.

#### IDENTIFY KEY FEATURES:

Whether considering in-house development or outsourcing, the analyst must create a detailed, clear list of system features. These features will serve as the overall specification for the system. By using the data gathered during fact-finding (discussed in Chapter 4), the analyst will list all system requirements and critical features, which will be included in the system requirements document—the outcome of the SDLC systems analysis phase.

#### CONSIDER NETWORK AND WEB-RELATED ISSUES:

When evaluating the system requirements, network and web-related issues must be addressed. The analyst needs to determine whether the system will run on a network, the Internet, or a company intranet, and incorporate these requirements into the system design. Additionally, it must be established whether the system will need to exchange data with external vendor or customer systems, ensuring compatibility.

#### **ESTIMATE VOLUME AND FUTURE GROWTH:**

The analyst must understand the current transaction volume and forecast future growth. For example, Figure 7-8 shows volume estimates for an order processing system. Along with current levels, it displays two forecasts—one based on existing order processing procedures and another assuming a new website is in operation. A comparison of the forecasts reveals that the new website will generate more new customers, process almost 80% more orders, and significantly reduce the need for sales and support staff. If in-house development is being considered, it's crucial to ensure that both software and hardware can handle the projected future transaction volumes and data storage needs. On the other hand, if outsourcing is being considered, volume and usage data is essential for analyzing ASP fee structures and estimating outsourcing costs.

| Online Order Processing System Estimated Activity During Next 12-Month Period |               |                                                    |                                                           |
|-------------------------------------------------------------------------------|---------------|----------------------------------------------------|-----------------------------------------------------------|
|                                                                               | CURRENT LEVEL | FUTURE GROWTH<br>(based on existing<br>procedures) | FUTURE GROWTH<br>(assuming new website is<br>operational) |
| Customers                                                                     | 36,500        | 40,150                                             | 63,875                                                    |
| Daily Orders                                                                  | 1,435         | 1,579                                              | 2,811                                                     |
| Daily Order Lines                                                             | 7,715         | 7,893                                              | 12,556                                                    |
| Sales Reps                                                                    | 29            | 32                                                 | 12                                                        |
| Order Processing<br>Support Staff                                             | 2             | 4                                                  | 3                                                         |
| Products                                                                      | 600           | 650                                                | 900                                                       |

**FIGURE 7-8** Volume estimates for an order processing system showing current activity levels and two forecasts: one based on the existing order processing procedures and another that assumes a new website is operational.

(Tilley and Rosenblatt, 2024)

#### **SPECIFY HARDWARE, SOFTWARE, OR PERSONNEL CONSTRAINTS:**

The analyst must determine if any existing hardware, software, or personnel issues could impact the acquisition decision. For example, if the company has a large number of legacy systems or has adopted an Enterprise Resource Planning (ERP) strategy, these factors will influence the decision-making process. Additionally, the company's policy on outsourcing IT functions needs to be reviewed to assess whether outsourcing is part of the long-term strategy. Personnel issues should also be considered; specifically, the in-house staffing requirements needed for developing, acquiring, implementing, and maintaining the system.

The company must determine whether it is willing to commit to the necessary staffing levels for in-house development versus considering an outsourcing option.

### **PREPARE A REQUEST FOR PROPOSAL (RFP) OR REQUEST FOR QUOTATION (RFQ):**

To gather the information required to make an informed decision, the analyst should prepare either a Request for Proposal (RFP) or a Request for Quotation (RFQ). Although the documents are similar, they are used in different situations depending on whether a specific software product has been selected.

An **RFP** is a document that outlines the company's needs, including a description of the business, the IT services or products required, and the desired features. It helps ensure that the organization's business needs will be met. The RFP also specifies the required service and support levels. Vendors then review the RFP to determine if their product meets the company's needs. RFPs can vary significantly in size and complexity, depending on the system they describe. For large systems, an RFP might contain dozens of pages detailing unique requirements and features. It may also classify some features as essential and others as desirable. The RFP should request specific pricing and payment terms. Templates for RFPs can be found online.

When evaluating responses to an RFP, an **evaluation model** can be used to objectively measure and compare vendor proposals. An evaluation model provides a standardized approach to rating vendors. For example, Figure 7-9 shows two evaluation models for a network project. The first model simply lists the key elements and the score for each vendor. The second model introduces a weight factor, assigning different levels of importance to each element. While vendor A has the highest total score in the first model, vendor C comes out on top in the weighted model.

#### **Unweighted Evaluation Model for a Network Project**

| Instructions: Rate each vendor on a scale from 1 (low) to 10 (high), then add vendor scores to calculate total points. |          |          |          |
|------------------------------------------------------------------------------------------------------------------------|----------|----------|----------|
|                                                                                                                        | VENDOR A | VENDOR B | VENDOR C |
| Price                                                                                                                  | 6        | 5        | 9        |
| Completion Date                                                                                                        | 2        | 5        | 8        |
| Layout/Design                                                                                                          | 8        | 8        | 5        |
| References                                                                                                             | 10       | 6        | 3        |
| <b>TOTAL POINTS</b>                                                                                                    | 26       | 24       | 25       |

#### **Weighted Evaluation Model for a Network Project**

| Instructions: Rate each vendor on a scale from 1 (low) to 10 (high), then multiply the vendor's score by the weight factor. Add vendor scores to calculate total points. |               |               |              |              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|---------------|--------------|--------------|
|                                                                                                                                                                          | WEIGHT FACTOR | VENDOR A      | VENDOR B     | VENDOR C     |
| Price                                                                                                                                                                    | 25            | 6 * 25 = 150  | 5 * 25 = 125 | 9 * 25 = 225 |
| Completion Date                                                                                                                                                          | 25            | 2 * 25 = 50   | 5 * 25 = 125 | 8 * 25 = 200 |
| Layout/Design                                                                                                                                                            | 35            | 8 * 35 = 280  | 8 * 35 = 280 | 5 * 35 = 175 |
| References                                                                                                                                                               | 15            | 10 * 15 = 150 | 6 * 15 = 90  | 3 * 15 = 45  |
| <b>TOTAL POINTS</b>                                                                                                                                                      | 100           | 630           | 620          | 645          |

**FIGURE 7-9** The three vendors have the same initial ratings, but the two evaluation models produce different results. In the unweighted model at the top of the figure, vendor A has the highest total points. However, after applying weighting factors, vendor C is the winner, as shown in the model at the bottom of the figure.

(Tilley and Rosenblatt, 2024)

There is no standard method for assigning weight factors in evaluation models, as each firm tends to develop its own approach based on the specific needs and context of the project. Typically, the analyst will gather as much input as possible from relevant stakeholders and then circulate proposed weight factors for further discussion and, ideally, consensus. These models are versatile tools that can be utilized throughout the SDLC. Often, analysts use a spreadsheet program to build and test different weighting factors and graph the results for clearer insights.

A **Request for Quotation (RFQ)** is more precise than a Request for Proposal (RFP). It is used when the specific product or service is already known, and only price quotations or bids are required. RFQs may cover options such as outright purchases, leasing arrangements, and terms for maintenance or technical support. Some vendors even provide RFQ forms directly on their websites. While the formats differ, both RFPs and RFQs share the same goal: to receive vendor responses that are clear, comparable, and responsive, thus enabling a well-informed selection decision.

## **Step 2: Identify Potential Vendors or Outsourcing Options**

The next step in the acquisition process is identifying potential vendors or outsourcing providers. The internet serves as a primary marketplace for IT products and services, where detailed information about major products and acquisition alternatives can be easily accessed. For industry-specific solutions, vertical applications can be found, and industry trade journals or websites can offer reviews of software tailored to particular sectors. Trade groups in specific industries may also provide referrals to companies offering specialized software solutions.

Another approach is to work with a consulting firm. Many IT consultants offer expertise in helping companies select software packages. The benefit of using a consultant is the ability to leverage their broad experience, which might be difficult for an individual company to gain on its own. While hiring a consultant adds additional costs, it can help avoid more expensive mistakes in the selection process.

Furthermore, online forums provide a valuable resource. These forums, whether hosted by private or public entities, or located within larger communities such as Google Groups or Reddit, allow users to engage in discussions, offer support, and share ideas. A simple web search can uncover forums relevant to specific topics, or a company's website (such as Microsoft) may provide forums, blogs, webcasts, and other helpful resources for IT professionals, as illustrated in Figure 7-10.



**FIGURE 7-10** Microsoft Community is a valuable resource for IT professionals.  
Source: Microsoft Corporation

(Tilley and Rosenblatt, 2024)

### Step 3: Evaluate the Alternatives

Once the potential alternatives have been identified, the analyst needs to choose the one that best meets the company's needs. This evaluation should be thorough, using information from a variety of sources such as vendor presentations, product documentation, trade publications, and independent software testing organizations. A comprehensive search on the internet using relevant keywords can also help uncover more details about specific software packages. Vendor websites, as well as those maintained by consultants and software publishers, often provide product references and links to vendors. To ensure a complete evaluation, feedback from existing users, product testing, and benchmarking are essential components.

#### Existing Users:

A great source of information is feedback from existing users of the software. Many software vendors, particularly those offering large-scale packages, provide references from their current clients. These references can give valuable insight into how well the software performs in real-world environments similar to the company's needs. However, be cautious, as some vendors may only offer references from satisfied customers, which can skew the feedback positively.

#### Application Testing:

Testing the application in a real-world scenario is an important part of the evaluation process. For smaller or horizontal applications, organizations may request demo copies of the software and test it by inputting a few sample transactions. Larger or vertical applications may require more extensive testing, potentially involving a team of IT staff and users over several days or weeks.

### Benchmarking:

Benchmarking is another useful evaluation method, especially when trying to assess how well a package can handle large transaction volumes. Benchmark tests measure the time it takes for the software to process a set number of transactions, such as posting 1,000 sales transactions. However, keep in mind that benchmarks are typically conducted in controlled environments that may not exactly mirror the day-to-day operations of the company. While benchmarking results may not predict exact performance in the company's specific context, they provide a valuable comparison of competing products under standardized conditions.

Many IT publications regularly publish reviews of software packages, which often include benchmark test results. These publications may also offer surveys covering various types of software. With the rise of digital media, many of these publications are now available online or through mobile apps, often featuring additional features and search capabilities. Additionally, independent firms provide benchmarking services, selling comparative analyses of different software packages. An example of such a nonprofit organization is the Transaction Processing Performance Council (TPC), which publishes performance standards and reports.

Finally, to complete the evaluation, each software package should be compared against the features outlined in the RFP, and the alternatives should be ranked. If some features are more critical than others, they should be given a higher weight, as illustrated in the evaluation model shown in Figure 7-11.



**FIGURE 7-11** The Transaction Processing Performance Council (TPC) is a nonprofit organization that publishes standards and reports for its members and the general public.

Source: Transaction Processing Performance Council (TPC)

(Tilley and Rosenblatt, 2024)

### Step 4: Perform Cost-Benefit Analysis

To effectively evaluate the options, the analyst should use a spreadsheet to identify and calculate the Total Cost of Ownership (TCO) for each alternative under consideration. This includes listing all relevant costs, using the volume forecasts prepared earlier. If outsourcing is one of the options, the analyst should study the fee structure models for each alternative.

Where possible, it's useful to present the results visually through charts and include "what-if" scenarios to assess the impact of potential changes in variables.

For software packages, acquisition options must be thoroughly considered. Purchasing software typically involves obtaining a software license, which grants the purchaser the right to use the software under specific terms. These licenses could limit usage to a single computer, a network, or an entire site, depending on the terms. In the case of large-scale systems, license agreements can sometimes be negotiated to better fit the company's needs. Additionally, it's essential to factor in user support costs, which can be a significant part of the TCO.

If outsourcing is selected, the contract will usually include technical support and maintenance, which must also be factored into the cost analysis. For in-house development, the costs of providing these services will need to be considered. If a software package is purchased, a maintenance agreement should be considered. This agreement may offer full support for a set period or charge fees for specific services. Some software vendors provide free support for a limited time, but after that period, support might be available for an additional cost—either per occurrence or based on the time spent. Vendors may also offer discounted prices for new software releases to registered owners.

### **Step 5: Prepare a Recommendation**

After completing the analysis, the systems analyst should prepare a formal recommendation that evaluates each alternative, highlighting the associated costs, benefits, advantages, and disadvantages. This recommendation should present a clear picture of the best option for the organization. It might also be necessary to submit a formal system requirements document along with a management presentation. Suggestions for preparing these documents, as well as tips for creating effective oral presentations, can be found in Part A of the Systems Analyst's Toolkit. Additional guidance for preparing the system requirements document and management presentations is provided in the following section.

## **1.9 COMPLETION OF SYSTEMS ANALYSIS TASKS**

### **1.9.1 System Requirements Document**

The system requirements document plays a critical role in finalizing the systems analysis phase. It contains the identified requirements for the new system, provides a summary of the alternatives that were considered, and makes a clear recommendation to management. This document is essential for measuring the performance, accuracy, and completeness of the finished system before transitioning to the systems design phase.

The system requirements document serves as a type of contract that outlines what the system developer must deliver to the users. During the fact-finding process, system requirements are gathered and compiled into a checklist. The document should be written in clear language so

that users can easily understand it, offer input, suggest improvements, and approve the final version.

The document should be well-organized and formatted for easy reading. It should include a cover page, a detailed table of contents, and potentially an index or glossary of terms for ease of reference. The content will vary depending on the complexity of the system and the company's specific needs.

### **1.9.2 Presentation to Management**

The presentation to management at the end of the systems analysis phase is one of the most important milestones in the system development process. At this point, key decisions are made regarding the future direction of the project. Before presenting to management, presentations should be given to key individuals in the IT department to keep them informed and to users to answer their questions and receive feedback.

The system requirements document (or a summary of it) should be distributed in advance for review. When preparing the presentation, keep in mind the following steps:

- Start with an overview of the project's purpose, objectives, and what decisions need to be made.
- Summarize the key alternatives, listing their costs, advantages, and disadvantages.
- Justify why the recommended alternative was chosen by the evaluation and selection team.
- Allow time for discussion and a Q&A session.
- Aim to obtain a final decision from management or establish a timeline for the next steps.

The objective of the presentation is to secure approval from management to move forward with system development and to secure the necessary financial resources. Management will typically choose from five options: develop an in-house system, modify an existing system, purchase or customize a software package, conduct more systems analysis, or discontinue the project entirely. The systems analyst's role will depend on which option is selected, such as working with an outsourcing provider if that option is chosen.

### **1.9.3 Transition to Systems Design**

Once the systems analysis phase is complete, the transition to systems design begins. Traditionally, systems design followed directly after the systems analysis phase, with the system requirements document serving as a blueprint for developers to turn the logical design into a working model. However, with the introduction of agile development, the process has become more dynamic and user-oriented. Agile methods offer more flexibility and speed, but in many cases, a blend of traditional and modern methods may be used depending on the specific needs of the project.

Regardless of the development method, systems design requires accurate documentation. While agile methods do not always mandate formal documentation, a successful development team must still capture and understand user requirements as they evolve throughout the project.

A logical design defines what must occur in the system but does not specify how it will be achieved. It represents the system's functionality and structure. In contrast, a physical design provides the "blueprints" for the system's construction, outlining the processes for data entry, verification, and storage, the physical layout of data files, sorting procedures, and report formats.

While logical and physical designs are closely related, accurate systems analysis is crucial for effective design. If new issues emerge during the design phase, such as overlooked details, evolving user needs, or changes in legal or governmental requirements, the analyst may need to revisit the fact-finding process to ensure the design is still valid. This integration between analysis and design helps ensure a successful system implementation.

### A QUESTION OF ETHICS

Stock.com/faberfoto\_8

A junior analyst at a medium-sized IT consulting firm has been asked by her manager to draft a response to an RFP from a large company that is seeking IT consulting services in connection with a new accounting system.

As the analyst worked on the RFP, she noticed a specific question about her firm's recent experience on this type of system. To the best of her knowledge, the firm has only worked on one other accounting project in the past three years. When the manager saw the analyst's draft response, he was upset about the way she answered the question. "You don't have to be quite that candid," he said. "Even though we only had one *formal* project, we do have several people who worked on accounting systems before they came here."

"Yes," the analyst replied, "But that isn't what the question is asking." As he left her office, the manager's final comment was, "If we want that job, we'll have to come up with a better answer." Thinking about it, the analyst isn't comfortable with anything but a straight answer. Is this an ethical question? What are her options?

(Tilley and Rosenblatt, 2024)

## 1. I O SUMMARY

This chapter covers the strategic decisions involved in system development, particularly the preparation and presentation of the system requirements document. It contrasts traditional systems, which often integrate with legacy systems and operate within specific company networks, with web-based systems that treat the web as the core platform. It also introduces modern development environments like .NET and MERN, as well as various outsourcing options such as Application Service Providers (ASPs) and Internet Backbone Services (IBSs).

### Key Points:

#### 1. Traditional vs. Web-Based Systems:

- **Traditional Systems:** These systems are designed to function within specific hardware and software environments and integrate with existing legacy systems. They also utilize Internet resources as enhancements but rely on more defined technical infrastructure.

- **Web-Based Systems:** These systems are more scalable, less dependent on specific hardware, and better suited for outsourcing the operations and support of the software. The evolution of the web, especially Web 2.0, has enabled platforms that emphasize collaboration, information sharing, and social networking.
- 2. **Cloud Computing and SaaS:**
  - **Cloud Computing:** This model allows companies to access software and data over the Internet using powerful cloud-based infrastructure, reducing the need for in-house servers and resources.
  - **SaaS (Software as a Service):** SaaS is a model in which software applications are provided as a service over the Internet, eliminating the need for traditional software installation and maintenance.
- 3. **In-House Development vs. Commercial Software Packages:**
  - **In-House Development:** This approach requires more resources but can be beneficial if a company needs a solution that is highly specific to its business requirements. It involves extensive involvement from systems analysts.
  - **Commercial Software Packages:** These are attractive alternatives as they tend to be cheaper, faster to implement, have a proven track record, and are regularly upgraded. Customization can be done if the package doesn't meet all business needs.
- 4. **Offshoring and Outsourcing:**
  - **Offshoring:** The practice of moving IT development and support overseas to lower costs, much like the outsourcing of manufacturing jobs in the past. While cost savings are the primary driver, offshoring introduces unique risks such as communication challenges and cultural differences.
- 5. **Role of the Systems Analyst:**
  - The systems analyst's role varies depending on the development strategy chosen. In-house development requires a deeper level of involvement in system design, testing, and implementation. On the other hand, outsourcing or using commercial software packages requires less involvement, as the vendor manages much of the development and implementation.
- 6. **Total Cost of Ownership (TCO):**
  - The most important factor when selecting a development strategy is TCO, which takes into account not just the initial cost of development but the long-term costs, including maintenance, support, and upgrades.
  - Financial analysis tools like **payback analysis**, **ROI (Return on Investment)**, and **NPV (Net Present Value)** are used to evaluate the financial impact of the different development options.
- 7. **Software Acquisition Process:**
  - This process involves several steps: evaluating system requirements, considering network and web-related issues, identifying potential vendors or outsourcing options, evaluating alternatives, conducting a cost-benefit analysis, preparing a recommendation, and implementing the solution.
  - **RFPs (Request for Proposals)** and **RFQs (Request for Quotations)** are key tools in the acquisition process. An **RFP** is used to invite vendors to respond with proposals that meet a set of system requirements, while an **RFQ** seeks bids for a specific product or service.

#### **8. System Requirements Document:**

- The system requirements document is the output of the systems analysis phase and provides detailed specifications for the system, along with cost and time estimates. It is the foundation for the management presentation, where decisions are made about whether to proceed with in-house development, modify the current system, purchase or customize software, or stop the project altogether.

#### **Conclusion:**

In this chapter, the focus was on the variety of development strategies, tools for evaluating software acquisition, and methods for presenting findings to management. The chapter emphasizes that the choice of development strategy—whether in-house development, customizing existing software, outsourcing, or choosing a SaaS model—depends heavily on the organization's needs, budget, and long-term objectives. A careful analysis of system requirements, costs, benefits, and risks is crucial for making an informed decision on the best path forward.

## Key Terms

**application service provider (ASP)** A firm that delivers a software application, or access to an application, by charging a usage or subscription fee.

**benchmark** A measure of the time a package takes to process a certain number of transactions.

**build or buy** A choice between developing in-house software and purchasing software, often called a build or buy, or make or buy, decision.

**business process outsourcing (BPO)** The outsourcing of a basic business process. *See also* outsourcing.

**cloud computing** An online software and data environment in which applications and services are accessed and used through an Internet connection rather than on a local computer; refers to the cloud symbol for the Internet.

**evaluation and selection team** A group of people involved in selecting hardware and software. The group includes systems analysts and users. A team approach ensures that critical factors are not overlooked and that a sound choice is made.

**evaluation model** A technique that uses a common yardstick to measure and compare vendor ratings.

**fixed fee model** A service model that charges a set fee based on a specified level of service and user support.

**forum** An online discussion on a particular topic, where people meet, offer support, and exchange ideas.

**global outsourcing** The practice of shifting IT development, support, and operations to other countries.

**horizontal application** A software package that can be used by many different types of organizations.

**in-house software** An information center or help desk within the IT department responsible for providing user support and offering services such as hotline assistance, training, and guidance to users who need technical help.

**Internet business services (IBSs)** Services that provide powerful web-based support for transactions such as order processing, billing, and customer relationship management.

**logical design** The definition of an information system's functions and features, and the relationships among its components.

**maintenance agreement** A specification of the conditions, charges, and time frame for users to contact the vendor for assistance when they have system problems or questions.

**make or buy** The choice between developing in-house software and purchasing software often is called a make or buy, or build or buy, decision.

**managed hosting** An operation is managed by the outside firm, or host. Another term for IBSs.

**middleware** Software that connects dissimilar applications and enables them to communicate and exchange data. For example, middleware can link a departmental database to a web server that can be accessed by client computers via the Internet or a company intranet.

**mobile device** Smartphones, tablets, and other computing devices that are not permanently tethered to a desk. They connect to the network wirelessly.

**net present value (NPV)** The total value of the benefits minus the total value of the costs, with both the costs and benefits being adjusted to reflect the point in time at which they occur.

**offshore outsourcing** The practice of shifting IT development, support, and operations to other countries.

**offshoring** *See* offshore outsourcing.

**outsourcing** The transfer of information systems development, operation, or maintenance to an outside firm that provides these services, for a fee, on a temporary or long-term basis.

**payback analysis** A determination of how long it takes an information system to pay for itself through reduced costs and increased benefits.

**physical design** A plan for the actual implementation of the system.

**request for proposal (RFP)** A written list of features and specifications given to prospective vendors before a specific product or package has been selected.

**request for quotation (RFQ)** Used to obtain a price quotation or bid on a specific product or package.

**return on investment (ROI)** A percentage rate that measures profitability by comparing the total net benefits (the return) received from a project to the total costs (the investment) of the project.  $ROI = (\text{total benefits} - \text{total costs}) / \text{total costs}$ .

**service desk** A centralized resource staffed by IT professionals that provides users with the support they need to do their jobs. Also called help desk.

**service provider** A firm that offers outsourcing solutions. Two popular outsourcing options involve ASPs and firms that offer IBSs.

**Software as a Service (SaaS)** A model of software delivery in which functionality is delivered on demand as a network-accessible service, rather than as a traditional software application that is downloaded and installed on the customer's computer.

**software license** A legal agreement that gives users the right to use the software under certain terms and conditions.

**software package** Software that is purchased or leased from another firm. A commercially produced software product, or family of products.

**system requirements document** A document that contains the requirements for the new system, describes the alternatives that were considered, and makes a specific recommendation to management. It is the end product of the systems analysis phase.

**software vendor** Company that develops software for sale.

**subscription model** A service model that charges a variable fee for an application based on the number of users or workstations that have access to the application.

**transaction model** A service model that charges a variable fee for an application based on the volume of transactions or operations performed by the application. Also called a usage model.

**usage model** See **transaction model**.

**user application** Programs that utilize standard business software, such as Microsoft Office, which has been configured in a specific manner to enhance user productivity.

**user interface** Includes screens, commands, controls, and features that enable users to interact more effectively with an application. See also **graphical user interface (GUI)**.

**value-added reseller (VAR)** A firm that enhances a commercial package by adding custom features and configuring it for a particular industry.

**vertical application** A software package that has been developed to handle information requirements for a specific type of business.

**Web 2.0** A second generation of the web that enables people to collaborate, interact, and share information much more dynamically, based on continuously available user applications rather than static HTML web pages. Interactive experience is a hallmark of Web 2.0.

(Tilley and Rosenblatt, 2024)

## EXERCISE

### Questions

1. List three characteristics each of traditional and web-based development.
2. How does cloud computing support Web 2.0 applications?
3. Why would a company choose in-house software development?
4. What is outsourcing?
5. List two reasons offshoring may be risky.
6. What is SaaS?
7. What is the primary objective of the evaluation and selection team in selecting a development strategy?
8. What are the five steps in the software acquisition process?
9. What is an RFP, and how does it differ from an RFQ?
10. Explain the relationship between logical and physical design.

### Discussion Topics

1. How has the proliferation of mobile devices affected IT professionals?
2. As more companies outsource systems development, will there be less need for in-house systems analysts? Why or why not?
3. Select a financial SaaS application, and describe its main features.
4. Suppose you tried to explain the concept of weighted evaluation models to a manager, and she responded by asking, "So, how do you set the weight factors? Is it just a subjective guess?" How would you reply?
5. What decisions might management reach at the end of the systems analysis phase, and what would be the next step in each case?

### Projects

1. Investigate the ROI of cloud-based software development environments.
2. Many financial tools are developed using Microsoft Excel. Identify three applications built using Excel, and describe how they are used in an organization.
3. Visit the IT department at your school or at a local company, and determine whether the systems were developed in-house or purchased. If packages were acquired, find out what customizing was done, if any. Write a brief memo describing the results.
4. Profile three companies that provide offshore software development services.
5. Various firms and organizations offer IT benchmarking. Locate an example on the Internet, and describe its services.

## CHAPTER 2: User Interface Design



### LEARNING OUTCOMES

After reading this Section of the guide, the learner should be able to:

- Explain user interfaces
- Explain the concept of human computer interaction, including user-friendly interface design
- Summarize the seven habits of successful interface designers
- Summarize the 10 guidelines for user interface design
- Design effective source documents and forms
- Explain printed output report design guidelines and principles
- Describe three types of printed output reports
- Discuss output and input technology issues
- Describe output and input security and control issues
- Explain emerging user interface trends, including modular design, responsive web design, and

### 2.1 USER INTERFACES

A user interface (UI) refers to how users interact with a computer system, encompassing the hardware, software, screens, menus, functions, outputs, and features that facilitate two-way communication between the user and the computer. The UI plays a crucial role in usability, which impacts user satisfaction, business function support, and system effectiveness.

In the past, chapters on UI design typically focused on output, since users interacted with and relied on this to perform their tasks. However, the landscape has changed for several reasons:

- Users now have the ability to customize their own outputs. System designers are more attuned to user needs, allowing systems to maintain data integrity while enabling users to filter, sort, and view data in ways that best help them with their work. In earlier times, the MIS department made these decisions, with limited input from users. Nowadays, successful applications are designed by first understanding user

requirements and then creating a design that fulfills both user needs and corporate goals.

- Centralized IT departments no longer produce large volumes of printed reports, which often went unused and gathered dust. Instead, the trend now is for output that is customized by the users themselves, whether individually or within a community, like a department. As noted in Chapter 4, IT teams must understand user needs before creating a solution.
- The user interface has evolved into a dynamic, two-way communication channel with powerful output capabilities. Users can now access data in ways that meet their needs—viewing, printing, or saving it directly from the screen. In contrast, earlier interfaces were often simple, character-based screens that might offer limited menu options, and an improperly entered command would result in an error message, frustrating users and reducing productivity. Many hardware-centric vendors of that era did not fully grasp the importance of user interface design.

Apple was a pioneer in UI development, introducing the graphical user interface (GUI) in the early 1980s, complete with a mouse and screen icons. While this concept initially faced resistance, Microsoft's adoption of the GUI with its Windows operating system made the idea widely accepted, with many users, including managers, wondering how they ever worked without it.

Industry experts generally believe that the best user interfaces are those that users don't even notice—they function seamlessly and intuitively. For example, Apple has long distinguished itself through its focus on creating intuitive interfaces, contributing to the company's market success. This approach suggests that consumers are willing to pay a premium for products that "just work."

In the past, when developing older systems, analysts typically focused first on designing printed and screen output and then worked on the necessary inputs to generate these results. This approach was effective for traditional systems that primarily transformed input data into structured output.



**FIGURE 8-1** Apple has long been a leader in creating elegant user interfaces for its products.  
Source: Apple Inc.

(Tilley and Rosenblatt, 2024)

As information management evolved from centralized data processing to dynamic, enterprise-wide systems, the focus shifted from being centered on the IT department to being centered on the users themselves. The IT department transitioned from being a provider of information to being a supplier of information technology. Today, the main emphasis is on how users—both within and outside the company—interact with the information system and how the system supports the business operations of the organization.

In a user-centered system, the lines between input, output, and the interface become less distinct. Most users engage with a combination of inputs, screen outputs, and data queries as they perform their daily job functions. Since all these tasks involve interaction with the computer system, the user interface becomes a crucial component of the system design process.

Designing a user interface requires an understanding of human-computer interaction and user-centered design principles, which are explored in the next section.

## **2.2 HUMAN-COMPUTER INTERACTION**

A user interface is built upon fundamental principles of human-computer interaction (HCI), which describes the relationship between people and computers as they perform tasks, like the worker shown in Figure 8-2. HCI concepts are applicable to everything from smartphones to global networks. In its broadest sense, HCI involves all the communication and instructions needed to input data into the system and receive output, whether through screen displays or printed reports.

Early user interfaces required users to type complex commands on a keyboard, which appeared as green text on a black screen. Then, the introduction of the graphical user interface (GUI) marked a major improvement, as it incorporated icons, graphical objects, and pointing devices. Today, designers aim to create interfaces that reflect user behavior, needs, and desires in a way that users hardly notice. As IBM has stated, the best user interfaces are "almost transparent—you can see right through the interface to your own work." In other words, a transparent interface does not distract the user and doesn't draw attention to itself.

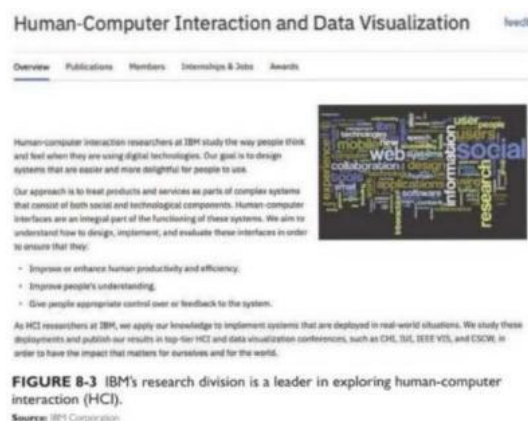


**FIGURE 8-2** HCI is essential to employee productivity, whether the work is done in a traditional office setting or on a construction site like the one shown in this figure.  
grodan/Shutterstock.com

(Tilley and Rosenblatt, 2024)

A systems analyst is responsible for designing user interfaces for software developed in-house and customizing interfaces for various commercial packages and productivity applications. The primary goal is to create designs that are user-friendly, making the software easy to learn and use.

Major industry players like Microsoft and IBM invest significant resources into user interface research. For instance, IBM Research focuses on human-computer interaction (HCI) with the goal of "designing systems that are easier and more delightful for people to use," as shown in Figure 8-3.



(Tilley and Rosenblatt, 2024)

Because human-computer interaction (HCI) significantly affects user productivity, it receives a great deal of attention, especially when it comes to high-stakes situations involving multimillion-dollar decisions. For example, in the article "*Human-Computer Interaction in Electronic Medical Records: From the Perspectives of Physicians and Data Scientists*" by E. Bologva et al., published in *Procedia Computer Science 100* (Elsevier, 2016), the authors discuss how software usability plays a crucial role in the medical field. Not everyone is satisfied with the current systems, particularly physicians who often struggle with poorly designed electronic health record (EHR) systems. Ms. Gardner, in her article, highlights that

physicians frequently multitask—answering questions about one patient while prescribing medication for another—but EHR software was not designed to accommodate this kind of workflow.

**CASE IN POINT 8.1: CASUAL OBSERVER SOFTWARE**

Casual Observer Software's main product is a program that monitors and analyzes user key-strokes and mouse clicks to learn more about the way employees use their computer systems. The problem is that some users feel this is an unwarranted intrusion into their privacy, and they prefer not to be observed. Some even fear that the data would be used for other reasons, including performance appraisal. You are a consultant who has been hired by a client firm that is trying to decide whether or not to use this software.

Before you advise the client, remember the Hawthorne effect, which suggests that employees might behave differently when they know they are being observed. Finally, think about the ethical issues that might be involved in this situation. What will you advise your client, and why?

(Tilley and Rosenblatt, 2024)

## 2.3 GUIDELINES FOR USER INTERFACE DESIGN

Although IT professionals may have varying opinions on interface design, most agree that good design is based on seven key principles. Successful interface designers consistently apply these principles, which become second nature over time. These principles are outlined in the following sections.

### 2.3.1 Understand the Business

A designer must comprehend the underlying business functions and how the system supports individual, departmental, and enterprise goals. The primary goal is to create an interface that aids users in performing their jobs effectively. A good starting point might be analyzing a Functional Decomposition Diagram (FDD), which visually represents business functions, breaking them down into various levels of detail. The FDD can serve as a checklist for the tasks that must be included in the interface design.

### 2.3.2 Maximize Graphical Effectiveness

Research indicates that people tend to learn better visually. The popularity of Apple's iOS and Microsoft Windows is largely due to their easy-to-use and visually intuitive GUIs. A well-designed interface can help users quickly learn new systems and become more productive. Additionally, graphical interfaces allow users to display and work with multiple windows on a single screen and transfer data between programs. If the interface involves data entry, it should adhere to established guidelines for designing data entry screens.

### 2.3.3 Think Like a User

A systems analyst should understand the users' experiences, knowledge, and skill levels. If there's a wide range of capabilities, the interface must be flexible enough to accommodate both novices and experienced users. To develop a user-centered interface, the designer should adopt the user's perspective, ensuring the system is easy to learn and uses terms and metaphors familiar to the users. Users will expect feedback based on their real-world experiences with other devices like cars, ATMs, and microwaves. The interface must be

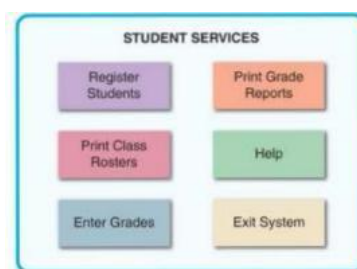
intuitive, forgiving of errors, and provide attractive, understandable output with the appropriate level of detail.

### 2.3.4 Use Models and Prototypes

For users, the interface is the most critical aspect of the system since it's where they interact with it, often for long periods. It's essential to create models and prototypes for user feedback. Initial designs can be presented as storyboards—sketches showing the general layout. Feedback can be gathered through interviews, questionnaires, and direct observation. Usability metrics, measured through software tracking user interactions, can also provide valuable data.

### 2.3.5 Focus on Usability

The user interface should include all necessary tasks, commands, and communications between users and the system. The opening screen should display the main options, with each option leading to more detailed choices. The goal is to provide a manageable number of options that are easy for users to understand. Too many options on one screen can overwhelm users, while too few can complicate navigation with multiple submenus. A common strategy is to present the most common choice as a default, but allow users to select other options if needed.



**FIGURE 8-4** The opening screen displays the main options for a student registration system. A user can click an option to see lower-level actions and menu choices.

(Tilley and Rosenblatt, 2024)

### 2.3.6 Invite Feedback

Even after the system is operational, it's important to monitor its usage and solicit user suggestions. Analysts can observe and survey users to ensure features are being used as intended. Sometimes, real-world operations expose problems not evident during prototype testing. Based on user feedback, help screens may need revisions, and design changes may be necessary to enhance system performance.

### 2.3.7 Document Everything

All screen designs should be thoroughly documented for later use by programmers. If a CASE tool or screen generator is used, designs should be numbered and saved in a hierarchical structure similar to a menu tree. User-approved sketches and storyboards can also serve as documentation for the user interface.

By following these basic user-centered design principles, a systems analyst can effectively plan, design, and deliver a successful user interface.

## 2.4 GUIDELINES FOR USER INTERFACE DESIGN

A system may have advanced technology and powerful features, but its true success depends on whether users like it and feel that it meets their needs. Below are general guidelines for successful user interface design, based on years of industry best practices. While there is some overlap between these guidelines, as many share common elements, they provide a solid foundation for traditional systems development. Keep in mind that user interface design for web applications or mobile apps requires additional considerations beyond these general guidelines. The most important takeaway is that not all of these recommendations must be followed—ultimately, the best interface is the one that works best for the users.

### 2.4.1 Create an Interface That Is Easy to Learn and Use

1. **Focus on system design objectives:** The goal should be on the system's functionality, not just drawing attention to the interface.
2. **Ensure the design is easy to understand and remember:** Consistency is key across all modules of the interface. This includes maintaining uniformity in color schemes, screen placements, fonts, and the overall "look and feel."
3. **Make commands, actions, and system responses consistent and predictable:** Users should be able to predict what will happen when they interact with the interface.
4. **Allow users to easily correct errors:** Design the system so users can easily fix mistakes without frustration.
5. **Label all controls, buttons, and icons clearly:** Labels should be intuitive and easy to understand.
6. **Use familiar images and provide clear on-screen instructions:** Select images that users can easily recognize, and provide concise, logical, and clear instructions. For example, in Figure 8-5, the top screen shows control buttons without clear meaning, while the bottom screen's instructions are more understandable.
7. **Show all available commands in a list, and dim unavailable ones:** Ensure that users can see all possible commands, with unavailable options clearly dimmed to avoid confusion.
8. **Make navigation easy:** Allow users to easily navigate through the menu structure and return to any level without difficulty.



**FIGURE 8-5** In the example at the top, the icons do not have a clear message. In the Help text examples at the bottom, only one message is understandable. The others would frustrate and annoy most users.

(Tilley and Rosenblatt, 2024)

## 2.4.2 Enhance User Productivity

The interface is where users interact with the system, and it can significantly impact their productivity. A well-designed interface can empower users to handle more complex tasks, thus increasing productivity. On the other hand, a poorly designed interface can frustrate users and lower their efficiency.

1. **Organize tasks and commands logically:** Group related tasks, commands, and functions in a way that mirrors actual business operations. Organizing them into a multilevel menu hierarchy or tree structure that reflects how users typically perform tasks makes navigation intuitive. For example, Figure 8-6 shows a logical menu hierarchy for an order tracking system.
2. **Menu organization:** Consider creating alphabetical menus or placing frequently used selections at the top. There's no universally accepted approach to menu item placement, so it's best to design a prototype and get feedback from users. Some systems allow recently used commands to appear first, though this feature may be distracting to some users. Offering a choice for users to decide works best.



**FIGURE 8-6** This menu hierarchy shows tasks, commands, and functions organized into logical groups and sequences. The structure resembles a functional decomposition diagram (FDD), which is a model of business functions and processes.

(Tilley and Rosenblatt, 2024)

3. **Provide shortcuts for experienced users:** Shortcuts, like hotkeys, can help experienced users navigate the system faster by skipping multiple menu levels. For example, using the "Alt" key and an underlined letter to access a command quickly.
4. **Use default values:** If most users will frequently enter the same data in a field, set a default value. For example, if most customers are from Albuquerque, set that city as the default value in the "City" field.
5. **Enable duplicate value functionality:** Allow users to automatically insert the value from the same field in the previous record. However, let users turn this feature on or off based on their preferences.
6. **Include a fast-find feature:** Implement a fast-find option that shows a list of possible values as soon as users type a few letters, making it easier to find the correct data.
7. **Consider natural language capabilities:** If available, consider adding a natural language feature that allows users to type commands or requests in regular text. Many applications already use this feature to enable users to request Help by typing a question, with the software using natural language processing to display relevant topics. This technology is also found in speech recognition systems, text-to-speech tools, automated voice responses, search engines, text editors, and language-learning applications.

### 2.4.3 Provide Flexibility

Imagine a scenario where a user wants to view all customer balances greater than \$5,000 in an accounts receivable system. One approach could be for the system to automatically check against a fixed value of \$5,000. While this is simple for both the programmer and user (no extra keystrokes are needed), it lacks flexibility. A better solution would be to allow the user to input the amount themselves, or start with a default value that is pre-filled. The user could then press ENTER to accept the default or enter a different value. In many cases, offering several options for users to choose from can be the best approach, enabling them to tailor the experience to their needs.

### 2.4.4 Provide Users with Help and Feedback

This guideline is crucial as it directly impacts the user experience. The goal is to make Help easy to access but unobtrusive when not needed.

1. **Make Help always available on demand:** Help should be easy to find whenever users need it, offering information on menu options, procedures, shortcuts, and error messages.
2. **Offer user-selected and context-sensitive help:**
  - **User-selected help** allows users to search for information by navigating through menus and submenus to reach relevant content.
  - **Context-sensitive help** offers immediate assistance based on the task the user is performing.



**FIGURE 8-7** The main Help screen for a student registration system.

(Tilley and Rosenblatt, 2024)

For example, Figure 8-7 shows a main Help screen for a student registration system.

3. **Provide a clear way for users to return to where they left off:** Every help screen should be clearly titled to identify the topic, and the content should be simple, concise, and easy to read. Use blank lines between paragraphs for better readability and include examples when appropriate.
4. **Include contact information:** Always provide users with a way to contact support (e.g., a phone extension or email address) if further help is needed.
5. **Confirm before deleting data:** Always ask users to confirm before deleting data (e.g., "Are you sure?") and provide a way to recover data if deleted by mistake. Ensure there are safeguards in place to prevent accidental changes or erasure of critical data.
6. **Provide an "Undo" option:** An "Undo" key or menu option allows users to reverse their most recent action or command.
7. **Highlight errors in user input:** If a user enters an incorrect command, highlight the part of the command that is incorrect and let the user correct it without re-entering the entire command.
8. **Use hypertext links:** Hyperlinks can help users navigate between related help topics efficiently.
9. **Display messages logically:** Ensure messages appear in logical locations on the screen and maintain consistency.
10. **Alert users to delays:** If a process is taking a long time, alert the user with an on-screen progress report. This is especially important for long delays.
11. **Allow messages to remain visible:** Messages should stay on the screen long enough for users to read them. In some cases, messages should remain until the user takes an action.
12. **Notify users of task success or failure:** After completing a task, inform users whether it was successful. Examples include "Update completed," "All transactions have been posted," or "ID Number not found."

13. **Provide text explanations for icons or images:** If an icon or image is used on a button, display a text explanation when the user hovers the mouse over it.
14. **Use clear, professional, and specific messages:** Avoid vague or cryptic messages. For example, instead of "ERROR – Unacceptable value," use a more specific message like "Enter a number between 1 and 5" or "Customer ID must be numeric."

### 2.4.5 Create an Attractive Layout and Design

Designing an attractive interface can be subjective, as people have different preferences. However, the analyst must focus on factors like color, layout, and ease of use. To get user feedback, screen mock-ups and menu trees can be tested. In case of uncertainty, it's often safer to choose simplicity over complexity. For instance, blinking messages may initially seem appealing, but they could be distracting. Similarly, too many fonts, styles, and sizes can confuse users. Every style used should convey a distinct message, such as indicating different levels of detail or differentiating between mandatory and optional actions.

Key Design Guidelines:

1. **Use appropriate colors:** Highlight different areas of the screen using colors that are not overly bright or gaudy.
2. **Use special effects sparingly:** While animations and sounds can be engaging in some contexts, too many special effects can become distracting, especially if the user has to interact with them repeatedly.
3. **Use hyperlinks:** Hyperlinks should be provided to allow users to easily navigate to related topics.
4. **Group related objects and information:** Organize elements so that they make sense to the user. The screen layout should be visualized from the user's perspective, simulating the tasks they will perform.
5. **Maintain appropriate screen density:** Keep the display uncluttered, with enough white space to make the design readable and attractive.
6. **Consistency in display:** Display titles, messages, and instructions in a consistent manner across all screens, and place them in similar locations for familiarity.
7. **Use consistent terminology:** Avoid using different terms like "delete," "cancel," and "erase" for the same action. Consistency in language helps users predict outcomes.
8. **Ensure commands have predictable effects:** A command like "BACK" should always perform the same action (e.g., returning to the previous screen) throughout the system.
9. **Consistency in mouse actions:** Ensure that pointing, clicking, and double-clicking produce consistent, predictable results across the application.
10. **Data entry flow:** When a user fills a field completely, don't automatically move to the next field. Instead, require the user to confirm the entry by pressing the Enter or Tab key after completing each field.
11. **Use familiar color patterns:** Stick to universally recognized color patterns, such as red for "stop," yellow for "caution," and green for "go," to reinforce on-screen instructions.
12. **Provide keyboard alternatives:** Offer keystroke alternatives for each menu command (e.g., File, Exit, Help), using easy-to-remember letters.

13. **Use familiar commands:** Use standard commands like Cut, Copy, and Paste, which are familiar to most users.
14. **Embrace the Windows look and feel:** If the users are familiar with Windows-based applications, design the interface to reflect that familiar aesthetic.
15. **Avoid jargon:** Don't use complex or technical terms. Instead, select everyday business terminology that users will easily understand.



**FIGURE 8-8** An example of a switchboard introducing TurboTax. The main options are clearly displayed on an uncluttered screen—something particularly important for users who are likely to be confused and/or nervous when beginning the tax preparation process.

Source: Intel.

(Tilley and Rosenblatt, 2024)

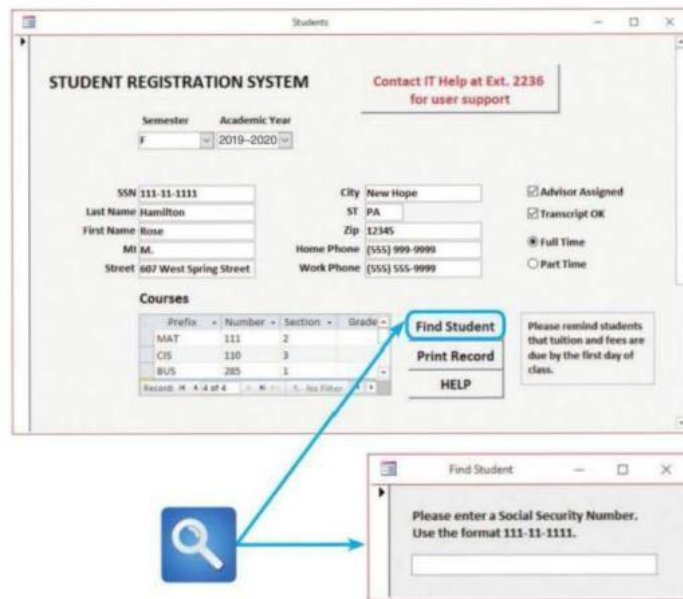
## 2.4.6 Enhance the Interface

A well-designed interface includes a variety of features to enhance user interaction, such as menu bars, toolbars, dialog boxes, text boxes, toggle buttons, list boxes, scroll bars, drop-down lists, option buttons, checkboxes, command buttons, and calendar controls. Creating an effective interface requires both aesthetic awareness and technical skills. It's crucial to gather user feedback regularly throughout the design process to ensure the interface meets user expectations and needs.

Key Guidelines for Enhancing the Interface:

1. **Start with a clear opening screen:** The opening screen is essential because it introduces the application and presents the main options to the user. A well-organized screen, such as a switchboard with clearly placed command buttons, helps users navigate the system easily. For instance, in TurboTax (Figure 8-8), the main options are clearly visible on an uncluttered screen, which is particularly important for users who may feel confused or nervous during a complex process like tax preparation.
2. **Use command buttons for actions:** Command buttons should initiate actions such as printing a form or requesting help. For example, clicking a "Find Student" button

might open a dialog box with additional instructions, guiding the user.



**FIGURE 8-9** A data entry screen for a student registration system. This screen uses several design features that are described in the text. When a user clicks the Find Student command button, a dialog box is displayed with instructions.

(Tilley and Rosenblatt, 2024)

3. **Customization options for software:** If you are using a commercial software package, check whether it supports the customization of menu bars and toolbars. Customization can make the system feel more user-friendly and tailored to specific needs.
4. **Implement shortcuts for efficiency:** Adding a shortcut feature lets users select menu commands by either clicking or pressing the Alt key combined with an underlined letter. You can also use toolbars containing icons or buttons to represent commonly used commands.
5. **Provide dialog boxes for input guidance:** If the system requires variable input data, use dialog boxes to explain what information is needed. This ensures users understand what is expected of them and helps avoid confusion.
6. **Toggle buttons for on/off status:** Toggle buttons are useful for switching between two states, such as turning an option on or off. They provide a clear visual cue about the current state.
7. **List boxes with scroll bars:** If the list of available choices exceeds the space provided, use scroll bars to allow users to navigate through the options. Additionally, ensure there is an alternative method for entering data that might not match any of the list choices.
8. **Option buttons (radio buttons) for exclusive choices:** Option buttons are ideal for allowing users to select one choice from a limited set of options. Use a clear message like "Choose one item" when only one option is allowed, or "Choose all that apply" when multiple selections are allowed. A black dot should indicate the selected option.
9. **Checkboxes for multiple selections:** When users can select one or more options from a list, checkboxes are an effective method. Use a checkmark or an "X" to show which options have been selected.

10. **Use calendar controls for date input:** When users need to enter dates, a calendar control is an excellent tool. It allows users to pick a date directly from a calendar interface, making the process easier and reducing errors.

## 2.4.7 Focus on Data Entry Screens

Data entry is crucial in many systems, as it's a key task for many users. Well-designed data entry screens can enhance productivity, minimize errors, and ensure that users feel comfortable interacting with the system. Here are some key guidelines for effective data entry design:

Key Guidelines for Data Entry Screens:

1. **Form Filling:** Whenever possible, use a form-filling method where the on-screen form resembles the source document. This makes data entry intuitive and easy for users by mirroring what they are familiar with.
2. **Restrict User Access to Data Entry Locations:** When a user is entering data, ensure the insertion point starts at the first required field and automatically moves to the next logical field as data is entered. Limit the ability to position the cursor to only where data can be entered, which prevents errors.

**FIGURE 8-10** In this data entry screen for creating events, the system generates start and end dates and times automatically, but these can be changed by the user at any time. A red asterisk, as shown beside Event Title, indicates required fields. Gray text within the data entry field lets the user know what information to provide. This text is replaced with user input.  
Source: GoodUI

3. **Cancel Option:** Always provide a way to leave the data entry screen without saving the current record, such as a "Cancel" button. This is especially important for web-based applications, where users can also navigate back via the browser's "Back" button.
4. **Descriptive Captions for Fields:** Each field should have a clear and descriptive caption indicating where the user should enter data. Additionally, specify any required or maximum field sizes, often done with text boxes, underscoring, or visual symbols.
5. **Allow Flexible Navigation Between Fields:** Users should be able to move among fields in a standard order or customize their navigation. In a GUI, users can override the default order and select fields using the mouse or arrow keys.
6. **Data Modification Options:** Allow users to add, change, delete, and view records. Always prompt for user confirmation when performing actions like deletions or changes with clear messages such as "Apply these changes? (Y/N)" or "Delete this record? (Y/N)." Highlighting the default response (usually "N") helps prevent accidental actions.

7. **Match the Layout to Source Documents:** If the source document has a specific field layout (e.g., fields running down in a column), replicate this layout on the screen. This makes it easier for users to follow and enter data correctly.
8. **Sample Formats:** When a specific data format is required (e.g., dates, phone numbers), provide on-screen instructions showing the correct format. This helps users avoid errors and ensures consistency.
9. **Input Masks for Format Consistency:** Use input masks, which are templates that restrict data entry to a specific format. For instance, Microsoft Access provides standard masks for dates, phone numbers, and social security numbers. Custom input masks can also be created for additional flexibility.
10. **Ending Keystroke for Each Field:** Ensure that an explicit keystroke (e.g., pressing Enter or Tab) is required to indicate the end of each field entry. Avoid automatic field transitions, as they can confuse users if they haven't finished their entry.
11. **No Leading Zeros for Numeric Fields:** Do not require users to type leading zeros in numeric fields. For example, for a project number like "045," users should only need to type "45." However, leading zeros might still be necessary for certain formats, like dates.
12. **No Trailing Zeros for Decimals:** Avoid requiring users to type trailing zeros for decimal numbers. For example, if a value like "98" should be interpreted as "98.00," the system should automatically format it correctly without needing extra input.
13. **Default Values:** Provide default values where applicable, so users can accept them by pressing the Enter key. This speeds up data entry but allows users to change the value if it's not appropriate.



**FIGURE 8-11** Microsoft Access provides various input masks for dates, phone numbers, and postcodes, among others. In addition, it is easy to create a custom mask using the characters shown here.

Sources: Microsoft Corporation.

(Tilley and Rosenblatt, 2024)

14. **Use Default Values for Repetitive Data:** When entering similar records (e.g., in a series of transactions), use default values for fields that are likely to remain constant (such as the date from the first record) until a new value is entered.
15. **Acceptable Values and Error Messages:** Provide a list of acceptable values for fields and display meaningful error messages if users enter invalid data. For fields with a limited set of valid entries, use drop-down lists to allow users to select the appropriate value.
16. **Confirm Data Before Submission:** Before finalizing data entry, give users the opportunity to review their entries and confirm their accuracy. For example, a message like "Add this record? (Y/N)" will allow the user to confirm before adding the record to the system.

#### Anticipating Future Needs

Design data entry screens with future scalability in mind. For instance, if a parts inventory database uses a one-character field to categorize items (e.g., "E" for electrical, "M" for mechanical), anticipate future changes that might require more specific categories. A more flexible design might allow for two-character codes, preventing future issues when expanding or changing the categorization system.

This proactive approach to designing data entry screens not only improves current usability but also prepares the system for future growth and changes in user requirements.

#### 2.4.8 Use Validation Rules

Validation rules are an essential part of improving data quality by preventing errors at the data entry stage. By using various types of validation checks, we can ensure that the data being entered into the system meets the required conditions, which ultimately prevents incorrect or inconsistent data from entering the system. Here are the key types of validation rules that can be applied:



**Sources:** Microsoft Corporation

### Types of Validation Rules:

- This ensures that data is entered in a specific sequence. For example, if work orders must be entered in numerical sequence, a validation rule will flag any order numbers entered out of sequence. Similarly, transactions should be entered chronologically, and any date entered out of order would trigger an error.
2. **Existence Check:**
- This is used to ensure that mandatory fields are not left blank. For instance, if an employee record requires a Social Security number, the system will not allow the record to be saved unless a valid Social Security number is entered.
3. **Data Type Check:**
- This rule ensures that the data entered matches the required type. For example, a field that expects numeric data will only allow numbers or numeric symbols, while an alphabetic field will only accept letters (A-Z or a-z). This prevents users from entering the wrong type of data.
4. **Range Check:**
- A range check ensures that data falls within a specified minimum and maximum range. For example, the number of hours worked by an employee

should fall between 0 and 24 hours. A similar rule can be applied for monetary values or other types of numeric data. If a value is outside of the acceptable range, it will trigger an error.

**5. Reasonableness Check:**

- This check flags values that are within acceptable limits but seem unusual or questionable. For example, a payment value of \$0.05 or \$5,000,000 might technically pass a range check, but both could be errors. Similarly, a daily-hours-worked value of 24 might pass a range check but may seem unreasonable, thus triggering a reasonableness check.

**6. Validity Check:**

- This type of check ensures that data entered must match one of a set of predefined valid values. For instance, if an inventory system accepts only 20 valid item classes, any input that doesn't match one of these valid classes would fail the check. Validity checks can also be used to ensure referential integrity—for example, ensuring a customer number in an order matches one in the customer file.

**7. Combination Check:**

- This rule is applied when two or more fields must be consistent when considered together. Even if individual fields pass their validation checks, the combination of their values might be inconsistent or unreasonable. For example, if an order for 30 units of an item applies a discount rate that only applies to orders of 100 units or more, this would be an invalid combination.

**8. Batch Controls:**

- Batch controls are used to verify the accuracy of batch data input. Before a batch of data (e.g., orders) is entered, the system can calculate totals (such as the total number of records or the sum of quantities). After the data is entered, the system recalculates these totals, and if they don't match, it indicates that there is an error in the batch. Batch controls do not identify specific errors but help to verify that the entire batch of data is correct.

## 2.4.9 Manage Data Effectively

Effective data management not only impacts users but also plays a critical role in enhancing company efficiency, productivity, and security. To minimize input errors, the system should process and verify data as soon as possible, ensuring each data element has a defined type (e.g., alphabetic, numeric, or alphanumeric) and an acceptable range of values.

Data should be captured as close to its source as possible. For example, using barcode scanners instead of manual forms in a warehouse, or having salespeople use tablets to record orders instead of paper forms, can significantly improve data accuracy. The most effective, accurate, and cost-efficient method of data entry is automated data capture.

An efficient system design ensures that data is entered only once. For instance, if payroll data is required for the human resources system, the system should allow data to be transferred automatically or have a central storage area accessible to both systems. Additionally, a secure system should include audit trails, logging every instance of data entry or change, such as when a customer's credit limit is set, by whom, and other relevant transaction history.

## 2.4.10 Reduce Input Volume

Reducing the volume of data entry is one of the most impactful guidelines, as it influences all other aspects. Minimizing unnecessary data entry reduces labor costs, speeds up data entry, and decreases the chance of errors. Therefore, analysts should begin by limiting the number of data fields required for each transaction.

**Only Enter Necessary Data:** Data should only be entered if it is needed by the system. For instance, if an order form includes the name of the clerk who took the order but the system doesn't require this information, it should not be entered.

**Avoid Entering Retrievable or Calculable Data:** Don't ask users to input data that can be fetched from system files or calculated from existing data. This prevents errors and ensures data consistency.

**Avoid Repeated Data Entry for Constant Information:** If certain data is constant across multiple entries, such as the order date for a batch of orders, only enter it once. For online orders, the system can automatically use the current system date.

**Use Codes:** Codes are shorter representations of data and can significantly reduce data entry time. For example, using a code for an item or customer instead of writing out the full name or description helps speed up the process and reduce errors.

**CASE IN POINT 8.2: BOOLEAN TOYS**

Suppose you are a systems analyst studying the order processing system at Boolean Toys, a fast-growing developer of software for preschool children. You know that many data entry users have complained about the input screens. Some users would prefer to rearrange the order of the fields, others would like to change the background color on their screens, and still others want shortcuts that would allow them to avoid a series of introductory screens.

What if Boolean's users could customize their own data entry screens without assistance from the IT staff by using a menu-driven utility program? What would be the pros and cons of such an approach? When should a systems analyst decide a design issue, and when should users be allowed to select what works best for them?

(Tilley and Rosenblatt, 2024)

## 2.5 SOURCE DOCUMENT AND FORM DESIGN

No matter how data enters an information system, the quality of the output is only as good as the quality of the input. The principle of "garbage in, garbage out" (GIGO) is well-known to IT professionals, as they understand that addressing issues at the data entry stage is the most effective

way to prevent problems later. The main goal is to ensure the accuracy, quality, and timeliness of the input data. Unfortunately, the ideal of a "paperless office" has yet to be fully realized. Despite advancements in RFID technology and automated data capture, we still rely on source documents and forms for data entry, meaning that system designers must address the challenge of the human-paper interface rather than just a human-computer one.

A **source document** serves several purposes: it collects input data, triggers or authorizes an action, and provides a record of the original transaction. During the input design phase, the systems analyst designs these documents to be user-friendly and easy to complete. Source documents are typically paper-based, but can also be provided electronically. Regardless of format, the design considerations remain the same.

Think back to instances when you struggled to fill out a poorly designed form—maybe the form lacked enough space, had confusing instructions, or was poorly organized. These are all signs of poor form layout.

A well-designed form layout makes it easy for users to fill out, providing sufficient space for data entry both vertically and horizontally. It should clearly indicate where data needs to be entered, often using blank lines or boxes, along with descriptive captions. Using checkboxes wherever possible is helpful, allowing users to quickly select options. However, it's important to also include an option for data that doesn't fit the predefined choices.

The way information is organized on the form is also critical. Source documents generally contain several zones, as illustrated in Figure 8-13. These include:

- **Heading Zone:** This section typically includes the company name or logo and the form's title and number.
- **Control Zone:** This zone contains codes, identification numbers, and dates needed for tracking or storing completed forms.
- **Instruction Zone:** This provides guidance for users on how to complete the form.
- **Body Zone:** The main portion of the form, often taking up at least half the space, where variable data is entered.
- **Totals Zone:** If applicable, this section displays totals that are relevant to the form.
- **Authorization Zone:** This area includes spaces for signatures, which are often necessary for validation or approval.

Designing source documents with these considerations in mind helps make data entry easier and more accurate.



FIGURE 8-13 Source document zones.

(Tilley and Rosenblatt, 2024)

Information on a form should flow from left to right and top to bottom, aligning with the natural reading pattern of users. This layout ensures the form is easy to complete for the person filling it out, as well as for those who will enter data into the system based on the completed form.

The same principles of user-friendly design apply to printed forms, such as invoices or monthly statements, though heading information is often pre-printed. Column headings should be concise yet descriptive, avoiding unconventional abbreviations, and there should be adequate spacing between columns for improved readability. The order and placement of fields on the form should be logical, with totals clearly labeled. When designing a preprinted form, it's helpful to consult with the vendor for advice on factors like paper size, font styles and sizes, colors, field placements, and other essential form details. The goal is to create a form that is both visually appealing and easy to read and use.

These design considerations also apply to web-based forms. Many resources are available to help design efficient and user-friendly online forms, including websites that adhere to the U.S. Federal Government's accessibility guidelines, which can be found at <http://www.section508.gov>.

## 2.6 PRINTED OUTPUT

Before designing printed output, several critical questions should be addressed:

- Why is the information being delivered as printed output rather than displayed on-screen, with options for users to view, print, or save as needed?
- Who is the audience for the information, why is it needed, and how will it be used?
- What specific information should be included?
- Will the printed output be designed for a specific device?
- When and how will the information be delivered, and how frequently must it be updated?
- Are there any security or confidentiality concerns, and how will these be managed?

The design process should not begin until these questions have been answered. Some of this information may have already been gathered during the systems analysis phase, but the analyst should meet with users to clarify exactly what output they need. Prototypes and mock-ups can be useful tools for gathering feedback throughout the design process.

### 2.6.1 Report Design

While many organizations strive to reduce the flow of paper and printed reports, few have fully eliminated printed output. Printed reports are portable, convenient, and necessary in certain situations. Many users find it helpful to view screen-based reports and print only the information they need for meetings or discussions. Additionally, printed output is used in turnaround documents, such as billing statements, which are later returned to the system with payment and processed accordingly.

Report designers use various styles, fonts, and images to make reports visually appealing and user-friendly. Whether displayed on-screen or printed, reports must be easy to read and well-organized. In many cases, managers judge the quality of a project based on the reports they receive.

Programs like Microsoft Access offer report design tools, such as the Report Wizard, which help designers create reports quickly. Many online database systems also provide similar guidelines for designing reports. Although most reports are graphically designed, some systems still produce character-based reports using fixed-spacing character sets. This method is often used to produce large-scale reports, such as payroll or inventory reports, particularly when multiple copies are needed.

Before finalizing a report design, users should approve it. The best approach is to create a sample report or prototype for users to review, including typical field values and enough records to showcase all design features. The sample can be created using a report generator or a tool like Microsoft Word.

### 2.6.2 Report Design Principles

Printed reports need to be attractive, professional, and easy to read. For instance, a well-designed report should include totals and subtotals for numeric fields. In the example shown, when a control field, such as "Store Number," changes, a control break occurs, which triggers actions like printing subtotals for a group of records. This type of report is called a control break report, and to produce it, records must be sorted in control field order.

Good report design requires careful attention to detail. Important design elements include:

- **Report Headers and Footers:** Every report should have a report header and footer. The header, which appears at the beginning of the report, includes the report title, date, and other relevant details. The footer, found at the end of the report, might contain grand totals or other concluding information.
- **Page Headers and Footers:** Each page should feature a page header at the top with column headings to identify the data. Column headings should be short but

descriptive, avoiding abbreviations unless easily understood by the users. The page footer can display the report title and page number.

| Store Number     | Employee Name        | Position | Regular Hours | Overtime Hours | Total Hours |
|------------------|----------------------|----------|---------------|----------------|-------------|
| 8                | Andres, Marguerite   | Clerk    | 20.0          | 0.0            | 20.0        |
| 8                | Bogema, Michelle     | Clerk    | 12.5          | 0.0            | 12.5        |
| 8                | Davenport, Kim       | Asst Mgr | 40.0          | 5.0            | 45.0        |
| 8                | Lemka, Susan         | Clerk    | 32.7          | 0.0            | 32.7        |
| 8                | Ramirez, Rudy        | Manager  | 40.0          | 8.5            | 48.5        |
| 8                | Ullery, Ruth         | Clerk    | 20.0          | 0.0            | 20.0        |
| Store 8 totals:  |                      |          | 165.2         | 13.5           | 178.7       |
| 17               | De Martini, Jennifer | Clerk    | 40.0          | 8.4            | 48.4        |
| 17               | Hall, Lisa           | Manager  | 40.0          | 0.0            | 40.0        |
| 17               | Rittenbery, Sandra   | Clerk    | 40.0          | 11.0           | 51.0        |
| 17               | Wyer, Elizabeth      | Clerk    | 20.0          | 0.0            | 20.0        |
| 17               | Zeigler, Cecile      | Clerk    | 32.0          | 0.0            | 32.0        |
| Store 17 totals: |                      |          | 172.0         | 19.4           | 191.4       |
| Grand totals:    |                      |          | 337.2         | 32.9           | 370.1       |

**FIGURE 8-14** The Employee Hours report is a detailed report with control breaks, subtotals, and grand totals. Note that a report header identifies the report, a page header contains column headings, a group footer contains subtotals for each store, a report footer contains grand totals, and a page footer identifies the page number.

(Tilley and Rosenblatt, 2024)

## Create a grouped or summary report

Access for Office 365, Access 2019, Access 2016, Access 2013, Access 2010, Access...

Information is often easier to understand when it is divided into groups. For example, a report that groups sales by region can highlight trends that otherwise might go unnoticed. In addition, placing totals (such as sums or averages) at the end of each group in your report can replace a lot of manual interaction with a calculator.

Access makes working with grouped reports easy. You can create a basic grouped report by using the Report Wizard, you can add grouping and sorting to an existing report, or you can revise grouping and sorting options that have already been defined.

**Note:** This article doesn't apply to Access web apps – the kind of database you design with Access and publish online.

### In this article

[Create a quick grouped or sorted report](#)

[Build a new grouped report by using the Report Wizard](#)

[Add or modify grouping and sorting in an existing report](#)

**FIGURE 8-15** Microsoft Access includes an easy-to-use tool for grouping data.

Source: Microsoft Corporation.

(Tilley and Rosenblatt, 2024)

- **Repeating Fields:** In report design, some elements may be repeated on every row, such as the store number. The decision of whether to repeat fields like this depends on user preference, so it's best to ask users for their input.
- **Consistent Design:** Reports should maintain a consistent look and feel across all documents. Common design elements, like the location of date and page numbers, should remain uniform across reports. Likewise, abbreviations and item layouts should be consistent to avoid confusion.

By following these principles, designers can create effective, easy-to-read reports that meet user needs and are visually consistent across the system.

### 2.6.3 Types of Reports

To be useful, a report must provide the information needed by the user. From a user's perspective, a report with too little information is not helpful, while one with too much information can be overwhelming and difficult to understand. The key to effective report design is matching the report to the specific needs of the user. Depending on their job functions, users may require different types of reports, such as:

- **Detail Reports:** A detail report provides one or more lines of output for each record processed. Since it includes detailed information for each record, these reports can become quite lengthy. For instance, in a large auto parts business with 3,000 items in stock, a detail report might include 3,000 lines across 50 pages. A user who wants to find parts that are in short supply would have to sift through all 3,000 lines. In such cases, an exception report could be a more efficient alternative.
- **Exception Reports:** An exception report displays only the records that meet specific conditions. These reports are helpful when a user only needs information on records that require action, rather than detailed information about all records. For example, a credit manager might use an exception report to identify customers with overdue accounts, or a customer service manager might want a report on all packages that were not delivered within the expected time frame.
- **Summary Reports:** Senior managers typically want to see aggregated figures rather than detailed information. For example, a sales manager might want to know the total sales for each sales representative but does not need to see a detailed report listing every individual sale. A summary report would be more appropriate in this case. Similarly, a personnel manager might need to know the total regular and overtime hours worked by employees at each location but might not need to see individual employee hours.

Each type of report serves a specific purpose and should be tailored to the user's needs to ensure it is both relevant and useful.

#### CASE IN POINT 8.3: LAZY EDDIE

Lazy Eddie is a furniture chain specializing in recliners. Their management has asked you to review the large number of printed reports that are distributed to Lazy Eddie's 35 store managers. Management is not sure that the managers actually read or use the reports, even though the store managers say they want them. Store visits have shown many of the reports end up stacked on top of filing cabinets, seemingly untouched.

To determine if store managers really use the printed reports, management has asked you to create a procedure that requires users to review and justify their information needs. You could design a form that asks if the information still is required, and why. You could try to get users to decide if a report is worth the cost of producing it. How do you proceed?

(Tilley and Rosenblatt, 2024)

## 2.7 TECHNOLOGY ISSUES

Unlike early innovations like the mouse and inkjet printer, most technological advancements today impact both output and input. In fact, output and input have become highly

interconnected, especially in user interfaces. It's challenging to pinpoint changes in one area that wouldn't also lead to or encourage changes in the other. For instance, new touch-screen input technologies generate output that must be carefully designed and sized to fit a particular device, whether it's a smartphone, tablet, or a 23-inch desktop monitor.

While the following sections will address output and input technology separately, interface designers must always be mindful of the potential linkages between the two. This awareness can help identify opportunities or potential issues that could arise from the interaction between input and output.

### **2.7.1 Output Technology**

While business information systems still predominantly deliver output through screen displays and printed documents, technology is significantly reshaping how information is communicated and accessed. This shift is particularly crucial for businesses that rely on information technology to reduce costs, boost employee productivity, and improve customer communication.

Apart from traditional screen displays and printed materials, there are many innovative ways to deliver output. The system requirements document will often specify user output needs. In the systems design phase, the analyst develops actual forms, reports, documents, and other types of output accessible from devices like workstations, notebooks, tablets, and smartphones. It's essential to consider how this information will be used, stored, and retrieved. The following subsections explore various types of output and the technologies behind them.

#### **Internet-Based Information Delivery:**

Millions of companies use the internet to reach customers globally, fueling the growth of e-commerce. Web designers must create user-friendly interfaces to display output and allow customers to interact. For example, businesses can link their inventory systems to their websites, allowing customers to browse products, view prices, and check availability. Another example is a system providing custom responses to inquiries, pulling relevant information from a knowledge base when users ask questions. The web also enables consumers to instantly download product brochures, user manuals, and quotes for financial services like mortgages and insurance.

Businesses also use live or pre-recorded webcasts to engage potential customers or investors. These audio or video files, distributed over the internet, are commonly used by radio and TV stations to broadcast programs.

#### **Email:**

Email remains a vital form of communication both internally and externally for businesses. Employees use it to exchange documents, data, schedules, and essential work-related information. Many companies have replaced traditional memos and printed correspondence with email. For example, financial institutions use email to confirm online stock trades, and businesses may send product updates or newsletters to customers.

**Blogs:**

Web-based blogs are a popular form of output, combining factual information with personal opinions. Blogs are useful for sharing news, reviewing current events, or promoting products, providing a unique perspective on a variety of topics.

**Instant****Messaging:**

Instant messaging is a fast-growing form of online communication, particularly valued for its ability to facilitate quick, real-time conversations. While some users find it distracting, others appreciate the constant flow of communication, especially in collaborative environments.

**Wireless****Devices:**

Messages and data can be transmitted to mobile devices, including tablets, smartphones, and other wireless products. These devices offer portability, multimedia capabilities, and internet access, making them essential for modern business communication.

**Digital****Audio,****Images,****and****Video:**

Digital audio, images, and video can be captured, stored, and transmitted as output to users who can then reproduce the content. Audio or video files can be attached to emails or inserted into documents like Microsoft Word files. Automated systems are also employed to handle voice transactions, providing customers with important information, such as airline seat availability or credit card balances, via phone.

Digital images and videos offer even more value. For example, an insurance adjuster can take a picture with a digital camera phone, submit it via a wireless device, and receive immediate approval for a claim. Video clips are especially useful in virtual tours, such as real estate walkthroughs, where users can explore properties interactively.

**Automated****Fax****Systems:**

Automated fax systems allow customers to request a fax via email, a company website, or a phone call, with the fax being transmitted quickly to the user's fax machine. Despite the rise of digital communication, fax systems remain a primary method of communication in certain industries like healthcare, insurance, and real estate.

**Podcasts:**

Podcasts are audio files that can be downloaded from the internet and played on a computer or portable media player. Companies use podcasts as marketing tools and to communicate internally. Podcasts may also include images, sound, and video, making them a versatile medium.

**Computer****Output****to****Digital****Media****(CODM):**

This process involves scanning and storing paper documents in a digital format for easy retrieval. For example, an insurance company might scan thousands of paper application forms and use software to extract specific data for further processing. Digital storage media, such as magnetic tapes, CDs, DVDs, and laser disks, are commonly used for this purpose.

**Specialized****Forms****of****Output:**

Various specialized devices and forms of output cater to the needs of specific industries, including:

- Portable, web-connected devices for running apps and handling multimedia output
- Retail point-of-sale terminals for credit card transactions and inventory updates
- Automated teller machines (ATMs) for bank transactions and printing slips
- Special-purpose printers for producing labels, ID cards, and other specialized documents
- Plotters for creating high-quality images like blueprints or maps
- Electronic data detection in credit cards, bank cards, and ID cards

### 8.7.2 Input Technology

Input technology has undergone significant advancements in recent years. In addition to traditional methods, businesses now have access to a wide range of new hardware and ways to capture and input data into systems. These innovations aim to speed up data entry, lower costs, and capture data in new formats such as digital signatures.

Input methods should prioritize cost-effectiveness, timeliness, and simplicity. Systems analysts typically analyze business operations and transactions to determine how and when data should be entered into a system. The first decision often involves choosing between batch input or online input methods. Each method comes with its own advantages and disadvantages, which analysts need to carefully consider based on various factors.

#### Batch

#### Input:

Batch input is when data entry is performed on a specified schedule, such as daily, weekly, or monthly. For example, payroll departments collect time cards at the end of each week and enter the data in batches. Similarly, schools may enter all grades for an academic term as a batch at the end of the term. Batch input is often used in situations where immediate data availability isn't necessary.

#### Online

#### Input:

While batch input has its place, online data entry is more commonly used in modern business activities. Online input offers major advantages, such as the immediate validation and availability of data. A popular method of online input is **source data automation**, which combines online data entry with automated data capture through devices like RFID tags, magnetic stripes, or even smartphones. Source data automation is fast, accurate, and minimizes human involvement in the data-entry process.

Large companies often combine source data automation with powerful communication networks to manage global operations instantly. Examples of source data automation include:

- Businesses using **point-of-sale (POS) terminals** with barcode scanners or magnetic swipe readers to input credit card data
- **ATMs** that read data strips on bank cards for transactions
- **Factory employees** using magnetic ID cards to track hours and production costs
- **Hospitals** employing barcode imprints on patient identification bracelets and using handheld scanners to gather treatment and medication data
- **Retail stores** using portable barcode scanners to log shipments and update inventory data

- **Libraries** using handheld scanners to read optical strips on books

#### **Trade-offs between Batch and Online Input:**

While online input has many advantages, it also presents certain challenges. For example, unless source data automation is implemented, manual data entry can be slower and more expensive than batch input because it typically occurs at the time the transaction happens, often when computer demand is high.

The decision to use batch or online input depends on the business's needs. For instance, hotel reservations must be entered and processed in real-time, but hotels can enter their monthly performance data in batches. Some forms of input naturally happen in batches—such as when a cable TV provider receives customer payments by mail in batches.

## **2.8 SECURITY AND CONTROL ISSUES**

A company must prioritize the protection of its data, not only for its own information but also for the data of its customers, employees, and suppliers. Corporate data holds immense value, and in many cases, it is priceless. Without secure, accurate, and reliable data, a company cannot effectively operate.

The following sections will explore the security and control measures required to safeguard both output and input data.

### **2.8.1 Output Security and Control**

Ensuring the security and accuracy of output is critical for any company. Output must be accurate, complete, current, and protected from unauthorized access. To maintain output integrity and security, companies implement various control methods.

#### **Key Control Methods for Output Security:**

- Every report should include vital information like a title, report number or code, printing date, and the time period it covers.
- Pages should be numbered consecutively (e.g., Page 1 of X) and the end of the report should be clearly labeled.
- Control totals and record counts should be reconciled with input totals and counts for accuracy.
- Random checks should be performed to ensure reports are correct and complete.
- Processing errors or interruptions should be logged for analysis and resolution.

#### **Security Measures for Output:**

- Output security safeguards privacy and protects proprietary data from theft or unauthorized access. This includes limiting the number of printed copies and using tracking procedures to account for each one.
- When printed output is distributed, specific protocols should ensure that it reaches only authorized recipients, especially when reports contain sensitive information like payroll data.

- Sensitive reports should be stored securely, and all pages should be properly labeled.
- Any outdated or unwanted reports, or output from interrupted print runs, should be shredded to prevent exposure of sensitive data.
- Blank check forms and signature stamps should be securely stored and regularly inventoried to ensure nothing goes missing.

**IT Department's Role:** In most organizations, the IT department is responsible for managing output security and control measures. They must ensure that security is built into the system through features like password protection, encryption of sensitive data, and user access controls. Physical security is also vital, particularly for printed output, which is tangible and can be easily accessed.

**Diskless Workstations:** To address security concerns related to enterprise-wide data access, many firms use diskless workstations. These are network terminals that provide a full user interface but limit the ability to print or copy data. The use of portable storage devices, like USB drives, is typically restricted on these terminals to help prevent unauthorized copying or data transfer.

By incorporating these output control and security practices, companies can protect their sensitive information and ensure that reports and data are handled safely and appropriately.



**FIGURE 8-17** To maintain output security, it is important to shred sensitive material.  
Gang Lu/Shutterstock.com

## 2.8.2 Input Security and Control

Input control ensures that the data entered into a system is accurate, complete, and secure. These measures are crucial at every stage of the input design process, beginning with the use of source documents that help ensure data quality. The following sections outline the key aspects of input security and control:

### Key Elements of Input Security and Control:

#### 1. **Audit Trails:**

An audit trail is essential for tracking and verifying input data. Each piece of information should be traceable back to the original input data that produced it. This requires documenting the source of each data item, the time it was entered into the system, and who entered it. The audit trail should also capture any changes made to

the data, detailing when and by whom those changes occurred. This log must be monitored carefully to maintain data integrity.

2. **Handling Source Documents:**

To avoid data loss before it enters the system, proper handling of source documents is critical. Any documents originating outside the organization should be logged when they are received. Additionally, transfers of source documents between departments should be documented to ensure that data is not lost or mishandled in transit.

3. **Data Security Policies:**

Data security policies and procedures protect data from loss or damage. While no system is 100% foolproof, data recovery utilities should be in place to restore lost or damaged data. Companies should also have protocols for the secure storage of source documents for a specified time period, in line with legal and business requirements. This is often part of a records retention policy that outlines how long data should be stored before it can be safely disposed of.

4. **Audit Trail Files and Reports:**

Audit trail files and reports must be saved and stored. If data files are damaged, these records can be used to reconstruct lost data. This ensures that even in the event of a system failure, the company can recover the critical information.

5. **Protecting Data from Unauthorized Access:**

Protecting data from unauthorized access is a vital component of input security. The system should require secure sign-on procedures to prevent unauthorized individuals from entering the system. Users should be encouraged to change their passwords regularly to reduce the risk of unauthorized access. Implementing multiple levels of access control is also a good practice— for example, a data entry person might be allowed to view a credit limit but not modify it.

6. **Encryption of Sensitive Data:**

Sensitive data should be encrypted (or coded) using encryption software so that only authorized users with the proper decoding software can access it. Encryption adds an additional layer of security, ensuring that even if data is intercepted, it cannot be read without the correct decryption key.

By employing these input control measures, companies can significantly reduce the risk of errors, data loss, or unauthorized access, ensuring that the data entering their systems remains secure and accurate.

## 2.9 EMERGING TRENDS

In modular design, individual components, or **modules**, are created to connect to a larger program or system. Each module represents a specific function, which is documented in a process description and can be seen on a **Data Flow Diagram (DFD)**. In **object-oriented design** (as explained in Chapter 6), these modules are referred to as **classes**.

The key advantages of modular design are:

- **Flexibility:** Independent modules can be developed, tested, and then combined or reused later.

- **Team Collaboration:** Large-scale systems often require multiple teams to work on different modules, allowing them to develop and integrate their parts without interfering with others.
- **Easier Maintenance:** Since each module performs a single function, it's easier to modify or update them individually without affecting the entire system.

Modular design is particularly important for large systems as it allows easier management and development processes by breaking down the system into smaller, more manageable pieces.

### 2.9.2 Responsive Web Design

With the increasing use of mobile devices, **responsive web design** (RWD) has become a critical development trend. This approach ensures that web content is displayed correctly, regardless of the device being used. The need for responsive design arises from the fact that devices like smartphones, tablets, and desktop computers all have different screen sizes and resolutions. Without RWD, the user experience can be compromised by having content that doesn't fit properly or requires excessive scrolling.

Key principles of **Responsive Web Design** include:

- **CSS3** for styling, ensuring flexibility in how elements are displayed.
- **Flexible images** that adapt to varying screen sizes.
- **Fluid grids**, where page elements are specified in relative terms (e.g., percentages), allowing them to adjust dynamically to different screens.

The goal of RWD is to **automatically adjust the layout** to the user's device, improving usability, performance, and maintainability. Developers no longer need to create separate versions of a website for each device, as the design adjusts automatically based on the device's screen size.

### 2.9.3 Prototyping

Prototyping is a method that involves creating an early version, or **prototype**, of a proposed system. This prototype is built quickly and then refined through a process of analysis, design, modeling, and testing. Prototyping is valuable because it allows users to interact with a working model of the system early in the development process, providing feedback that can be used to refine the final product.

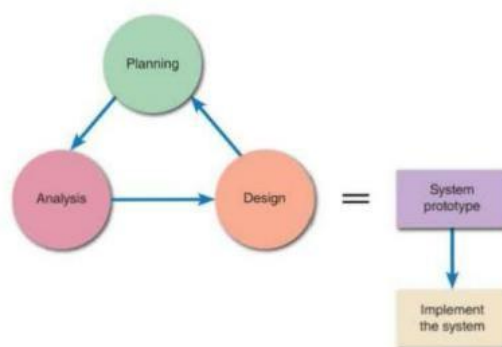


**FIGURE 8-18** Wind tunnel testing is a typical example of prototyping.  
*Courtesy of NASA*

User input and feedback are crucial at every stage of the system development process. Prototyping allows users to interact with a model that accurately represents the system's outputs, inputs, interfaces, and processes. This provides a risk-free environment where users can test the model, approve it, or suggest changes. In some cases, the prototype evolves into the final version of the system, while in others, it is only used to validate user requirements and is discarded afterward. Prototyping is especially important in agile development, where systems are built incrementally through prototypes that are continuously adjusted based on user feedback. As the process progresses, developers refine earlier versions, gradually merging them into the final product. Agile methods emphasize continuous feedback, where each step is influenced by insights gained from previous steps.

Systems analysts typically use two prototyping methods: system prototyping and design prototyping.

**System Prototyping:** This approach creates a fully functional model of the information system. When the prototype meets all requirements, it is ready for implementation. Since this model is on track for implementation, obtaining user feedback is essential to ensure it meets all user and management requirements.



**FIGURE 8-19** The end product of system prototyping is a working model of the information system, ready for proper implementation.

(Tilley and Rosenblatt, 2024)

**Design Prototyping:** In contrast, design prototyping focuses on verifying user requirements. After the prototype is used to capture feedback, it is discarded, and the actual

implementation continues. Design prototyping is useful for developing testing and training procedures before the final system is available.

Prototyping helps reduce the risks and potential financial losses that could arise if a completed system fails to meet business needs. However, there are potential issues to be aware of:

- The rapid development pace might lead to quality issues that only become apparent when the system is operational.
- Certain system requirements, like reliability and maintainability, may not be adequately tested in a prototype.
- In very complex systems, prototypes can become unwieldy and difficult to manage.
- Users might want to adopt the prototype with little or no changes, mistakenly believing it fully meets their needs. This can result in higher maintenance costs as further customization may be required later in the SDLC.

**Design Prototyping**, also known as throwaway prototyping, has more limited objectives, but these objectives are still significant. The goal of design prototyping is to create a user-approved model that documents and benchmarks the features of the final system. It helps capture user input and approval while continuing the system's development within the SDLC framework. Systems analysts commonly use design prototyping to create outputs, inputs, and user interfaces.

**Trade-offs of Prototyping:** Prototyping offers several advantages:

- It helps avoid misunderstandings between users and system developers.
- It allows developers to create accurate specifications for the final system based on the prototype.
- Managers can evaluate a working model more effectively than a paper specification.

### A QUESTION OF ETHICS

Stock.com/fabrizio\_8

One of the systems analysts on the project team thought that he did a good job of designing the company's tech support webpage, but his supervisor isn't so sure. His supervisor is concerned that the design is very similar to a page used by the company's major competitor, and she asked him whether he had used any HTML code from that site in his design. Although the analyst didn't copy any of the code, he did examine it in his web browser to see how they handled some design issues.

The supervisor asked the analyst to investigate webpage copyright issues and report back to her. In his research, the analyst learned that outright copying would be a copyright violation, but merely viewing other sites to get design ideas would be permissible. What is not so clear is the gray area in the middle. The analyst asked you, as a friend, for your opinion on this question: Even if no actual copying is involved, are there ethical constraints on how far you should go in using the creative work of others? How would you answer?

## 2.10 SUMMARY

The chapter began by introducing user interface (UI) design and human-computer interaction (HCI) concepts. A graphical user interface (GUI) uses visual elements and techniques to allow users to communicate effectively with the system. User-centered design principles focus on understanding the business, maximizing the effectiveness of graphics, thinking from the

user's perspective, utilizing models and prototypes, emphasizing usability, inviting user feedback, and documenting the process.

When designing a user interface, it should be transparent and easy to learn and use. The design should enhance user productivity, make help and error correction easily accessible, minimize input data errors, provide clear feedback, create an attractive layout, and use familiar terms and images. Various control features, such as menu bars, toolbars, drop-down lists, dialog boxes, toggle buttons, list boxes, option buttons, checkboxes, and command buttons, can also be added. These controls are placed on a main "switchboard," which functions like a graphical version of the main menu.

The section on input design began with a description of source documents, outlining different zones within a document, such as the heading zone, control zone, instruction zone, body zone, totals zone, and authorization zone. The chapter then covered data entry screen design, explaining how input masks and validation rules can reduce data errors. Input masks are templates that allow only specific character combinations, while validation rules prevent inappropriate data from entering the system. These checks include sequence, existence, range, reasonableness, and validity checks.

The chapter also discussed various types of printed reports, such as detail, exception, and summary reports. It explained the key features of reports, including control fields, control breaks, report headers and footers, page headers and footers, and group headers and footers. Other types of output, such as web-based delivery, audio output, instant messaging, podcasts, email, and other specialized forms of output, were also covered.

The chapter described batch and online input methods, input media and procedures, and input volume. Data capture, which can be automated, involves identifying and recording source data, while data entry involves converting source data into a computer-readable format. New technologies like optical and voice recognition systems, biological feedback devices, motion sensors, and various graphical input devices are also emerging.

Security and control measures were discussed as well. Output control includes physical protection of data and reports, and managing unauthorized ports or devices that might extract data from the system. Input controls involve audit trails, encryption, password security, data security, and setting access levels to restrict who can view or use data.

Finally, the chapter highlighted emerging trends in modular design, responsive web design, and prototyping.

## Key Terms

**audit trail** A record of the source of each data item and when it entered a system. In addition to recording the original source, an audit trail must show how and when data is accessed or changed, and by whom. All these actions must be logged in an audit trail file and monitored carefully.

**authorization zone** Part of a form that contains any required signatures.

**automated fax** A system that allows a customer to request a fax using email, the company website, or a telephone. The response is transmitted in a matter of seconds back to the user's fax machine. See *faxback*.

**batch** A group of data, usually inputted into an information system at the same time.

**batch control** A total used to verify batch input. Batch controls might check data items such as record counts and numeric field totals. For example, before entering a batch of orders, a user might calculate the total number of orders and the sum of all the order quantities. When the batch of orders is entered, the order system also calculates the same two totals. If the system totals do not match the input totals, then a data entry error has occurred.

**batch input** A process where data entry is performed on a specified time schedule, such as daily, weekly, monthly, or longer. For example, batch input occurs when a payroll department collects time cards at the end of the week and enters the data as a batch.

**blog** An online journal. The term is a contraction of "web log."

**calendar control** A calendar control allows the user to select a date that the system will display and store as a field value.

**character-based report** A report created using a single mono-spaced character set.

**check box** Used to select one or more choices from a group. A check mark, or an X, represents selected options.

**combination check** A type of data validation check that is performed on two or more fields to ensure that they are consistent or reasonable when considered together. Even though all the fields involved in a combination check might pass their individual validation checks, the combination of the field values might be inconsistent or unreasonable.

**command button** Onscreen button that initiates an action such as printing a form or requesting Help.

**context-sensitive** A feature that is sensitive to the current conditions when it is invoked. For example, context-sensitive help offers assistance for a task in progress.

**control break** A control break usually causes specific actions to occur, such as printing subtotals for a group of records.

**control break report** A detail report that focuses on control breaks.

**control field order** In a control break report, the records are arranged or sorted in the same order as the control fields.

**data security** Protection of data from loss or damage and recovers data when it is lost or damaged.

**data type check** A type of data validation check that is used to ensure that a data item fits the required data type. For example, a numeric field must have only numbers or numeric symbols, and an alphabetic field can contain only the characters A through Z or the characters a through z.

**data validation rule** A mechanism to improve input quality by testing the data and rejecting any entry that fails to meet specified conditions.

**default value** A value that a system displays automatically.

**design prototyping** Creating a prototype of user requirements, after which the prototype is discarded and implementation continues. Also called *throwaway prototyping*.

**detail report** A detail report produces one or more lines of output for each record processed.

**dialog box** Allows a user to enter information about a task that a system will perform.

**diskless workstation** A network terminal that supports a full-featured user interface but limits the printing or copying of data, except to certain network resources that can be monitored and controlled more easily.

**electronic health record (EHR)** An electronic record of a patient's health information generated as the patient encounters various health-care providers and shared among multiple facilities and agencies.

**encryption** A process where data is coded (converted into unreadable characters) so that only those with the required authorization can access the data (usually via decoding software).

**exception report** A document displaying only those records that meet a specific condition or conditions. Exception reports are useful when the user wants information only on records that might require action but does not need to know the details.

**existence check** A type of data validation check that is used for mandatory data items. For example, if an employee record requires a Social Security number, an existence check would not allow the user to save the record until he or she enters a suitable value in the SSN field.

**faxback** *See* automated fax.

**form filling** A very effective method of online data entry where a blank form that duplicates or resembles the source document is completed on the screen. The user enters the data and then moves to the next field.

**form layout** The physical appearance and placement of data on a form. Form layout makes the form easy to complete and provides enough space, both vertically and horizontally, for users to enter the data.

**garbage in, garbage out (GIGO)** The concept that the quality of the output is only as good as the quality of the input.

**graphical user interface (GUI)** The use of graphical objects and techniques allowing users to communicate with a system. A well-designed GUI can help users learn a new system rapidly and work with the system effectively.

**hash totals** Not meaningful numbers themselves but are useful for comparison purposes. Also known as batch control totals.

**human-computer interaction (HCI)** A description of the relationship between computers and the people who use them to perform business-related tasks. HCI concepts apply to everything from a PC desktop to the main menu for a global network.

**input control** The necessary measures to ensure that input data is correct, complete, and secure. A systems analyst must focus on input control during every phase of input design, starting with source documents that promote data accuracy and quality.

**input mask** Template or pattern that makes it easier for users to enter data. Often used in automated forms to guide an unfamiliar user.

**limit check** Occurs when a validation check involves a minimum or a maximum value, but not both. Checking that a payment amount is greater than zero, but not specifying a maximum value, is an example of a limit check.

**list box** An output mechanism that displays a list of choices that the user can select.

**menu bar** A set of user-selectable software application options, usually located across the top of the screen.

**mock-up** When designing a report, a sample report is prepared, which is a mock-up, or prototype, for users to review. The sample should include typical field values and contain enough records to show all the design features.

**modular design** A design that can be broken down into logical blocks. Also known as partitioning, or top-down design.

**module** Related program code organized into small units that are easy to understand and maintain. A complex program could have hundreds or even thousands of modules.

**natural language** A software feature that allows users to type commands or requests in normal English (or other language) phrases.

**online data entry** A data entry method used for most business activity. The online method offers major advantages, including the immediate validation and availability of data.

**option button** Radio buttons that represent groups of options. The user can select only one option at a time; a selected option contains a black dot. *See also* **radio button**.

**output control** Methods to maintain output integrity and security. For example, every report should include an appropriate title, report number or code, printing date, and time period covered. Reports should have pages that are numbered consecutively, identified as Page xx of xx, and the end of the report should be labeled clearly.

**output security** Output security protects privacy rights and shields the organization's proprietary data from theft or unauthorized access.

**page footer** Appears at the bottom of the page and is used to display the name of the report and the page number.

**page header** Appears at the top of the page and includes the column headings that identify the data.

**podcast** A web-based broadcast that allows a user to receive audio or multimedia files using music player software such as iTunes, and listen to them on a PC or download them to a portable MP3 player or smart phone.

**prototype** An early, rapidly constructed working version of the proposed information system.

**prototyping** The method by which a prototype is developed. It involves a repetitive sequence of analysis, design, modeling, and testing. It is a common technique that can be used to design anything from a new home to a computer network.

**radio button** Buttons that represent groups of options. The user can select only one option at a time; a selected option contains a black dot. *See also* **option button**.

**radio frequency identification (RFID) tag** An input device used in source data automation.

**range check** A type of data validation check that tests data items to verify that they fall between a specified minimum and maximum value. The daily hours worked by an employee, for example, must fall within the range of 0 to 24.

**reasonableness check** A type of data validation check that identifies values that are questionable but not necessarily wrong. For example, input payment values of \$0.05 and \$5,000,000.00 both pass a simple limit check for a payment value greater than zero, and yet both values could be errors.

**records retention policy** Rules designed to meet all legal requirements and business needs for keeping records.

**report footer** Appears at the end of the report, can include grand totals for numeric fields and other end-of-report information.

**report header** Appears at the beginning of a report and identifies the report as well as the report title, date, and other necessary information.

**scroll bar** In user interface design, a scroll bar allows the user to move through the available choices for an input field.

**sequence check** A type of data validation check that is used when the data must be in some predetermined sequence. If the user must enter work orders in numerical sequence, for example, then an out-of-sequence order number indicates an error. If the user must enter transactions chronologically, then a transaction with an out-of-sequence date indicates an error.

**source data automation** A popular online input method that combines online data entry and automated data capture using input devices such as magnetic data strips or swipe scanners.

**source document** A form used to request and collect input data, trigger or authorize an input action, and provide a record of the original transaction. During the input design stage, you develop source documents that are easy to complete and inexpensive.

**storyboard** Sketches used during prototyping to show the general screen layout and design.

**summary report** A report used by individuals at higher levels in the organization that includes less detail than reports used by lower-level employees.

**switchboard** The use of command buttons in a user interface to enable users to navigate a system and select from groups of related tasks.

**system prototyping** Producing a full-featured, working model of the information system being developed. |

**throwaway prototyping** See *design prototyping*. |

**toggle button** A GUI element used to represent on or off status. Clicking the toggle button switches to the other status.

**toolbar** A GUI element that contains icons or buttons that represent shortcuts for executing common commands.

**totals zone** If a form has data totals, they will appear in this section of the form. |

**transparent interface** A user interface that users don't really notice—a user-friendly interface that does not distract the user and calls no attention to itself. |

**turnaround document** Output document that is later entered back into the same or another information system. A telephone or utility bill, for example, might be a turnaround document printed by the company's billing system. When the bill is returned with payment, it is scanned into the company's accounts receivable system to record the payment accurately. |

**usability** In user interface design, includes user satisfaction, support for business functions, and system effectiveness.

**usability metrics** Data that interface designers can obtain by using software that can record and measure user interactions with the system. |

**user-centered** A term that indicates the primary focus is upon the user. In a user-centered system, the distinction blurs between input, output, and the interface itself.

**user interface (UI)** The mechanism through which the user interacts with the system. The interface can be graphical, textual, aural, or a combination of different modes of interaction. |

**validity check** A type of data validation check that is used for data items that must have certain values. For example, if an inventory system has 20 valid item classes, then any input item that does not match one of the valid classes will fail the check. |

**webcast** A one-way transmission of information or training materials, such as a Webinar session, available on demand or for a specific period to online participants.

## EXERCISES

1. Explain Apple's view of user interface design, especially for apps.
2. What is HCI (Human-Computer Interaction)?
3. Why is a transparent interface desirable?
4. What are the seven habits of successful interface designers?
5. How would you rank the 10 guidelines for user interface design in order of importance? Explain your answer.
6. What are the main principles of source document design?
7. What is the difference between a detail report, a summary report, and an exception report?
8. How has input technology changed in recent years?
9. What is output security?
10. What are three emerging trends in user interface design?

## Discussion Topics:

1. Some systems analysts argue, "Give users what they ask for. If they want lots of reports and reams of data, then that is what you should provide. Otherwise, they will feel that you are trying to tell them how to do their jobs." Others say, "Systems analysts should let users know what information can be obtained from the system. If you listen to users, you'll never get anywhere because they really don't know what they want and don't understand information systems." What do you think of these arguments?
2. Some systems analysts maintain that source documents are unnecessary. They say that all input can be entered directly into the system, without wasting time in an intermediate step. Do you agree? Can you think of any situations where source documents are essential?
3. Suppose your network support company employs 75 technicians who travel constantly and work at customer sites. Your task is to design an information system that provides technical data and information to the field team. What types of output and information delivery would you suggest for the system?
4. A user interface can be quite restrictive. For example, the interface design might not allow a user to exit to a Windows desktop or to log on to the Internet. Should a user interface include such restrictions? Why or why not?
5. How is the increased use of smartphones and tablets, with their smaller screen size, affecting user interface design practices?

**Projects:**

1. Visit the administrative office at your school or a local company. Ask to see examples of input screens. Analyze the design and appearance of each screen and try to identify at least one possible improvement.
2. Search the web to find an especially good example of a user interface that includes guidelines in this chapter. Document your research and discuss it with your class.
3. Review Section 8.2 and the comments about EHR usability. Research the current status of EHR usability and describe all noteworthy developments.
4. Suggest at least two good examples and two bad examples of source document design.
5. Explore the emerging area of wearable computing, such as the Apple Watch, and comment on the impact of these devices on user interface design.

## CHAPTER 3: Data Design



### LEARNING OUTCOMES

After reading this Section of the guide, the learner should be able to:

1. Explain basic data design concepts, including data structures, DBMSs, and the evolution of the relational database model.
2. Explain the main components of a DBMS.
3. Define the major characteristics of web-based design.
4. Define data design terminology.
5. Draw entity-relationship diagrams.
6. Apply data normalization.
7. Utilize codes to simplify output, input, and data formats.
8. Explain data storage routes and techniques, including logical versus physical storage.
9. Explain data coding.
10. Explain data control measures.
11. Summarize the tasks involved in completing the systems analysis phase of the Software Development Life Cycle (SDLC)

### 3.1 DATA DESIGN CONCEPTS

Systems analysts need to have a solid grasp of fundamental data design principles, such as data structures and the development of the relational database model.

#### 3.1.1 Data Structures

A data structure is a system for organizing, storing, and managing data. Data structures typically consist of files or tables that interact in different ways. Each file or table contains information about people, places, things, or events. For example, one file or table may store data about customers, while others may contain information about products, orders, suppliers, or employees.

Older legacy systems often relied on file processing because it was compatible with mainframe hardware and batch input. Some companies still use this method for managing large volumes of structured data regularly, as it can be cost-effective in certain cases. For instance, a credit card company might use file processing to update account balances from daily transactions stored in a TRANSACTIONS file to a CUSTOMERS file, as shown in Figure 9-1. In such a relatively simple process, file processing could be a viable option.



**FIGURE 9-1** A credit card company that posts thousands of daily transactions might consider a file processing option.

As time went on, the modern relational database became the standard model for systems developers. The following example of an auto service shop will illustrate a comparison between the two concepts.

### 3.1.2 Mario and Danica: A Data Design Example

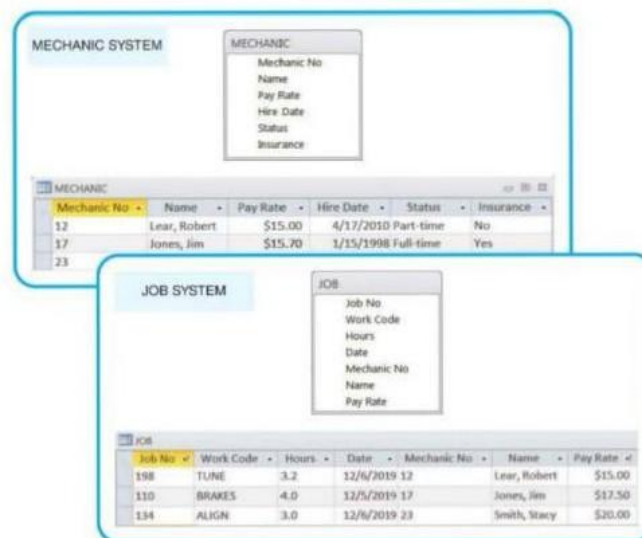
**MARIO'S AUTO SHOP:** Mario relies on two file-oriented systems, also known as file processing systems, to manage his business. These systems store data in separate files that are not connected or linked. The following describes how Mario's file-oriented systems work:

- The **MECHANIC SYSTEM** uses the **MECHANIC** file to store data about shop employees.
- The **JOB SYSTEM** uses the **JOB** file to store data about the work performed at the shop.

In this setup, as shown in Figure 9-2, data about the mechanic, the customer, and the brake job could be stored either in a file-oriented system or in a database system. However, using two separate systems means that some data is stored in two different places, and there is a chance that the data may not be consistent. For example, three data items—Mechanic No, Name, and Pay Rate—are stored in both files. This redundancy is a significant disadvantage of file-oriented systems because it threatens data quality and integrity. A typical issue, as seen in Figure 9-3, is the discrepancy in Jim Jones' pay rate: it is listed as \$18.90 in the **MECHANIC SYSTEM** file but \$19.80 in the **JOB SYSTEM** file.



**FIGURE 9-2** In the example shown here, data about the mechanic, the customer, and the brake job might be stored in a file-oriented system or in a database system.  
Lisa F. Young/Shutterstock.com

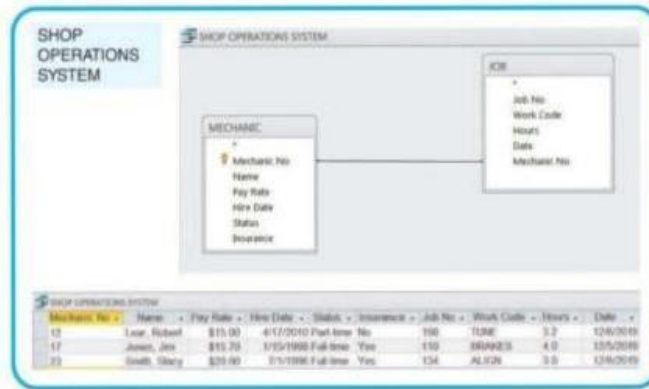


**FIGURE 9-3** Mario's shop uses two separate systems, so certain data must be entered twice. This redundancy is inefficient and can produce errors.

(Tilley and Rosenblatt, 2024)

**DANICA'S AUTO SHOP:** Danica uses a database management system (DBMS) with two separate tables that are joined together, making them function like one large table, as shown in Figure 9-4. In Danica's **SHOP OPERATIONS SYSTEM**, the tables are linked by the **Mechanic ID** field, known as a common field because it connects the tables. Unlike Mario's system, no other data items are duplicated in Danica's system, except for the common field. This design is called a relational database or relational model, which was introduced in the 1970s and remains the standard method for organizing, storing, and managing business data.

In contrast to Mario's file-oriented systems, which show two different pay rates for Jim Jones due to a possible data entry error, Danica's relational database prevents such errors. In her system, an employee's pay rate is stored in only one place. However, it's important to note that DBMSs are not entirely immune to data entry issues, which will be discussed in more detail later in this chapter.



**FIGURE 9-4** Danica's SHOP OPERATIONS SYSTEM uses a database design, which avoids duplication. The data can be viewed as if it were one large table, regardless of where the data is physically stored.

(Tilley and Rosenblatt, 2024)

### 3.1.3 Database Management Systems

A database provides a comprehensive framework that eliminates data redundancy and supports a real-time, dynamic environment. Figure 9-5 illustrates a company-wide database that supports four distinct information systems.

A **database management system (DBMS)** is a collection of tools, features, and interfaces that allow users to add, update, manage, access, and analyze data. From the user's perspective, the main benefit of a DBMS is that it offers timely, interactive, and flexible data access.

Some specific advantages of a DBMS include the following:



**FIGURE 9-5** In this example, a sales database can support four separate business systems.

(Tilley and Rosenblatt, 2024)

- **Scalability:** Scalability refers to a system's ability to grow, change, or shrink easily to meet the evolving needs of a business. For instance, if a company wants to include data about secondary suppliers, a new table can be added to the relational database and connected via a common field.
- **Economy of scale:** Database design improves hardware utilization. With an enterprise-wide database, processing costs are lower because powerful servers and communication networks are used. The efficiency of processing large volumes of data on larger computers is referred to as the economy of scale.
- **Enterprise-wide application:** A DBMS is usually managed by a database administrator (DBA), who evaluates overall needs and maintains the database for the entire

organization, not just a single department or user. Database systems support enterprise-wide applications more effectively than file processing systems.

- **Stronger standards:** Good database management ensures that standards for data naming, formats, and documentation are consistently followed across the organization.
- **Better security:** The DBA can establish authorization protocols to ensure that only authorized users can access the database, and can assign different access levels to various users. Most DBMSs offer advanced security features.
- **Data independence:** Systems interacting with a DBMS are largely independent of how the physical data is stored. This flexibility allows the DBA to modify data structures without impacting the systems that use the data.

Although the trend is moving toward enterprise-wide database designs, many companies still use a mix of centralized DBMSs and smaller, department-level database systems. This is because large businesses typically view data as a company-wide resource that needs to be accessible to users across the organization. However, there are factors that encourage a more decentralized approach, such as the cost of network infrastructure, a reluctance to move away from smaller, more flexible systems, and the understanding that enterprise-wide DBMSs can be complex and costly to maintain.

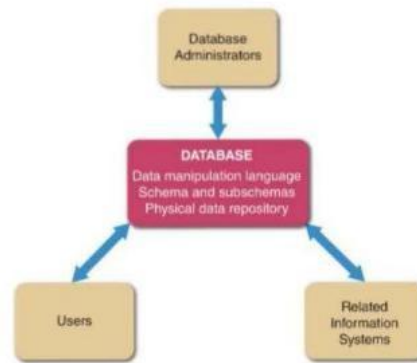
As a result, many companies adopt a **client/server design**, where data processing is distributed across several computers. Client/server systems are explained in more detail in Chapter 10. Ultimately, the best solution depends on the specific needs and circumstances of the organization.

### 3.2 DBMS COMPONENTS

A DBMS (Database Management System) serves as an interface between a database and users who need access to the data. While users are mainly concerned with an intuitive interface and support for their business needs, a systems analyst must have a comprehensive understanding of all the components of a DBMS. In addition to interfaces for users, database administrators (DBAs), and related systems, a DBMS includes several other components, such as:

- A **data manipulation language:** This allows users and applications to interact with the database, performing operations like querying, updating, and deleting data.
- A **schema and subschemas:** The schema defines the structure of the database, while subschemas represent different views or perspectives of the data based on user needs.
- A **physical data repository:** This is where the actual data is stored and organized within the system.

These components work together to enable effective database management and data access, as shown in Figure 9-6.



**FIGURE 9-6** In addition to interfaces for users, database administrators, and related information systems, a DBMS also has a data manipulation language, a schema and subschemas, and a physical data repository.

(Tilley and Rosenblatt, 2024)

### 3.2.1 Interfaces for Users, Database Administrators, and Related Systems

When users, DBAs, and related information systems request data and services, the DBMS processes those requests, manipulates the data, and provides the appropriate response. A **data manipulation language (DML)** controls database operations such as storing, retrieving, updating, and deleting data. Most commercial DBMSs, like Oracle and IBM's DB2, use a DML. Some database products, such as Microsoft Access, also offer a user-friendly graphical interface that allows users to control operations through menu-driven commands.

#### USERS:

Users typically work with predefined queries and switchboard commands, but they also use query languages to access stored data. A query language lets users specify what they want to accomplish without needing to know how the task will be carried out. Some query languages, like **Query By Example (QBE)**, allow users to provide an example of the data they need. Many database programs also use **Structured Query Language (SQL)**, which facilitates communication between client workstations and servers or mainframes. For example, Figure 9-7 shows a QBE request for all 2019 Ford Fusions of the Lime Squeeze or Blue Candy color with a power moonroof, and the SQL commands generated by that request.

#### DATABASE ADMINISTRATORS (DBAs):

The DBA is responsible for managing and maintaining the DBMS. DBAs focus on ensuring data security and integrity, preventing unauthorized access, providing backup and recovery solutions, maintaining audit trails, and supporting users' needs. Many DBMSs come with utility programs that help DBAs create and update data structures, monitor database usage patterns, and detect and report irregularities within the database.

#### RELATED INFORMATION SYSTEMS:

A DBMS can support multiple related information systems that provide input to the DBMS and require specific data from it. Unlike user interfaces, no human intervention is needed for

two-way communication between the DBMS and these related systems. These systems can exchange data directly with the DBMS in a seamless manner.



**FIGURE 9-7** Using QBE, a user can display all 2019 Ford Fusions that have a power moonroof and are either Lime Squeeze or Blue Candy color.

(Tilley and Rosenblatt, 2024)

### 3.2.2 Schema

The complete definition of a database, which includes descriptions of all fields, tables, and relationships, is called a **schema**. A **subschema** is a subset of the schema, representing a specific view of the database that is used by one or more systems or users. A subschema defines only the parts of the database that a particular system or user is authorized to access or needs. For example, to protect individual privacy, a project management system should not be allowed to access employee pay rates. In this case, the subschema for the project management system would exclude the pay rate field. Subschemas are also used to control access levels, specifying what users, systems, or locations can create, retrieve, update, or delete data, in accordance with the company's security policies.

### 3.2.3 Physical Data Repository

In Chapter 5, we discussed a data dictionary that describes all the data elements in the logical design. At this stage in the development process, the data dictionary is transformed into a **physical data repository**, which includes the schema, subschemas, and the actual stored data. The physical repository can be centralized in one location or distributed across several locations. Additionally, the stored data may be managed by a single DBMS or multiple systems.

To ensure connectivity and access across various systems and DBMSs, companies use **ODBC-compliant software** (Open Database Connectivity). ODBC is an industry-standard protocol that enables software from different vendors to communicate and exchange data. ODBC uses SQL statements, which the DBMS can understand and execute, similar to the examples in Figure 9-7. Another common standard for database connectivity is **JDBC** (Java Database Connectivity), which allows Java applications to interact with any database that uses SQL statements and is ODBC-compliant.

Physical design issues are further explored in **Chapter 10**, which covers system architecture, and in **Chapter 11**, which focuses on system implementation and data conversion.

## 3.3 WEB-BASED DESIGN

**Figure 9-8** lists some major characteristics of web-based design. In a web-based design, the

**Internet** serves as the front end, or interface, for the DBMS. Internet technology provides significant power and flexibility because the related information system is not tied to any specific combination of hardware and software. Access to the database only requires a web browser and an Internet connection.

Web-based systems are particularly popular because they offer several advantages:

- **Ease of access:** Users can connect to the system from anywhere with an internet connection.
- **Cost-effectiveness:** They eliminate the need for specialized hardware and software for each user.
- **Worldwide connectivity:** Businesses can reach global audiences, making it easier to operate in the global economy.

These features make web-based systems highly attractive for companies needing flexible, scalable solutions in a competitive, interconnected world.

| CHARACTERISTIC      | EXPLANATION                                                                                                                                                                                                                                                                                                        |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Global access       | The Internet enables worldwide access, using existing infrastructure and standard telecommunications protocols.                                                                                                                                                                                                    |
| Ease of use         | Web browsers provide a familiar interface that is user-friendly and easily learned.                                                                                                                                                                                                                                |
| Multiple platforms  | Web-based design is not dependent on a specific combination of hardware or software. All that is required is a browser and an Internet connection.                                                                                                                                                                 |
| Cost effectiveness  | Initial investment is relatively low because the Internet serves as the communication network. Users require only a browser, and web-based systems do not require powerful workstations. Flexibility is high because numerous outsourcing options exist for development, hosting, maintenance, and system support. |
| Security issues     | Security is a universal issue, but Internet connectivity raises special concerns. These can be addressed with a combination of good design, software that can protect the system and detect intrusion, stringent rules for passwords and user identification, and vigilant users and managers.                     |
| Adaptability issues | The Internet offers many advantages in terms of access, connectivity, and flexibility. Migrating a traditional database design to the web, however, can require design modification, additional software, and some added expense.                                                                                  |

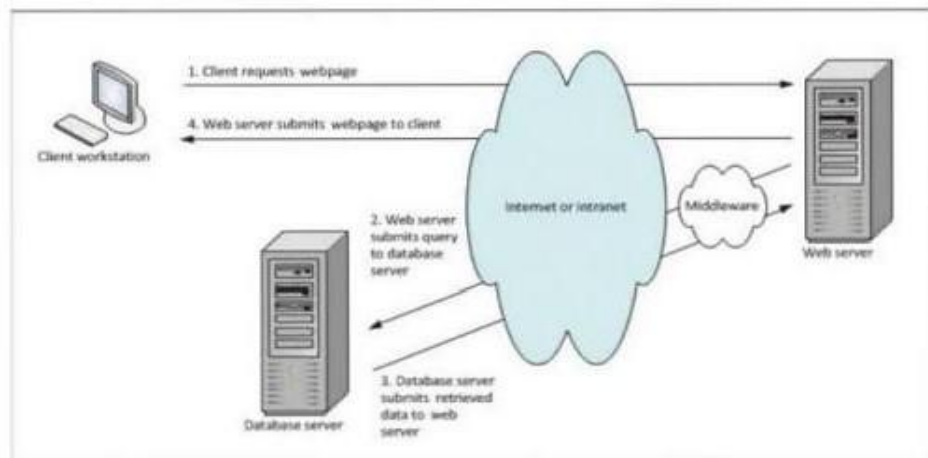
**FIGURE 9-8** Web-based design characteristics include global access, ease of use, multiple platforms, cost-effectiveness, security issues, and adaptability issues. In a web-based design, the Internet serves as the front end, or interface, to the database management system. Access to the database requires only a web browser and an Internet connection.

(Tilley and Rosenblatt, 2024)

To access data in a web-based system, the database must be connected to the **Internet** or **intranet**. However, the database and the Internet use different languages. Databases are managed using various specialized languages and commands, which are separate from **HTML**, the language used on the web. The goal is to connect the database to the web and enable data to be viewed and updated.

To bridge this gap, **middleware** is used. Middleware is software that integrates different applications and enables them to exchange data. It can interpret client requests made in HTML form and translate those requests into commands the database can understand and execute. When the database responds to those commands, middleware translates the results into HTML pages, which can then be displayed in the user's browser, as shown in Figure 9-9. The entire process can occur through the **Internet** or a company **intranet**, serving as the

communication channel between the database and the web. **Middleware** is explored further in **Chapter 10**, where its role in system integration is discussed in more detail.



**FIGURE 9-9** When a client workstation requests a web page (1), the web server uses middleware to generate a data query to the database server (2). The database server responds (3), and the middleware translates the retrieved data into an HTML page that can be sent by the web server and displayed by the user's browser (4).

Web-based data must be both secure and easily accessible to authorized users. To achieve this, well-designed systems implement security at three levels:

1. **The Database Itself:** Protecting the data stored in the database, ensuring only authorized access and actions (like reading, writing, or updating).
2. **The Web Server:** Securing the web server to prevent unauthorized access and attacks that could compromise the system's integrity.
3. **Telecommunication Links:** Ensuring the security of the communication channels that connect the database, web server, and other system components to prevent data interception or tampering.

These levels of security work together to safeguard data and maintain its integrity. **Data security** is discussed in more detail in **Section 9.9** and **Chapter 12**, where various measures and practices for securing web-based systems are explored.

### 3.4 DATA DESIGN TERMS

Using the concepts discussed earlier, a systems analyst can select a design approach and begin constructing the system. The first step is to understand key data design terminology. Here are some of the important terms:

#### 3.4.1 Definitions

- **ENTITY:** An entity is a person, place, thing, or event for which data is collected and maintained. For example, in an online sales system, entities might include **CUSTOMER**, **ORDER**, **PRODUCT**, and **SUPPLIER**. During the systems analysis phase, data flow diagrams (DFDs) were used to identify various entities and data stores. Now, the relationships among these entities will be considered.
- **TABLE OR FILE:** Data is organized into **tables** or **files**. A table or file contains a set of related records that store data about a specific entity. Tables and files are typically represented as two-dimensional structures with vertical columns and horizontal rows.

Each **column** represents a **field** (a characteristic of the entity), and each **row** represents a **record** (an individual instance of the entity). For example, if a company has 10,000 customers, the **CUSTOMER** table will have 10,000 records, each representing a specific customer. The terms **table** and **file** are often used interchangeably, depending on the context.

- **FIELD:** A **field**, also called an **attribute**, is a single characteristic or fact about an entity. For example, a **CUSTOMER** entity might include fields such as **Customer ID**, **First Name**, **Last Name**, **Address**, **City**, **State**, **Postal Code**, and **Email Address**. A **common field** is an attribute that appears in more than one entity. Common fields can be used to link entities in various types of relationships.
- **RECORD:** A **record**, also called a **tuple**, is a set of related fields that describes one instance or occurrence of an entity (e.g., one customer, one order, or one product). A record can consist of one or more fields, depending on the information needed.

These terms form the foundation for understanding data design and help in structuring and organizing the database efficiently.

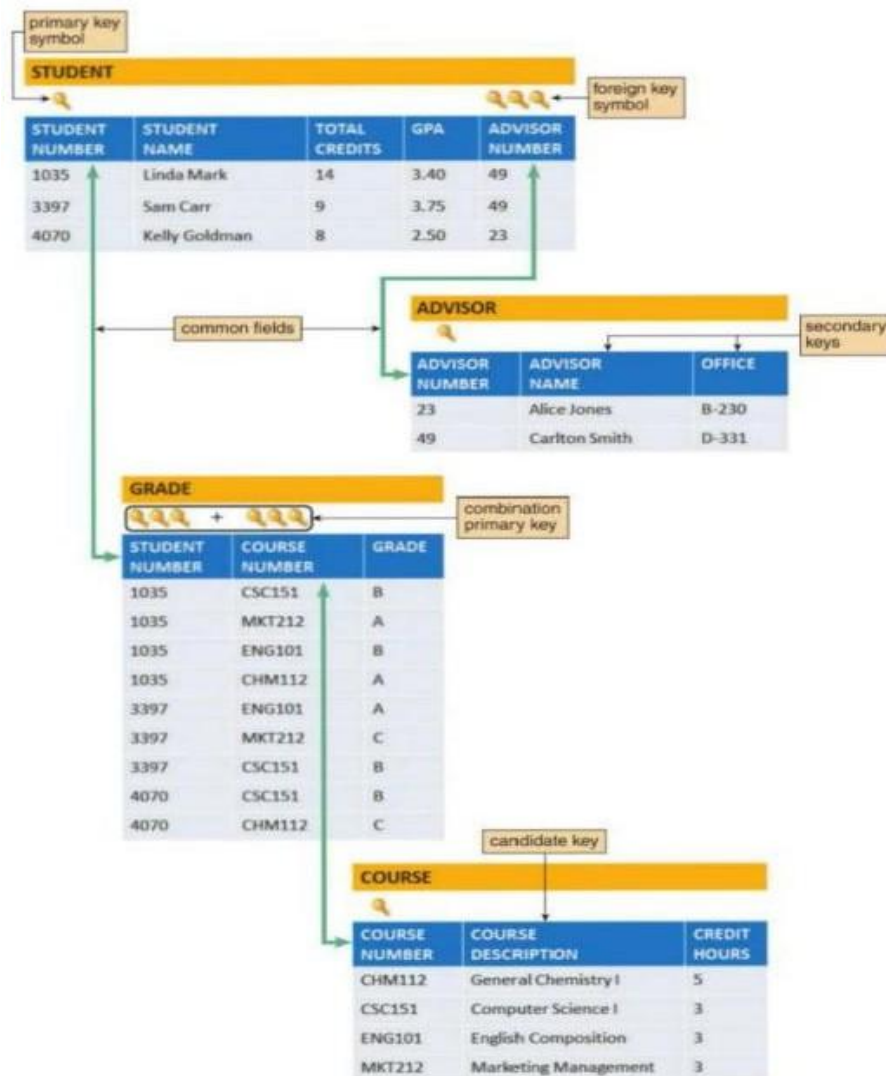


FIGURE 9-10 Examples of common fields, primary keys, candidate keys, foreign keys, and secondary keys.

(Tilley and Rosenblatt, 2024)

A **candidate key** refers to any field or combination of fields that could potentially be used as the primary key in a table. For example, if each employee has a unique employee number, that could serve as a primary key. However, a Social Security number would not make a good candidate key, as it is not always unique. When selecting a primary key, the field chosen should contain the least amount of data and be the easiest to use. Any field that isn't selected as a primary or candidate key is called a nonkey field.

In some cases, there are multiple candidate keys that could serve as the primary key. For example, the "COURSE DESCRIPTION" field in the COURSE table could also be a candidate key. However, the **OFFICE** field in the ADVISOR table cannot be a candidate key because multiple advisors could share the same office.

A **foreign key** is a field in one table that links to the primary key in another table, establishing a relationship between the two. For example, "ADVISOR NUMBER" appears in both the "STUDENT" and "ADVISOR" tables, joining them together. In the ADVISOR table, the ADVISOR NUMBER is the primary key, but in the STUDENT table, it acts as a foreign key. Foreign keys

don't need to be unique; the same value can appear multiple times in the referencing table. For example, "ADVISOR NUMBER 49" may appear many times in the STUDENT table, but it must be unique in the ADVISOR table.

Additionally, **foreign keys** can create a **composite primary key** in another table. For instance, in the "GRADE" table, both the "STUDENT NUMBER" and "COURSE NUMBER" fields are foreign keys. While student numbers and course IDs can appear multiple times, the combination of a specific student and a specific course will be unique. This unique combination ensures that the grade assigned is correctly linked to the right student and course.

A **secondary key** is a field or combination of fields used to access or retrieve records, but unlike the primary key, secondary keys don't need to be unique. For example, you could use the "POSTAL CODE" field as a secondary key to find all customers from a certain area. Secondary keys can also be used to sort records, such as by GPA in the STUDENT table, to organize students in order of their grades.

Since a table can only have one primary key, the need for secondary keys arises. For example, in a CUSTOMER file, the **CUSTOMER NUMBER** is the primary key and must be unique. However, if you need to find a customer by name, you could use the **CUSTOMER NAME** field as a secondary key to search for all records matching that name.

In summary, **secondary keys** can help retrieve, sort, or display records in various ways, even though they are not unique like primary keys. In the example in Figure 9-10, both the student name and advisor names are secondary keys, but other fields, like the **ADVISOR NUMBER** in the STUDENT table, could also be used as secondary keys to find all students with a specific advisor.

Sure! Here's the revised version with the numbering included:

### 3.4.2 Referential Integrity

Validity checks help avoid data input errors. One type of validity check, called referential integrity, is a set of rules that prevents data inconsistency and quality issues. In a relational database, referential integrity means that a foreign key value cannot be entered in one table unless it matches an existing primary key in another table.

For example, referential integrity would prevent a customer order from being entered in an order table unless that customer already exists in the customer table. Without referential integrity, an order could exist without a corresponding customer, which would create an orphan record.

In the example shown in Figure 9-10, referential integrity will not allow a user to enter an advisor number (foreign key value) in the STUDENT table unless a valid advisor number (primary key value) already exists in the ADVISOR table.

Referential integrity can also prevent the deletion of a record if it has a primary key that is referenced by foreign keys in another table. For example, if an advisor resigns to accept a position at another school, the advisor cannot be deleted from the ADVISOR table if there are

still student records in the STUDENT table that reference that advisor number. Deleting the advisor record without reassigning the students would create orphan student records. To avoid this issue, the students must be reassigned to other advisors by changing the value in the ADVISOR NUMBER field, and then the advisor record can be deleted.

When creating a relational database, referential integrity can be built into the design. Figure 9-11 shows a Microsoft Access screen that identifies a common field and allows the user to enforce referential integrity rules.

### 3.5 ENTITY RELATIONSHIP DIAGRAMS

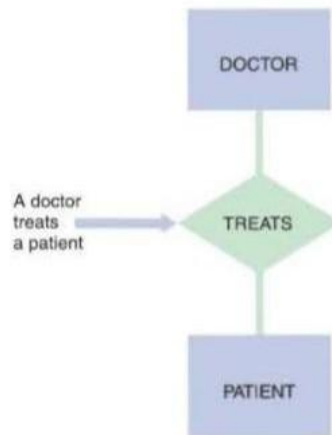
An **entity** refers to a person, place, thing, or event for which data is collected and maintained. Examples of entities include customers, sales regions, products, or orders. In an information system, it's essential to recognize the relationships between entities. For instance, a **CUSTOMER** entity can have multiple instances of an **ORDER** entity, and an **EMPLOYEE** entity may have one (or none) instance of a **SPOUSE** entity.

An **entity-relationship diagram (ERD)** is a model that illustrates the logical relationships and interactions among the system's entities. The ERD offers a high-level view of the system and serves as a blueprint for developing the physical data structures.

#### 3.5.1 Drawing an ERD

The first step in creating an Entity-Relationship Diagram (ERD) is to list the entities identified during the systems analysis phase and consider the nature of the relationships linking them. At this stage, a simplified approach can be used to illustrate the relationships between the entities.

Although there are various methods for drawing ERDs, a common approach is to represent entities as rectangles and relationships as diamond shapes. The entity rectangles are labeled with singular nouns, while the relationship diamonds are labeled with verbs, typically arranged from top to bottom or left to right. For example, in Figure 9-12, a DOCTOR entity treats a PATIENT entity. Unlike data flow diagrams, ERDs focus on depicting relationships between entities, rather than data or information flows.



**FIGURE 9-12** In an entity-relationship diagram, entities are labeled with singular nouns and relationships are labeled with verbs. The relationship is interpreted as a simple English sentence.

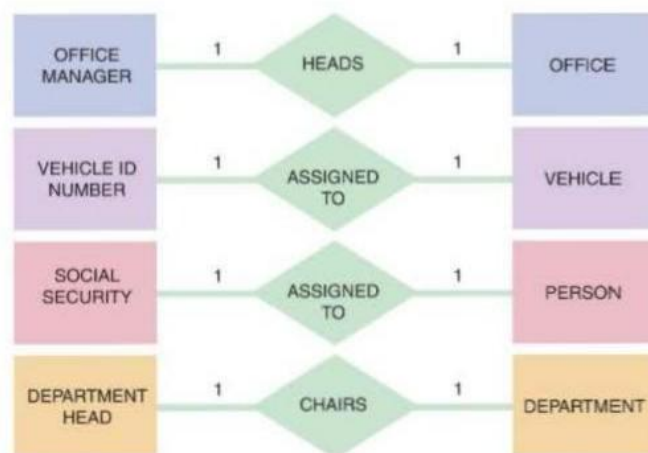
(Tilley and Rosenblatt, 2024)

### 3.5.2 Types of Relationships

There are three types of relationships that can exist between entities: one-to-one, one-to-many, and many-to-many.

#### 1. One-to-One Relationship (1:1)

A one-to-one relationship occurs when exactly one instance of the second entity is associated with each instance of the first entity. For example, if each doctor has exactly one office and each office is assigned to one doctor, this would be a 1:1 relationship. In an ERD, the number "1" is placed next to the lines connecting the two entities to indicate this relationship, as shown in Figure 9-13.



**FIGURE 9-13** Examples of one-to-one (1:1) relationships.

(Tilley and Rosenblatt, 2024)

## 2. One-to-Many Relationship (1:M)

A one-to-many relationship exists when one instance of the first entity can relate to many instances of the second entity, but each instance of the second entity can only relate to one instance of the first entity. For example, in the relationship between **DEPARTMENT** and **EMPLOYEE**, one department can have many employees, but each employee works in only one department at a time. In an ERD, the line connecting the "many" side is labeled with "M," and the line connecting the "one" side is labeled with "1," as shown in Figure 9-14.

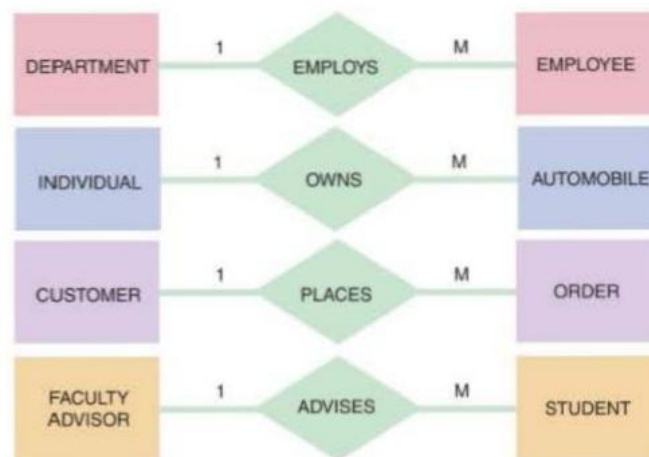


FIGURE 9-14 Examples of one-to-many (1:M) relationships.

(Tilley and Rosenblatt, 2024)

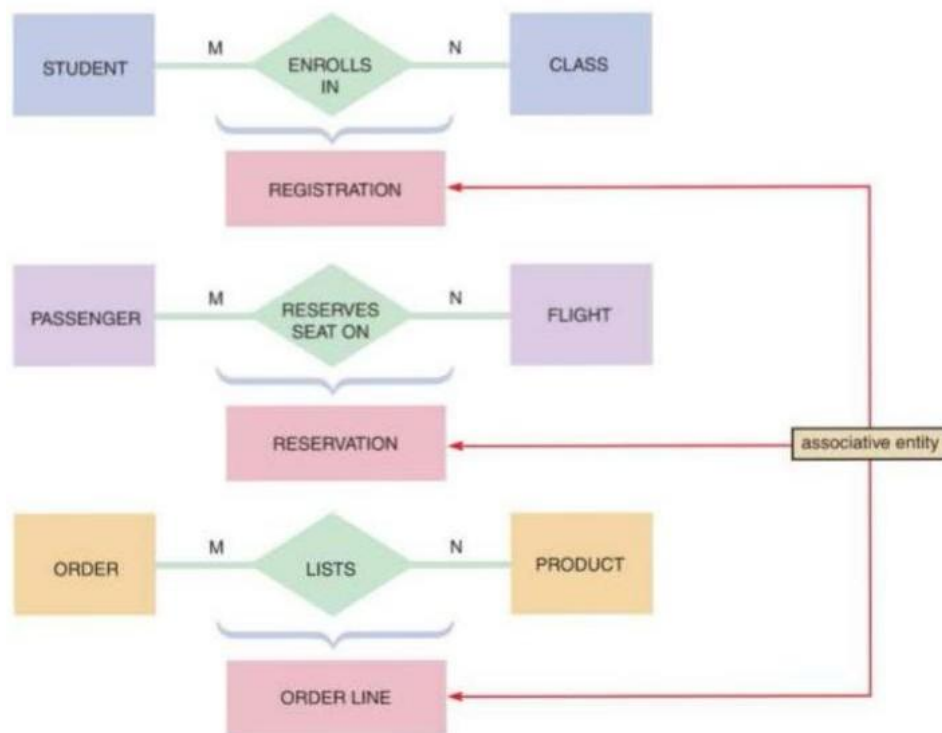
### How many is many?

In some cases, the "many" can refer to any number, including zero. For example, an individual might own multiple automobiles, one automobile, or none at all. Therefore, "many" can represent any number, including zero, as shown in the **INDIVIDUAL** and **AUTOMOBILE** relationship in Figure 9-14.

## 3. Many-to-Many Relationship (M:N)

A many-to-many relationship occurs when one instance of the first entity can relate to many instances of the second entity, and one instance of the second entity can relate to many instances of the first entity. For example, the relationship between

**STUDENT** and **CLASS** is many-to-many: a student can take many classes, and each class can have many students enrolled. In an ERD, both connecting lines are labeled with the letter "M" and "N" to indicate the many-to-many relationship, as shown in Figure 9-15.

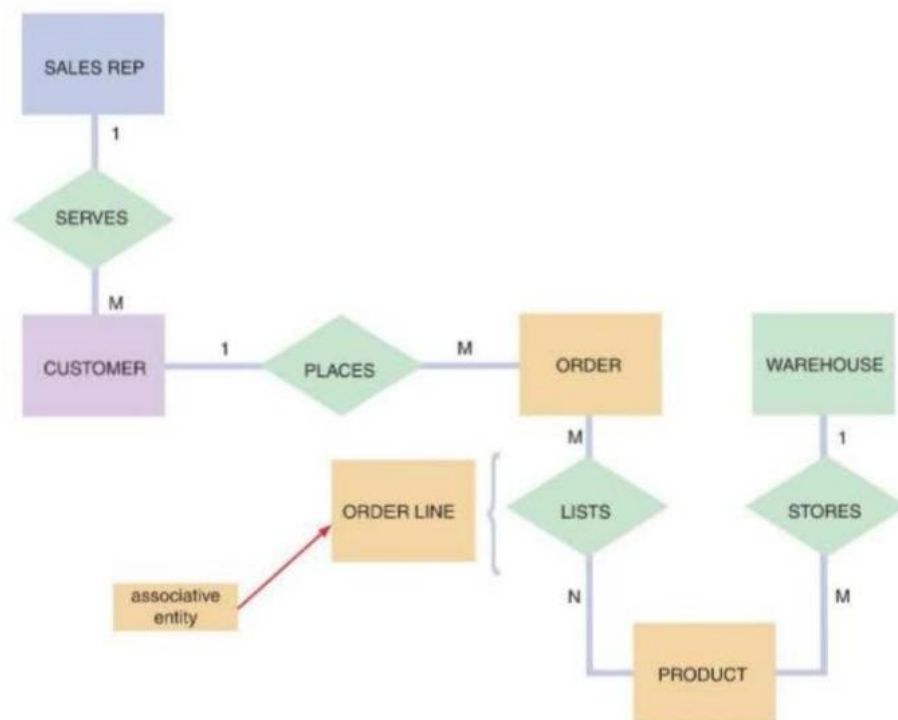


**FIGURE 9-15** Examples of many-to-many (M:N) relationships. Notice that the event or transaction that links the two entities is an associative entry with its own set of attributes and characteristics.

(Tilley and Rosenblatt, 2024)

An **M:N** relationship differs from **1:1** or **1:M** relationships because the event or transaction linking the two entities is actually a third entity, known as an **associative entity**, which has its own characteristics. In the first example in Figure 9-15, the **ENROLL IN** symbol represents a **REGISTRATION** entity, which records each instance of a specific student enrolling in a specific course. Similarly, the **RESERVES SEAT ON** symbol represents a **RESERVATION** entity, which records each instance of a specific passenger reserving a seat on a specific flight. In the third example, the **LISTS** symbol represents an **ORDER LINE** entity, which records each instance of a specific product being listed in a specific customer order.

Figure 9-16 shows an ERD for a sales system, highlighting various entities and relationships, including the associative entity called **ORDER LINE**. The detailed nature of these relationships is referred to as **cardinality**. To create an accurate data design that reflects all relationships among system entities, an analyst must fully understand cardinality.



**FIGURE 9-16** An entity-relationship diagram for SALES REP, CUSTOMER, ORDER, PRODUCT, and WAREHOUSE. Notice that the ORDER and PRODUCT entities are joined by an associative entity named ORDER LINE.

(Tilley and Rosenblatt, 2024)

## Normalization

Normalization is the process of designing tables by assigning specific fields or attributes to each table in a database. A table design defines the fields and specifies the primary key for each table. By working with an initial set of table designs, normalization helps create a more efficient database design that is straightforward, adaptable, and eliminates data redundancy. It involves applying a series of rules to detect and resolve issues and complexities in table designs. The concept of normalization was developed by Edgar Codd, a British computer scientist, who laid out the fundamental principles of relational database design.

### 3.6 Normalization Stages

The normalization process typically includes four stages: unnormalized design, first normal form, second normal form, and third normal form. These stages represent a progression, with third normal form being the optimal design. Most business databases are designed using third normal form, though higher normal forms exist, they are rarely needed in business systems.

#### 3.6.1 Standard Notation Format

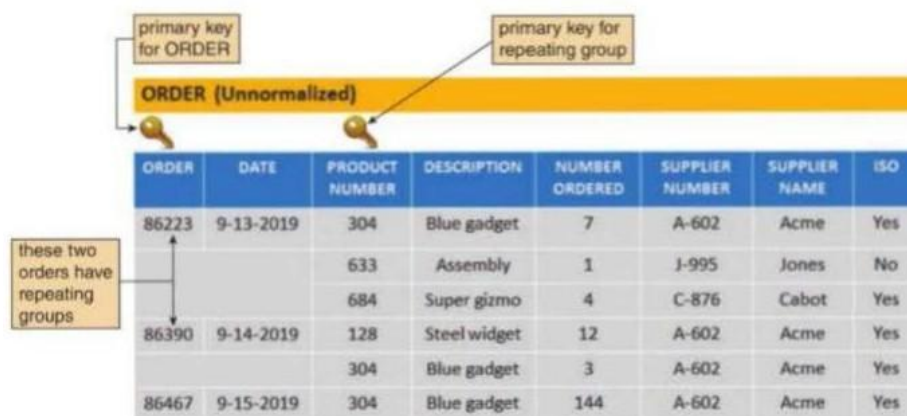
Designing tables is easier when using a standardized notation format to represent a table's structure, fields, and primary key. The standard notation for an ORDER system starts with the table name, followed by a set of fields in parentheses, separated by commas. The primary key

field(s) are underlined, like this:

NAME (FIELD 1, FIELD 2, FIELD 3).

## Repeating Groups

During data design, the analyst must identify repeating groups of fields. A repeating group is a set of one or more fields that can appear multiple times within a single record, with each occurrence having different values. For example, in the case of a company using written source documents to record orders, a repeating group might consist of multiple products listed within the same order number. In this case, fields such as product number, description, quantity ordered, supplier number, supplier name, and status would repeat within the same order. A repeating group can be thought of as a set of child records within a parent record.



The diagram shows a table titled "ORDER (Unnormalized)". Above the table, two callouts identify keys: "primary key for ORDER" points to the ORDER column, and "primary key for repeating group" points to the PRODUCT NUMBER column. The table itself has columns: ORDER, DATE, PRODUCT NUMBER, DESCRIPTION, NUMBER ORDERED, SUPPLIER NUMBER, SUPPLIER NAME, and ISO. It contains six rows. The first two rows (86223 and 86390) are grouped by a bracket on the left labeled "these two orders have repeating groups". The first row (86223) has three rows of product data (304, 633, 684). The second row (86390) has two rows of product data (128, 304). The third row (86467) has one row of product data (304).

| ORDER | DATE      | PRODUCT NUMBER | DESCRIPTION  | NUMBER ORDERED | SUPPLIER NUMBER | SUPPLIER NAME | ISO |
|-------|-----------|----------------|--------------|----------------|-----------------|---------------|-----|
| 86223 | 9-13-2019 | 304            | Blue gadget  | 7              | A-602           | Acme          | Yes |
|       |           | 633            | Assembly     | 1              | J-995           | Jones         | No  |
|       |           | 684            | Super gizmo  | 4              | C-876           | Cabot         | Yes |
| 86390 | 9-14-2019 | 128            | Steel widget | 12             | A-602           | Acme          | Yes |
|       |           | 304            | Blue gadget  | 3              | A-602           | Acme          | Yes |
| 86467 | 9-15-2019 | 304            | Blue gadget  | 144            | A-602           | Acme          | Yes |

**FIGURE 9-20** In the ORDER table design, two orders have repeating groups that contain several products. ORDER is the primary key for the ORDER table, and PRODUCT NUMBER serves as a primary key for the repeating group. Because it contains repeating groups, the ORDER table is unnormalized.

(Tilley and Rosenblatt, 2024)

## Unnormalized Table Design

A table design that contains a repeating group is referred to as unnormalized. The standard notation method for representing an unnormalized design is to enclose the repeating group of fields within a second set of parentheses. An example of an unnormalized table looks like this:

**NAME (FIELD 1, FIELD 2, FIELD 3, (REPEATING FIELD 1, REPEATING FIELD 2))**

Now, let's look at the unnormalized ORDER table design shown in Figure 9-20. According to the notation guidelines, the design can be described as follows:

**ORDER (ORDER, DATE, (PRODUCT NUMBER, DESCRIPTION, NUMBER ORDERED, SUPPLIER NUMBER, SUPPLIER NAME, ISO))**

This notation indicates that the ORDER table design contains eight fields, which are listed within the outer parentheses. The ORDER field is underlined to indicate it is the primary key. The fields PRODUCT NUMBER, DESCRIPTION, NUMBER ORDERED, SUPPLIER NUMBER, SUPPLIER NAME, and ISO are enclosed within an inner set of parentheses to show they are part of a repeating group. Note that PRODUCT NUMBER is also underlined because it serves as the primary key of the repeating group. If a customer orders three different products in one order, these six fields will repeat for each product.

### 3.6.2 First Normal Form

A table is in first normal form (1NF) if it does not contain a repeating group. To convert an unnormalized design to 1NF, the table's primary key must be expanded to include the primary key of the repeating group.

For instance, in the ORDER table shown in Figure 9-20, the repeating group consists of six fields: PRODUCT NUMBER, DESCRIPTION, NUMBER ORDERED, SUPPLIER NUMBER, SUPPLIER NAME, and ISO. Of these fields, only PRODUCT NUMBER can act as a primary key because it uniquely identifies each instance of the repeating group. DESCRIPTION cannot be a primary key because it may not be unique; for example, many products might share the same description, like "washer," but be identified uniquely by their part numbers.

When the primary key of the ORDER table is expanded to include PRODUCT NUMBER, the repeating group is eliminated, and the ORDER table is now in 1NF. The new design looks like this:

**ORDER (ORDER, DATE, PRODUCT NUMBER, DESCRIPTION, NUMBER ORDERED, SUPPLIER NUMBER, SUPPLIER NAME, ISO)**

In this 1NF design, the repeating group is removed, and more records are generated—one for each combination of a specific order and a specific product. For example, the repeating group for order number 86223 now becomes three separate records, and the repeating group for order number 86390 becomes two separate records. As a result, each record in the 1NF design represents a single instance of a specific order and product.

It is important to note that the primary key in the 1NF design is a combination of the ORDER and PRODUCT NUMBER fields. Neither ORDER nor PRODUCT NUMBER alone can be the primary key, as ORDER does not uniquely identify each product in a multiple-item order, and PRODUCT NUMBER can appear more than once across different orders. Therefore, both fields together form a composite primary key to uniquely identify each record in the table.

in 1NF, the primary key is a **unique** combination of a specific ORDER and a specific PRODUCT NUMBER

**ORDER in 1NF**

🔑 + 🔑

| ORDER | DATE      | PRODUCT NUMBER | DESCRIPTION  | NUMBER ORDERED | SUPPLIER NUMBER | SUPPLIER NAME | ISO |
|-------|-----------|----------------|--------------|----------------|-----------------|---------------|-----|
| 86223 | 9-13-2019 | 304            | Blue gadget  | 7              | A-602           | Acme          | Yes |
| 86223 | 9-13-2019 | 633            | Assembly     | 1              | J-995           | Jones         | No  |
| 86223 | 9-13-2019 | 684            | Super gizmo  | 4              | C-876           | Cabot         | Yes |
| 86390 | 9-14-2019 | 128            | Steel widget | 12             | A-602           | Acme          | Yes |
| 86390 | 9-14-2019 | 304            | Blue gadget  | 3              | A-602           | Acme          | Yes |
| 86467 | 9-15-2019 | 304            | Blue gadget  | 144            | A-602           | Acme          | Yes |

in 1NF

- There are no repeating groups
- The primary key is a **unique** combination of two foreign key values: ORDER and PRODUCT NUMBER
- All fields depend on the primary key, but some fields do not depend on the **whole** key — only part of it

**FIGURE 9-21** The ORDER table as it appears in 1NF. The repeating groups have been eliminated. Notice that the repeating group for order 86223 has become three separate records, and the repeating group for order 86390 has become two separate records. The 1NF primary key is a combination of ORDER and PRODUCT NUMBER, which uniquely identifies each record.

(Tilley and Rosenblatt, 2024)

### 3.6.3 Second Normal Form

To understand second normal form (2NF), it's essential to grasp the concept of functional dependence. A field A is functionally dependent on field B if the value of field A depends on field B. For example, in Figure 9-21, the DATE value is functionally dependent on the ORDER because for a specific order number, there can only be one date. In contrast, the product description is not dependent on the order number because there might be multiple product descriptions for a specific order number, each representing a different item ordered.

A table design is in second normal form (2NF) if it is in 1NF and if all fields that are not part of the primary key are functionally dependent on the entire primary key. If any field in a 1NF table depends on only part of a combination primary key, the table is not in 2NF.

Note that if a 1NF design has a primary key consisting of only one field, the issue of partial dependence does not arise, because the entire primary key is a single field. Therefore, a 1NF table with a single-field primary key is automatically in 2NF.

Now, let's reexamine the 1NF design for the ORDER table shown in Figure 9-21:

**ORDER (ORDER, DATE, PRODUCT NUMBER, DESCRIPTION, NUMBER ORDERED, SUPPLIER NUMBER, SUPPLIER NAME, ISO)**

Recall that the primary key is the combination of the order number and the product number. The NUMBER ORDERED field depends on the entire primary key because it refers to a specific product number and a specific order number. However, the DATE field depends only on the order number, which is only part of the primary key. Similarly, the DESCRIPTION field depends on the product number, which is also only part of the primary key. Since some fields do not depend on the entire primary key, the design is not in 2NF.

## Converting 1NF to 2NF

There is a standard process for converting a table from 1NF to 2NF. The objective is to break the original table into two or more new tables and reassign the fields so that each nonkey field will depend on the entire primary key in its table. The following steps should be followed:

1. **Create and name a separate table for each field in the existing primary key.**  
For example, in Figure 9-21, the ORDER table's primary key has two fields, ORDER and PRODUCT NUMBER, so two tables must be created. The result is:

|                                      |                |             |
|--------------------------------------|----------------|-------------|
| <b>ORDER</b>                         | <b>(ORDER,</b> | <b>...)</b> |
| <b>PRODUCT (PRODUCT NUMBER, ...)</b> |                |             |

2. **Create a new table for each possible combination of the original primary key fields.**  
In the Figure 9-21 example, a new table is created with a combination primary key of ORDER and PRODUCT NUMBER. This table describes individual lines in an order, so it is named **ORDER LINE (ORDER, PRODUCT NUMBER)**.

3. **Study the three tables and place each field with its appropriate primary key, which is the minimal key on which it functionally depends.**  
After placing the fields in the appropriate tables, remove any table that does not have any additional fields assigned to it. The remaining tables represent the 2NF version of the original table.

The three tables in 2NF would be:

|                                                                                   |                |              |
|-----------------------------------------------------------------------------------|----------------|--------------|
| <b>ORDER</b>                                                                      | <b>(ORDER,</b> | <b>DATE)</b> |
| <b>PRODUCT (PRODUCT NUMBER, DESCRIPTION, SUPPLIER NUMBER, SUPPLIER NAME, ISO)</b> |                |              |
| <b>ORDER LINE (ORDER, PRODUCT NUMBER)</b>                                         |                |              |

Figure 9-22 shows the 2NF table designs. By following the steps, the original 1NF table has been converted into three 2NF tables.

**Why is it important to move from 1NF to 2NF?**  
There are four kinds of problems with 1NF designs that do not exist in 2NF:

### 1. Cumbersome updates:

Suppose there are 500 current orders for product number 304, and the product description needs to be updated. In 1NF, this would require modifying 500 records, which is labor-intensive and costly.

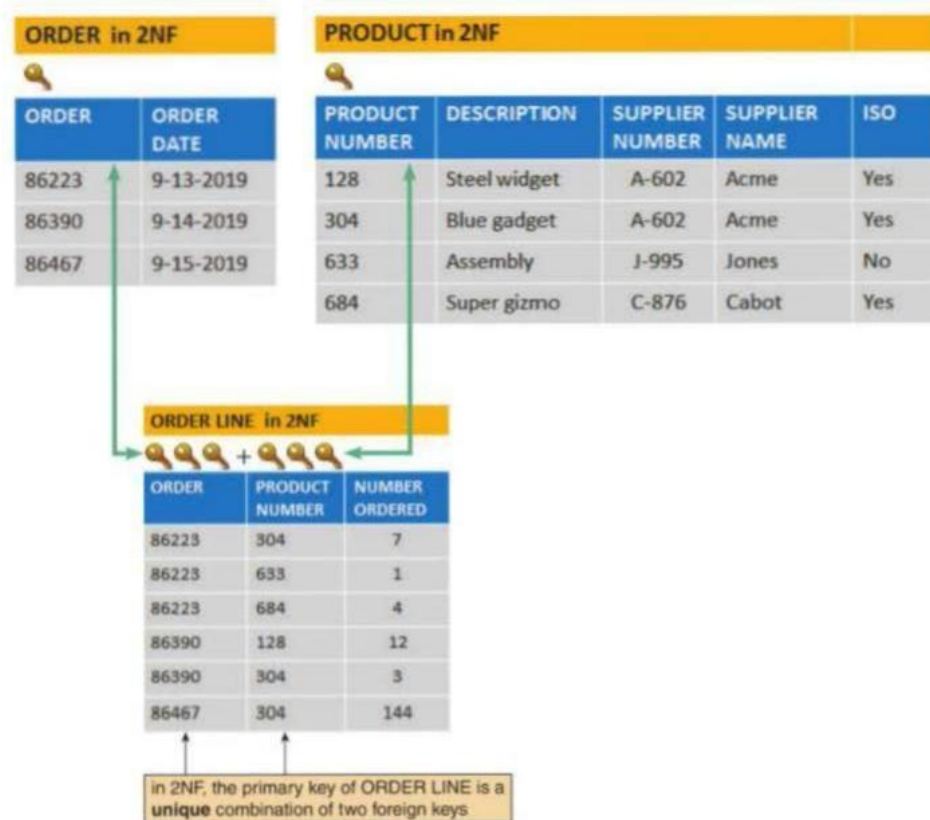
### 2. Inconsistent

**data:**

In 1NF, there could be inconsistent product descriptions across records. If product number 304 appears in many order records, some descriptions might be inaccurate or misspelled. This inconsistency is difficult to manage and results in data quality problems.

### 3. Problems adding a new product:

In 1NF, the primary key requires both an order number and a product number. This creates difficulty when adding a new product that has not been ordered yet, as there is no order number to use.



**FIGURE 9-22** ORDER, PRODUCT, and ORDER LINE tables in 2NF. All fields are functionally dependent on the primary key.

(Tilley and Rosenblatt, 2024)

#### 4. Problems deleting a product:

If a product is deleted, and all related records are removed after an order is completed, the product's details may be lost if no orders currently reference it.

#### Has the 2NF design eliminated all potential problems?

Yes, in the 2NF design:

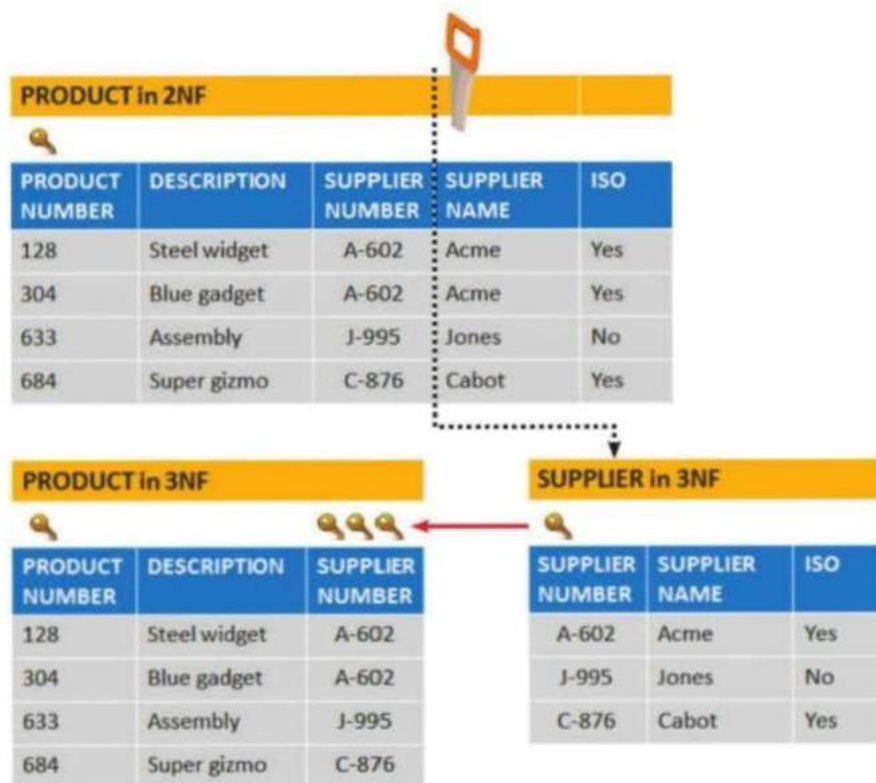
- Changing a product description now only requires updating one **PRODUCT** record, not multiple records.
- Multiple, inconsistent product descriptions are avoided because the description appears only once in the **PRODUCT** table.
- Adding a new product is easier because a new **PRODUCT** record is created without needing an order number.
- Deleting a product is also simplified. Even if the last **ORDER LINE** record for a product is removed, the **PRODUCT** record remains, ensuring the product's description is not lost.

Thus, the four potential problems from 1NF are eliminated, and the three 2NF designs are more efficient and reliable than both the original unnormalized and 1NF designs.

### 3.6.4 Third Normal Form

A popular rule of thumb is that a design is in **Third Normal Form (3NF)** if every nonkey field depends on the key, the whole key, and nothing but the key. A 3NF design eliminates redundancy and data integrity issues that can still exist in 2NF designs.

Continuing with the **ORDER** example, let's review the **PRODUCT** table design in Figure 9-23: **PRODUCT (PRODUCT NUMBER, DESCRIPTION, SUPPLIER NUMBER, SUPPLIER NAME, ISO)**



**FIGURE 9-23** When the PRODUCT table is transformed from 2NF to 3NF, the result is two separate tables: PRODUCT and SUPPLIER. Note that in 3NF, all fields depend on the key alone.

(Tilley and Rosenblatt, 2024)

This **PRODUCT** table is in 1NF because it has no repeating group, and it is in 2NF because the primary key is a single field. However, there are still potential problems in this design:

1. **Cumbersome updates:**  
If a supplier's name needs to be changed, every record in which that name appears must be updated. If there are hundreds or thousands of records, this becomes slow, expensive, and prone to errors.
2. **Inconsistent data:**  
The 2NF design allows a supplier to have different names or ISO statuses in different records, leading to potential inconsistencies.
3. **Problems adding new suppliers:**  
Since the supplier name is included in the **ORDER** table, a dummy **ORDER** record must be created to add a new supplier who has not yet received any orders.
4. **Loss of supplier information:**  
If all orders for a supplier are deleted, the supplier's number and name will also be lost.

These issues arise because the design is not in **3NF**. A table design is in **3NF** if it is in **2NF** and if no nonkey field is dependent on another nonkey field. A **nonkey field** is any field that is not a candidate key for the primary key.

In the **PRODUCT** table, **SUPPLIER NAME** and **ISO** are dependent on **SUPPLIER NUMBER**, which is a nonkey field. Since these two nonkey fields depend on another nonkey field, the table is not in **3NF**.

To convert the table to **3NF**, we need to remove all fields from the **2NF** table that depend on another nonkey field and place them in a new table where the nonkey field becomes the primary key. In the **PRODUCT** example, **SUPPLIER NAME** and **ISO** depend on **SUPPLIER NUMBER**, so they need to be removed from the **PRODUCT** table and placed in a new **SUPPLIER** table.

As shown in Figure 9-23, the **3NF** version divides the **2NF** table into two separate **3NF** tables:

- **PRODUCT (PRODUCT NUMBER, DESCRIPTION, SUPPLIER NUMBER)**
- **SUPPLIER (SUPPLIER NUMBER, SUPPLIER NAME, ISO)**

This transformation eliminates redundancy and the potential problems associated with the **2NF** design, resulting in a more efficient and consistent structure.

#### CASE IN POINT 9.2: CYBERTOYS

You handle administrative support for CyberToys, a small chain that sells computer hardware and software and specializes in personal service. The company has four stores located at malls and is planning more. Each store has a manager, a technician, and between one and four sales reps.

The owners want to create a personnel records database, and they asked you to review a table that they had designed. They suggested fields for store number, location, store telephone, manager name, and manager home telephone. They also want fields for technician name and technician home telephone and fields for up to four sales rep names and sales rep home telephones.

Draw their suggested design and analyze it using the normalization concepts you learned in the chapter. What do you think of their design and why? What would you propose?

### 3.6.5 Two Real-World Examples

A good way to understand normalization is to apply the rules to real-world scenarios. This section presents two different examples: a school and a technical service company. By following a step-by-step process, data designs can be created that are efficient, maintainable, and resistant to errors.

EXAMPLE I: Crossroads College

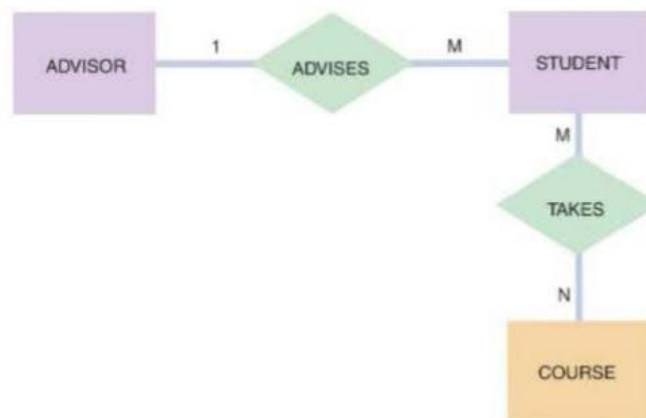
Consider the typical situation depicted in **Figure 9-24**, which shows several entities in the **Crossroads College advising system: ADVISOR, COURSE, and STUDENT.**



**FIGURE 9-24** A faculty advisor, who represents an entity, can advise many students, each of whom can register for one or many courses.  
Monkey Business Images/Shutterstock.com

(Tilley and Rosenblatt, 2024)

The relationships among the three entities are shown in the **Entity-Relationship Diagram (ERD)** in **Figure 9-25**. The following sections will discuss the normalization rules for these entities.



**FIGURE 9-25** An initial entity-relationship diagram for ADVISOR, STUDENT, and COURSE.

(Tilley and Rosenblatt, 2024)

Before starting the normalization process, it is noted that the **STUDENT** table contains fields related to the **ADVISOR** and **COURSE** entities. As a result, a decision is made to begin with the initial design for the **STUDENT** table, shown in **Figure 9-26**. The table design includes the following fields:

- **Student Number**
- **Student Name**
- **Total Credits Taken**
- **Grade Point Average (GPA)**
- **Advisor Number**
- **Advisor Name**

- **Advisor Office**
- **Course Number**
- **Course Description**
- **Course Credits**
- **Grade Received**

The **STUDENT** table in **Figure 9-26** is not yet normalized because it has a repeating group (related to the courses the student has taken). This repeating group consists of **Course Number**, **Credits**, and **Grade** fields.

The **STUDENT** table design can be written as follows:

- **STUDENT (STUDENT NUMBER, STUDENT NAME, TOTAL CREDITS, GPA, ADVISOR NUMBER, ADVISOR NAME, OFFICE, (COURSE NUMBER, CREDIT HOURS, GRADE))**

To convert this **STUDENT** table into **1NF**, the primary key must be expanded to include the primary key of the repeating group. This results in a new design that eliminates the repeating group by turning each course entry into a separate record for each student. The **1NF** version of the **STUDENT** table is:

- **STUDENT (STUDENT NUMBER, STUDENT NAME, TOTAL CREDITS, GPA, ADVISOR NUMBER, ADVISOR NAME, OFFICE, COURSE NUMBER, CREDIT HOURS, GRADE)**

Now, each record contains data about one course taken by one student, and the table is in **1NF**. The primary key has been expanded to include **STUDENT NUMBER** and **COURSE NUMBER**, ensuring that each record is unique for each student and course combination.

| STUDENT (Unnormalized) |              |               |      |                |              |        |               |              |       |
|------------------------|--------------|---------------|------|----------------|--------------|--------|---------------|--------------|-------|
| STUDENT NUMBER         | STUDENT NAME | TOTAL CREDITS | GPA  | ADVISOR NUMBER | ADVISOR NAME | OFFICE | COURSE NUMBER | CREDIT HOURS | GRADE |
| 1035                   | Linda        | 47            | 3.60 | 49             | Smith        | B212   | CSC151        | 4            | B     |
|                        |              |               |      |                |              |        | MKT212        | 3            | A     |
|                        |              |               |      |                |              |        | ENG101        | 3            | B     |
|                        |              |               |      |                |              |        | CHM112        | 4            | A     |
|                        |              |               |      |                |              |        | BUS105        | 2            | A     |
| 3397                   | Sam          | 29            | 3.00 | 49             | Smith        | B212   | ENG101        | 3            | A     |
|                        |              |               |      |                |              |        | MKT212        | 3            | C     |
|                        |              |               |      |                |              |        | CSC151        | 4            | B     |
| 4070                   | Kelly        | 14            | 2.90 | 23             | Jones        | C333   | CSC151        | 4            | B     |
|                        |              |               |      |                |              |        | CHM112        | 4            | C     |
|                        |              |               |      |                |              |        | ENG101        | 3            | C     |
|                        |              |               |      |                |              |        | BUS105        | 2            | C     |

**FIGURE 9-26** The **STUDENT** table is unnormalized because it contains a repeating group that represents the course each student has taken.

(Tilley and Rosenblatt, 2024)

Figure 9-27 displays the **1NF** version of the sample **STUDENT** data. In this table, certain fields

depend only on a part of the primary key. Specifically, the **student name**, **total credits**, **GPA**, **advisor number**, and **advisor name** are associated only with the **student number**, and do not rely on the **course number**. The **course description** is dependent solely on the **course number**, not the **student number**. Only the **GRADE** field depends on the full primary key (both **student number** and **course number**).

in 1NF, the primary key is a **unique** combination of a specific STUDENT NUMBER and a specific COURSE NUMBER

**STUDENT in 1NF**

| STUDENT NUMBER | STUDENT NAME | TOTAL CREDITS | GPA  | ADVISOR NUMBER | ADVISOR NAME | OFFICE | COURSE NUMBER | CREDIT HOURS | GRADE |
|----------------|--------------|---------------|------|----------------|--------------|--------|---------------|--------------|-------|
| 1035           | Linda        | 47            | 3.60 | 49             | Smith        | B212   | CSC151        | 4            | B     |
| 1035           | Linda        | 47            | 3.60 | 49             | Smith        | B212   | MKT212        | 3            | A     |
| 1035           | Linda        | 47            | 3.60 | 49             | Smith        | B212   | ENG101        | 3            | B     |
| 1035           | Linda        | 47            | 3.60 | 49             | Smith        | B212   | CHM112        | 4            | A     |
| 1035           | Linda        | 47            | 3.60 | 49             | Smith        | B212   | BUS105        | 2            | A     |
| 3397           | Sam          | 29            | 3.00 | 49             | Smith        | B212   | ENG101        | 3            | A     |
| 3397           | Sam          | 29            | 3.00 | 49             | Smith        | B212   | MKT212        | 3            | C     |
| 3397           | Sam          | 29            | 3.00 | 49             | Smith        | B212   | CSC151        | 4            | B     |
| 4070           | Kelly        | 14            | 2.90 | 23             | Jones        | C333   | CSC151        | 4            | B     |
| 4070           | Kelly        | 14            | 2.90 | 23             | Jones        | C333   | CHM112        | 4            | C     |
| 4070           | Kelly        | 14            | 2.90 | 23             | Jones        | C333   | ENG101        | 3            | C     |
| 4070           | Kelly        | 14            | 2.90 | 23             | Jones        | C333   | BUS105        | 2            | C     |

in 1NF

- There are no repeating groups
- The primary key is a **unique** combination of two foreign key values: STUDENT NUMBER and COURSE NUMBER
- All fields depend on the primary key, but some fields do not depend on the **whole** key — only part of it

**FIGURE 9-27** The student table in 1NF. Notice that the primary key has been expanded to include STUDENT NUMBER and COURSE NUMBER.

(Tilley and Rosenblatt, 2024)

Figure 9-27 displays the **1NF** version of the sample **STUDENT** data. In this table, certain fields depend only on a part of the primary key. Specifically, the **student name**, **total credits**, **GPA**, **advisor number**, and **advisor name** are associated only with the **student number**, and do not rely on the **course number**. The **course description** is dependent solely on the **course number**, not the **student number**. Only the **GRADE** field depends on the full primary key (both **student number** and **course number**).

in 1NF, the primary key is a **unique** combination of a specific STUDENT NUMBER and a specific COURSE NUMBER

**STUDENT in 1NF**

| STUDENT NUMBER | STUDENT NAME | TOTAL CREDITS | GPA  | ADVISOR NUMBER | ADVISOR NAME | OFFICE | COURSE NUMBER | CREDIT HOURS | GRADE |
|----------------|--------------|---------------|------|----------------|--------------|--------|---------------|--------------|-------|
| 1035           | Linda        | 47            | 3.60 | 49             | Smith        | B212   | CSC151        | 4            | B     |
| 1035           | Linda        | 47            | 3.60 | 49             | Smith        | B212   | MKT212        | 3            | A     |
| 1035           | Linda        | 47            | 3.60 | 49             | Smith        | B212   | ENG101        | 3            | B     |
| 1035           | Linda        | 47            | 3.60 | 49             | Smith        | B212   | CHM112        | 4            | A     |
| 1035           | Linda        | 47            | 3.60 | 49             | Smith        | B212   | BUS105        | 2            | A     |
| 3397           | Sam          | 29            | 3.00 | 49             | Smith        | B212   | ENG101        | 3            | A     |
| 3397           | Sam          | 29            | 3.00 | 49             | Smith        | B212   | MKT212        | 3            | C     |
| 3397           | Sam          | 29            | 3.00 | 49             | Smith        | B212   | CSC151        | 4            | B     |
| 4070           | Kelly        | 14            | 2.90 | 23             | Jones        | C333   | CSC151        | 4            | B     |
| 4070           | Kelly        | 14            | 2.90 | 23             | Jones        | C333   | CHM112        | 4            | C     |
| 4070           | Kelly        | 14            | 2.90 | 23             | Jones        | C333   | ENG101        | 3            | C     |
| 4070           | Kelly        | 14            | 2.90 | 23             | Jones        | C333   | BUS105        | 2            | C     |

in 1NF

- There are no repeating groups
- The primary key is a **unique** combination of two foreign key values: STUDENT NUMBER and COURSE NUMBER
- All fields depend on the primary key, but some fields do not depend on the **whole** key — only part of it

**FIGURE 9-27** The student table in 1NF. Notice that the primary key has been expanded to include STUDENT NUMBER and COURSE NUMBER.

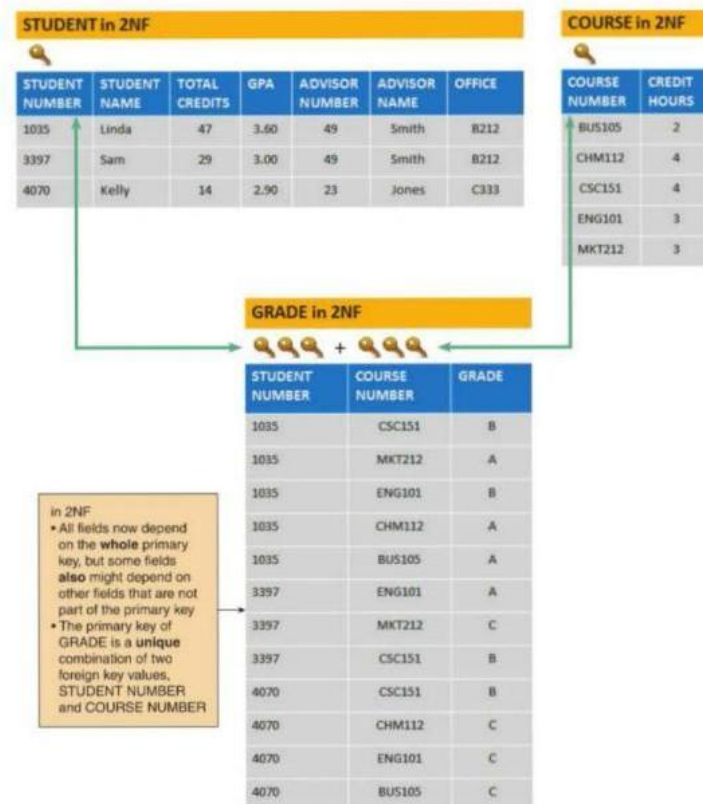
(Tilley and Rosenblatt, 2024)

Following the **1NF to 2NF** conversion process, new tables are created for each field and combination of fields within the primary key. The other fields are assigned to their respective keys. The resulting tables are as follows:

- **STUDENT** (STUDENT NUMBER, STUDENT NAME, TOTAL CREDITS, GPA, ADVISOR NUMBER, ADVISOR NAME, OFFICE)
- **COURSE** (COURSE NUMBER, CREDIT HOURS)
- **GRADE** (STUDENT NUMBER, COURSE NUMBER, GRADE)

The original **1NF STUDENT** table has now been converted into three tables, all in **2NF**. In each of these tables, every non-key field depends on the entire primary key.

However, are all three tables in **3NF**? The **COURSE** and **GRADE** tables are in **3NF**, but the **STUDENT** table is not. This is because the **ADVISOR NAME** and **OFFICE** fields depend on the **ADVISOR NUMBER**, which is not part of the **STUDENT** table's primary key. To convert the **STUDENT** table to **3NF**, the **ADVISOR NAME** and **OFFICE** fields are removed and placed into a separate table where **ADVISOR NUMBER** serves as the primary key.



**FIGURE 9-28** The STUDENT, COURSE, and GRADE tables in 2NF. Notice that all fields are functionally dependent on the entire primary key of their respective tables.

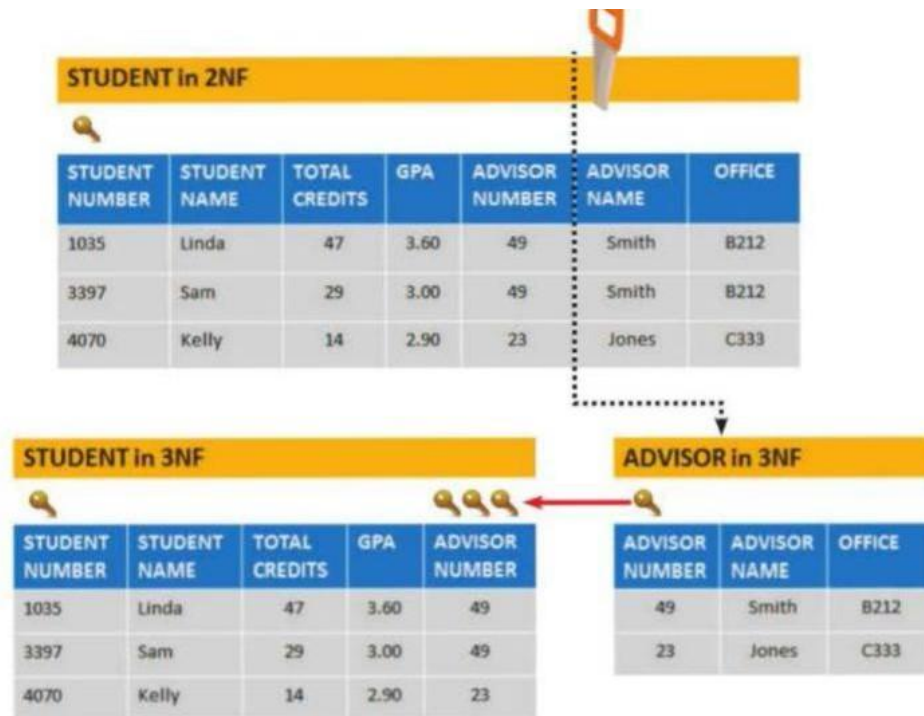
(Tilley and Rosenblatt, 2024)

Figure 9-29 illustrates the **3NF** versions of the sample data for the **STUDENT**, **ADVISOR**, **COURSE**, and **GRADE** tables. The final **3NF design** is as follows:

- **STUDENT** (STUDENT NUMBER, STUDENT NAME, TOTAL CREDITS, GPA, ADVISOR NUMBER)
- **ADVISOR** (ADVISOR NUMBER, ADVISOR NAME, OFFICE)
- **COURSE** (COURSE NUMBER, CREDIT HOURS)
- **GRADE** (STUDENT NUMBER, COURSE NUMBER, GRADE)

In this design:

- Each table now adheres to the principles of **3NF**, meaning all non-key fields depend only on the primary key, and there is no transitive dependency between non-key fields.



**FIGURE 9-29** STUDENT, ADVISOR, COURSE, and GRADE tables in 3NF. When the STUDENT table is transformed from 2NF to 3NF, the result is two tables: STUDENT and ADVISOR.

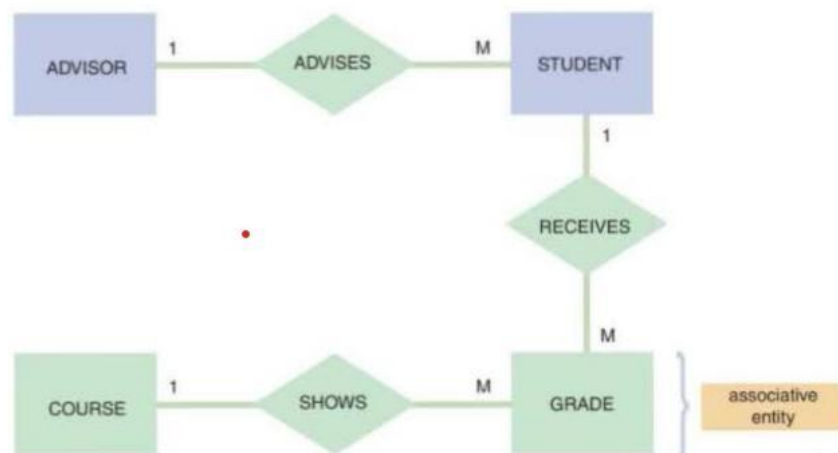
(Tilley and Rosenblatt, 2024)

Figure 9-30 illustrates the **complete Entity-Relationship Diagram (ERD)** after normalization. There are now **four entities**: **STUDENT**, **ADVISOR**, **COURSE**, and **GRADE** (which is an associative entity).

It's important to note that in **Figure 9-25**, which was drawn before **GRADE** was identified as an entity, the **M:N relationship** (many-to-many) between **STUDENT** and **COURSE** was depicted. After normalization, this M:N relationship has been transformed into **two 1:M relationships** (one-to-many):

- One relationship between **STUDENT** and **GRADE**.
- The other relationship between **COURSE** and **GRADE**.

This approach eliminates redundancy and creates more efficient data relationships.



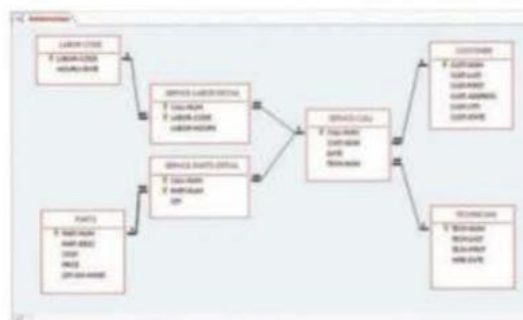
**FIGURE 9-30** The entity-relationship diagram for STUDENT, ADVISOR, and COURSE after normalization. The GRADE entry was identified during the normalization process. GRADE is an associative entity that links the STUDENT and COURSE tables.

(Tilley and Rosenblatt, 2024)

To create **3NF designs**, it's essential to understand the concepts of **first**, **second**, and **third normal forms**. A **systems analyst** will often encounter designs far more complex than the examples in this chapter.

#### EXAMPLE 2: Magic Maintenance

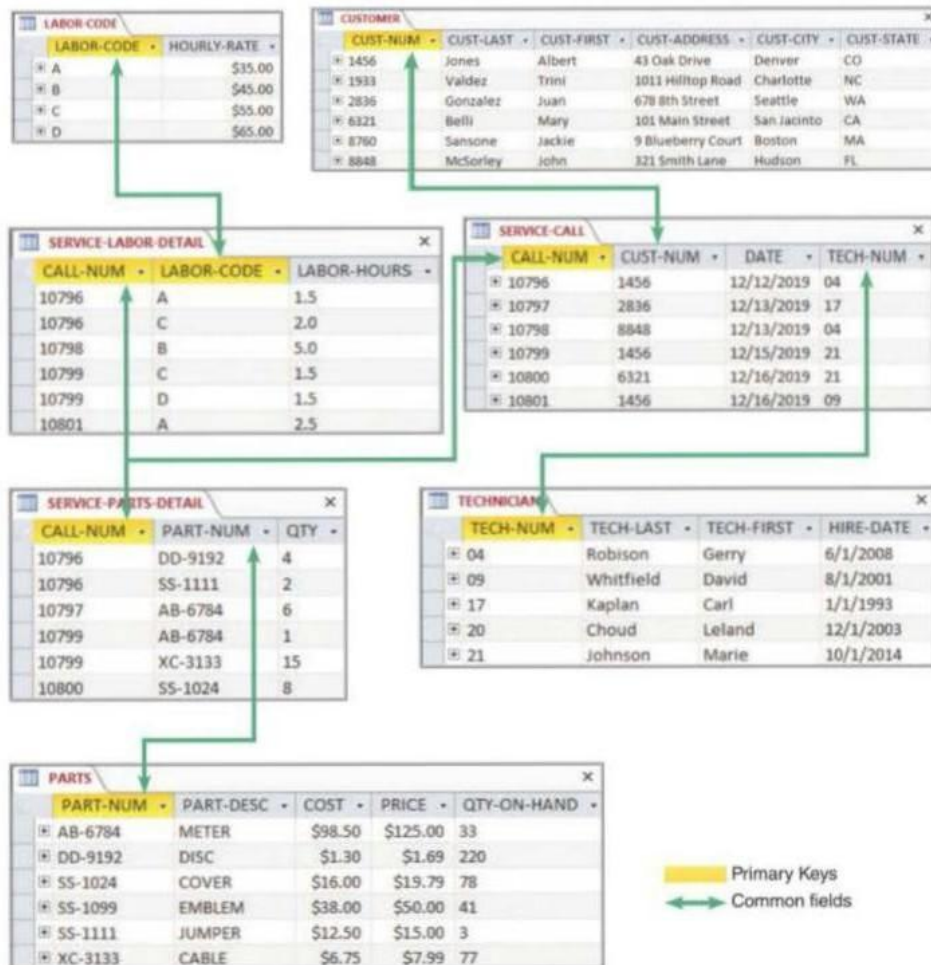
Magic Maintenance is a company that provides on-site service for electronic equipment. **Figure 9-31** illustrates the overall **database design** that such a firm might use. This diagram incorporates many of the concepts described earlier. The database consists of **seven separate tables**, all connected by common fields, making them part of an integrated data structure.



**FIGURE 9-31** A relational database design for a computer service company uses common fields to link the tables and form an overall data structure. Notice the one-to-many notation symbols and the primary keys, which are indicated with gold-colored key symbols.

(Tilley and Rosenblatt, 2024)

**Figure 9-32** offers further details, including sample data, **primary keys**, and common fields. It's worth noting that the entities in the database include **customers**, **technicians**, and **service calls**, among others. This example highlights how databases for real-world businesses can become intricate, involving multiple entities that need to be normalized to ensure efficient and accurate data storage.



**FIGURE 9-32** Sample data, primary keys, and common fields for the database shown in Figure 9-31. The design is in 3NF. Notice that all nonkey fields functionally depend on a primary key alone.

(Tilley and Rosenblatt, 2024)

In addition to customers, technicians, and service calls, the database also stores data related to **parts**. Other tables store information about **labor** and **parts** that are used on specific service calls.

It's also important to note that all tables use a **single field** as the **primary key**, except for the **SERVICE-LABOR-DETAIL** and **SERVICE-PARTS-DETAIL** tables. These two tables require a **combination of two fields** to uniquely identify each record, as their primary key is a composite of those fields. This design allows the database to track and manage specific details of labor and parts used during service calls more efficiently.

### 3.7 CODES

Codes are sequences of letters or numbers used to represent data items, offering a simplified way to handle and format data for input, output, and storage.

#### 3.7.1 Overview of Codes

Codes are an integral part of daily life, simplifying data representation. For example, a **student number** is a unique identifier for students in a school system. Multiple students named *John Turner* might exist, but each will have a unique student number, such as **268960**, which ensures their identification.

Similarly, **postal codes** are used to encode geographic and delivery information. A **9-digit postal code** is structured to provide multiple layers of detail about a location. For example, in the **32901-6975** ZIP code:

- The first digit, **3**, represents a broad geographical area (southeastern U.S.).
- The next two digits, **29**, pinpoint the area east of Orlando, Florida.
- The following two digits, **01**, identify Melbourne, Florida.
- The last four digits, **6975**, direct to a specific address, such as the Florida Institute of Technology's campus.

Benefits of Codes:

1. **Space and Cost Efficiency:** Codes, being shorter than the full data they represent, save storage space and reduce associated costs.
2. **Faster Data Processing:** Codes enable faster data transmission and entry by reducing the volume of information.
3. **Concealment or Revelation of Information:** Codes can be used to hide or reveal specific details. For instance, the last digits of a part number might represent a supplier number or a salesperson's discount capability.
4. **Error Reduction:** Using codes can minimize data entry mistakes, especially when the code is easier to recall than the full data. Moreover, by restricting inputs to valid codes, the chances of entering incorrect data are minimized, and codes can verify their correctness during data input

|                                                |                 |           |                                         |
|------------------------------------------------|-----------------|-----------|-----------------------------------------|
| Broad geographical location in southeastern US | FL Orlando East | Melbourne | Florida Tech<br>150 W. University Blvd. |
| 3                                              | 29              | 01        | 6975                                    |

**FIGURE 9-33** A zip code is an example of a significant digit code that uses groups and subgroups to store data. This example is for the zip code 32901-6975, which is the location of the Florida Institute of Technology in Melbourne, Florida.

### 3.7.2 Types of Codes

Companies use various coding methods for data representation. These methods help ensure that codes are easy for users to learn and apply. When creating or altering codes, it's important to gather user feedback. Below are seven common types of codes:

1. **Sequence Codes:**
  - Sequence codes are numbers or letters assigned in a specific order. These codes don't carry extra information beyond indicating the order in which items are entered into the system.

- For example, in a human resources system, employees are assigned consecutive numbers. The number **584** indicates that this employee was hired after employee **433**, but it doesn't provide the exact hire date.



**FIGURE 9-34** Abbreviations for some of the world's busiest airports.

BLANKArtist/Shutterstock.com

## 2. Block Sequence Codes:

- Block sequence codes use blocks of numbers to represent different classifications.
- For instance, college course numbers might be assigned using this method. A **100-level** course (e.g., Chemistry 110) indicates a freshman-level course, while a **200-level** course (e.g., Mathematics 225) indicates a sophomore-level course. Numbers within a block can also carry additional meaning, such as prerequisites (e.g., English 151 is a prerequisite for English 152).

## 3. Alphabetic Codes:

- These codes use letters to categorize or distinguish items, often based on a category or an easy-to-remember abbreviation, also known as a mnemonic code.

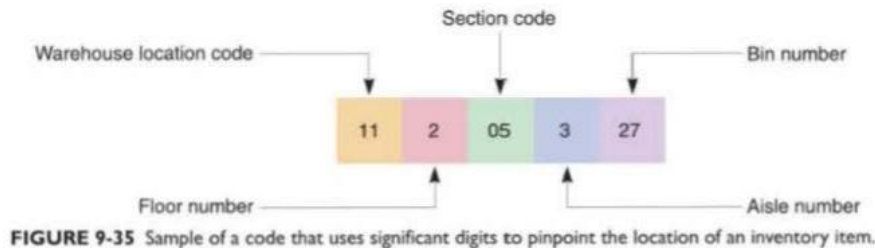
There are several variations:

- **Category Codes:** Used to identify related items, such as department codes in a store (e.g., **GN** for gardening supplies, **HW** for hardware, **EL** for electronics).
- **Abbreviation Codes:** These are alphabetic abbreviations, such as state codes like **NY** for New York, **ME** for Maine, and **MN** for Minnesota.
- **Mnemonic Codes:** These are alphabetic abbreviations designed to be easy to remember. For example, three-character airport codes like **ATL** for Atlanta and **MIA** for Miami. Some airport codes are not mnemonic, like **ORD** for Chicago O'Hare or **MCO** for Orlando.

## 4. Significant Digit Codes:

- These codes use subgroups of digits to represent different levels of classification. Each digit or group of digits has its significance.
- For example, postal codes are significant digit codes, as are inventory location codes. An inventory location code might be something like **11205327**, which is broken down into:
  - **11** for the warehouse number,
  - **0** for the floor number,

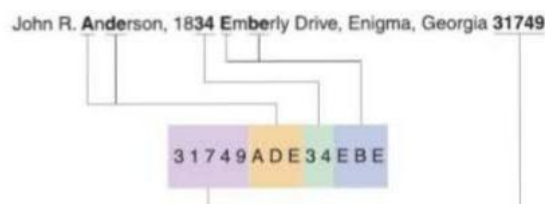
- **53** for the section code,
- **2** for the aisle number, and
- **7** for the bin number.
- Each of these subgroups holds meaning and helps pinpoint a specific location within a warehouse.



**FIGURE 9-35** Sample of a code that uses significant digits to pinpoint the location of an inventory item.

### 5. Derivation Codes:

- Derivation codes are created by combining various attributes or characteristics of an item. These codes are commonly used in magazine subscription systems, for example.
- A typical derivation code for a magazine subscription might include:
  - The subscriber's **five-digit postal code**,
  - The **first, third, and fourth** letters of the subscriber's last name,
  - The **last two digits** of the subscriber's house number, and
  - The **first, third, and fourth** letters of the subscriber's street name.
- This combination of elements creates a unique code for each subscriber, drawing on multiple pieces of personal information.



**FIGURE 9-36** A magazine subscriber code is derived from various parts of the name and address.

### 6. Cipher Codes:

- Cipher codes use a keyword to encode numbers. Each letter in the keyword corresponds to a specific number.
- For instance, a store may use a **10-letter word** (e.g., **CAMPGROUND**) to encode wholesale prices. In this case, each letter corresponds to a number:
  - **C = 1, A = 2, M = 3**, and so on.
- A code like **GRAND** could indicate that the store paid **\$562.90** for an item, where the word is translated into a number using the cipher.

### 7. Action Codes:

- Action codes specify the action that should be taken with an associated item.
- In a student records system, for example, action codes could include:
  - **D** for displaying a record,
  - **A** for adding a record, and
  - **X** for exiting the program.
- These codes guide the system in determining what function should be performed.

These types of codes help to organize and streamline data processing, making it easier to identify, manage, and execute actions in different systems.

### 3.7.3 Designing Codes

Here are some important suggestions for designing codes effectively:

1. **Keep codes concise:** Codes should not be unnecessarily long. For example, if you need to identify 250 customers, there's no need to create a six-digit code. A smaller code will suffice and be more efficient.
2. **Allow for expansion:** A coding scheme should accommodate future growth. If a company has eight warehouses today, using a single-digit code might work, but as the number of warehouses grows, the code must be expanded. For example, a two-digit code or even an alphanumeric code might be necessary in the future. Airlines use six-character codes, allowing for millions of combinations, which is an example of accommodating future needs.
3. **Keep codes stable:** Consistency is key. Changing codes frequently can create problems, especially with stored data and documents. During the transition, both old and new codes may need to be used for a time, and special procedures will be required to manage both. For example, when telephone area codes change, both the old and new codes may be valid for a period.
4. **Make codes unique:** Codes used for identification must be unique. If a code can represent multiple things, it loses its usefulness. For example, the code **HW** could represent hardware or houseware, but it's not specific enough to be useful on its own.
5. **Use sortable codes:** When grouping items by code, ensure the codes are sortable. For example, products with codes in the 100s and 300s should be of one type, while products with codes in the 200s should be of another type. Be mindful of the sorting order—sometimes adding leading zeros (e.g., 01, 02, 03) will ensure that single-digit codes are sorted properly with double-digit codes.
6. **Use a simple structure:** Keep code structures simple and uniform. Avoid mixing formats, such as using two letters, a hyphen, and a single digit in one part number, and one letter, a hyphen, and two digits in another. Consistent format makes codes easier to understand and manage.
7. **Avoid confusion:** Certain characters can be easily confused, such as the number zero (0) and the uppercase letter O, or the number one (1) and the uppercase letter I. Ensure your code design avoids using characters that might be misread, reducing the likelihood of errors.
8. **Make codes meaningful:** Codes should be intuitive, user-friendly, and easy to interpret. A code like **SW** for the southwest sales region is much easier to understand and remember than a code like **14**. Similarly, **ENG** for the English department is easier to interpret than something like **XVA** or **132**.
9. **Use a code for a single purpose:** Avoid combining unrelated attributes into a single code. For example, using one code for both an employee's department and insurance plan type can lead to confusion. It's better to have separate codes for each attribute.
10. **Keep codes consistent:** If one system (e.g., payroll) uses two-digit department codes, don't create a completely different coding scheme for another system (e.g., personnel). Consistency across systems is important for clarity and ease of use.

By following these guidelines, codes can be designed in a way that is efficient, effective, and user-friendly.

### CASE IN POINT 9.3: MADERA TOOLS

Madera Tools operates a small business that specializes in hard-to-find woodworking tools. The firm advertises in various woodworking magazines and currently accepts mail and telephone orders. Madera is planning a website that will be the firm's primary sales channel. The site will feature an online catalog, powerful search capabilities, and links to woodworking information and resources.

Madera has asked you, an IT consultant, whether a set of codes would be advantageous. What codes would you suggest? Provide at least two choices for a customer code and at least two choices for a product code. Be sure to describe your choices and provide some specific examples. Also include an explanation of why you selected these particular codes and what advantages they might offer.

## 3.8 DATA STORAGE AND ACCESS

Data storage and access include strategic business tools like data warehousing and data mining software. They also encompass logical and physical storage challenges, the selection of appropriate data storage formats, and specific considerations for storing date fields.

### 3.8.1 Tools and Techniques

In the realm of data storage and access, tools like data warehousing and data mining play a crucial role in helping businesses manage vast amounts of data essential for operations and decision-making. These tools, along with considerations related to data storage formats and storage of date fields, are vital for strategic business success.

#### Data Warehousing

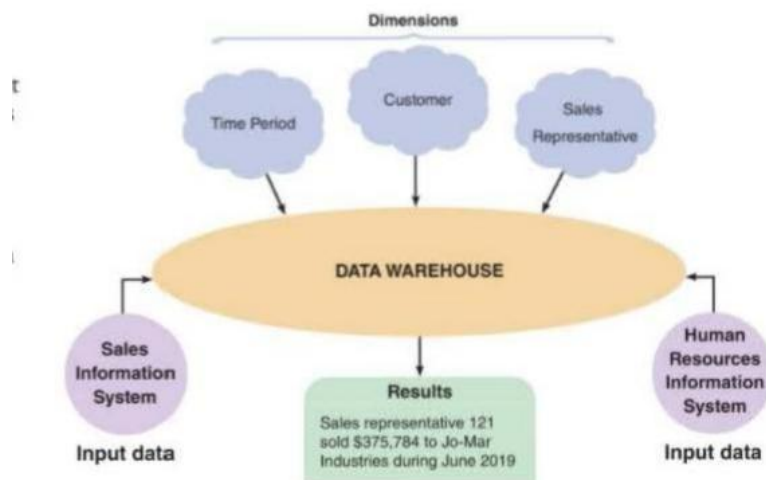
Large companies typically maintain numerous databases, and these databases may or may not be interconnected. To facilitate quick and easy access to this vast amount of information, businesses use specialized software that organizes and stores data in a way known as a *data warehouse*.

A **data warehouse** is an integrated collection of data that can include a variety of information, often from different sources within the organization. This allows for a cohesive, enterprise-wide perspective on the data, which aids in supporting management analysis and decision-making.

The primary feature of a data warehouse is its ability to combine and store data from different systems, making it easier for users to access relevant information for their analysis. For instance, data from order processing, inventory, and payroll systems can all be stored together in a data warehouse, making it easier to retrieve data across different systems.

A data warehouse is organized in such a way that users can analyze multidimensional data by selecting specific characteristics or dimensions. For example, a user might want to retrieve sales data for a particular month or for a specific sales representative across multiple systems. Without a data warehouse, this would require accessing multiple databases and systems, which could be time-consuming and complex. However, with a data warehouse, data from

different systems (such as sales data and employee information) can be linked and retrieved with ease.



**FIGURE 9-37** A data warehouse stores data from several systems. By selecting data dimensions, a user can retrieve specific information without having to know how or where the data is stored.

## Data Mart

While a data warehouse covers the entire enterprise's data needs, some companies opt for **data marts**. A data mart is a smaller, more focused version of a data warehouse. It serves the needs of a specific department, such as sales, marketing, or finance. Each data mart includes only the data that the department needs to function effectively.

Data marts have their advantages, such as faster access times due to their smaller size, and they are more tailored to specific departmental needs. However, there are trade-offs, and the choice between using a data warehouse or a data mart often depends on the company's particular situation and data requirements.

## Data Storage and Access:

Regardless of the overall approach, storing large amounts of data is a complex process that requires careful planning and structure, much like building a house. A well-organized data warehouse needs an architecture with detailed planning and specifications.

## Data Mining:

Data mining software is designed to identify meaningful patterns and relationships in data. For example, data mining can help a consumer products company identify potential customers based on their previous purchases. Although data about customer behavior is valuable, it also raises ethical and privacy concerns, as highlighted in this chapter's "Question of Ethics" section. The growth of e-commerce has made data mining a key marketing tool. Dan R. Greening, in his article "Data Mining on the Web," discusses how web hosts can collect data about visitors, but much of it is of little value. He mentions that marketers and analysts are now using machine learning algorithms to uncover hidden patterns in databases and take action based on those insights. He also suggests measurable goals for web marketers, such

as:

- Increasing the number of pages viewed per session
- Increasing the number of referred customers
- Reducing clicks to close (i.e., the number of page views needed to complete a purchase or obtain desired information)
- Increasing checkouts per visit
- Increasing average profit per checkout

This type of data collection is often called *clickstream storage*, and it can help create profiles of typical customers, including new customers, returning customers, and those who browse but don't buy. However, it raises legal and privacy concerns if companies misuse this data by linking online behaviors to personal information.

Data mining helps uncover patterns and trends in large datasets, making it a valuable tool for managers. For instance, there's a well-known story about a supermarket chain discovering that beer and diapers were often purchased together. Whether true or not, the conclusion is clear: retailers can optimize store layout by grouping such products together. This technique, known as *market basket analysis*, uses association rule learning.

### Logical vs. Physical Storage:

It's important to distinguish between logical and physical storage. Logical storage refers to how users view and access data, regardless of where or how it is actually stored. On the other hand, physical storage involves the hardware that reads and writes binary data to media like hard drives, CDs/DVDs, or network storage devices. For example, a document may be stored in different physical locations on a hard drive, but the user sees it as a single logical entity on the screen.

Logical storage involves characters (alphabetic or numeric) that form fields, which describe a specific attribute of a person, place, thing, or event. When designing fields, space should be allocated for the largest anticipated values, but without wasting storage for unused capacity. For example, a customer order entry system for 800 customers should use a five-character field (00001 to 99999), not a three-character field.

A logical record is a set of related field values that describes a person, place, thing, or event. Application programs interact with these logical records without concern for where or how the data is stored physically. The operating system handles the physical storage location.

### Data Coding:

Computers use binary digits (bits), which are either 1 or 0, to represent data. Various coding schemes, such as EBCDIC, ASCII, and binary, are used to encode and store data. Unicode, a newer standard, uses two bytes per character, allowing it to represent over 65,000 unique characters. This is especially useful in global systems that need to support multiple languages.

- **EBCDIC (Extended Binary Coded Decimal Interchange Code)** is commonly used on mainframes and high-capacity servers, while **ASCII (American Standard Code for Information Interchange)** is used on most personal computers.
- **Binary storage** is more efficient because it represents numbers directly as binary values. For instance, an integer uses only 16 bits to store a number like 12,345, whereas **ASCII** or **EBCDIC** would need more space.

**Unicode** enables the representation of a vast range of multilingual characters, which is essential for multinational systems or software used in various regions. Previously, software was developed in English and translated later, which was costly and error-prone. Unicode allows for content to be developed in a way that's easily translatable, simplifying this process and supporting languages worldwide. Most modern operating systems and platforms now support Unicode.



**FIGURE 9-38** Unicode is an international coding format that represents characters as integers, using 16 bits (two bytes) per character. The Unicode Consortium maintains standards and support for Unicode.

(Tilley and Rosenblatt, 2024)

## Storing Dates:

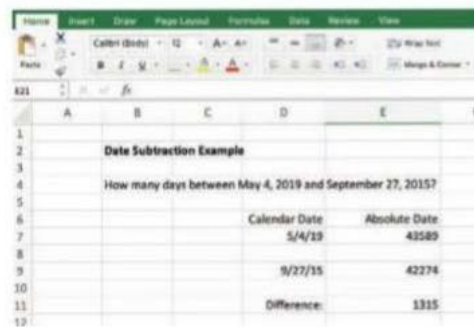
The best way to store dates depends on how they will be displayed and whether they will be used in calculations.

At the turn of the twenty-first century, many organizations faced a major problem known as the Y2K issue. This occurred because dates were often stored using only two digits for the year. In response to that experience, most date formats now follow the model set by the International Organization for Standardization (ISO), which uses a four-digit year, two digits for the month, and two digits for the day (YYYYMMDD). For example, a date stored in the ISO format, such as 20150504, represents May 4, 2015. Using this format allows for easy sorting and comparisons of dates. If one date in this format is larger than another, the first date is later. For example, 20150504 (May 4, 2015) is later than 20130927 (September 27, 2013).

However, problems arise when dates need to be used in calculations. For example, if a manufacturing order placed on June 23 takes three weeks to complete, when will the order be ready? Or if a payment due on August 13 is not paid until April 27 of the following year, how late is the payment and how much interest is owed? In these cases, it is easier to use **absolute dates**.

An **absolute date** is the total number of days since a specific base date. To calculate the number of days between two absolute dates, you simply subtract one date from the other. For example, if the base date is January 1, 1900, then May 4, 2015, would have an absolute date of 42,128. Similarly, September 27, 2013, would have an absolute date of 41,544. Subtracting the earlier date from the later one (42,128 - 41,544) results in 584 days. This

method allows easy calculation of time differences, and spreadsheets can be used to determine and display absolute dates as shown in Figure 9-39.



The screenshot shows an Excel spreadsheet with the following content:

|    | A | B | C | D | E | F |
|----|---|---|---|---|---|---|
| 1  |   |   |   |   |   |   |
| 2  |   |   |   |   |   |   |
| 3  |   |   |   |   |   |   |
| 4  |   |   |   |   |   |   |
| 5  |   |   |   |   |   |   |
| 6  |   |   |   |   |   |   |
| 7  |   |   |   |   |   |   |
| 8  |   |   |   |   |   |   |
| 9  |   |   |   |   |   |   |
| 10 |   |   |   |   |   |   |
| 11 |   |   |   |   |   |   |
| 12 |   |   |   |   |   |   |

The spreadsheet contains the following text and values:

- Row 2: "Date Subtraction Example"
- Row 3: "How many days between May 4, 2019 and September 27, 2015?"
- Row 6: "Calendar Date" (in column D) and "Absolute Date" (in column E)
- Row 7: "5/4/19" (in column D) and "43589" (in column E)
- Row 9: "9/27/15" (in column D) and "42274" (in column E)
- Row 11: "Difference:" (in column D) and "1315" (in column E)

**FIGURE 9-39** Microsoft Excel uses absolute dates in calculations. In this example, May 4, 2019, is displayed as 43589, and September 27, 2015, is displayed as 42274. The difference between the dates is 1315 days.

Scott Tilley

(Tilley and Rosenblatt, 2024)

### 3.9 DATA CONTROL

It is crucial to secure the physical aspect of the system, as illustrated in Figure 9-40. File and database management must encompass all necessary measures to ensure that data storage is accurate, complete, and protected. This control of files and databases is also linked to the input and output methods discussed previously.



**FIGURE 9-40** In addition to network monitoring, system security includes access codes, data encryption, passwords, and audit trails.

Gorodenko/Shutterstock.com

(Tilley and Rosenblatt, 2024)

A well-designed Database Management System (DBMS) must include built-in controls and security features, such as subschemas, passwords, encryption, audit trail files, and backup and recovery procedures to maintain the integrity of the data. The analyst's main role is to ensure that these features are used correctly.

As mentioned earlier, a subschema can offer a restricted view of the database to certain users or groups. Limiting access to files and databases is the most common method for protecting stored data. Users need to provide a valid user ID and password to access a file or database. Different privileges, or roles, can be assigned to users, allowing some employees read-only access while others may have permission to update or delete data. For sensitive information, additional access codes can be set up to restrict specific records or fields within those records. Encryption can also be used to protect stored data by converting readable data into unreadable characters to prevent unauthorized access.

System files and databases should be regularly backed up, and multiple backup copies must be kept for a defined period. In case of a data loss, recovery procedures can restore the database to its last backup state. Audit log files, which track every access and modification to the file or database, can help in recovering changes made since the last backup. Additionally, audit fields within records can be included to provide extra control or security, such as the creation or modification date, the user who made the change, and the number of times a record has been accessed.

### A QUESTION OF ETHICS

stock.com/abertito\_18

Tip Top Toys is a relatively small division of Worldwide Enterprises. Worldwide has nine other divisions, which include insurance, health-care products, and financial planning services, to name a few.

The corporate marketing director for Worldwide has requested Tip Top's customer shopping data to target people who might be likely to purchase items or services from other Worldwide divisions. The database manager is not totally comfortable with this and pointed out Tip Top's web privacy policy, which states "Tip Top Toys, a division of Worldwide Enterprises, will not share personal data with other companies without a customer's consent."

The marketing director replied that the statement only applies to outside companies—not other Worldwide divisions. He said he checked with the corporate legal department, and they agreed. The database manager replied, "Even if it is legally OK, it's not the *right* thing to do. Many people take our statement to mean that their data does not leave Tip Top. At the very least, we should give customers a choice, and share the data only with their consent."

Do you agree with the marketing director? Why or why not?

(Tilley and Rosenblatt, 2024)

### 3.10 SUMMARY

This chapter continues the exploration of the systems design phase in the Systems Development Life Cycle (SDLC). It explains that files and tables contain data about people, places, things, or events that impact the information system. File-oriented systems, also known as file processing systems, manage data stored separately.

A database consists of interconnected tables that form an overall data structure. A Database Management System (DBMS) is a suite of tools, features, and interfaces that enables users to add, update, manage, access, and analyze data within a database. DBMS designs offer greater power and flexibility compared to traditional file-oriented systems, providing benefits like scalability, organization-wide access, cost efficiency, data sharing, conflict resolution, standard enforcement, controlled redundancy, security, flexibility, improved programmer

productivity, and data independence. Large databases require complex security and backup/recovery features.

DBMS components include user, database administrator (DBA), and system interfaces, a Data Manipulation Language (DML), schemas, and a physical data repository. Other data management techniques include data warehousing, which organizes data for easy access, and data mining, which seeks patterns and relationships in data. Data mining includes techniques like clickstream storage (tracking user interactions with a site) and market basket analysis (identifying product relationships and consumer purchasing patterns).

In web-based systems, the Internet serves as the user interface for the DBMS. Access to the database is provided through a web browser and an internet connection. Middleware can interpret HTML client requests and convert them into database commands. Web-based data must be secure and easily accessible to authorized users, with security measures implemented at three levels: the database, the web server, and the telecommunications links between components.

In an information system, an entity represents a person, place, thing, or event for which data is gathered and maintained. A field or attribute represents a characteristic of an entity, while a record or tuple contains related fields that describe a single instance of an entity. Data is stored in files (in file-based systems) and tables (in a DBMS environment).

A primary key is the field or combination of fields that uniquely and minimally identifies a specific record, while a candidate key is any field that could serve as a primary key. A foreign key matches the primary key of another file or table. A secondary key is used for sorting or retrieving records.

Entity-Relationship Diagrams (ERDs) graphically represent all entities in a system and their relationships. The basic relationships in an ERD are one-to-one (1:1), one-to-many (1:M), and many-to-many (M:N), with M:N relationships linked by an associative entity. Cardinality refers to the relationship between two entities, and crow's foot notation is a common method to represent cardinality.

Normalization is the process of refining data design to avoid problems. A record is in 1st Normal Form (1NF) if it has no repeating groups. It is in 2nd Normal Form (2NF) if it is in 1NF and all non-key fields depend on the primary key. A record is in 3rd Normal Form (3NF) if it is in 2NF and no non-key field depends on another non-key field.

Data design tasks include creating an initial ERD, assigning data elements to entities, normalizing table designs, and completing the data dictionary for files, records, and fields.

A code is a set of letters or numbers used to represent data. Codes help speed up data entry, reduce storage requirements, and cut transmission time. They can also reveal or conceal information. Types of codes include sequence codes, block sequence codes, classification codes, alphanumeric codes (e.g., category, abbreviation, mnemonic), significant digit codes, denotation codes, cipher codes, and action codes.

Logical storage refers to how users perceive information, regardless of how or where it is stored. Physical storage involves hardware-related processes such as reading and writing binary data to physical media. A logical record is a set of related field values describing an entity. Data can be stored in various formats, such as EBCDIC, ASCII, binary, Unicode, ISO, and absolute formats.

File and database control measures include restricting data access, encrypting data, performing backups and recovery, maintaining audit trail files, and using internal audit fields.

## EXERCISES

Questions:

1. What is a data structure?
2. Briefly describe the components of a DBMS.
3. List the major characteristics of web-based design.
4. Explain primary key, candidate key, secondary key, and foreign key.
5. What are ERDs and how are they used?
6. How do you convert an unnormalized design to 1NF? In your answer, refer to specific pages and figures in this chapter.
7. How are codes used in data design?
8. What is data warehousing and data mining?
9. How would a specific date, such as March 15, 2019, be represented as an absolute date?
10. How are permissions used to control access to data?

Discussion Topics:

1. In the auto shop examples in Section 9.1.2, what are some problems that might arise in Mario's system? Why won't Danica run into the same problems? Provide specific examples in your answer.
2. Many large organizations have had their database system hacked and customer data stolen. How should the security for the database be different than security for the rest of the system? Does it make a difference for web-based data designs? If so, how?
3. Suggest three typical business situations where referential integrity avoids data problems.
4. We use lots of codes in our personal and business lives. How many can you name?
5. Are there ethical issues to consider when planning a database? For example, should sensitive personal data (such as medical information) be stored in the same DBMS that manages employee salary and benefits data? Why or why not?

Projects:

1. Consider an automobile dealership with three locations. Identify the possible candidate keys, the likely primary key, a probable foreign key, and potential secondary keys.
2. Visit a bookstore and draw an ERD describing the bookstore's operations.

3. Cludadwy Chairs sells a patented seat that spectators can take to youth soccer games. The company operates a factory in Kansas and also contracts its manufacturing projects to small firms in Canada and Mexico. An unusual problem has occurred for this small multinational company: People are getting confused about dates in internal memos, purchase orders, and email. When the company's database was originally designed, the designer was not aware that the format for dates in Canada and Mexico was different from the format used in the United States. For example, in Canada and Mexico, the notation 7/1/19 indicates January 7, 2019, whereas in the United States, the same notation indicates July 1, 2019. Although it seems like a small point, the date confusion has resulted in several order cancellations. Cludadwy Chairs has asked for your advice. You could suggest writing a simple program to convert the dates automatically or design a switchboard command that would allow users to select a date format as data is entered. You realize, however, that Cludadwy Chairs might want to do business in other countries in the future. What would be the best course of action? Should the company adapt to the standard of each country, or should it maintain a single international format? What are the arguments for each option?
4. Use Microsoft Access or similar database software to create a DBMS for the imaginary company called TopTex Publishing, which is described in Case in Point 9.1. Add several sample records to each table and report to the class on your progress.
5. Search the internet to find information about date formats. Determine whether the date format used in the United States is the most common format.

## CHAPTER 4: System Architecture



### LEARNING OUTCOMES

After reading this Section of the guide, the learner should be able to:

1. **Create a checklist of factors to consider when choosing a system architecture.**
2. **Outline the development of system architecture from mainframes to modern designs.**
3. **Describe the concept of client/server architecture.**
4. **Discuss how the Internet has influenced system architecture.**
5. **Compare developing e-commerce solutions in-house versus using packaged solutions and service providers.**
6. **Differentiate between online and batch processing.**
7. **Provide an overview of network models such as hierarchical, bus, ring, star, and mesh topologies.**
8. **Explain the role of network devices like routers, gateways, and proxy servers.**
9. **Discuss wireless networking, including standards, topologies, and trends.**
10. **Outline the final steps in the systems design phase.**

### 4.1 ARCHITECTURE CHECKLIST

At this stage of the SDLC, the goal is to establish the overall architecture for implementing the information system. System architecture takes the logical design of a system and translates it into a physical structure that encompasses hardware, software, network support, processing methods, and security features. The outcome of the systems design phase is the system design specification, and if approved, the next step will be systems implementation.

Similar to how an architect begins with the client's requirements, a system analyst must consider various factors that will influence the choice of architecture. This is done using an overall architecture checklist, which includes:

- Corporate organization and culture
- Enterprise Resource Planning (ERP)
- Initial and total cost of ownership (TCO)
- Scalability
- Web integration
- Legacy system interface requirements
- Processing options

- Security concerns
- Corporate portals

### Corporate Organization and Culture

An information system needs to fit well within a company's organization and culture to be successful. For instance, consider two large bicycle brands: Green Bikes and Blue Bikes. Both companies operate across three divisions: a manufacturing unit in Asia, a bike accessory and clothing factory in Los Angeles, and a Canadian plant for bike carriers and custom trailers. Although the companies appear similar, their organizational cultures differ. Green Bikes is highly centralized, managing daily operations from its Los Angeles office, whereas Blue Bikes allows its divisions to operate independently with minimal oversight.

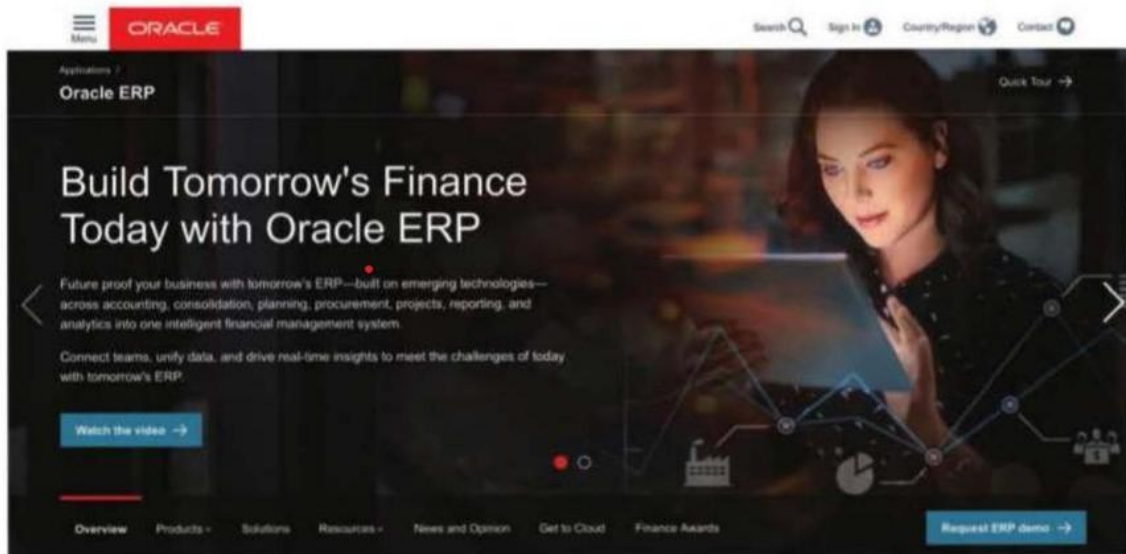
If both companies sought advice from a consultant about an IT architecture to enhance productivity and reduce costs, the consultant would have to consider how the companies' organizational structures and cultures influence the architecture recommendation. The best approach would likely involve studying daily business functions, engaging with users at all levels, and addressing operational feasibility, much like earlier stages in the development process.

### Enterprise Resource Planning (ERP)

ERP software helps companies implement a unified IT strategy across the organization, with specific guidelines for architecture, data standards, processing, networking, and user interfaces. The key benefit of ERP is its ability to define a platform, consisting of both hardware and software, that ensures easy connectivity and integration with future systems, including both internal software and third-party applications.

Many companies are extending their ERP systems to suppliers and customers through supply chain management (SCM). In a fully integrated supply chain, a customer order might automatically trigger a work order in the manufacturing system, which then prompts an order for more parts from suppliers. In competitive and dynamic markets, SCM allows businesses to respond more quickly, improve customer service, and reduce costs.

Oracle, for example, provides ERP solutions as cloud-based services. These services support employee collaboration, mobile access to information, and data analytics to gain insights into business processes.



**FIGURE 10-1** Oracle offers ERP solutions as a cloud-based service.  
Source: Oracle Corporation.

(Tilley and Rosenblatt, 2024)

### 4.1.3 Initial Cost and TCO

Considering economic feasibility and Total Cost of Ownership (TCO) during the system planning and analysis phase is crucial. TCO includes both hard costs (tangible expenses like purchases, fees, and contracts) and soft costs (such as management, support, training, and downtime), which are often harder to measure but just as significant.

A TCO analysis should address the following questions:

- Is in-house development still the best option? Are the necessary technical skills available, and does the initial cost estimate still seem realistic?
- If a specific package was selected earlier, is it still the best choice? Are newer versions or competing products available, and have there been any changes in pricing or support?
- Are there new outsourcing options?
- Have any economic, governmental, or regulatory changes occurred that could impact the project?
- Have any technical advancements occurred that could influence the project?
- Have any major assumptions changed since the company made the "build versus buy" decision?
- Are there merger or acquisition considerations that might require compatibility with a specific environment?
- Have any new trends emerged in the marketplace? Are new products or technologies about to be introduced?
- Have the original TCO estimates been updated? Are there significant differences?

The answers to these questions might alter the initial cost and TCO, and the system's requirements and alternatives should be reviewed before moving forward with the system architecture design.

#### **4.1.4 Scalability**

Scalability, or extensibility, refers to a system's ability to expand, change, or downsize to meet a business's evolving needs. It's especially important in systems like transaction processing systems, which must be capable of handling increasing volumes over time. A scalable network, for instance, should support anywhere from a few nodes to thousands of them, while a scalable Database Management System (DBMS) should handle the addition of new business divisions or data volumes. Scalability is a key consideration when investing heavily in a project, as management is concerned about the system's long-term viability and adaptability.

#### **4.1.5 Web Integration**

Information systems often include applications that manage input, processing logic, and output. A systems analyst must assess whether a new application will be part of an e-commerce strategy and how it integrates with other web-based components. Web-centric architecture follows internet design protocols, allowing seamless integration into an e-commerce strategy. Even if e-commerce is not involved, a web-based application can still operate on the internet or a company's intranet or extranet, avoiding many connectivity and compatibility issues that arise with different hardware environments. In a web-based environment, external business partners can easily exchange data using standard web browsers.

#### **4.1.6 Legacy Systems**

When a new system is introduced, it might need to interface with existing legacy systems, which are older but still functional. For example, a new marketing system might need to send sales data to an accounting system or retrieve product cost data from a legacy manufacturing system. Interfacing new systems with legacy systems involves analyzing data formats and ensuring compatibility. In some cases, legacy data may need to be converted, a process that can be both costly and time-consuming. Middleware can sometimes be used to facilitate communication between new and legacy systems. It's also important to determine whether the new application will eventually replace the legacy system or coexist with it.

#### **4.1.7 Processing Options**

During system architecture planning, one of the key considerations is how data will be processed: online or in batches. For example, transaction processing systems that handle large volumes of orders will require more network capacity, processing power, and storage than a monthly billing system that processes data in batches. If the system is required to operate online 24/7, provisions for backup and quick recovery in case of failure are essential. The characteristics of both online and batch processing methods, along with examples of each, are further discussed later in this chapter.

#### 4.1.8 Security Issues

Security is a significant concern in system design, ranging from simple password protection to more complex intrusion detection systems. As the design is translated into specific hardware and software, security measures must be thoroughly addressed. This is particularly important when data or processing occurs remotely, rather than in a centralized facility. In mission-critical systems, security considerations will significantly impact the system's architecture and design. For web-based systems, additional security measures are necessary to protect critical data in the internet environment. E-commerce applications, in particular, must ensure the security of customers' personal data. The prevalence of high-profile security breaches in recent years highlights the growing importance of incorporating robust security into system architecture. More detailed security concepts and strategies are covered in Chapter 12.



**FIGURE 10-2** User IDs and passwords are traditional elements of system security.  
JMika/Shutterstock.com

(Tilley and Rosenblatt, 2024)

#### 4.1.9 Corporate Portals

Some systems may include a corporate portal in the planned architecture. A portal serves as an entry point to a multifunctional website, providing access to different tools and features for users. A corporate portal can serve various user groups, such as customers, employees, suppliers, and the public. A well-designed portal can integrate with other systems and deliver a consistent experience across different organizational divisions, making it easier for users to navigate and access necessary information.

##### CASE IN POINT 10.1: ABC SYSTEMS

You are a systems analyst at ABC Systems, a fast-growing IT consulting firm, that provides a wide range of services to companies that want to establish e-commerce operations. During the past 18 months, ABC acquired two smaller firms and set up a new division that specializes in SCM. Aligning ABC's internal systems was quite a challenge, and top management was not especially happy with the integration cost or the timetable. To avoid future problems, you have decided to suggest an ERP strategy, and you plan to present your views at the staff meeting tomorrow. ABC's management team is very informal and prefers a loose, flexible style of management. How will you persuade them that ERP is the way to go?

## 4.2 THE EVOLUTION OF SYSTEM ARCHITECTURE

Every business information system must fulfill three core functions:

- Managing applications that execute processing logic
- Handling data storage and access
- Providing an interface for user interaction

Depending on the system architecture, these functions may be carried out on a server, a client, or shared between the two. During the design phase, the system analyst must decide where each function will be performed and evaluate the benefits and drawbacks of each architectural approach.

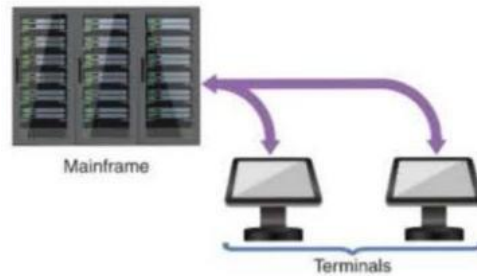
### 4.2.1 Mainframe Architecture

A server is a computer that provides data, processing services, or other support to one or more client computers. The earliest servers were mainframe computers, and a system where the server handles all the processing is often referred to as mainframe architecture. While the server does not have to be a traditional mainframe, this term typically refers to a multi-user setup where the server is significantly more powerful than the client machines.

To understand the server's role in modern systems, it's helpful to know the history of mainframe architecture. In the 1960s, mainframe architecture was the only option available. These early systems centralized data processing, meaning all data input and output were handled at a central location, often known as a data processing center. Physical data would be delivered to this center, where it was manually entered into the system. Users did not interact directly with the system except through printed reports distributed by the IT department.

As network technology evolved, companies installed terminals at remote locations, enabling users to input and access data from anywhere within the organization, regardless of the location of the central server. A terminal consisted of a keyboard and a display screen for input and output, but it lacked independent processing capabilities. In a centralized design, as depicted in Figure 10-3, keystrokes from the remote user's terminal would be sent to the mainframe, which would then return the processed output to the user's screen.

Today, mainframe architecture is still utilized in industries requiring large-scale processing at a central location. For example, banks may rely on mainframe servers to update customer balances nightly. In a mix of old and new technologies, many organizations are transitioning some data processing from mainframes to the cloud, with mainframes also being part of a cloud computing infrastructure.



**FIGURE 10-3** In a centralized design, the remote user's keystrokes are transmitted to the mainframe, which responds by sending screen output back to the user's screen.

(Tilley and Rosenblatt, 2024)

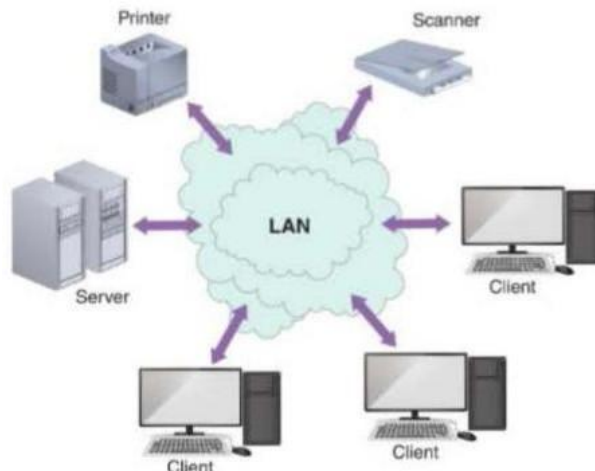
### 4.2.2 Impact of the Personal Computer

In the 1990s, the rise of personal computer (PC) technology brought powerful microcomputers to corporate desktops. Users quickly discovered they could run applications like word processing, spreadsheets, and databases independently, without needing assistance from the IT department. This mode of computing, known as stand-alone computing, soon led to the creation of networks that allowed these stand-alone PCs to exchange data and perform local processing.

In stand-alone mode, each individual workstation acted as its own server, performing tasks such as storing, accessing, and processing data, while also providing a user interface. Although stand-alone PCs increased productivity and empowered employees to perform tasks that previously required IT department support, this approach was often inefficient and costly. Moreover, relying on individual workstations to store data raised significant concerns about security, data integrity, and consistency. Without a centralized storage system, valuable business data could not be adequately protected or backed up, exposing companies to considerable risks. In some cases, frustrated by the lack of support from IT, users took matters into their own hands by creating and managing their own databases. This not only compounded security issues but also led to data inconsistency and unreliability.

### 4.2.3 Network Evolution

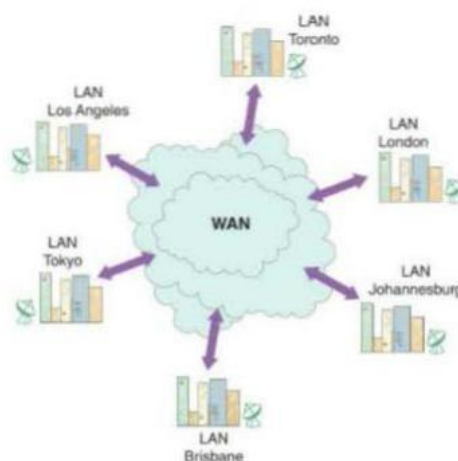
As technology progressed, companies addressed the challenges of stand-alone computing by connecting clients into a local area network (LAN), which enabled data and hardware resource sharing, as illustrated in figure 10-4. Multiple LANs could then be connected to a centralized server. Further advancements in technology allowed for the creation of more powerful networks capable of using satellite links, high-speed fiber-optic lines, or the Internet to share data.



**FIGURE 10-4** A LAN allows sharing of data and hardware, such as printers and scanners.

(Tilley and Rosenblatt, 2024)

A wide area network (WAN) extends over large distances and can connect LANs that are geographically far apart, even on different continents, as shown in Figure 10-5. When users access data on a LAN or WAN, the network operates transparently, making the data appear as though it's stored locally on the user's own workstation. Systems that connect multiple LANs or WANs across a company are known as distributed systems. The effectiveness of a distributed system depends on the power and capacity of the underlying data infrastructure. In comparison to mainframe architecture, distributed systems introduce greater concerns about data security and integrity. This is because multiple individual clients require access to the system in order to perform processing, increasing the complexity of managing and protecting the data across various nodes in the network.



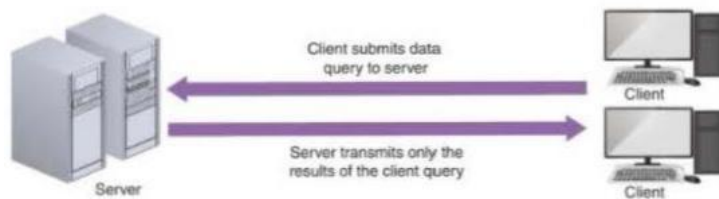
**FIGURE 10-5** A WAN can connect many LANs and link users who are continents apart.

(Tilley and Rosenblatt, 2024)

### 4.3 CLIENT/SERVER ARCHITECTURE

Client/server architecture is a distributed computing model commonly used today in interconnected systems. It divides processing responsibilities between one or more clients and a central server. In this setup, the client is responsible for handling the user interface, which includes tasks like data entry, querying, and displaying information. The server, on the other hand, stores the data and manages database access.

In a typical client/server interaction, the client sends a request for data or a service, and the server processes that request before sending the result back to the client. Importantly, the actual data file remains on the server; only the request and response are transmitted over the network. The server may even contact other servers to gather the necessary data or processing power, though this interaction is invisible to the client.



**FIGURE 10-6** In a client/server design, data is stored and usually processed on the server.

(Tilley and Rosenblatt, 2024)

Figure 10-7 highlights key differences between client/server and traditional mainframe systems. Early client/server systems did not always yield the expected savings, primarily due to the lack of clear standards at the time. Moreover, the development costs were often higher than anticipated. The implementation process was also expensive, as the client machines required powerful hardware and software to handle the processing tasks shared with the server.

Additionally, many companies had legacy data—older data stored in legacy systems—that was challenging to access and migrate into a client/server setup. This further complicated the transition and implementation of client/server systems, making them less cost-effective in some cases.

| Comparison of Client/Server and Mainframe Systems |                                                               |                                                                            |
|---------------------------------------------------|---------------------------------------------------------------|----------------------------------------------------------------------------|
| Characteristics                                   | Client/Server                                                 | Mainframe                                                                  |
| Basic architecture                                | Very flexible                                                 | Very rigid                                                                 |
| Application development                           | Flexible<br>Fast<br>Object-oriented                           | Highly structured<br>Slow<br>Traditional                                   |
| User environment                                  | PC-based<br>GUI<br>Empowers the user<br>Improves productivity | Uses terminals<br>Text interface<br>Constrains the user<br>Limited options |
| Security and control features                     | Decentralized<br>Difficult to control                         | Centralized<br>Easier to control                                           |
| Processing options                                | Can be shared and configured in any form desired              | Extensive and programmable                                                 |
| Data storage options                              | Can be distributed to place data closer to users              | All data is stored centrally                                               |
| Hardware/software integration                     | Very flexible<br>Multivendor model                            | Very rigid<br>Single proprietary vendor                                    |

**FIGURE 10-7** Comparison of the characteristics of client/server and mainframe systems.

(Tilley and Rosenblatt, 2024)

### 4.3 CLIENT/SERVER ARCHITECTURE

1. As large-scale networks grew more powerful, client/server systems became increasingly cost-effective. Many companies adopted client/server systems to leverage a unique combination of computing power, flexibility, and support for evolving business operations. Today, client/server architecture remains a popular choice in system design, especially as it incorporates Internet protocols and network models discussed later in this chapter.
2. With the rise of business alliances and collaboration with customers and suppliers, the client/server concept has expanded to include clients and servers beyond organizational boundaries. **Service-Oriented Architecture (SOA)** exemplifies this expansion, where a service can act as both a client and a server simultaneously, even existing outside of corporate networks. Some view **cloud computing** as a completely new concept, while others see it as the evolution of client/server architecture. In cloud computing, the Internet-based platform handles processing tasks, serving as the server while replacing traditional network infrastructures. Regardless of its classification, the key takeaway is that successful systems must support business requirements, and system architecture remains a crucial aspect of the development process.

#### 4.3.1 The Client's Role:

3. In a client/server system, it's essential to define the division of processing tasks between the client and the server. A **fat client**, or **thick client**, places most or all of the application processing logic on the client, while a **thin client** design places the processing logic on the server.
4. In the late 1990s, **Sun Microsystems** (now part of Oracle) strongly advocated for thin-client computing, which they referred to as **net-centric computing**. These thin clients were Java-powered terminals that communicated with servers via standard Internet protocols. Thin clients were believed to offer a lower **Total Cost of Ownership (TCO)** due to centralized maintenance. However, many users found the functionality of thin clients limited (e.g., lacking software like Microsoft Office) and encountered latency issues due to remote application and data access. As a result, **fat clients** (e.g., regular PCs) continued to dominate, despite their higher management demands and TCO.
5. Today, **laptops**, **tablets**, and **smartphones** have grown powerful enough that the appeal of thin clients has waned, except in specific cases like accessing the cloud for large datasets or specialized processing needs. The app ecosystem has also shifted the TCO balance in favor of powerful client devices at the edge of the system.

#### 4.3.2 Client/Server Tiers:

6. Early client/server designs were called **two-tier** designs. In a two-tier design, the user interface resides on the client, the data is stored on the server, and the application logic can be distributed between the client and server.
7. A more popular design today is the **three-tier** design, where the user interface remains on the client, and the data is stored on the server, similar to the two-tier model. However, a middle layer is added between the client and server, responsible for processing client requests and translating them into data access commands that the server can execute. This middle layer is often called an **application server** as it houses the business logic required by the system. A three-tier design may also be referred to as a **multi-tier** design, especially when there are multiple intermediate layers involved.

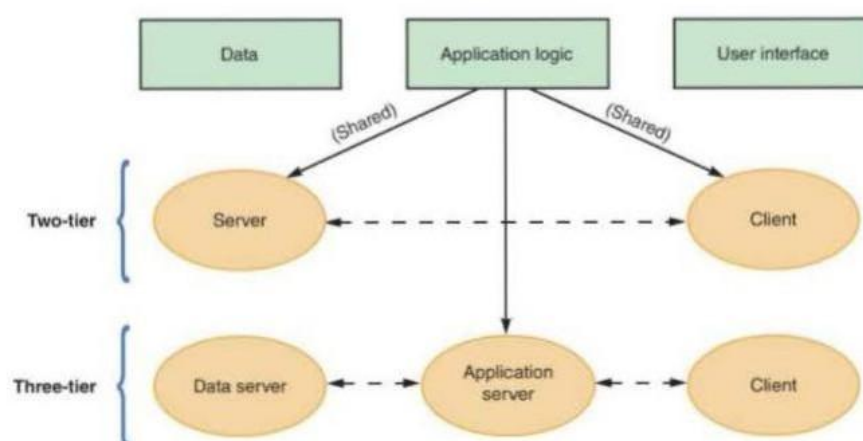


FIGURE 10-8 Characteristics of two-tier versus three-tier client/server design.

(Tilley and Rosenblatt, 2024)

The application logic layer in a three-tier architecture helps improve performance by reducing the workload on the data server. By isolating the application logic, it also frees clients from handling complex processing tasks. Since this layer can run on a server that is more powerful than typical client machines, it becomes more efficient and cost-effective, particularly in large-scale systems. Figure 10-9 illustrates the placement of data, application logic, and user interface across different architectures. In a client/server setup, the tiers communicate through software known as middleware, as explained in the next section.

| Architecture                         |                    | Data | Application Logic | User Interface |
|--------------------------------------|--------------------|------|-------------------|----------------|
| Central data processing center       | Server             | X    | X                 | X              |
|                                      | Client             |      |                   |                |
| Central server with remote terminals | Server             | X    | X                 |                |
|                                      | Client             |      |                   | X              |
| Stand-alone client                   | Server             |      |                   |                |
|                                      | Client             | X    | X                 | X              |
| Two-tier client/server               | Server             | X    | X                 |                |
|                                      | Client             |      | X                 | X              |
| Three-tier client/server             | Data server        | X    |                   |                |
|                                      | Application server |      | X                 |                |
|                                      | Client             |      |                   | X              |

**FIGURE 10-9** The location of the data, the application logic, and the user interface depend on the type of architecture.

(Tilley and Rosenblatt, 2024)

### 4.3.3 Middleware

In a multitier system, special software called middleware facilitates communication between the tiers and allows data to be exchanged. Sometimes referred to as "glueware," middleware connects various software components in a federated system. It plays a crucial role in integrating legacy systems with web-based or cloud-based applications. Middleware also represents the "slash" in the term client/server, acting as the intermediary between the client and server.

### 4.3.4 Cost-Benefit Issues

To meet business requirements, information systems must be scalable, powerful, and flexible. Client/server systems typically offer the best mix of features to fulfill these needs. Whether a business is growing or shrinking, client/server systems enable the company to scale the system to accommodate changes. It is often easier to adjust the number of clients and the tasks they handle than to upgrade a large central server.

Client/server computing allows businesses to shift applications from expensive mainframes to less expensive client platforms, sometimes even moving resource-heavy processes to the cloud. Additionally, by using common programming languages like SQL, clients and servers can communicate across various platforms, which is vital for businesses with significant investments in different hardware and software environments.

Client/server systems can also improve network load and response times. For instance, when a user at company headquarters requests sales figures, the server retrieves the data, processes it, and immediately sends the results back to the client. The client is unaware of the data retrieval and processing, as these tasks occur on the server.

#### **4.3.5 Performance Issues**

Although client/server architecture has several advantages, it also comes with performance challenges due to the separation of server-based data and networked clients that access the data.

In a centralized environment, a server-based program sends commands executed by the server's CPU. Processing speed is enhanced because both the data and program instructions travel on an internal system bus, which is more efficient than external networks. In contrast, a client/server design separates applications from data. Clients submit data requests to the server, which sends the data back. As the number of clients and demand for services increase, the network's capacity becomes a limiting factor, causing system performance to decline.

According to IBM, client/server systems have different performance characteristics from centralized systems. Response times in client/server systems increase gradually as more requests are made, but then rise sharply when the system nears its capacity. This point, known as the "knee of the curve," marks a dramatic drop in speed and efficiency. To maintain acceptable performance, developers must anticipate the number of users, network traffic, server size and location, and design a system that supports both current and future needs.

To optimize performance, client/server systems must be designed so that clients only contact the server when necessary and minimize communication trips. This is one goal of the HTTP/2 protocol used between servers and web browsers.

Another factor affecting client/server performance is data storage. Just as processing can occur at different locations, data can be stored in multiple places through a distributed database management system (DDBMS). Using a DDBMS offers several advantages, such as reducing network traffic by storing data closer to users, scalability (allowing new data sites to be added without reworking the system), and increased fault tolerance due to data being stored in multiple locations. However, data security can become a challenge when data is stored in various places, as it is harder to enforce controls and standards. Additionally, the architecture of a DDBMS is more complex and difficult to manage. From a design perspective, companies often desire the control of centralization and the flexibility of decentralization.

### **4.4 THE IMPACT OF THE INTERNET**

The internet has had a profound effect on system architecture. It has evolved beyond being just a communication channel, with many IT experts now viewing it as a completely new environment for developing systems.

#### **4.4.1 Internet-Based Architecture**

The internet has revolutionized system architecture, evolving from a communication channel to a fundamentally different environment for system development. In traditional client/server systems, the client handles the user interface, while the server (or servers in a multitier system) manages the data and application logic. This division means part of the system runs on the client and part on the server. However, in an Internet-based architecture, the web server provides the entire user interface in the form of HTML documents, which are displayed by the client's browser. By shifting the responsibility for the interface from the client to the server, data transmission becomes simpler, reducing hardware costs and complexity.

The rise of Internet-based architecture has changed fundamental concepts in system design, pushing us towards a more connected, online environment. Meanwhile, millions of people now use web-based collaboration and social networking applications to accomplish tasks that were once done in person, over the phone, or through traditional internet methods.

#### **4.4.2 Cloud Computing**

Cloud computing refers to the use of the cloud symbol to represent the Internet. The concept involves a network of remote computers that provide a comprehensive online software and data environment, hosted by third-party providers. In this model, the user's computer does not handle all the processing; instead, the cloud performs some or all of the tasks. This contrasts with traditional computing models, where processing and data are distributed across various systems within an enterprise. Essentially, the cloud acts as a single large computer that handles tasks for users.

Cloud computing eliminates compatibility issues, as the Internet itself serves as the platform. This architecture also allows for scaling on demand, where resources can be adjusted based on the current need. For instance, during peak usage, additional cloud servers might be brought online automatically to support the increased workload.

Cloud computing is particularly well-suited for Software as a Service (SaaS) applications, which do not require users to purchase software but instead pay for it as a service, similar to how one pays for utilities like electricity or cable TV each month. Service providers can update and modify the software without requiring user involvement.

Despite the significant advantages of cloud computing, there are some concerns. First, it may require more bandwidth than traditional client/server networks, as it involves transferring large amounts of data. Second, because cloud computing relies on the Internet, users cannot access cloud services if their Internet connection fails. Security issues also arise from transmitting large data over the Internet, along with challenges in securely storing this data. Lastly, there's the issue of control. Since the service provider manages data storage and access, they have full control over the system, which can be problematic for companies hesitant to entrust mission-critical data to third-party providers, especially when these providers operate in different countries or jurisdictions.

As technology continues to evolve, cloud computing will become more feasible, secure, and attractive. As the IT industry shifts toward a web-based architecture, cloud computing is rapidly growing and has become a cornerstone of enterprise system architecture. It is expected to remain a key element of system design in the foreseeable future.

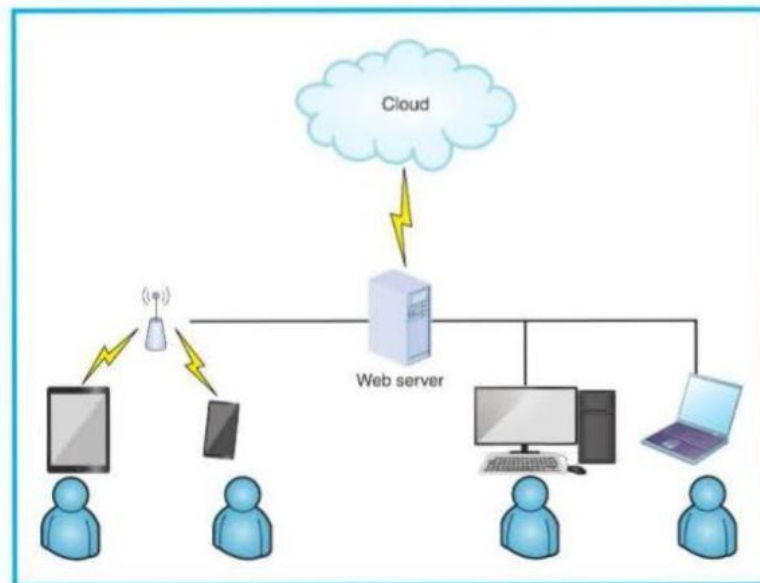


FIGURE 10-10 Cloud computing.

(Tilley and Rosenblatt, 2024)

#### 4.4.1 Web 2.0

The shift toward internet-based collaboration has been so impactful that it has been coined "Web 2.0." This term doesn't refer to a more technically advanced version of the current web but instead envisions a second generation of the web designed to enable people to collaborate, interact, and share information in a more dynamic way. Some see Web 2.0 as a stepping stone towards the "semantic web" (or Web 3.0), where documents on the internet have meanings (semantics) rather than just structure (syntax, like HTML markup).

Social networking sites such as Facebook, Twitter, and LinkedIn have experienced explosive growth in the Web 2.0 environment. Another form of social collaboration is the "wiki," a web-based repository of information that anyone can access, contribute to, or modify. A wiki represents the collective knowledge of a group. One of the most well-known wikis is Wikipedia, but smaller-scale wikis are also growing rapidly in businesses, schools, and other organizations aiming to compile and share information.

One of Web 2.0's goals is to enhance creativity, interaction, and the sharing of ideas. In this regard, Web 2.0 mirrors concepts like agile development and the open-source software movement. Communities and services in the Web 2.0 environment are built on data created by users. As users collaborate, new layers of information—such as text, audio, images, and video clips—are added to an overall environment known as the "Internet operating system," shared within the user community.

## 4.5 E-COMMERCE ARCHITECTURE

The rapid growth of online commerce is significantly altering the IT landscape. E-commerce solutions must be efficient, reliable, and cost-effective. When planning an e-commerce architecture, analysts can consider in-house development, packaged solutions, or service providers. The following sections explore these options.

### 4.5.1 In-House Solutions

Chapter 7 discussed how to evaluate the advantages and disadvantages of in-house development versus purchasing a software package. The same principles apply when designing a system.

If the decision is made to proceed with an in-house solution, a comprehensive plan is necessary to ensure the project's goals are met. Figure 10-11 provides guidelines for companies developing e-commerce strategies. In-house solutions typically require a higher initial investment but offer greater flexibility for companies that need to adapt quickly in a dynamic e-commerce environment. By developing in-house, a company has more control over integration with customers and suppliers and is less reliant on vendor-specific solutions.

| Guidelines for In-house E-commerce Site Development                                                                                                                                           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Analyze the company's business needs and develop a clear statement of your goals. Consider the experience of other companies with similar projects.                                           |
| Obtain input from users who understand the business and technology issues involved in the project. Plan for future growth, but aim for ease of use.                                           |
| Determine whether the IT staff has the necessary skills and experience to implement the project. Consider training, additional resources, and the use of consultants if necessary.            |
| Consider integration requirements for existing legacy systems or enterprise resource planning. Select a physical infrastructure carefully, so it will support the application, now and later. |
| Develop the project in modular form so users can test and approve the functional elements as you go along.                                                                                    |
| Connect the application to existing in-house systems and verify interactivity.                                                                                                                |
| Test every aspect of the site exhaustively. Consider a preliminary rollout to a pilot group to obtain feedback before a full launch.                                                          |

**FIGURE 10-11** Guidelines for companies developing e-commerce strategies.

(Tilley and Rosenblatt, 2024)

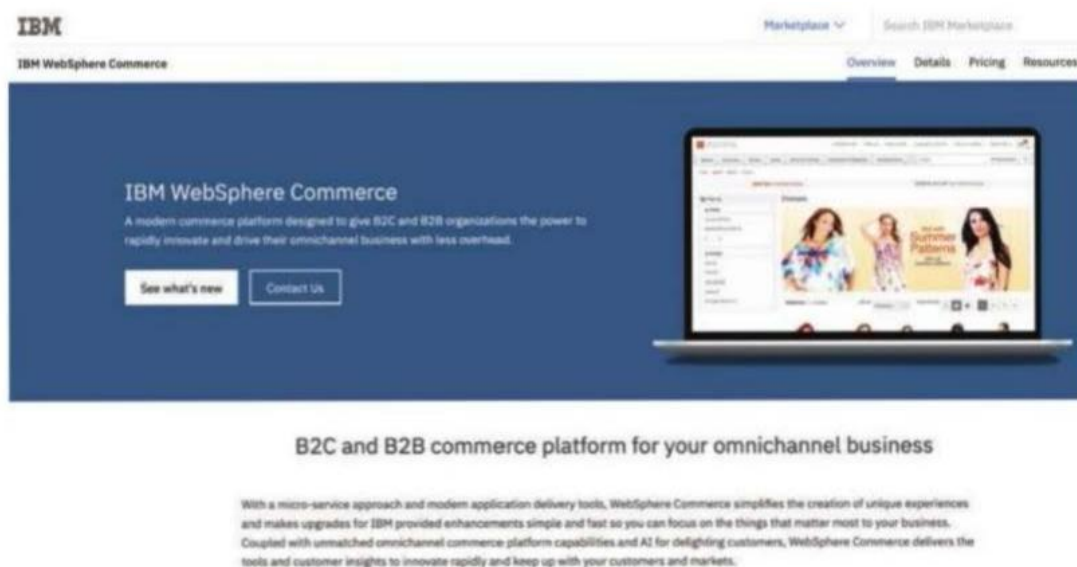
For smaller companies, deciding on in-house web development is especially crucial, as this approach demands financial resources and management attention that many small businesses may not be able or willing to commit. However, an in-house strategy offers several valuable benefits, including:

- A unique website that aligns with the company's other marketing efforts in terms of look and feel.
- Full control over the site's structure, including the number of pages and file sizes.
- A scalable framework that can handle future increases in sales and product offerings.
- Greater flexibility to modify and manage the site as the company evolves.
- The opportunity to integrate the company's web-based business systems with its other information systems, potentially leading to cost savings and improved customer service.

Regardless of whether a company chooses an in-house or packaged solution, the decision regarding web hosting is a separate matter. While internal hosting offers benefits like greater control and security, it can be significantly more expensive, especially for small to medium-sized businesses.

#### 4.5.2 Packaged Solutions

If a small company is hesitant to take on the challenge and complexity of developing an e-commerce website in-house, a packaged solution can be an alternative. This is true even for medium- to large-sized companies. Many vendors, including IBM and Microsoft, offer turnkey systems for companies that want to launch an e-business quickly (as shown in Figure 10-12). However, for large-scale systems that need to integrate with existing applications, packaged solutions may be less attractive.



**FIGURE 10-12** IBM WebSphere Commerce offers software solutions for companies that want to get an e-business up and running quickly.

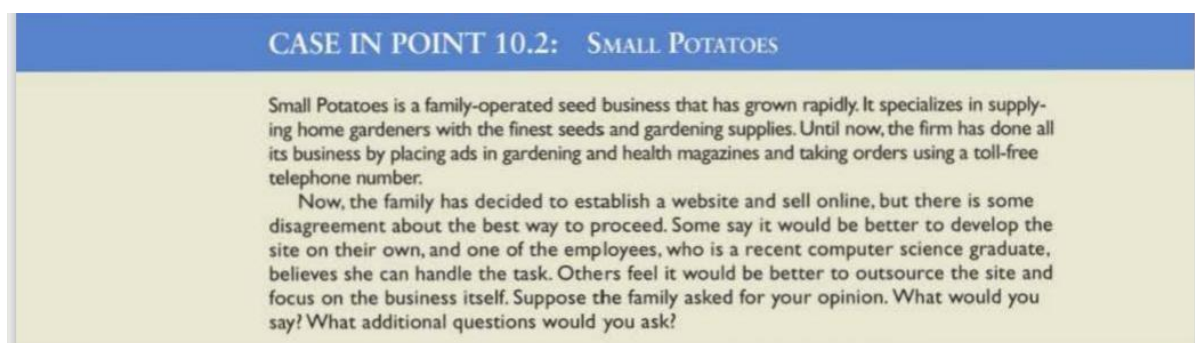
Source: IBM Corporation

(Tilley and Rosenblatt, 2024)

#### 4.5.3 Service Providers

Another option is to use an application service provider (ASP). As discussed in Chapter 7, an ASP delivers applications or access to applications through a subscription or usage-based fee. Many ASPs now offer comprehensive internet business services for companies that prefer to outsource these functions.

A systems analyst faces a wide range of products and strategies when implementing internet-based systems. A good starting point may be to consider the experiences of other companies within the same industry. Many service providers share the names of their clients and success stories. While this information may not always be completely reliable, it can offer valuable insights into the vendor's products and services.



**CASE IN POINT 10.2: SMALL POTATOES**

Small Potatoes is a family-operated seed business that has grown rapidly. It specializes in supplying home gardeners with the finest seeds and gardening supplies. Until now, the firm has done all its business by placing ads in gardening and health magazines and taking orders using a toll-free telephone number.

Now, the family has decided to establish a website and sell online, but there is some disagreement about the best way to proceed. Some say it would be better to develop the site on their own, and one of the employees, who is a recent computer science graduate, believes she can handle the task. Others feel it would be better to outsource the site and focus on the business itself. Suppose the family asked for your opinion. What would you say? What additional questions would you ask?

(Tilley and Rosenblatt, 2024)

## 4.6 PROCESSING METHODS

When selecting an architecture, the systems analyst must identify which transactions should be handled in real-time through **online processing** and which functions, if any, can be executed using **batch processing**. Online processing involves handling transactions immediately as they occur, providing immediate feedback to users. In contrast, batch processing involves gathering data over a period of time and processing it in groups, typically during off-peak hours to optimize system resources.

The choice between online and batch processing depends on factors like the nature of the transaction, urgency, and system performance requirements. The systems analyst will evaluate the need for real-time interactions versus the efficiency of processing large volumes of data in batches.

### 4.6.1 Online Processing

When selecting an architecture, the systems analyst must determine which transactions will be handled online and which, if any, can be managed through batch processing.

#### Online Processing

In the early days of computer systems, data was handled in batches, processing records as a group. While this batch processing model has become less common, even the most advanced

online systems still perform maintenance tasks, process large amounts of data during off-peak hours, and handle housekeeping functions similar to legacy systems. This section focuses on the core concept of online processing in modern systems, while the next section will discuss the evolution of batch processing.

An online system processes transactions immediately when they occur, providing direct output to users. Since online systems are interactive, they avoid delays and facilitate continuous communication between the user and the system.

A common example of online processing is an airline reservations system. When customers visit the airline's website, they can input their origin, destination, travel dates, and times. The system then searches the database and displays available flights, times, and prices. The customer can proceed to make a reservation, entering their name, address, and credit card details. The system immediately creates the reservation, assigns a seat, and updates the flight database.

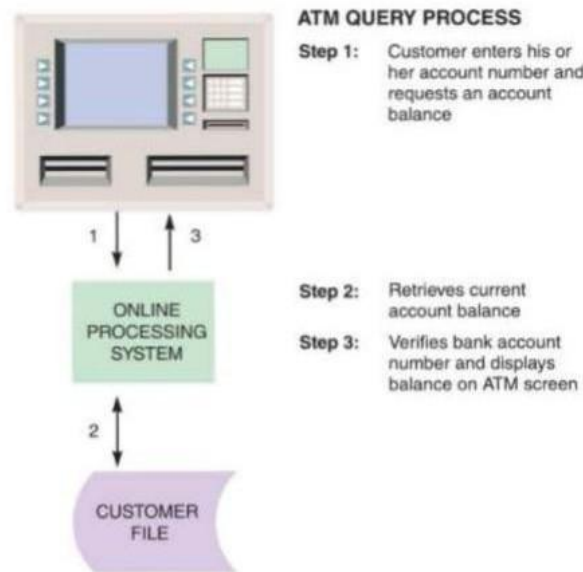
Online processing can also be applied to file-oriented systems. For example, when a customer uses an ATM to inquire about their account balance, the system follows these steps:

1. The customer enters their account number and requests the balance.
2. The ATM verifies the customer's card and password, then retrieves the account record using the account number as the primary key.
3. The system verifies the account number and displays the balance.
4. The system transmits the current balance to the ATM, which prints it for the customer.

### **Characteristics of Online Processing Systems**

Online processing systems typically have four key characteristics:

1. **Immediate Transaction Processing:** The system processes transactions completely when and where they occur.
2. **User Interaction:** Users interact directly with the information system.
3. **Random Data Access:** Users can access data randomly.
4. **System Availability:** The information system must be available whenever needed to support business functions.



**FIGURE 10-13** When a customer requests a balance, the ATM system verifies the account number, submits the query, retrieves the current balance, and displays the balance on the ATM screen.

(Tilley and Rosenblatt, 2024)

#### 4.6.2 Batch Processing

Batch processing means that data is managed in groups or batches. This method was a common and acceptable choice in the 1960s, and for most firms, it was the only option. Today, while real-time information is crucial for most businesses, batch processing is still useful in certain situations. For example, it is often applied to large volumes of data that need to be processed on a routine schedule, such as weekly payroll, daily credit card transaction updates, or closing stock data that needs to be calculated and published in the next day's news media.

The **advantages of batch processing** include:

- **Tasks can be planned and run on a predetermined schedule**, without user involvement.
- **Batch programs requiring major network resources** can be scheduled to run during off-peak times when costs are lower and impact on other traffic is minimized.
- **Security, audit, and privacy:** Batch methods are well-suited for handling tasks that involve sensitive data, as they run in a relatively controlled environment.

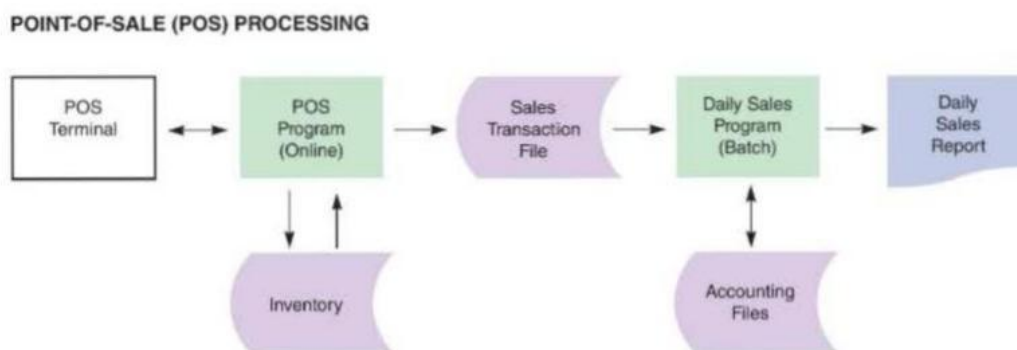
#### 4.6.3 Example

The diagram in **Figure 10-14** shows how a point-of-sale (POS) terminal, such as that used in a supermarket, might trigger a series of online and batch processing events. The system uses online processing to handle data entry and inventory updates, while reports and accounting entries are handled in a batch. A company would choose a mix of online and batch processing when it makes good business sense.

Consider the following scenario in a typical retail store:

- During business hours, a salesperson enters a sale on a POS terminal, which is part of an online system that handles daily sales transactions and maintains an up-to-date inventory file.
- When the salesperson enters the transaction, **online processing** occurs. The system performs calculations, updates the inventory file, and produces output on the POS terminal in the form of a screen display and a printed receipt. At the same time, each sales transaction creates input data for day-end **batch processing**.
- When the store closes, the system uses the sales transactions to produce the daily sales report, perform the related accounting entries, and analyze the data to identify slow- or fast-moving items, sales trends, and related issues (such as store discounts for the next day).

This mix of online and batch processing is an example of how businesses can leverage both methods to meet different needs efficiently. Online processing ensures real-time updates and interaction, while batch processing is used for handling large volumes of data that don't require immediate attention.



**FIGURE 10-14** Many retailers use a combination of online and batch processing. When a salesperson enters the sale on the POS terminal, the online system retrieves data from the item file, updates the quantity in stock, and produces a sales transaction record. At the end of the day, a batch processing program produces a daily sales report and updates the accounting system.

(Tilley and Rosenblatt, 2024)

Online and batch processing are fundamentally different but can work well together in certain scenarios. In the case of the retail store example, an **online system** handles the POS processing, which must be done in real-time as transactions occur, ensuring that the data is entered and validated immediately. This provides up-to-date information, which is crucial for maintaining an accurate inventory and ensuring smooth operations. However, **online processing** can be expensive for smaller firms, especially when dealing with a high volume of transactions, and the additional costs related to data backup and recovery can further increase IT expenditures.

On the other hand, **batch processing** is well-suited for routine, overnight tasks such as generating sales reports, performing accounting entries, and analyzing marketing data. It can be cost-effective and less vulnerable to system disruptions, as it doesn't require real-time interaction with users. Batch processing can be scheduled to run when network traffic is

lower, minimizing its impact on the business. This method is more scalable and can handle large volumes of data with less immediate resource demand.

The combination of online and batch processing allows businesses to balance real-time operations with efficient, cost-effective handling of large volumes of data. By using both methods strategically, companies can ensure their systems are both up-to-date and scalable while minimizing operational costs.

## **4. 7 NETWORK MODELS**

A network allows sharing of resources such as hardware, software, and data to reduce costs and provide more capabilities to users. When designing a network, systems analysts need to understand several key concepts, such as the OSI model, network topology, network protocols, and wireless networks. These concepts help in configuring the network efficiently and effectively.

### **4.7.1 The OSI Model**

The OSI (Open Systems Interconnection) model is essential for understanding how data moves between applications on different networked computers. The model consists of seven layers, each responsible for a specific function. These layers ensure seamless network connectivity across different hardware environments, providing a standardized way for different systems to communicate.

The layers in the OSI model are:

1. **Physical Layer:** Deals with the hardware transmission of data over the network.
2. **Data Link Layer:** Manages error detection and correction from the physical layer.
3. **Network Layer:** Handles the routing of data packets across the network.
4. **Transport Layer:** Ensures error-free data transfer between devices.
5. **Session Layer:** Manages sessions between applications.
6. **Presentation Layer:** Translates data between the application and transport layers.
7. **Application Layer:** Provides the interface for software applications to communicate over the network.

### **4.7.2 Network Topology**

Network topology refers to how the network is physically or logically arranged. Physical topology describes the actual layout of cables and devices, while logical topology describes how data travels between devices.

A physical topology might not always match the logical topology, as cabling might be set up for practical reasons (e.g., cost) but could still use a different pattern for logical data flow. Network topologies can generally be grouped into four main types: hierarchical, bus, ring, and star.

#### **Hierarchical Network Topology**

In a hierarchical network, one or more powerful servers control the entire network. Lower-level servers handle departmental tasks or more localized operations. This type of topology is commonly used in larger organizations, such as retail chains, where a central computer tracks sales and inventory levels while local computers handle day-to-day operations at individual stores.

In this example:

- The **central computer** in the retail store chain analyzes sales trends and determines stock levels.
- **Local computers** at each store handle sales transactions and update the central computer.

This topology reflects the operational flow of the organization, where higher levels manage broader tasks, while lower levels handle specific, localized tasks.

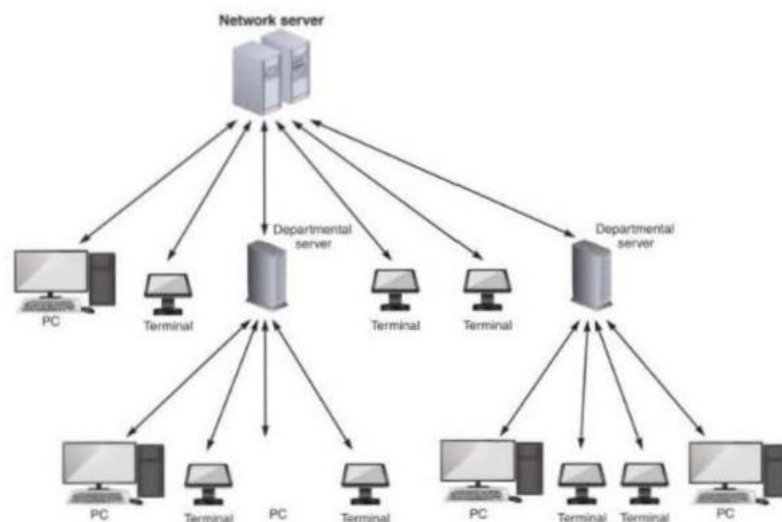


FIGURE 10-15 A hierarchical network with a single server that controls the network.

(Tilley and Rosenblatt, 2024)

## Bus Network

In a **bus network**, a single communication path connects all devices, including the central server, departmental servers, workstations, and peripheral devices. Data is transmitted in both directions along the central bus. Key advantages of a bus network include:

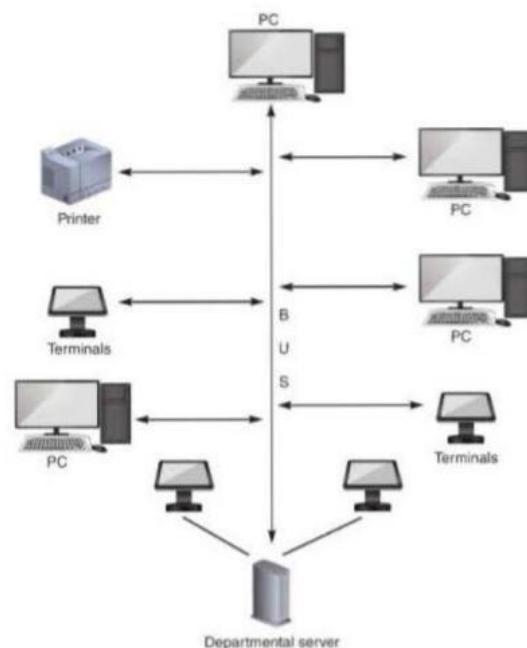
- **Lower cabling requirements** compared to other topologies, as only one cable is needed.
- **Flexibility**: Devices can be added or removed without disrupting the entire network.

- **Fault tolerance:** Failure in one workstation does not affect other devices on the network.

However, the bus network has significant disadvantages:

- If the central bus fails, the entire network goes down.
- **Performance issues:** As more devices are added, the network's performance can degrade because all traffic passes through the same bus.

While **bus networks** were once a common LAN topology, they are now less popular due to newer technologies. Some businesses still use bus networks to avoid the cost of new wiring and hardware.



**FIGURE 10-16** A bus network with all the devices connected to a single communication path.

(Tilley and Rosenblatt, 2024)

## Ring Network

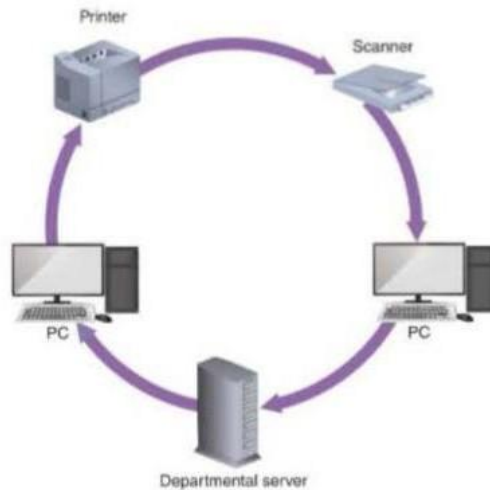
A **ring network** is arranged in a circle where data flows in only one direction, passing through each device until it reaches the destination. IBM's **token ring** LANs were an example of this setup, which was primarily used in large companies using IBM mainframe systems.

Advantages of a **ring network**:

- Structured data flow, as data only travels in one direction, reducing the risk of network collisions.

Disadvantages:

- If any device fails, the entire network can be disrupted because data cannot bypass the failed device.
- **Outdated**: While still in use by some legacy systems, ring networks are less common today.



**FIGURE 10-17** A ring network with a set of computers that sends and receives data flowing in one direction.

(Tilley and Rosenblatt, 2024)

## Star Network

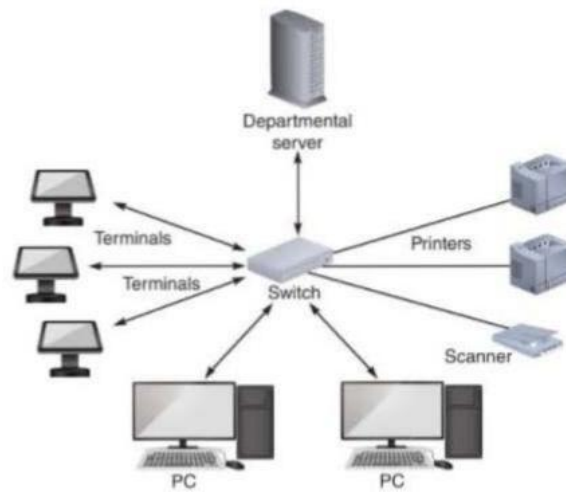
The **star network** is one of the most popular LAN topologies due to its speed and versatility. It has a central device (usually a **switch**) that manages the network and handles all data traffic between devices. Older **hub**-based star networks have been largely replaced by switches, which offer improved performance.

Advantages of a **star network**:

- **Central control**: All traffic flows into and out of the switch, making it easier to manage and monitor the network.
- **Better performance**: A **switch** ensures that data is only sent to the devices that need it, rather than broadcasting to all devices, as a hub does.
- **Easy to expand**: New devices can be added by connecting them to the central switch.

Disadvantages:

- The **switch** is a single point of failure; if it fails, the entire network can go down. However, in large star networks, **backup switches** can mitigate this risk.



**FIGURE 10-18** A star network with a switch, departmental server, and connected computers and devices.

(Tilley and Rosenblatt, 2024)

## Mesh Network

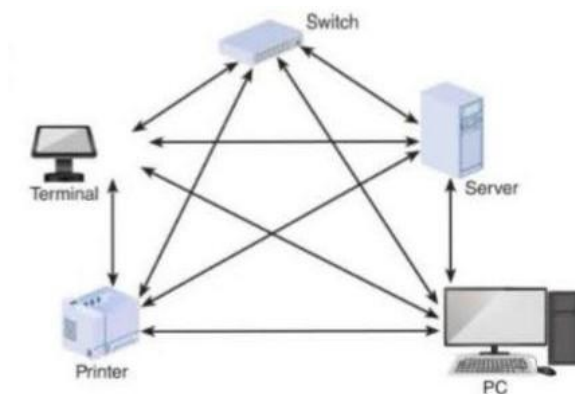
In a **mesh network**, every device is connected to every other device, providing multiple paths for data to travel. This design is highly **reliable** and offers **redundancy**, as there are several backup routes for data if one path fails.

Advantages of a **mesh network**:

- **Redundancy**: Multiple paths allow for uninterrupted communication if any node fails, which is particularly useful for mission-critical applications.
- **Reliability**: This design is resistant to network outages, as data can find alternative paths.

Disadvantages:

- **Expensive**: Setting up and maintaining a mesh network is costly due to the large number of connections required.
- **Complexity**: It is difficult to manage due to the numerous connections between devices.



**FIGURE 10-19** A mesh network is used in situations where a high degree of redundancy is needed, such as military applications. The redundant design provides alternate data paths but is expensive to install and maintain.

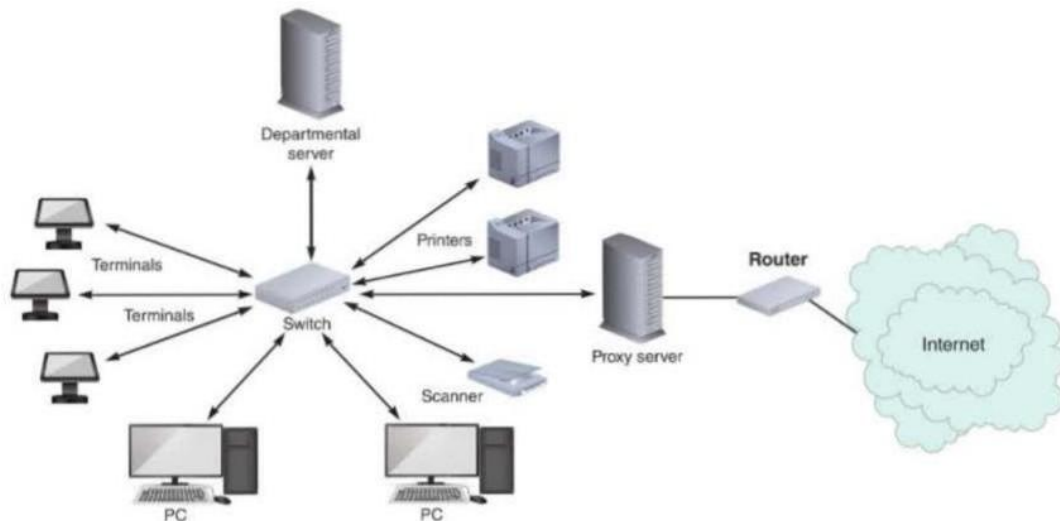
(Tilley and Rosenblatt, 2024)

### 4.7.3 Network Devices

Networks are interconnected using **routers**, devices that connect different network segments and determine the most efficient data path for information to travel.

- **Routers** connect networks, making it possible to link multiple topologies into a larger network, including external networks like the Internet. This type of connection is called a **gateway**.
- **Proxy servers** provide internet connectivity for internal users by routing traffic between the LAN and the external network.

The **star network** example with a **switch** and **router** is a typical configuration for many business networks. The switch manages the LAN traffic, while the router links the internal network to the Internet, allowing for communication between devices across the global network.



**FIGURE 10-20** Routers can be used to create gateways between different network topologies and large, dissimilar networks such as the Internet.

(Tilley and Rosenblatt, 2024)

## 4.8 WIRELESS NETWORKS

Although a wired LAN provides enormous flexibility, the **cabling cost** can be substantial, and changes to the wiring are inevitable in a dynamic organization. Many companies find **wireless technology** to be an attractive alternative. A **wireless local area network (WLAN)** is relatively inexpensive to install and is well-suited to workgroups and users who are not anchored to a specific desk or location. Many **notebook computers** and other **mobile devices** come with built-in **wireless capability**, and it is relatively simple to add this feature to existing desktop computers and workstations to set up a wireless network.

Like their wired counterparts, wireless networks also follow certain **standards** and **topologies**, which are discussed in the following sections.

### 4.8.1 Standards

Wireless networks are governed by a variety of **standards** and protocols, with **IEEE 802.11** being the most common. This family of standards, developed by the Institute of Electrical and Electronics Engineers (IEEE), is used for wireless local area networks (**WLANs**). The standard has evolved over time, with each version designed to improve **bandwidth**, **range**, and **security**. For example:

- **802.11b (1999)**: Had an average speed of **11 Mbps**.
- **802.11g**: Increased bandwidth to **54 Mbps**.
- **802.11n**: Reached **450 Mbps**, using **MIMO (Multiple Input Multiple Output)** technology to boost performance.
- **802.11ac**: Offers speeds of up to **7 Gbps** using **MIMO** and other advanced technologies.

These advances in wireless speed have made WLANs an attractive alternative to wired networks, especially as they become more cost-effective and secure. However, **wireless security** remains a significant concern and will be explored in detail in **Chapter 12**.

### 4.8.2 Topologies

Just like wired networks, **wireless networks** can be configured in various **topologies**. The two primary WLAN topologies defined in the **IEEE 802.11** standard are:

1. **Basic Service Set (BSS):**

This is also called **Infrastructure Mode**. In this topology, a **central wireless device**, called an **access point (AP)** or **wireless access point (WAP)**, serves all wireless clients. It acts like a **hub** in a **star topology** for wired networks, except that it connects wireless clients instead of wired devices. The AP broadcasts all traffic to all connected clients, typically providing a connection to a **wired network**.

2. **Extended Service Set (ESS):**

This topology expands the range of wireless access. An **ESS** consists of two or more BSSs, which means that as a client moves through the network's coverage area, it can **roam** from one AP to another. Roaming automatically switches the client to the access point with the stronger signal, ensuring uninterrupted service. This is particularly useful in large spaces, like office buildings or campuses.

### 4.8.3 Trends

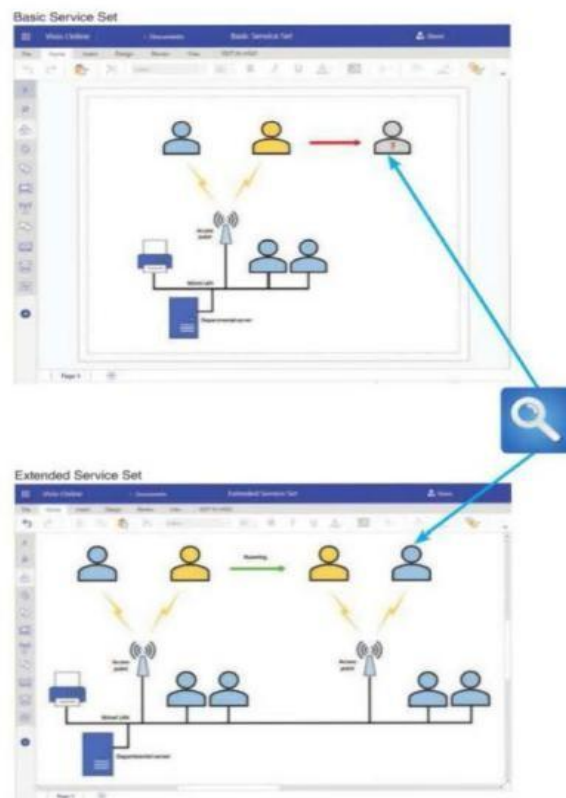
Wireless technology has transformed the IT industry and will continue to have a major impact on businesses and individuals. With the growing popularity of **802.11** wireless standards, the **Wi-Fi Alliance** has played a critical role in certifying product interoperability. Founded in **1999**, the Wi-Fi Alliance ensures that products meet the **IEEE 802.11** specifications. Devices that are certified as **Wi-Fi compatible** can work seamlessly with each other, providing a smoother user experience.

While **Wi-Fi** networks offer numerous benefits, such as **flexibility** and ease of installation, they come with certain limitations, including:

- **Interference:** Devices operating in the **2.4 GHz band** can experience interference from household appliances like **microwave ovens** and **cordless telephones**, which also use the same frequency range.
- **Security concerns:** Wireless networks are inherently more vulnerable to **interception** and **intrusion** than wired networks. Security measures are crucial to protect sensitive data in wireless environments.

Additionally, **Bluetooth** has gained popularity for short-distance wireless communication. It is commonly used for low-power devices like **wireless keyboards**, **mice**, **headsets**, and

**printers.** Bluetooth is designed for small, **localized networks** and is particularly useful for exchanging information between devices over short distances (e.g., a **smartphone** and a **tablet**)



**FIGURE 10-21** The user in the upper screen has moved out of the BSS coverage area and cannot communicate. In the lower screen, the user roams into another ESS coverage area and the transition is seamless.

(Tilley and Rosenblatt, 2024)

### CASE IN POINT 10.3: SPIDER IT SERVICES

Spider IT Services specializes in custom network design and installation. Firms hire Spider to do an overall analysis of their network needs, including a detailed cost-benefit study. Recently, a problem arose. One of Spider's clients complained that the relatively new network was too slow and lacked sufficient capacity. Reviewing the case, Spider's top management realized that the rapidly growing client had simply outgrown the network much earlier than anticipated. How could this problem have been avoided?

## 4. 9 SYSTEMS DESIGN COMPLETION

**System architecture** marks the conclusion of the systems design phase within the SDLC. During the **systems analysis phase**, all the functional components were identified and documented with detailed process descriptions. The goal at this stage was to outline the system's functions and determine the roles of each module without addressing how those functions would be executed. Transitioning from analysis to design, the development process

then focused on aspects such as output design, user interface design, data design, and system architecture issues.

With a clear understanding of the system's requirements and design, the next step is to move towards **software application development**, which will be documented and prepared for the **systems implementation phase** of the SDLC, as described in Chapter 11.

Additionally, developers must plan for **system management and support tools** to monitor system performance, handle fault management, ensure proper backup procedures, and prepare for disaster recovery. These areas are discussed in more detail in Chapter 12.

The final tasks in the systems design phase involve creating a **system design specification**, obtaining **user approval**, and delivering a **presentation to management**.

#### 4.9.1 System Design Specification

The **system design specification** serves as a document that outlines the entire design for the new information system. This includes detailed information about costs, staffing, and scheduling necessary for the next phase of the SDLC—systems implementation. The system design specification serves as the baseline for the operational system and differs from the system requirements document, as it is geared towards programmers who will use it to create the necessary programs. While some sections from the system requirements document, such as process descriptions, data dictionary entries, and data flow diagrams, may be repeated, the content is tailored to support the development phase.

A typical system design specification will include:

1. **Management Summary:** A brief overview for managers and executives, including the project's current status, costs, benefits, implementation schedule, and key issues requiring attention.
2. **System Components:** A detailed description of the system design, including the user interface, inputs, outputs, files, databases, network specifications, and any necessary support processing such as backup and recovery.
3. **System Environment:** The section describing constraints and conditions affecting the system, such as transaction volumes, data storage needs, processing schedules, and reporting deadlines.
4. **Implementation Requirements:** Specifications for setup processing, initial data entry, user training requirements, and software test plans.
5. **Time and Cost Estimates:** Detailed schedules, cost estimates, and staffing needs for system development, as well as comparisons of projected costs versus actual expenditures.
6. **Additional Material:** Any other relevant documentation from previous phases that might assist in the review and understanding of the design.

#### 4.9.2 User Approval

User approval is crucial throughout the systems design phase. Users must review and approve the interface design, reports, menus, data entry screens, and other elements affecting their interaction with the system. This approval process continues throughout the design phase, with the systems analyst meeting with users to review prototypes, make necessary adjustments, and obtain written approval. Securing user approval early ensures that the project stays on track and meets user expectations, while also providing valuable feedback about the system's design.

IT department members, including management, programmers, and the operations group, also need to review the system design specification. They will be concerned with staffing, costs, hardware/software requirements, network impacts, and other operational considerations. Clear communication from the systems analyst ensures that all stakeholders are informed, their input is considered, and approvals are obtained efficiently.

Once the system design specification is completed, it should be distributed to all relevant stakeholders, allowing at least one week for review before the presentation.

#### 4.9.3 Presentations

The **presentation** phase marks the end of the systems design phase. Several presentations are typically conducted to explain the system, address questions, gather feedback, and obtain final approval.

1. **First Presentation:** Presented to the technical team, including systems analysts, programmers, and technical support staff. This presentation is focused on technical details and prepares the team for the next phases of the project.
2. **Second Presentation:** Directed at department managers and users from affected departments. The goal is to gain support and approval from those who will be using the system and those responsible for overseeing its implementation. This presentation emphasizes the impact on the user experience and operational aspects of the system.
3. **Final Presentation:** Given to company management, this presentation aims to secure management's approval and support for moving forward with the next SDLC phase—systems implementation. By this point, all necessary approvals should have been secured from users and IT, and the management presentation focuses on obtaining a commitment for the resources required for implementation.

## A QUESTION OF ETHICS



stock.com/iberkatz\_8

The new accounting system is operational, but feedback from users has been negative. The most common complaint is that the system is not user-friendly. Some people in the IT department think that more user training would solve the problem. However, the IT manager is opposed to a fresh round of training. "Let's just set up the network to monitor the users' keystrokes and mouse clicks, and see what the patterns are," he suggested. "We can analyze the data and come up with tips and suggestions that would make the system easier to use."

Your initial reaction is that the IT manager is wrong for two reasons. First, you believe that monitoring would not be an effective method to learn what users really want. In your view, that should have been done in the system requirements phase. Second, you are bothered by an ethical question: Even though the proposed monitoring would involve company business, the company network, and company time, you feel that many users would resent the unannounced monitoring and might feel that their performance or other computing activities were being appraised without their knowledge.

The IT manager has asked to you to write up a recommendation. What will you say about the ethical question that troubles you?

### 4.10 SUMMARY

An information system integrates hardware, software, data, procedures, and people into an architecture that transforms the system's logical design into a physical structure. This structure includes all necessary hardware, software, and processing methods. The software includes application programs that handle input, manage processing logic, and produce the required output.

When selecting a system architecture, analysts must consider factors such as ERP, initial costs, total cost of ownership (TCO), scalability, web integration, legacy system interfaces, processing options, security, and corporate portals. ERP establishes a company-wide IT strategy, defining standards for data, processing, networks, and user interface design. Companies can extend ERP systems to suppliers and customers via Supply Chain Management (SCM). A systems analyst should assess costs and ensure the system is scalable—able to be expanded, modified, or downsized easily to meet business demands. Analysts must also consider whether the system should be web-based, follow internet protocols, and interface with legacy systems. Security is a major concern, especially for e-commerce applications handling sensitive data. Processing methods influence system design and the resources required, while corporate portals offer a gateway for various stakeholders to access different services.

System architecture relies on servers and clients. Servers provide data and processing services to clients. In mainframe architecture, the server handles all processing, and terminals communicate with the centralized system. Clients can connect to distributed systems forming local or wide area networks (LANs or WANs). Client/server architecture distributes processing between clients and a central server. Typically, the client manages the user interface (input, queries, and screen display), while the server stores data and handles database management. Application logic is distributed between the client and server. Clients request information from the server, which performs the necessary operation and returns the result. Client/server systems are more scalable and flexible compared to file server designs.

A fat (or thick) client design places most application processing at the client side, while a thin client design shifts most processing to the server. Fat clients may incur higher TCO due to hardware and software needs, plus maintenance costs, but are simpler to develop. Thin clients have a central server that handles most processing.

Client/server systems can be two-tier or three-tier. In a two-tier design, the client handles the user interface, and the server stores data. Application logic can be on either the client or server. In a three-tier design, the client handles the user interface, and the server stores data, with an additional middle layer, known as the application server, that processes client requests and converts them into commands for the server. Middleware connects different applications and enables them to share data. When designing a system, analysts also need to consider cost-benefit and performance issues.

The internet has significantly influenced system architecture. Analysts must consider e-commerce architecture, packaged solutions, and service providers when implementing designs. Concepts such as cloud computing and Web 2.0 are shaping the future of internet computing. Cloud computing uses remote servers for processing, and Web 2.0 promotes dynamic interaction and collaboration online, contributing to the growth of social networking and group communication.

The primary method for processing today is online processing, where users interact with systems that process transactions in real-time. Batch systems, on the other hand, process transactions in scheduled groups. Many online systems also use batch processing for tasks like reporting and accounting.

Networks enable sharing of hardware, software, and data to reduce costs and enhance user capabilities. The OSI model, a seven-layer framework, represents how networks function. Network configurations (topologies) include hierarchical, bus, ring, star, and mesh designs. Wireless networks (WLANs), based on IEEE 802.11 standards, have become widely popular for their flexibility. However, wireless networks can face issues such as interference and security concerns.

A system design specification outlines the complete system architecture and serves as the foundation for final presentations at the end of the design phase. After these presentations, the project may move to the system development phase, require more design work, or be canceled.

## **EXERCISE**

### **Questions**

1. If you had to rank the items in the architecture checklist, from most important to least important, what would your list look like?
2. What are the three functions that every business information system must carry out, irrespective of system architecture?
3. What is client/server architecture?
4. What has been the impact of the Internet on system architecture?
5. What are the differences between in-house e-commerce development with packaged solutions and service providers?
6. What are the advantages of online and batch processing, respectively?
7. Explain the five main network models.
8. What functions do routers, gateways, and proxy servers serve in a network?
9. What role do standards play in wireless networking?
10. List the sections of a system design specification and describe the contents.

### **Discussion Topics**

1. How is the proliferation of mobile devices that are locally powerful, use apps instead of full-fledged applications, and rely on wireless network connectivity changing system architecture design considerations?
2. E-commerce has seen explosive growth in recent years. What are the most important reasons for this trend? Will it continue? Why or why not?
3. Is batch processing still relevant? Why or why not?
4. What are the main differences between the BSS and ESS wireless topologies?
5. One manager says, "When a new system is proposed, I want a written report, not an oral presentation, which is like a sales pitch. I only want to see the facts about costs, benefits, and schedules." Do you agree with that point of view?

### **Projects**

1. Visit the IT department at your school or a local company to learn about the network they use. Describe the network and draw a sketch of the configuration.
2. Prepare a 10-minute talk explaining Web 2.0 and cloud computing to a college class. Using the text and your own Internet research, briefly describe the five most important points you will include in your presentation.
3. Perform research on the Internet to identify a service provider that offers web-based business solutions, and write a report describing the firm and its services.
4. Perform research on the Internet to learn about emerging trends in wireless networking and typical costs involved in the installation of a wireless LAN.
5. Examine the role wireless networks are having in the developing world. Why are some places bypassing LANs and physical cabling altogether and moving to a wireless system architecture? What are the advantages and disadvantages of this?

## CHAPTER 5: Managing Systems Implementation



### LEARNING OUTCOMES

After reading this Section of the guide, the learner should be able to:

1. Explain quality assurance and three techniques to help improve the finished product.
2. Outline application development.
3. Apply structured development.
4. Apply object-oriented development.
5. Apply agile development.
6. Explain coding.
7. Explain unit, integration, and system testing.
8. Differentiate between program, system, operations, and user documentation.
9. Explain the role of online documentation.
10. Describe the five tasks involved in system installation.

### 5.1 QUALITY ASSURANCE

In today's competitive business world, companies are deeply focused on the quality of their products and services. For an organization to succeed, it must ensure quality across all aspects, including its information systems. It is essential that top management offers the leadership, motivation, and support necessary to maintain high-quality IT resources.

Despite careful system design and implementation, issues can still arise. While thorough testing can identify problems during the implementation phase, it is significantly cheaper to address mistakes earlier in the development process. The primary goal of quality assurance (QA) is to prevent issues or detect them as early as possible. Poor quality can stem from

unclear requirements, design flaws, coding mistakes, faulty documentation, or ineffective testing.

To enhance the finished product, software developers should follow best practices in software engineering, systems engineering, and internationally recognized quality standards.

## **Software Engineering**

Software engineering involves applying engineering principles to create complex, long-lasting applications. It involves a combination of people, processes, and technology. Software engineering encompasses more than just development and includes five technical areas: requirements, design, construction, testing, and maintenance/evolution. It also involves non-technical tasks like cost estimation, project management, and process improvement.

The Software Engineering Institute (SEI) at Carnegie Mellon University is a key player in the field, offering quality standards and recommended practices for software developers and systems analysts. SEI's primary goal is to discover better, faster, and more affordable methods of software development. To achieve this, SEI introduced the Capability Maturity Model (CMM), which has been widely adopted by organizations globally. The model's aim is to enhance software quality, reduce development time, and cut costs. The CMM has five maturity levels, ranging from an unpredictable, poorly controlled state (Level 1) to a level of optimized process improvement (Level 5).

After the original CMM was updated, other variations were introduced. Eventually, the SEI developed the Capability Maturity Model Integration (CMMI), which integrates both software and systems development into a broader framework. CMMI tracks an organization's processes across five levels, from Level 1, which is chaotic and reactive, to Level 5, where process improvement is continuously optimized.

## **Systems Engineering**

Systems engineering extends beyond software engineering to include other components of the overall system, such as hardware, networks, and interfaces. Professional organizations like INCOSE and the IEEE Systems Council provide guidance on best practices and emerging technologies in systems engineering. By adopting a more comprehensive approach, systems analysis can benefit from a broader perspective, enabling more effective solutions to complex, large-scale problems.



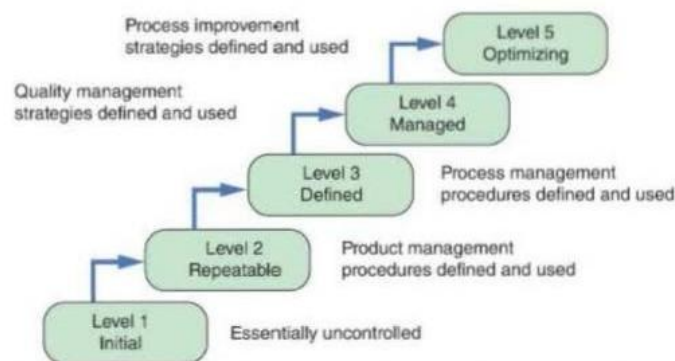
### Annual SEI Architecture Technology User Network Conference

As systems grow in complexity, architecture's role becomes increasingly important at the enterprise, systems, and software levels. Architecture practitioners rely on technology, research, and the knowledge and experience of peers to build predictable, high-quality systems.

The SATURN Conference brings together an international audience of practicing software architects, industry thought leaders, developers, technical managers, and researchers to share ideas, insights, and experience about effective architecture-centric practices for developing and maintaining software-intensive systems.

**FIGURE 11-1** The Software Engineering Institute at Carnegie Mellon University has had profound influence on software engineering research and practice.

Source: Carnegie Mellon University



**FIGURE 11-2** The CMM has five maturity levels, from Level 1 (initial), which is essentially uncontrolled development, to Level 5 (optimizing), in which process improvement strategies are defined and used.

Source: Scott Tilley

(Tilley and Rosenblatt, 2024)

## WHAT IS SYSTEMS ENGINEERING?

**Systems Engineering** is a discipline focused on designing, creating, and managing complex systems throughout their life cycle. It plays a central role in the development of successful systems by being responsible for defining the system concept, architecture, and design. Systems engineers assess and manage complexity and risk, ensuring that all components of the system work together effectively.

They are responsible for various aspects of system creation, including how to measure whether the deployed system functions as intended. Systems engineering provides the tools, techniques, methods, knowledge, standards, principles, and concepts necessary to successfully launch and operate complex systems. Effective systems engineering practices are key to delivering successful systems that meet their goals and function properly.



**FIGURE 11-3** INCOSE's view of systems engineering.  
Source: INCOSE - International Council on Systems Engineering

(Tilley and Rosenblatt, 2024)

### 5.1.3 International Organization for Standardization

What do automobiles, water, and software have in common? Along with thousands of other products and services, they are all covered by standards from the International Organization for Standardization (ISO), which was discussed in Chapter 9. ISO standards include everything from internationally recognized symbols, such as those shown in Figure 11-4, to the ISBN numbering system that identifies this text. In addition, ISO seeks to offer a global consensus of what constitutes good management practices that can help firms deliver consistently high-quality products and services—including software.

Because software is so important to a company's success, many firms seek assurance that software systems, either purchased or developed in-house, will meet rigid quality standards. In 2014, ISO updated a set of guidelines, called ISO 9000-3:2014, that provided a QA framework for developing and maintaining software.

A company can specify ISO standards when it purchases software from a supplier or use ISO guidelines for in-house software development to ensure that the final result measures up to ISO standards. ISO requires a specific development plan, which outlines a step-by-step process for transforming user requirements into a finished product. ISO standards can be quite detailed. For example, ISO requires that a software supplier document all testing and

maintain records of test results. If problems are found, they must be resolved, and any modules affected must be retested. Additionally, software and hardware specifications of all test equipment must be documented and included in the test records.



**FIGURE 11-4** ISO symbols include internationally recognized symbols.  
bytedust/Shutterstock.com

(Tilley and Rosenblatt, 2024)

## 5.2 APPLICATION DEVELOPMENT

Application development refers to the process of building the programs and code modules that make up the core of an information system. As explained in Chapter 1, there are several development methodologies, including structured analysis, object-oriented (OO) analysis, and agile methods. Regardless of the chosen approach, the ultimate goal is to translate the system design into executable programs and code modules that work as intended.

In application development, even small projects can involve hundreds or even thousands of modules, making the process complex and difficult to manage. This is why effective project management becomes crucial during this phase to control schedules and budgets.

Users and managers are eager to see the new system implemented, so it is critical to set realistic schedules, meet project deadlines, control costs, and ensure quality. To achieve these goals, project managers or systems analysts should use project management tools and techniques, similar to those discussed in Chapter 3, to monitor and manage the development process.

### 5.2.3 Review the System Design

At this point, it is helpful to revisit the tasks involved in creating the system design:

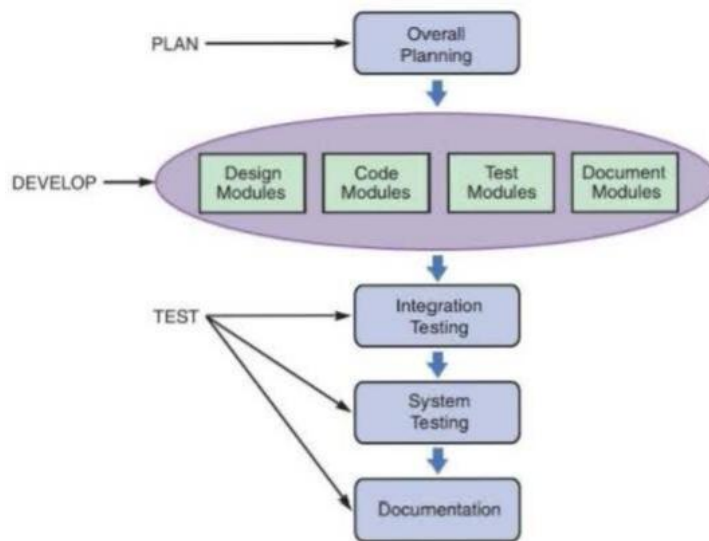
- **Chapter 4:** Focused on requirements modeling and functional decomposition diagrams (FDDs) to break down complex business operations into smaller, manageable functions.
- **Chapter 5:** Covered structured data and process modeling, including data flow diagrams (DFDs), and the creation of process descriptions for documenting business logic and processing requirements.
- **Chapter 6:** Focused on the object-oriented (OO) model of the new system, which included use case diagrams, class diagrams, sequence diagrams, state transition diagrams, and activity diagrams.
- **Chapter 7:** Addressed the selection of a development strategy.
- **Chapter 8:** Focused on designing the user interface.
- **Chapter 9:** Discussed data design issues, analyzing relationships between system entities, and creating entity-relationship diagrams (ERDs).
- **Chapter 10:** Covered considerations for overall system architecture.

Together, these tasks contributed to creating a comprehensive design and implementation plan for the system.

### 5.2.4 Application Development Tasks

Once system design is completed, the next step is to begin translating the design into a functioning application. The process differs depending on the methodology used:

- **Traditional Methods:** In traditional structured or OO methods, development begins after system design is completed. The modules, which are related units of program code, are then designed, coded, tested, and documented. After individual modules are developed and tested, further integration testing is done, and the system is thoroughly documented.
- **Agile Methods:** If an agile development method is chosen, the development starts with planning the project, laying the groundwork, assembling the team, and preparing to interact with customers. This iterative process focuses on quick, incremental development cycles, allowing for continuous feedback and adaptation throughout the project.



**FIGURE 11-5** The main steps in traditional application development.

(Tilley and Rosenblatt, 2024)

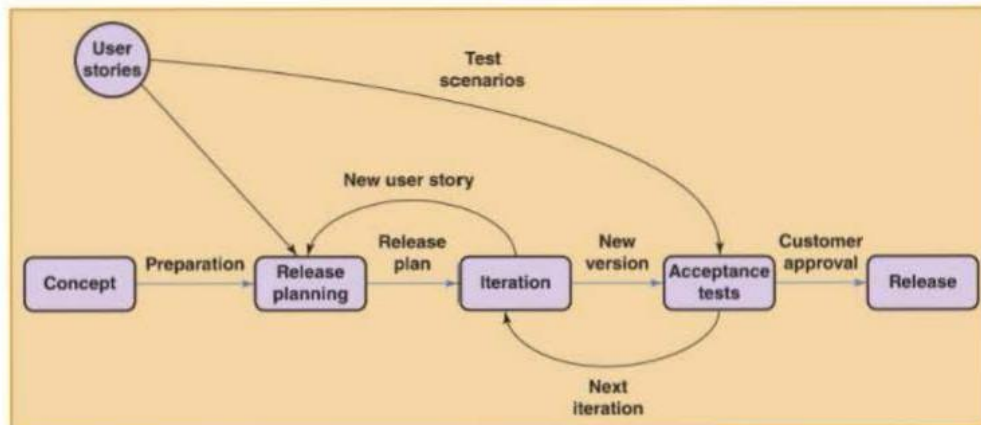
When program modules are created using structured or object-oriented (OO) methods, the process begins with reviewing the requirements documentation from previous phases of the Systems Development Life Cycle (SDLC). A valuable repository of information can be found if the documentation file was created early on and regularly updated. The centerpiece of this documentation is the system design specification, accompanied by diagrams, source documents, screen layouts, report designs, data dictionary entries, and user feedback. If a CASE (Computer-Aided Software Engineering) tool was used during systems analysis and design, the analyst's job will be made easier. From here, coding and testing tasks begin. While programmers typically write the actual code, IT managers often assign systems analysts to work with them as a team.

## AGILE METHODS

If an agile approach is selected, intense communication and collaboration between the IT team and the users or customers begins. The goal is to create the system using an iterative process of planning, designing, coding, and testing. Agile projects often use iterative and incremental models, including Extreme Programming (XP), as shown in Figure 11-6. Agile development and XP are explored in more detail later in this chapter.

### 5.2.5 Systems Development Tools

Different systems development approaches come with their own sets of tools that work well with their respective methodologies. For example, structured development heavily relies on Data Flow Diagrams (DFDs) and structure charts. OO methods utilize various Unified Modeling Language (UML) diagrams, including use case, class, sequence, and state transition diagrams. Agile methods typically use spiral or other iterative models like the one shown in Figure 11-6.



**FIGURE 11-6** Simplified model of an XP project. Note the emphasis on iteration and testing.

(Tilley and Rosenblatt, 2024)

In addition to methodology-specific tools, developers also have access to multipurpose tools that help translate system logic into functioning program modules. These generic tools include:

### ENTITY-RELATIONSHIP DIAGRAMS (ERDs)

During data design (Chapter 9), ERDs were discussed as a way to show how system entities and objects interact. ERDs are useful regardless of the methodology employed, as they help map out the relationships between entities (such as one-to-one, one-to-many, and many-to-many), which must be implemented in the application development process.

### FLOWCHARTS

As discussed in Chapter 5, flowcharts are useful for representing program logic and are especially helpful in visualizing modular design. Flowcharts graphically depict logical rules and interactions, using symbols connected by arrows. By using flowcharts, developers can break large systems into smaller subsystems and modules that are easier to understand and code.

### PSEUDOCODE

Pseudocode is a technique used to represent program logic in a simple and structured form of English. Unlike programming languages, pseudocode is not language-specific, allowing it to describe program actions in plain language without requiring strict syntax. This tool allows both systems analysts and programmers to describe software modules that can be implemented in any programming language. Figure 11-7 provides an example of pseudocode for a sales promotion policy.

### DECISION TABLES AND DECISION TREES

As explained in Chapter 5, decision tables and decision trees can be used to model the business logic of an information system. In addition to serving as modeling tools, decision tables and decision trees can be used during the system development process as code

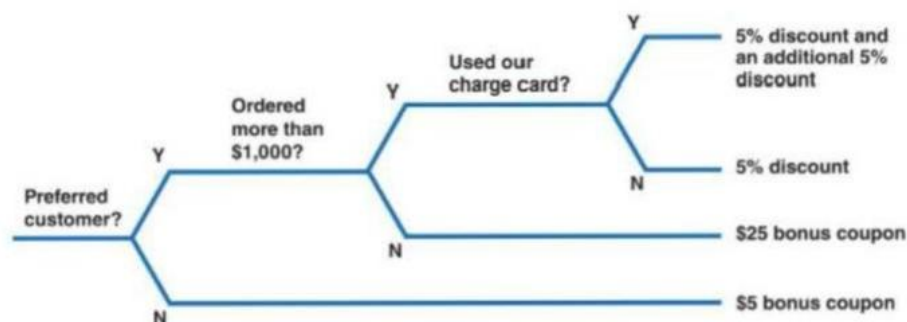
modules that implement the logical rules. Figure 11-8 shows an example of a decision tree that documents the sales promotion policy shown in Figure 11-7. The decision tree accurately reflects the sales promotion policy, which contains three separate conditions and four possible outcomes.

| SAMPLE OF A SALES PROMOTION POLICY |                                                                                                                                            |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| •                                  | Preferred customers who order more than \$1,000 are entitled to a 5% discount, and an additional 5% discount if they used our charge card. |
| •                                  | Preferred customers who do not order more than \$1,000 receive a \$25 bonus coupon.                                                        |
| •                                  | All other customers receive a \$5 bonus coupon.                                                                                            |

| PSEUDOCODE VERSION OF THE SALES PROMOTION POLICY |                           |
|--------------------------------------------------|---------------------------|
| IF customer is a preferred customer, and         |                           |
| IF customer orders more than \$1,000 then        |                           |
| Apply a 5% discount, and                         |                           |
| IF customer uses our charge card, then           |                           |
| Apply an additional 5% discount                  |                           |
| ELSE                                             | Award a \$25 bonus coupon |
| ELSE                                             | Award a \$5 bonus coupon  |

**FIGURE 11-7** Sample of a sales promotion policy with logical rules (top) and a pseudocode version of the same policy (bottom). Note the alignment and indentation of the logic statements in the pseudocode.



**FIGURE 11-8** Sample decision tree that reflects the sales promotion policy in Figure 11-7. Like a decision table, a decision tree shows the action to be taken based on certain properties.

(Tilley and Rosenblatt, 2024)

### 5.3 STRUCTURED DEVELOPMENT

Structured application development typically follows a top-down approach, which begins with a general design and progresses to a more detailed structure. After a systems analyst documents the system's requirements, they break the system down into subsystems and modules in a process called partitioning. This approach, also referred to as modular design, is similar to constructing a hierarchical set of Data Flow Diagrams (DFDs). By assigning modules to different programmers, multiple areas of development can progress simultaneously. As discussed in Chapter 3, project management software can help monitor work on each module, estimate overall development time, forecast the required human and technical resources, and calculate a critical path for the project.

Since all the modules must function together correctly, the analyst must proceed carefully, with continuous input from programmers and IT management to ensure a solid, well-

integrated structure. The analyst also needs to make sure that integration capability is built into each design and thoroughly tested.

### 5.3.3 Structure Charts

Structure charts illustrate the program modules and their relationships. A structure chart consists of rectangles that represent program modules, along with arrows and other symbols that provide additional information. Generally, a higher-level module, known as a control module, oversees lower-level modules, which are referred to as subordinate modules. The structure chart uses symbols to represent various actions or conditions.

- **MODULE:** A rectangle symbolizes a module, as shown in Figure 11-9. In this figure, vertical lines at the edges of the rectangle indicate that module 1.3 is a *library module*. A library module contains reusable code that can be invoked from multiple points within the chart.
- **DATA COUPLE:** An arrow with an empty circle represents a *data couple*. A data couple indicates the data passed from one module to another. For example, in Figure 11-10, the "Look Up Customer Name" module exchanges data with the "Maintain Customer Data" module via a data couple.

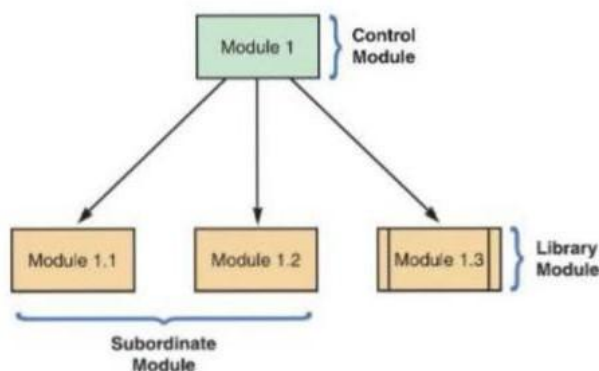


FIGURE 11-9 An example of structure chart modules.

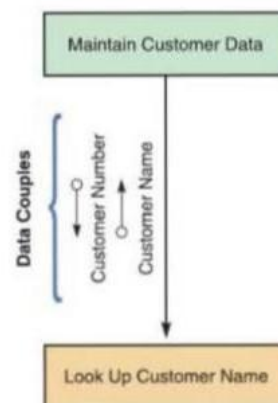


FIGURE 11-10 An example of a structure chart data couple.

**CONTROL COUPLE:** An arrow with a filled circle represents a *control couple*. A control couple conveys a message, also known as a *status flag*, from one module to another. In the example shown in Figure 11-11, the "Update Customer File" module sends an "Account Overdue" flag back to the "Maintain Customer Data" module. The purpose of this flag is to signal a specific condition or action that another module should respond to. Control couples are useful for indicating when certain conditions are met or when certain actions need to be taken in the system.

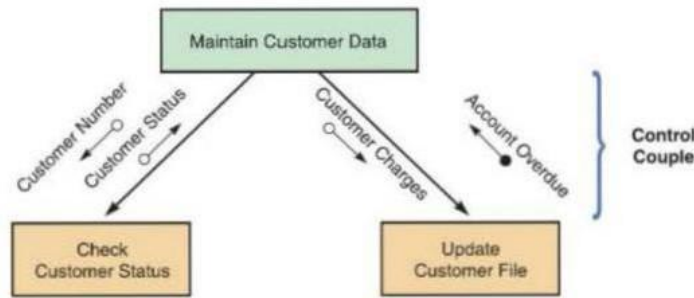


FIGURE 11-11 An example of a structure chart control couple.

(Tilley and Rosenblatt, 2024)

**CONDITION:** A line with a diamond shape at one end represents a *condition*. This line indicates that a control module decides which subordinate modules will be triggered based on a specific condition. In the example shown in Figure 11-12, "Sort Inventory Part" is a control module that uses a condition line to determine which of the three subordinate modules to invoke, depending on the condition. The condition line helps manage decision-making within the structure, ensuring that the correct module is executed based on certain criteria.

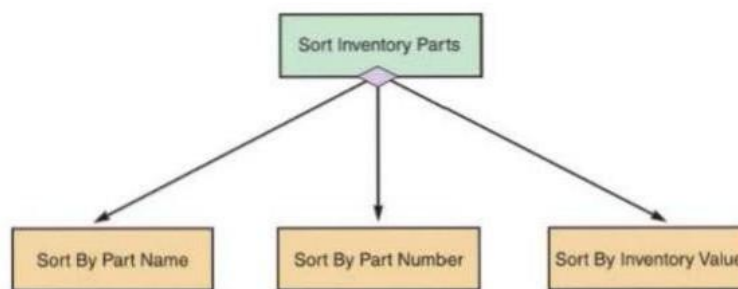
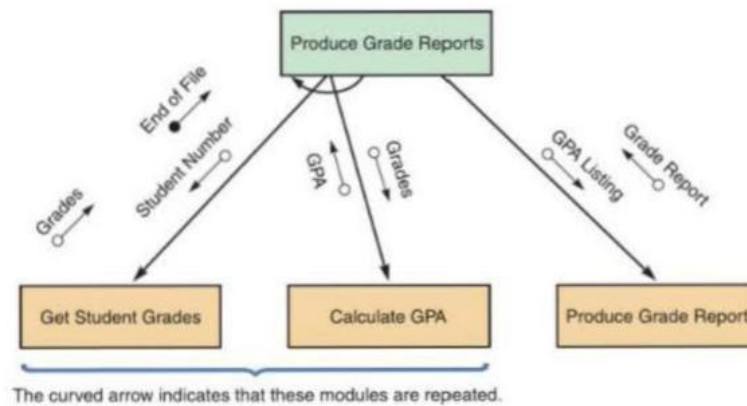


FIGURE 11-12 The diagram shows a control module that triggers three subordinate modules.

(Tilley and Rosenblatt, 2024)

**LOOP:** A curved arrow represents a *loop*. A loop signifies that one or more modules are executed repeatedly. In the example shown in Figure 11-13, the "Get Student Grades" and "Calculate GPA" modules are repeated as part of the loop. The loop allows for repetitive execution of certain processes, such as processing multiple records or performing iterative calculations until a specific condition is met.



**FIGURE 11-13** The diagram shows a structure chart loop with two repeating modules.

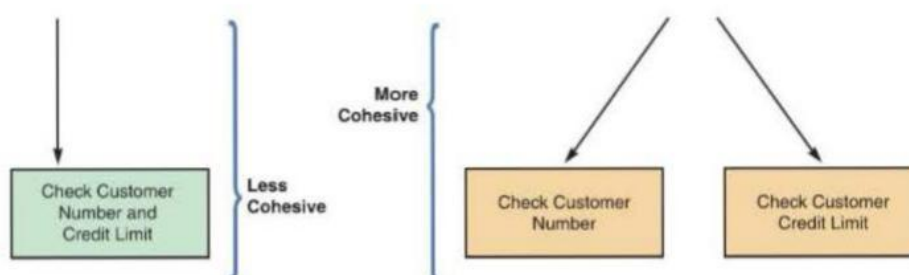
(Tilley and Rosenblatt, 2024)

### 5.3.4 Cohesion and Coupling

Cohesion and coupling are important tools for evaluating the overall design. As explained in the following sections, it is desirable to have modules that are highly cohesive and loosely coupled. Otherwise, system maintenance becomes more costly due to difficulties in making changes to the system's structure.

**Cohesion** measures a module's scope and processing characteristics. A module that performs a single function or task has a high degree of cohesion, which is desirable. Because it focuses on a single task, a cohesive module is much easier to code and reuse. For example, a module named *Verify Customer Number* is more cohesive than a module named *Calculate and Print Statements*. If the word *and* is found in a module name, this implies that more than one task is involved.

If a module must perform multiple tasks, more complex coding is required, and the module will be more difficult to create and maintain. To make a module more cohesive, split it into separate units, each with a single function. For example, by splitting the module *Check Customer Number and Credit Limit* in Figure 11-14 into two separate modules—*Check Customer Number* and *Check Customer Credit Limit*—cohesion is greatly improved.



**FIGURE 11-14** Two examples of cohesion. Note that the single module on the left is less cohesive than the two modules on the right.

### 5.3.2 Coupling

Coupling describes the degree of interdependence among modules. Modules that are independent are loosely coupled, which is desirable. Loosely coupled modules are easier to maintain and modify because the logic in one module does not affect other modules. If a programmer needs to update a loosely coupled module, they can accomplish the task in a single location.

If modules are tightly coupled, one module is linked to internal logic contained in another module. For example, Module A might refer to an internal variable contained in Module B. In that case, a logic error in Module B will affect the processing in Module A. For that reason, passing a status flag down as a message from a control module is generally regarded as poor design. It is better to have subordinate modules handle processing tasks as independently as possible, to avoid a cascade effect of logic errors in the control module.

In **Figure 11-15**, the tightly coupled example on the left shows that the subordinate module *Calculate Current Charges* depends on a status flag sent down from the control module *Update Customer Balance*. It would be preferable to have the modules loosely coupled and logically independent. In the example on the right, a status flag is not needed because the subordinate module *Apply Discount* handles discount processing independently. Any logic errors are confined to a single location: the *Apply Discount* module.

### 5.3.3 Drawing a Structure Chart

If a structured analysis method was used during system design, the structure charts will be based on the DFDs created during data and process modeling.

Typically, three steps are followed when creating a structure chart:

1. **Review DFDs:** Identify the processes and methods.
2. **Identify Program Modules:** Determine control-subordinate relationships.
3. **Add Symbols for Couples and Loops:** Include the necessary symbols to represent data and control couples, as well as loops.

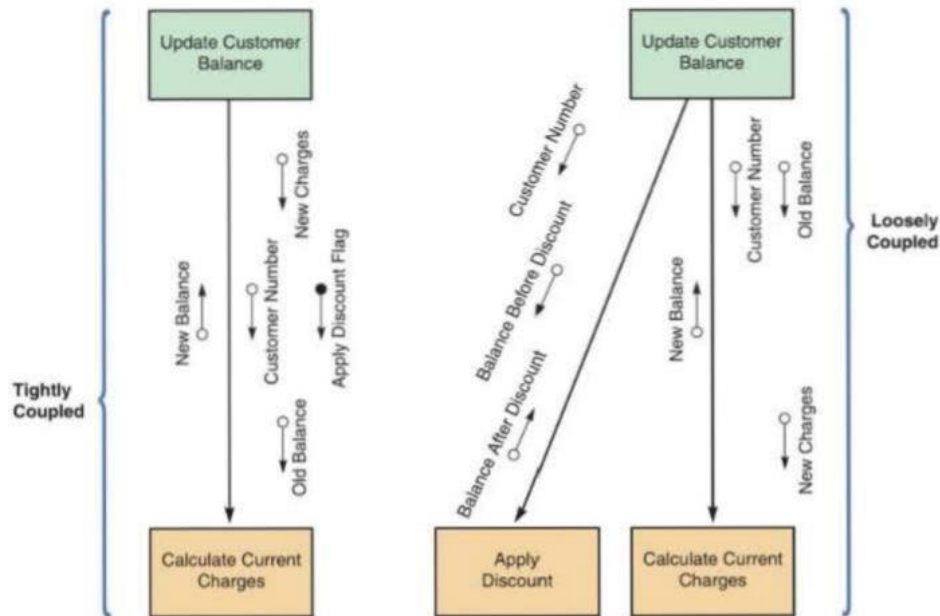


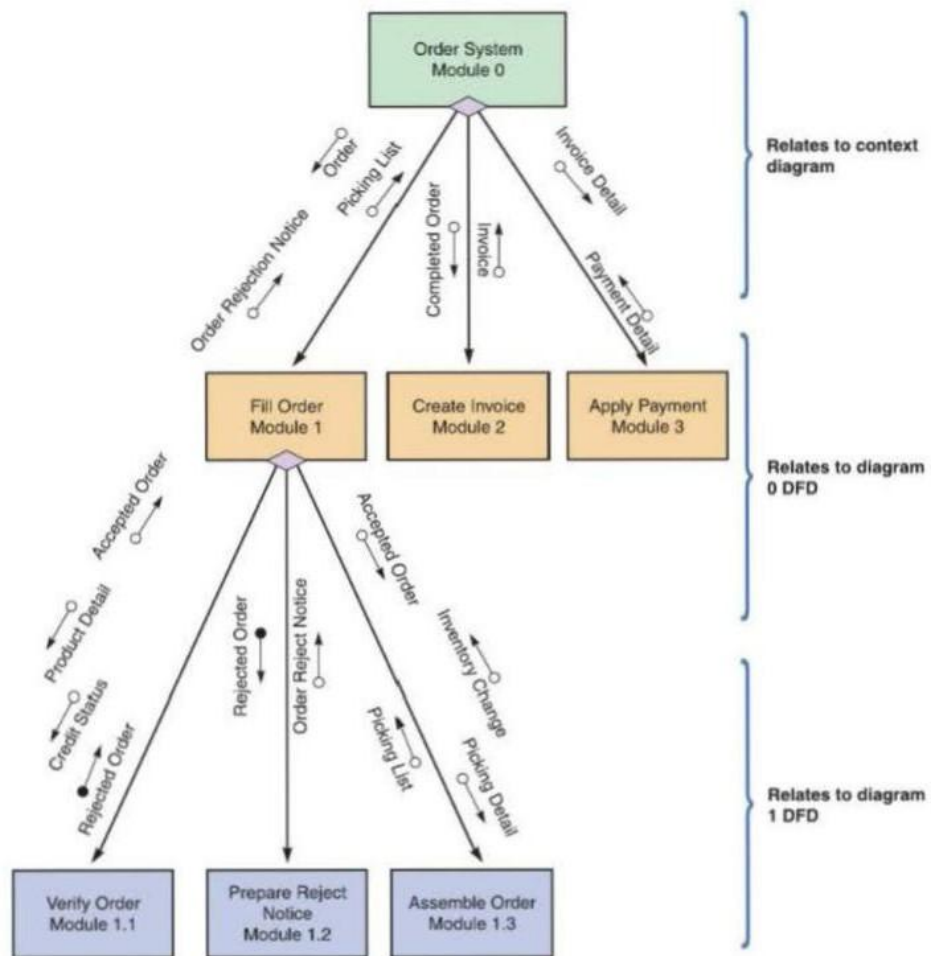
FIGURE 11-15 An example of tightly coupled and loosely coupled structure charts.

(Tilley and Rosenblatt, 2024)

**STEP 1. REVIEW THE DFDs:** The first step is to review all DFDs (Data Flow Diagrams) for accuracy and completeness, especially if changes have occurred since the systems analysis phase. If object models were also developed, they should be analyzed to identify the objects, the methods that each object must perform, and the relationships among the objects. A method is similar to a functional primitive and requires code to implement the necessary actions.

**STEP 2. IDENTIFY MODULES AND RELATIONSHIPS:** Working from the logical model, functional primitives or object methods are transformed into program modules. When analyzing a set of DFDs, remember that each DFD level represents a processing level. If DFDs are being used, one works way down from the context diagram to the lower-level diagrams, identifying control modules and subordinate modules until the functional primitives are reached. If more cohesion is desired, processes can be divided into smaller modules that handle a single task. **Figure 11-16** shows a structure chart based on the order system from Chapter 5. Note how the three-level structure chart relates to the three DFD levels.

**STEP 3. ADD COUPLES, LOOPS, AND CONDITIONS:** Next, couples, loops, and conditions are added to the structure chart. If DFDs are being used, the data flows and the data dictionary can be reviewed to identify the data elements that pass from one module to another. In addition to adding the data couples, control couples are added where a module is sending a control parameter, or flag, to another module. Loops and condition lines that indicate repetitive or alternative processing steps are also added, as shown in **Figure 11-16**. If an object model was developed, the class diagrams and object relationship diagrams can be reviewed to ensure that the interaction among the objects is fully understood.



**FIGURE 11-16** A structure chart based on the order system DFDs in Chapter 5. The three-level structure chart relates to the three DFD levels.

(Tilley and Rosenblatt, 2024)

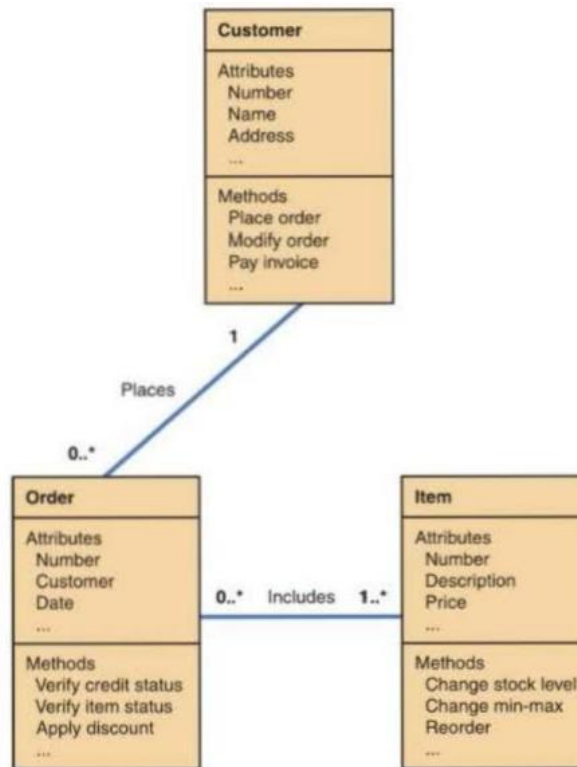
At this point, the structure chart is ready for careful analysis. Each process, data element, or object method should be checked to ensure that the chart accurately reflects all previous documentation and that the logic is correct. All modules should be strongly cohesive and loosely coupled. This step might require drawing several versions of the chart to fine-tune the design and ensure its effectiveness. Additionally, some CASE (Computer-Aided Software Engineering) tools can assist in analyzing the chart and identifying potential problem areas. These tools can help streamline the process by highlighting design flaws or inconsistencies that may require further attention.

## 5.4 OBJECT-ORIENTED DEVELOPMENT

Object-Oriented Development (OOD) is a methodology that builds on the principles of object-oriented analysis (OOA). As discussed in Chapter 6, OOA focuses on organizing the system into objects that contain both data and methods (program logic). When implementing an object-oriented design, the relationships and interactions between these objects are already defined, unlike in structured design where a structure chart is used to describe the interaction between program modules. In OOD, the object model itself represents the structure of the application.

In object-oriented design, classes are used to group similar objects together. Each class contains attributes, which define the characteristics of the objects, and methods, which define the behavior or actions that the objects can perform. For example, the *Customer* class might have attributes like *Number*, *Name*, and *Address*, as well as methods such as *PlaceOrder*, *ModifyOrder*, and *PayInvoice*. The interactions between these classes are represented using class diagrams, which also show how different classes, like *Customer* and *Order*, communicate with each other.

This design approach offers a more natural mapping to real-world systems, and the object interactions defined during the OOA phase simplify the translation into a programming language. The relationships between objects are well-defined, enabling a clearer and more maintainable structure than traditional modular design methods.



**FIGURE 11-17** A simplified class diagram for a customer order processing system.

(Tilley and Rosenblatt, 2024)

In addition to class diagrams, object-oriented development often involves using object relationship diagrams to provide a clear overview of object interactions. These diagrams were created during the object-oriented analysis phase and help to visually represent how objects within the system interact to carry out business functions and processes.

For example, an object relationship diagram for a fitness center might illustrate different objects, such as *Customer*, *Membership*, *Trainer*, *Workout*, and *Payment*. These objects would interact with one another to support processes such as membership sign-ups, workout scheduling, payment processing, and tracking customer progress.

In such a diagram, the relationships between objects, such as *Customer* interacting with *Membership* to register or *Trainer* interacting with *Workout* to create a training plan, are clearly shown. This diagram helps developers understand the overall flow of information and actions within the system, ensuring that object interactions are well-structured and aligned with the business goals.

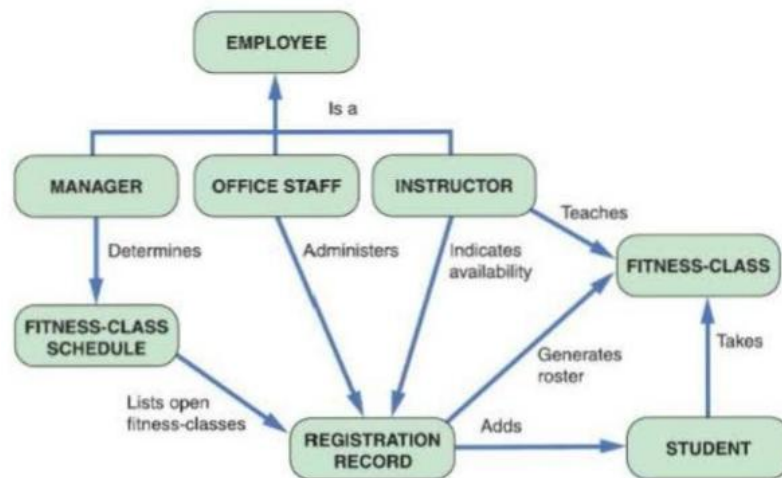


FIGURE 11-18 An object-relationship diagram for a fitness center.

(Tilley and Rosenblatt, 2024)

In object-oriented development (OOD), the implementation of the design must be handled carefully to fully realize its potential benefits, such as reduced costs, faster development times, and improved overall quality. However, achieving these benefits requires a thoughtful approach and sufficient preparation. Many organizations set unrealistic expectations, skipping the necessary steps in analyzing, preparing for, and executing the OOD process, which can lead to disappointing results.

Just as you would never build a bridge without a thorough analysis and blueprint, OOD projects require careful planning and attention to detail throughout the process. This includes proper analysis of requirements, careful design, thorough implementation, and exhaustive testing. Without this attention to detail, the resulting system may fail to meet the intended specifications, and maintenance can become difficult and costly.

### 5.4.2 Implementation of Object-Oriented Designs

When translating an object-oriented design into a working application, programmers focus on the elements described in the object model, such as classes, attributes, methods, and messages. The key task is to analyze the design documents, including class diagrams, sequence diagrams, state transition diagrams, and activity diagrams, to ensure that each part of the design is properly translated into code.

The programmer must pay special attention to the event-driven nature of OOD applications. Every event, transaction, or message triggers an action, and this flow of events is critical in ensuring the system behaves as expected. Sequence and state transition diagrams play an important role in understanding the event flow and identifying the necessary actions. These event-driven systems are often built using pseudocode initially, or by leveraging CASE tools and code generators that can automatically create code based on the object model.

### 5.4.3 Object-Oriented Cohesion and Coupling

Cohesion and coupling are just as crucial in object-oriented development as they are in traditional development approaches. Classes should be designed to be as **loosely coupled** as possible, meaning they should operate independently from one another. Additionally, an object's **methods** should also be loosely coupled and highly cohesive. This means that each method should perform a specific set of closely related tasks, and the methods should interact minimally with methods of other objects.

By following these principles, the object model remains modular, easy to understand, and easy to modify. When cohesion and coupling are neglected, the codebase becomes tangled and difficult to maintain. A lack of independence among classes and methods makes it harder to make changes, which can lead to higher costs and greater complexity over time.

In summary, object-oriented development is a powerful methodology that can significantly enhance the efficiency and maintainability of software systems. However, to ensure its effectiveness, it's essential to rigorously follow the principles of OOD, including designing with high cohesion and low coupling, translating the design properly into code, and maintaining clear communication throughout the development process.

## 5.5 AGILE DEVELOPMENT

As mentioned in Chapter 1, agile development is a unique approach to systems development. While it follows many of the same steps as traditional development, it relies on a highly iterative process. The development team maintains continuous communication with the primary user, referred to as the customer, to refine and shape the system according to the customer's needs. Agile development is named for its quick and flexible nature, allowing it to easily adapt to changes. The method emphasizes small teams, frequent communication, and fast-paced development cycles. The four core principles of agile software development are illustrated in Figure 11-19.



**FIGURE 11-19** The manifesto for agile software development.  
Source: Agile Software Development.

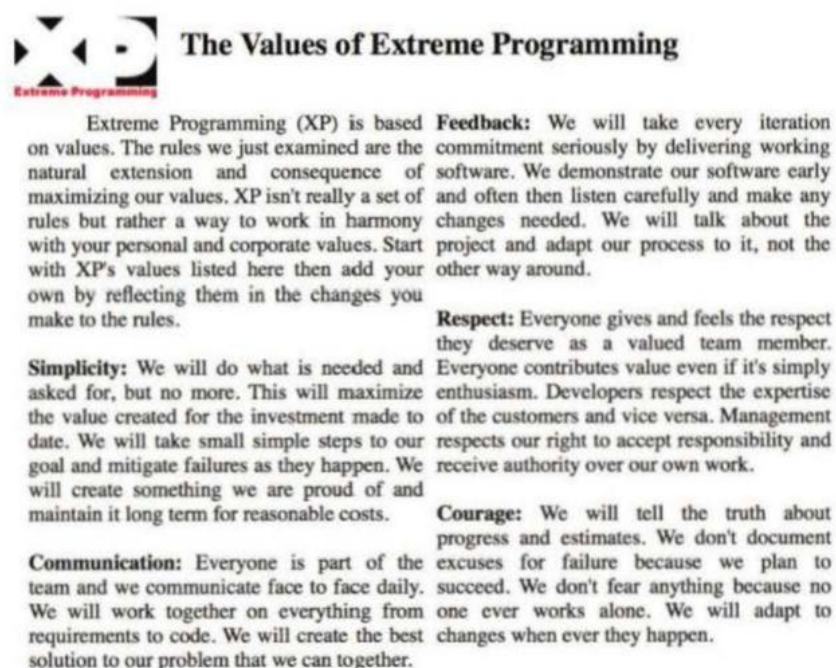
(Tilley and Rosenblatt, 2024)

Programmers can utilize popular programming languages that are agile-friendly, such as Python, Ruby, and Perl. However, agile methods do not mandate a specific language, and developers can also use various object-oriented languages like Java, C++, C#, and others. Like traditional methodologies, agile development has both advantages and disadvantages. Currently, agile methodology is widely adopted in software projects. Its advocates claim it accelerates development, delivers exactly what the customer wants when needed, encourages teamwork, and empowers employees. However, critics argue that its emphasis on quick iterations and rapid releases can lead to a lack of discipline and result in systems of questionable quality. Moreover, agile methods may not be suitable for larger, more complex projects due to their focus on flexibility rather than a clearly defined final product.

Before adopting agile development, it's crucial to carefully assess the proposed system and development approach. As experienced IT professionals understand, there is no one-size-fits-all solution. For more details on agile methods, refer to the discussion of systems development methods in Chapter 1 and agile methods like Scrum in Chapter 4.

### 5.5.3 Extreme Programming (XP)

XP is an agile development method that follows an iterative approach, as shown in Figure 11-20. In XP, a team of users and developers immerse themselves in system development. XP emphasizes values such as simplicity, communication, feedback, respect, and courage. Achieving success requires strong commitment to the process, corporate support, and dedicated team members.



**FIGURE 11-20** The five core values of XP.  
Source: ©2009, Don Wells

(Tilley and Rosenblatt, 2024)

XP introduces a practice called **pair programming**, where two programmers collaborate on the same task using the same computer. One programmer, known as the **driver**, writes the code, while the other, called the **navigator**, observes and provides strategic guidance. The navigator focuses on the big picture, seeing the overall structure and potential issues, while the driver is focused on writing the specific lines of code. Throughout the process, the two programmers engage in continuous discussions, sharing ideas and insights.

Another key concept in XP is **test-driven development** (TDD), where unit tests are created before any code is written. This approach ensures that the development process stays focused on the intended outcomes right from the start, helping prevent programmers from deviating from the project's goals. Given the fast-paced and iterative nature of agile development, testing in XP heavily relies on automated testing methods to keep the process efficient and manageable.

#### 5.5.4 User Stories

In agile development, the customer initiates the process by providing **user stories**. A user story is a brief, simple description of a system requirement from the perspective of the user. These stories help the development team understand the project's requirements, priorities, and scope. They do not include technical details and are typically written on index cards. Here are a few examples of user stories:

- **Sales Manager:** "I want to identify fast- or slow-moving items so I can manage our inventory more effectively."
- **Store Manager:** "I need enough lead time to replenish my stock, so I don't run out of hot items."
- **Sales Representative:** "I want to offer the best selection of fast-selling items and clear out the old stock that is not moving."

Each user story is prioritized by the customer, and programmers assess its complexity by assigning a score that estimates the difficulty of implementing the requirement. These user stories collectively form **epics**, which are broader sets of functionality that help in estimating the overall scope, time, and difficulty of the project. As the project progresses, frequent meetings with the customer help refine the user stories and add more detail.

#### 5.5.5 Iterations and Releases

Along with defining user stories, the development team must create a **release plan** that outlines when the user stories will be implemented and when the system will be released. Each release typically functions as a prototype that can be tested and adjusted as needed.

**Iterations** are the cycles during which user stories are implemented. Each iteration cycle typically lasts about two weeks and involves several stages: planning, designing, coding, and testing. At the start of each iteration, the team conducts a planning meeting to break down user stories into specific tasks, which are assigned to team members. If new user stories are added, the release plan is updated accordingly.

The success of the project is determined by the customer's approval. The team frequently meets with the customer to present prototype releases, and the customer tests these versions as they become available. As the project progresses, the customer may suggest additional user stories or changes, which are addressed in subsequent iteration cycles. Obsolete code is removed, and the remaining code is restructured to ensure the system is up to date. This iterative process continues until all user stories have been implemented, tested, and accepted by the customer.

## 5.6 CODING

### Coding Process and Tools

**Coding** is the process of converting program logic into executable instructions that a computer can understand. Based on a specific design, a programmer uses a programming language to transform the logic into code. For small programs, a single developer can handle the coding, while larger programs are typically divided into **modules** that multiple individuals or teams can work on simultaneously.

Each programmer has their own preferred development environment and coding standards to make the process more efficient. Commonly used programming languages include **Visual Basic**, **Java**, and **Python**. Additionally, with the rise of **Internet-based** and **mobile applications**, web-centric languages like **HTML**, **XML**, **JavaScript**, and **Swift** have become increasingly important.

To streamline the integration of system components and reduce coding time, many programmers use an **Integrated Development Environment (IDE)**. IDEs simplify the process of developing software by providing various built-in tools and features such as real-time error detection, syntax highlighting, code navigation (class browsers), and version control. Examples of popular commercial IDEs include **Apple Xcode** and **Microsoft .NET**. Additionally, open-source IDEs like **Eclipse** are also widely used.

Earlier chapters highlighted how **systems analysts** utilize **application generators**, **report writers**, **screen generators**, **fourth-generation languages**, and other **CASE (Computer-Aided Software Engineering) tools** to produce code directly from design specifications. Certain commercial applications can generate editable code based on user actions such as **macros**, **keystrokes**, or **mouse clicks**. For example, **IBM's Rational tools** can generate code fragments from **UML design documents**, which helps improve **quality assurance (QA)** by ensuring that design and implementation remain consistent.

## 5.7 TESTING

### Testing Process After Coding

After coding a program, the next critical step is **testing** to ensure that it functions correctly. This process is carried out in stages: initially, individual programs are tested, then modules are tested together, and eventually, the entire system is tested.

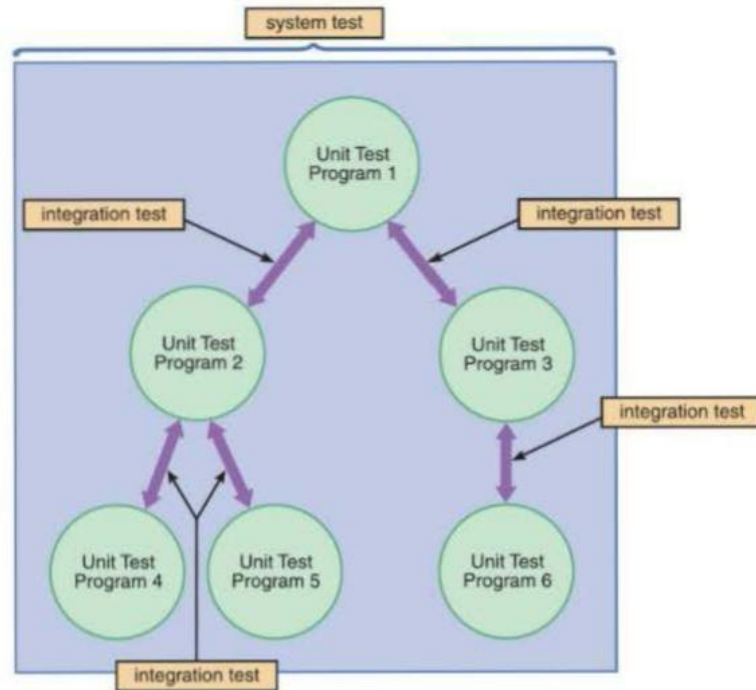
1. **Compiling and Syntax Checking:** The first step is to compile the program using a **CASE tool** or a **language compiler**. This identifies **syntax errors**, which are errors in the language grammar. The programmer works to fix these errors until the program compiles and executes correctly.
2. **Desk Checking:** Next, the programmer performs **desk checking**, which involves manually reviewing the program code to identify **logic errors** that could lead to incorrect results. This step can be carried out by the original programmer or other team members. Many organizations formalize this step into a **structured walk-through** or **code review**.

In a **code review**, a group of typically **three to five IT staff members** participates in a meeting where the group carefully examines the code. This peer group usually includes the project team, along with other programmers and analysts who were not involved in the project. The goal is to identify errors, apply quality standards, and ensure the program aligns with the design specifications. Errors discovered at this stage are easier to fix because the program is still in the development phase.

3. **Design Walk-through:** In addition to reviewing the program code, the team typically conducts a **design walk-through** with end-users or a representative group of people who will be interacting with the system. This helps ensure that the user interface is appropriate and that all necessary features are included. This is a continuation of the **modeling** and **prototyping** efforts that started earlier in the systems development process.
4. **Testing Stages:** After the desk checking and code review, the project team moves on to more formal testing phases, starting with **unit testing**, then progressing to **integration testing**, and finally **system testing**, as shown in Figure 11-21.

### 5.7.3 Unit Testing

**Unit testing** is the process of testing an individual program or module to identify and eliminate **execution errors** that could cause abnormal program termination and **logic errors** that might have been overlooked during the desk checking phase. This ensures that each part of the program works as expected before integration with other parts of the system.



**FIGURE 11-21** The first step in testing is unit testing, followed by integration testing and then system testing.

(Tilley and Rosenblatt, 2024)

## Test Data

Test data plays a crucial role in identifying both **correct** and **erroneous** data, ensuring the system functions correctly in all possible situations. For instance, if a program expects numeric input within a certain range, the test data should cover:

- Minimum values
- Maximum values
- Out-of-range values
- Alphanumeric characters, to test error handling

Software tools are often employed during testing to identify the location and possible causes of program errors.

## Unit Testing

In **unit testing**, individual programs or modules are tested separately before they are integrated into the system. One common technique used during this phase is **stub testing**. In stub testing, programmers simulate other program outcomes or results, displaying messages to indicate whether the program executes successfully. This ensures that individual components work as intended before they are connected to the rest of the system.

Typically, **test data** is created by someone other than the original programmer, often systems analysts, as part of a larger **test plan** that outlines how, when, and by whom testing will be

conducted. This plan also identifies what test data will be used. A **comprehensive test plan** should cover every possible situation the program might encounter.

## Integration Testing

**Integration testing** involves testing programs that depend on each other. For example, if there is a program that checks and validates customer credit status and another that updates the customer master file, **integration testing** ensures the correct transfer of data between these programs. Without integration testing, there's no guarantee that the data passed from one program to another will be correct.

During integration testing, test data must account for both **normal** and **unusual** conditions. For instance, one might test by passing typical records followed by blank or incomplete records to simulate unusual situations.

It's important that integration tests only begin after all unit tests have been completed successfully.

## System Testing

**System testing** is the final stage of testing, where the entire system is tested as a whole. It involves simulating actual operating conditions by entering data (including real data) and performing tasks like queries and report generation. The goal is to verify that the system performs as expected under all conditions. This includes checking:

- All input handling, both valid and invalid
- User interaction with the system
- Proper integration of all system components
- The system's ability to handle expected data volumes efficiently

Commercial software packages also undergo system testing, though unit and integration tests may not be performed in the same manner. Successful system testing is often referred to as **acceptance testing** since it signifies that the system is ready for deployment.

## Challenges and Considerations

- **Testing Effort:** The amount of testing required depends on the project, and decisions must be made carefully. Pressure from users, tight budgets, and management deadlines often push for quick testing and deployment, which can compromise the quality of the final product.
- **Cost of Errors:** Testing helps identify errors early, potentially saving time and money by preventing costly issues later. However, no system is 100% error-free, and some issues may not be discovered until the system is operational. Critical errors affecting data integrity must be addressed immediately, while minor issues (e.g., typographical errors) can be corrected post-deployment.
- **User Expectations:** Users may have differing expectations, with some wanting a completely finished product while others may be willing to accept minor updates after

installation. Ultimately, the decision to proceed with the installation or delay it due to discovered issues is made by management after weighing the pros and cons.

#### CASE IN POINT 11.1: YOUR MOVE, INC.

You work at Your Move, Inc., a large retailer specializing in games of all kinds. The company is in the final stages of developing a new inventory management system, and the IT manager wants you to handle the testing.

"Be sure you put lots of errors into the test data," she said. "Users are bound to make mistakes, and we need to design built-in safeguards that will catch the mistakes, and either fix them automatically or alert the user to the problem."

Of course, this comment makes a lot of sense, but you've never done this before, and you wonder how to proceed. Should you try to invent every possible data error? How will you know that you've thought of every situation that could occur? Consider the problem, develop an approach, and write up your plan in a brief memo.

(Tilley and Rosenblatt, 2024)

## 5.8 DOCUMENTATION

Documentation plays a crucial role in describing an information system and assisting users, managers, and IT staff who need to interact with it. Proper documentation helps minimize system downtime, reduce costs, and accelerate maintenance tasks. For instance, the Rigi research environment, shown in Figure 11-22, can automate the documentation process, allowing software developers to produce accurate and detailed reference materials through in-depth source code analysis.

Effective documentation is vital for the smooth operation and upkeep of a system. It not only supports users but also provides essential information for IT staff who are responsible for system modifications, adding new features, or performing maintenance. The different types of documentation include program documentation, system documentation, operations documentation, and user documentation.



(Tilley and Rosenblatt, 2024)

### 5.8.3 Program Documentation

Program documentation outlines the inputs, outputs, and processing logic for all program modules. This process begins during the systems analysis phase and continues throughout implementation. Analysts typically create overall documentation, such as process descriptions and report layouts, early in the SDLC to guide programmers. These guides ensure that modules are well-supported with clear internal and external comments, making them easier to understand and maintain. A systems analyst typically verifies that the program documentation is both complete and accurate. Additionally, defect tracking software, also known as bug tracking software, is used to document and monitor program defects, code changes, and patch replacements.

### 5.8.4 System Documentation

System documentation explains the functions of the system and how they are implemented. It includes data dictionary entries, DFDs, object models, screen layouts, source documents, and the system's original request. This type of documentation is essential for programmers and analysts who must maintain and support the system. Most of the system documentation is prepared during the analysis and design phases of the SDLC. During implementation, an analyst ensures that prior documentation is complete, accurate, and up-to-date, making necessary updates whenever changes occur (e.g., screen or report modifications).

### 5.8.5 Operations Documentation

If the information system involves a mainframe or centralized servers, documentation must be prepared for the IT group responsible for centralized operations. This type of environment may require scheduling batch jobs or distributing printed reports. The IT operations staff becomes the first point of contact when users encounter system issues. Operations documentation includes all the necessary information for processing and distributing both online and printed output. Key components include:

- Identification of programs, systems analysts, programmers, and the system
- Scheduling for printed output, including report frequency and deadlines
- Input files and their sources, output files and their destinations
- Distribution lists for emails and reports
- Required forms, including online forms
- Error messages, informational messages, and restart procedures
- Special instructions, including security requirements

Operations documentation should be clear, concise, and preferably available online. It is important to involve the operations group early in the SDLC to help identify any issues, ensuring the documentation develops alongside the project.

#### **5.8.6 User Documentation**

User documentation provides instructions for users who interact with the system, including user manuals, help screens, and online tutorials. Programmers or systems analysts typically create program and system documentation, but user documentation requires a different set of skills. Experts in technical writing are needed to develop clear and effective user documentation. This is particularly true for online documentation, which needs to work in coordination with print materials, intranet, and internet-based information.

User documentation cannot be added at the end of the project and must be developed in parallel with the system. This is especially true for online help and context-sensitive help features, which are an integral part of the program and must also be tested. In large companies, a technical support team, including technical writers, may assist in creating user documentation and training materials. Regardless of the delivery method, the documentation must be clear, accessible, and understandable for all users.

#### **Key Components of User Documentation:**

- An overview of the system, detailing major features, capabilities, and limitations
- Descriptions of source documents, including content, preparation, and processing
- An overview of menus, data entry screens, options, and instructions
- Examples of reports regularly generated or available upon request
- Security and audit trail information
- Clarification of responsibilities for specific input, output, or processing tasks
- Procedures for reporting problems or requesting changes
- Examples of exceptions and error situations
- Frequently asked questions (FAQs)

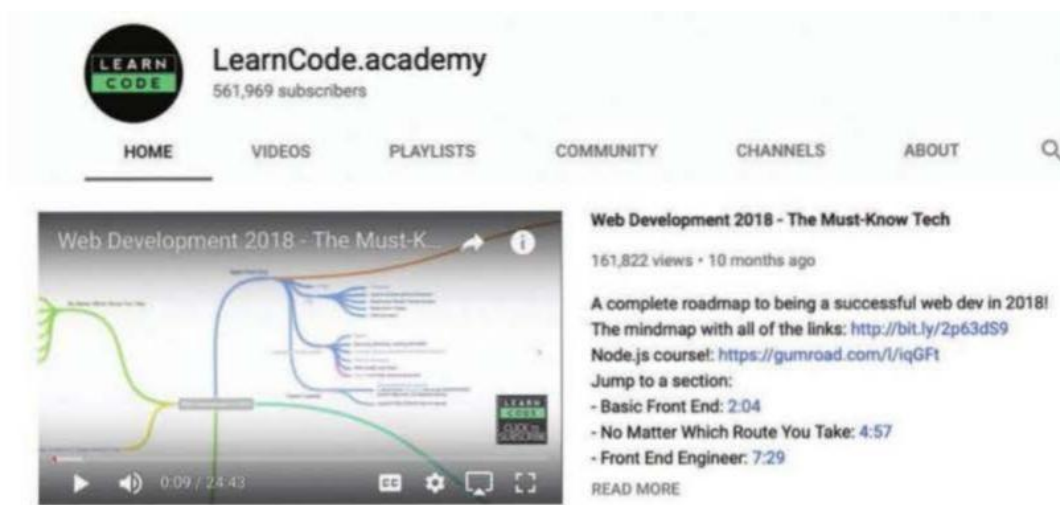
- Guidance on how to get help and update the user manual

### 5.8.7 Online Documentation

With many users preferring immediate help, online documentation has become increasingly important. Context-sensitive help, on-screen demos, hints, and tips are expected by users and are commonly found in popular software packages. For in-house systems, these features should be part of the system requirements. If someone other than the analysts is developing the documentation, they should be involved early on to familiarize themselves with the software and begin creating the necessary materials.

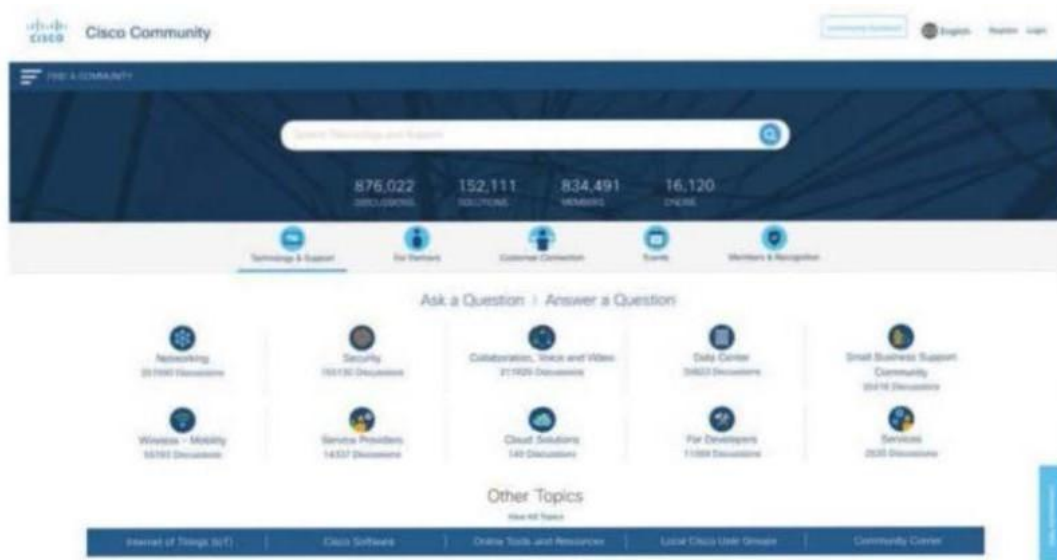
Online documentation can reduce the need for IT staff to assist with issues through telephone, email, or face-to-face support. Interactive tutorials, often using visual media, are especially popular. For example, platforms like YouTube provide accessible tutorials, and websites, intranet sites, and internet-based technical support can offer additional resources.

Though online documentation is essential, written documentation still holds value, especially for training and reference purposes. Systems analysts or technical writers usually prepare the manual, with input from users during the development process. The development of documentation can take time, and it should be planned well in advance to ensure that a complete package, including documentation, is ready for release once the software coding is finished.



**FIGURE 11-23** LearnCode.academy is a popular YouTube channel offering free web development tutorials, website design tutorials, and more.

Source: Learn Code



**FIGURE 11-24** The Cisco Support Community invites users to contribute valuable experience and documentation using social media.

Source: Cisco Support Community

(Tilley and Rosenblatt, 2024)

The completion of the project must include providing user documentation, and addressing this issue from the start of the project is essential. Determining the user documentation requirements early and identifying who will be responsible for creating it are critical for ensuring a timely project release.

If user documentation is neglected until after the program is completed, it often leads to one of two outcomes: (1) the documentation will be rushed and poorly put together just to meet the deadline, which will likely result in inadequacy; or (2) it will be completed properly, but the product release will be significantly delayed.

User training is usually scheduled at the time of system installation. These training sessions provide an ideal opportunity to distribute the user manual and explain the procedures for future updates to the documentation. Training for users, managers, and IT staff will be covered later in this chapter.

## 5.9 INSTALLATION

Once system testing is complete, the results are presented to management. This includes describing the test results, updating the status of all required documentation, and summarizing input from users who participated in the system testing. Detailed time schedules, cost estimates, and staffing requirements for making the system fully operational should also be provided. If the system testing reveals no technical, economical, or operational issues, management will decide on a schedule for system installation.

For every information systems project—whether developed in-house or purchased as a commercial package—the following system installation tasks are performed:

- Preparing a separate operational and test environment
- Performing system changeover
- Conducting data conversion
- Providing training for users, managers, and IT staff
- Carrying out post-implementation tasks

### **5.9.3 Operational and Test Environments**

An environment, or platform, refers to a specific combination of hardware and software. The environment used for the actual system operation is called the *operational environment* or *production environment*. The environment used by analysts and programmers to develop and maintain programs is called the *test environment*.

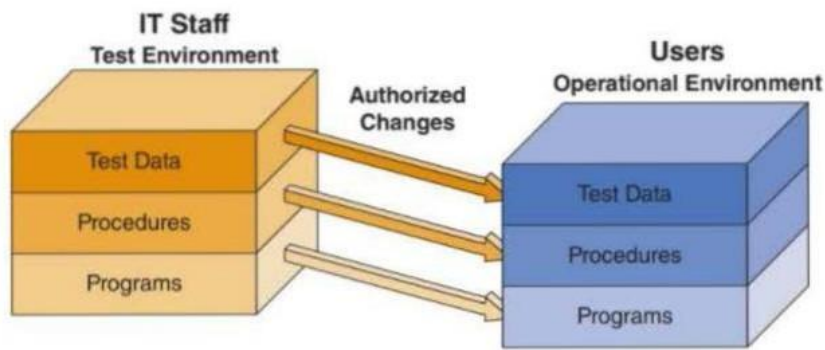
A separate test environment is necessary to maintain system security, integrity, and to protect the operational environment. Typically, the test environment resides on a limited-access workstation or server located in the IT department.

Access to the operational environment is restricted to users and must be strictly controlled. Systems analysts and programmers should not have access to the operational environment except to correct system problems or make authorized modifications. Otherwise, IT department members should not be accessing the day-to-day operational system.

The test environment for an information system contains copies of all programs, procedures, and test data files. Before making any changes to the operational system, the analyst must verify them in the test environment and obtain user approval.

An effective testing process is essential to ensuring product quality. Every experienced systems analyst can recall stories about seemingly innocent program changes that, when introduced without proper testing, ended up causing unexpected problems. After any modification, the same acceptance tests that were run during the system's development should be repeated. By restricting access to the operational environment and conducting all tests in a separate environment, the system can be protected, and issues that could damage data or disrupt operations can be avoided.

The operational environment includes hardware and software configurations, system utilities, telecommunications resources, and any other components that might affect system performance. Since network capabilities are crucial in a client/server environment, connectivity, specifications, and performance must be verified before installing applications. All communication features in the test environment should be carefully checked and rechecked after the applications are loaded into the operational environment. Documentation should specify all network settings, technical, and operational requirements for communication hardware and software. If network resources need to be built or upgraded to support the new system, the platform must be rigorously tested before system installation begins.



**FIGURE 11-25** The test environment versus the operational environment. Note that access to the test environment is limited to IT staff, while the operational environment is restricted to users.

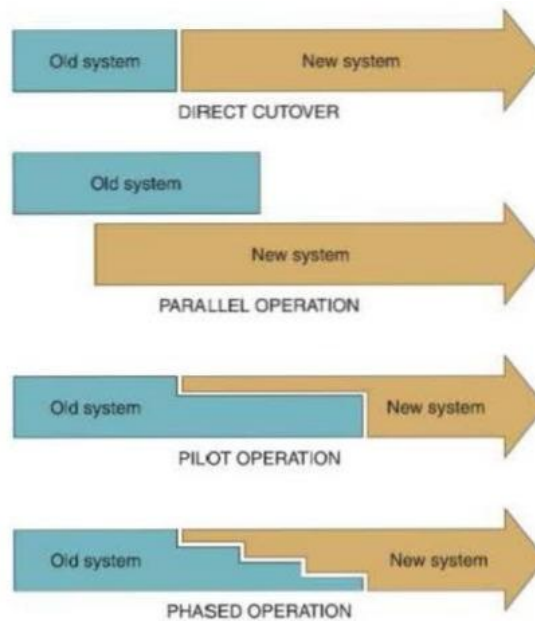
(Tilley and Rosenblatt, 2024)

#### 5.9.4 System Changeover

System changeover is the process of transitioning from the old system to the new one and making the new system operational. The changeover process can either be rapid or gradual, depending on the method chosen. The four primary changeover methods are:

1. **Direct Cutover**
2. **Parallel Operation**
3. **Pilot Operation**
4. **Phased Operation**

Each method has different risk, cost, and timing considerations, and the best choice depends on the specific situation.



**FIGURE 11-26** The four system changeover methods.

(Tilley and Rosenblatt, 2024)

### Direct Cutover

The direct cutover method involves switching from the old system to the new one immediately once the new system becomes operational. This method is typically the least expensive because only one system needs to be operated and maintained at a time.

However, direct cutover carries significant risks. Despite thorough testing and training, problems may arise during actual system operation due to unforeseen data situations or user errors. Since live data is often more complex and larger in volume than test data, these issues can be harder to anticipate.

Detecting errors is also more difficult with direct cutover because there is no old system to compare with the new one to check for discrepancies. Additionally, major system errors could cause the system to terminate abnormally, and with direct cutover, reverting to the old system is not an option.

Although it is risky, companies often choose direct cutover for implementing commercial software packages because these packages are typically less prone to total failure. For systems developed in-house, direct cutover is generally used only for non-critical systems or when the operating environment cannot support both old and new systems.

When using direct cutover, timing is crucial. For example, systems with periodic cycles (weekly, monthly, or yearly) should ideally be switched over at the beginning of a quarter or fiscal year to minimize disruptions.

## **Parallel Operation**

Parallel operation involves running both the old and new systems simultaneously for a specified period. Data is input into both systems, and the output from the new system is compared with the output from the old system. The old system is terminated once the new system is confirmed to be working correctly.

Parallel operation is the safest method because it provides a backup (the old system) if the new system fails. The output from both systems can be compared, making it easier to detect problems.

However, this method is the most expensive. Running both systems incurs the cost of operating and maintaining two systems, and users must work in both systems. In some cases, temporary staff might be required to handle the extra workload. Additionally, running both systems could cause delays in processing.

Parallel operation is not practical if the systems are incompatible or if they perform different functions. It's also inappropriate if the new system changes the business operations significantly.

## **Pilot Operation**

Pilot operation involves implementing the new system at a selected location (e.g., one branch or department) while the old system continues to operate in the rest of the organization. Once the system proves successful at the pilot site, it is rolled out across the rest of the organization, often using direct cutover.

Pilot operation reduces the risk of system failure compared to direct cutover since only a small portion of the company is affected at first. It also allows the system to be tested in a real-world environment without impacting the entire organization. Additionally, the parallel operation at the pilot site is less expensive than running both systems across the entire company.

Once the system is proven successful at the pilot site, it can be fully implemented with minimal changeover time.

## **Phased Operation**

The phased operation method involves implementing the new system in stages or modules. For instance, one part of the system (e.g., materials management) may be implemented first, followed by other parts (e.g., production control). Each module is implemented using one of the other three changeover methods.

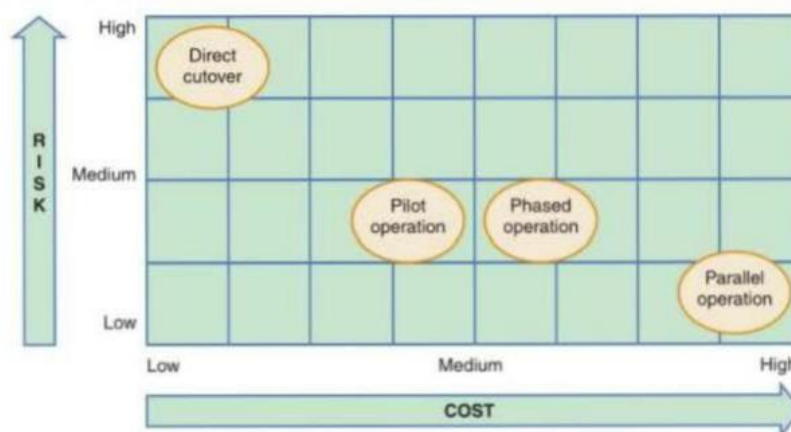
Phased operation limits the risk of failure to the module being implemented at the time. If one module fails, it will not affect the rest of the system. This is a more cost-effective option than full parallel operation since only one part of the system is operational at a time.

However, phased operation is not feasible if the system cannot be divided into logical modules or if the number of modules is large. In these cases, it might become more expensive than pilot operation.

### Choosing the Right Method

Each changeover method has its own set of advantages and disadvantages. A systems analyst must weigh these factors carefully and recommend the best method based on the specific business situation, the system's complexity, the organization's tolerance for risk, and the resources available.

The final decision will be influenced by input from the IT staff, users, and management, with the ultimate goal of minimizing risk while ensuring a smooth transition from the old system to the new one.



**FIGURE 11-27** Relative risk and cost characteristics of the four changeover methods.

### CASE IN POINT 11.2: GLOBAL COOLING

You are a systems analyst at Global Cooling, a leading manufacturer of air conditioning units. You are leading a team that is developing a new production scheduling system. The project is now in the application development stage. You are in the final stages of integration testing. Your supervisor is eager to implement the new application ahead of schedule and asked if you could trim system testing from two weeks to three days and use a direct cutover method instead of the parallel changeover method that originally was planned. Write a brief memo expressing your views.

## 5.9.5 Data Conversion

Data conversion is a crucial part of the system installation process, where data from the old system is transferred to the new system. The conversion process can happen before, during, or after the new system becomes fully operational. It is important to develop a data conversion plan early and test it once the test environment is set up.

When replacing an existing system with a new one, data conversion should ideally be automated. The old system may be able to export data in a format compatible with the new system or use a standard format like A CII or ODBC. ODBC (Open Database Connectivity) is a

widely used industry standard that enables databases from different vendors to communicate and exchange data. Most database vendors provide ODBC drivers to facilitate this communication.

If a standard format is unavailable, a program will need to be created to extract the data and convert it into a suitable format for the new system. Data conversion can be more challenging when transitioning from a manual system, as all data may need to be manually entered unless scanning is used. Even in automated conversions, the new system may require additional data that must be entered manually.

During the conversion process, strict input controls must be in place to protect the data from unauthorized access and prevent incorrect data input. Errors are inevitable despite careful preparation. For example, issues like duplicate customer records or inconsistent data that were tolerated in the old system may cause the new system to fail. Organizations often require that users verify all data, fix errors, and provide any missing information during the conversion. Although this process can be costly and time-consuming, it is essential that the new system is populated with accurate and error-free data.

### **5.9.6 Training**

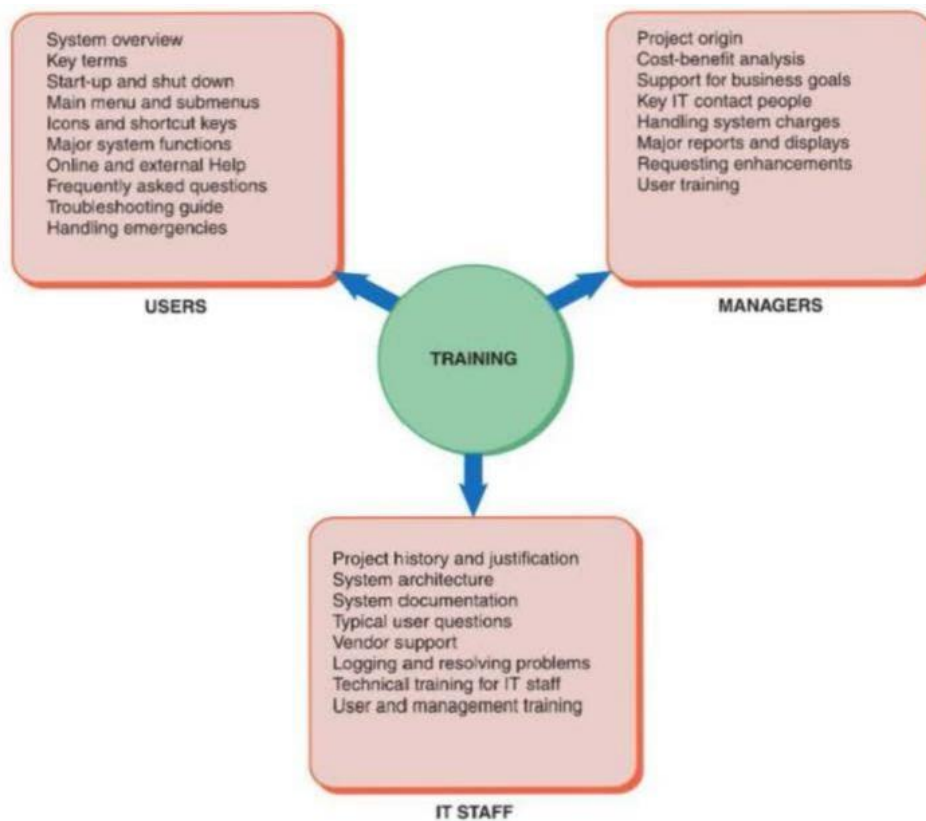
No system can be effective without proper training, whether it involves software, hardware, or manufacturing processes. For an information system to succeed, it requires training for users, managers, and IT staff members. The success of the entire system development can depend on whether people understand and know how to effectively use the system.

#### **Training Plan**

A training plan should be considered early in the system development process. As documentation is created, it should be assessed for how it can be used in future training sessions. When the system is implemented, it's important to provide the right training for the right people at the right time. The first step is identifying who needs training and what kind of training is required. The organization should be carefully analyzed to determine how the system supports business operations and who will be involved or affected by the system.

The three primary groups that require training are users, managers, and IT staff. A manager doesn't need to know every feature or detail of the system but should have an overview to ensure proper user training. Users, on the other hand, need to know how to perform their specific job functions but do not need to understand the full system's design or cost allocation. IT staff members need to have the most comprehensive understanding of the system to support it, as they must understand its operation, how it supports business needs, and the skills users need to perform their tasks.

Once training needs are identified, decisions must be made about how to deliver the training. Training can be sourced from vendors, outside training firms, or from internal resources, such as IT staff.



**FIGURE 11-28** Examples of training topics for three different groups. Users, managers, and IT staff members have different training needs.

(Tilley and Rosenblatt, 2024)

## Vendor Training

When the system involves purchasing software or hardware, vendor-supplied training should be part of the request for proposal (RFP) or request for quotation (RFQ). Many vendors offer training programs either for free or at a low cost, which may be included as part of the product package. In some cases, companies may negotiate training costs depending on their relationship with the vendor.

Vendor training typically focuses on the products the vendor has developed and often provides the best value. However, this training may be limited to the standard version of the product, and additional in-house training may be necessary if the product has been customized for the organization.

## Webinars, Podcasts, and Tutorials

Vendors often offer web-based training options, such as webinars, podcasts, and tutorials. A webinar is an online seminar that offers an interactive training experience. These sessions are typically scheduled events with a group of pre-registered users and a presenter. They can also

be prerecorded and accessed as a webcast, which is a one-way transmission for on-demand viewing.

A podcast is an audio or multimedia file available for download, which users can listen to at their convenience. Podcasts can be pre-scheduled, made available on demand, or updated automatically, depending on user preferences.

A tutorial is an online, interactive lesson that guides users through learning materials. These can be developed by software vendors, an organization's IT team, or third parties. Tutorials are often available for popular software packages and may be sold separately online.

## Outside Training Resources

Independent training firms can also provide hardware or software training if vendor training is not feasible or internal resources are unavailable. The growth of information technology has led to an expansion in the computer training industry. Many training firms, institutes, and consultants offer both standardized and customized training programs. For example, platforms like Udemy provide online courses to fulfill various training needs. Additionally, universities, industry associations, and other nonprofit organizations may offer training resources.



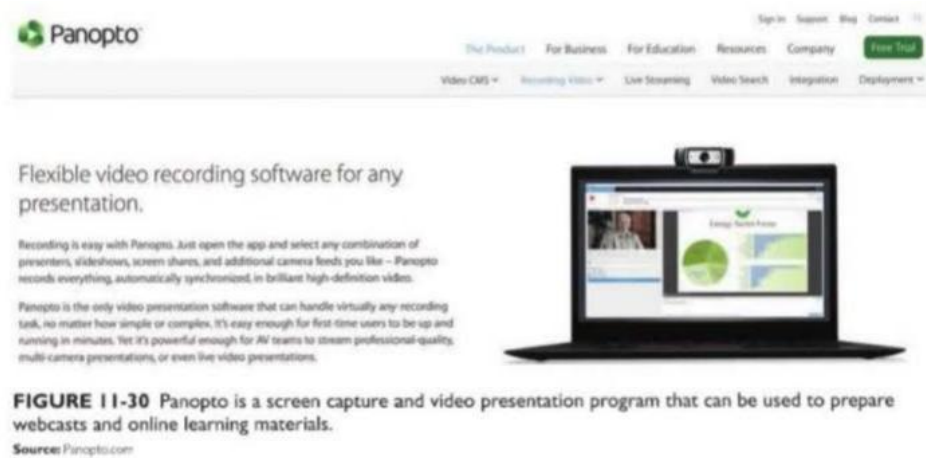
**FIGURE 11-29** Udemy is one of many sources of online training.  
Source: Udemy.com

(Tilley and Rosenblatt, 2024)

## Training Tips

In many cases, both IT staff and user departments are responsible for developing and conducting training for internally developed software. If the organization has a service desk, their staff might assist with user training.

Multimedia can be an effective training tool. Tools like Microsoft PowerPoint or Apple Keynote can be used to create engaging presentations with slides, animations, and sound. Software that records keystrokes and mouse actions, such as Camtasia or Panopto, can provide demonstrations of the system in action. If a media or graphics team is available, they can help produce training materials, including videos and instructional charts.



(Tilley and Rosenblatt, 2024)

- **Train People in Groups:** Group training is an efficient way to utilize time and resources. It also provides the opportunity for trainees to learn from one another through shared questions and challenges. Training should be tailored to the job interests and skills of different participant groups. For example, IT staff and users require different types of training, and a single program may not meet everyone's needs.
- **Select the Most Effective Training Location:** Conducting training at the company's location offers benefits such as no travel costs, the ability to handle local emergencies, and the convenience of training in the actual environment where the system will operate. However, there are potential downsides: employees may be distracted by other duties, and using company facilities could disrupt normal operations and limit hands-on training opportunities.
- **Provide Multiple Learning Methods:** People learn in different ways. Some prefer lectures, discussions, or Q&A sessions, while others learn best by watching demonstrations or reading materials. Most people benefit most from hands-on experience, so training should include methods that cater to hearing, seeing, and doing.
- **Leverage Previous Trainees:** After one group has been trained, they can assist others. Peer learning can be effective, as users often understand the system more quickly when trained by colleagues with similar experiences and job responsibilities. The "train-the-trainer" strategy can be used, where knowledgeable users train others. In

such cases, initial training should include not only system usage but also instruction on how to teach the material effectively.

### **Interactive Training**

Training methods and costs are often correlated. For example, training an airline pilot in a simulator differs greatly from training corporate users on an inventory system. While budgets are a business consideration, IT staff often work with available resources, which may not be ideal. Hands-on training is generally preferred, but other more cost-effective methods, such as training manuals, printed handouts, and online materials, can also be used.

If formal training materials are not available for a new system, help information can be embedded in dialog boxes that provide suggestions whenever users select different menu items. A good user interface with helpful error messages and hints can also aid in training. However, the most effective training is interactive, self-paced, and multimedia-based, such as online training and video tutorials.

### **Online Training**

Regardless of the method, training lessons should provide step-by-step instructions for using system features. Materials should resemble real screens, and tasks should be relevant to daily work. Video lessons are popular, such as online tutorials from platforms like Lynda.com.

Advanced online training systems may offer interactive sessions where users perform practice tasks and receive feedback. Training materials should include a reference section summarizing all options, commands, error messages, and recommended actions for problems.

When training concludes, many organizations conduct a full-scale test or "dress rehearsal" for both users and IT support staff. This simulation includes all procedures, even those executed only at specific times (e.g., monthly, quarterly, or yearly). Participants can consult system documentation, help screens, or each other to address any issues. This test provides valuable hands-on experience and boosts confidence for all involved.

### **5.9.7 Post-Implementation Tasks**

Once the new system is operational, two tasks remain:

1. **Post-Implementation Evaluation:** Conduct an evaluation to assess the success of the implementation and identify any areas for improvement.
2. **Final Report to Management:** Deliver a comprehensive final report to management, summarizing the project's success, challenges, and any lessons learned.

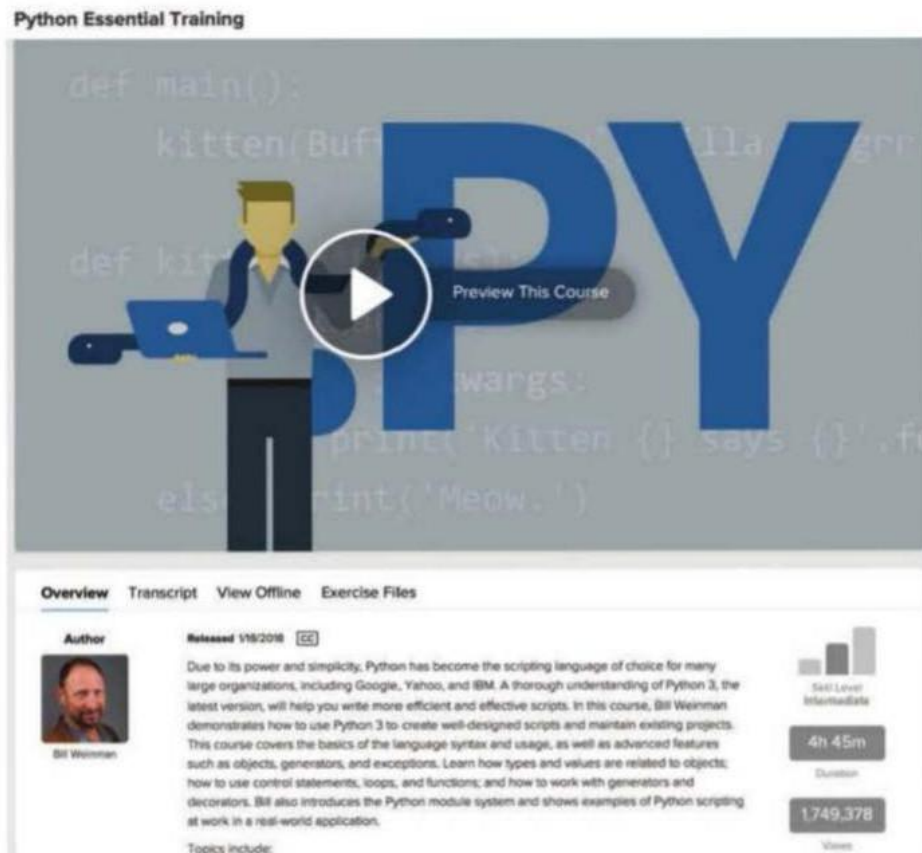


FIGURE 11-31 A sample lesson on learning Python in an online tutorial from lynda.com.

Source: LinkedIn Corporation.

(Tilley and Rosenblatt, 2024)

## Post-Implementation Evaluation

A post-implementation evaluation assesses the overall quality of the information system to ensure that it meets the specified requirements, aligns with user objectives, and delivers the expected benefits. It provides essential feedback to the development team, helping them improve IT development practices for future projects.

The post-implementation evaluation examines all aspects of the system and development process. Key areas evaluated include:

- **Accuracy, Completeness, and Timeliness of System Output:** Ensuring that the system provides correct, complete, and timely information.
- **User Satisfaction:** Gauging how satisfied users are with the system.
- **System Reliability and Maintainability:** Evaluating how well the system performs under normal operating conditions and how easy it is to maintain.
- **Adequacy of System Controls and Security Measures:** Ensuring that proper controls are in place to safeguard the system and its data.
- **Hardware Efficiency and Platform Performance:** Assessing the performance of the hardware and system platform.
- **Effectiveness of Database Implementation:** Evaluating how effectively the database supports the system's functionality.

- **Performance of the IT Team:** Reviewing how well the IT team handled the development and implementation process.
- **Completeness and Quality of Documentation:** Ensuring that all system documentation is complete, accurate, and user-friendly.
- **Quality and Effectiveness of Training:** Reviewing how well users were trained and whether the training materials were effective.
- **Accuracy of Cost-Benefit Estimates and Development Schedules:** Comparing the actual costs and timelines to the initial estimates.

The same fact-finding techniques used during the system analysis phase can be applied here. To evaluate the system, the following activities should be conducted:

- **Interviews:** Conduct interviews with management and key users to gather their feedback.
- **Observation:** Observe users and IT personnel working with the new system.
- **Review of Documentation:** Read all system documentation and training materials to ensure they are complete and clear.
- **Examine Source Documents and Output:** Review source documents, output reports, and screen displays to verify their accuracy and relevance.
- **Surveys/Questionnaires:** Use questionnaires to collect feedback from a large number of users to get a broad perspective.
- **Analyze Logs:** Review maintenance and help desk logs to identify recurring issues or challenges.

A sample user evaluation form for the new information system typically includes numerical ratings for various system elements, allowing easy tabulation of results. Additionally, open-ended sections are provided to capture users' comments and suggestions.

PDF DOCUMENT LIBRARY:

User Evaluation Form

System: \_\_\_\_\_ Evaluator: \_\_\_\_\_ Date: \_\_\_\_\_

Please evaluate the information systems project by circling the one number for each factor that best represents your assessment.

|                                      | 1              | 2 | 3          | 4 | 5 | 6         |
|--------------------------------------|----------------|---|------------|---|---|-----------|
|                                      | Unsatisfactory |   | Acceptable |   |   | Excellent |
| <b>SYSTEM OUTPUT</b>                 |                |   |            |   |   |           |
| 1. Accuracy of information _____     | 1              | 2 | 3          | 4 | 5 | 6         |
| 2. Completeness of information _____ | 1              | 2 | 3          | 4 | 5 | 6         |
| 3. Ease of use _____                 | 1              | 2 | 3          | 4 | 5 | 6         |
| 4. Timeliness of information _____   | 1              | 2 | 3          | 4 | 5 | 6         |
| <b>USER INTERFACE</b>                |                |   |            |   |   |           |
| 5. Clarity of instructions _____     | 1              | 2 | 3          | 4 | 5 | 6         |
| 6. Quality of Help messages _____    | 1              | 2 | 3          | 4 | 5 | 6         |
| 7. Ease of use _____                 | 1              | 2 | 3          | 4 | 5 | 6         |
| 8. Appropriateness of options _____  | 1              | 2 | 3          | 4 | 5 | 6         |
| 9. Clarity of error messages _____   | 1              | 2 | 3          | 4 | 5 | 6         |
| 10. Prevention of input errors _____ | 1              | 2 | 3          | 4 | 5 | 6         |
| <b>INFORMATION TECHNOLOGY STAFF</b>  |                |   |            |   |   |           |
| 11. Cooperation _____                | 1              | 2 | 3          | 4 | 5 | 6         |
| 12. Availability _____               | 1              | 2 | 3          | 4 | 5 | 6         |
| 13. Knowledge _____                  | 1              | 2 | 3          | 4 | 5 | 6         |
| 14. Reporting of progress _____      | 1              | 2 | 3          | 4 | 5 | 6         |
| 15. Communication skills _____       | 1              | 2 | 3          | 4 | 5 | 6         |
| <b>TRAINING</b>                      |                |   |            |   |   |           |
| 16. Completeness _____               | 1              | 2 | 3          | 4 | 5 | 6         |
| 17. Appropriateness _____            | 1              | 2 | 3          | 4 | 5 | 6         |
| 18. Schedule _____                   | 1              | 2 | 3          | 4 | 5 | 6         |

**FIGURE 11-32** Sample user evaluation form. The numerical scale allows easy tabulation of results. Following this section, the form provides space for open-ended comments and suggestions.

(Tilley and Rosenblatt, 2024)

Whenever possible, the **post-implementation evaluation** should be conducted by individuals who were not directly involved in the system's development. While IT staff and users typically perform the evaluation, some organizations use an internal audit group or independent auditors to ensure the accuracy and thoroughness of the evaluation.

**Timing of Evaluation:** The timing of the post-implementation evaluation is critical. If the evaluation occurs too soon after the system's implementation, users may not have had enough time to fully experience and assess the system's strengths and weaknesses. On the other hand, waiting too long to perform the evaluation can cause users to forget key details about their initial experience with the system. Ideally, evaluations should take place at least **six months after implementation** to allow users to gain sufficient experience, but sometimes earlier evaluations are performed due to pressure to close out the project quickly and move on to other tasks.

A **post-implementation evaluation** should be conducted while users can still recall specific incidents, successes, and problems, offering constructive feedback for improvement. If done too early, users may not have enough experience with the new system to provide sufficient feedback. If done too late, they may forget important details. In any case, these evaluations should be **standard practice** for all information systems projects.

Sometimes, evaluations are skipped because users are eager to start working with the new system, or because IT staff members have more pressing priorities. Management may also not fully appreciate the importance of post-implementation evaluations. Despite this, they are essential because they provide valuable insights into what worked well and what didn't, preventing the same errors from being repeated in future projects.

#### Final Report to Management

At the conclusion of each phase in the Systems Development Life Cycle (SDLC), a **final report** is submitted to management, and this is also the case for the systems implementation phase. The final report should include the following elements:

- **Final versions of all system documentation**
- **Planned modifications and enhancements** that have been identified
- **Recap of all systems development costs and schedules**
- **Comparison of actual costs and schedules** with the original estimates
- **Post-implementation evaluation** (if performed)

**CASE IN POINT 11.3: YORKTOWN INDUSTRIES**

You like your new job as lead systems analyst at Yorktown Industries. You were pleased that your development team completed the new human resources system ahead of schedule and under budget. You looked forward to receiving the post-implementation evaluation because you were confident that both the system and the development team would receive high marks from users and managers.

After the system operated for one month, you received a call from your supervisor, who told you that you would have to handle the evaluation—even though you headed the development effort. You told your supervisor that you did not feel comfortable evaluating your own team's work. You explained that someone who was not involved in its development should do an independent evaluation. Your supervisor responded that he had full confidence in your ability to be objective. He explained that no one else was available and he needed the evaluation quickly so he could move forward with the next stage in the corporate development plan.

You are troubled about the situation. What should you do, and why?

(Tilley and Rosenblatt, 2024)

## 5.10 SUMMARY

The **system implementation phase** involves the application development, testing, installation, and evaluation of the new system. During this phase, analysts decide on the overall design strategy and collaborate with programmers to complete the design, coding, testing, and documentation. Quality assurance (QA) is crucial throughout this process, and many organizations adhere to software engineering principles and quality standards set by organizations like ISO.

Different development approaches use various tools. For instance, structured development heavily relies on **Data Flow Diagrams (DFDs)** and **structure charts**, which depict program modules, data flows, control flows, conditions, and loops. **Object-Oriented (OO) methods** use tools such as **use case diagrams**, **class diagrams**, **sequence diagrams**, and **state transition diagrams**. **Agile methods** typically use **iterative and incremental models** to create the system.

Generic tools can also aid system developers in translating system logic into working program modules. These tools include **Functional Requirements Documents (FRDs)**, **flowcharts**, **pseudocode**, **decision tables**, and **decision trees**.

**Cohesion** measures the scope and focus of a module's processing characteristics. A module with a single, specific function has high cohesion, which is desirable. **Coupling** measures the relationship and interdependence between modules. Loosely coupled modules, which are relatively independent, are also desirable. These concepts of cohesion and coupling apply to both structured and object-oriented development.

In creating a structure chart, typically, three steps are followed: 1) Review **DFDs** and object models to identify processes and methods; 2) Identify program modules and their control-subordination relationships; and 3) Add symbols for coupling and loops. The structure chart is then reviewed to ensure consistency with the system documentation.

For agile development, **user stories** are created by the customer to define the features required and their priority levels. In agile methodologies, system releases are iterative, and each release undergoes careful testing by the customer.

During application development, programmers perform tasks like **desk checking**, **code reviews**, and **unit testing**. Analysts design the initial test plans, including test steps and data for **integration testing** and **system testing**. **Integration testing** is necessary for programs that interact with one another, while **system testing** is the final step, which includes users in the testing process.

In addition to system documentation, analysts and technical writers prepare **operations documentation** for the IT operations team and **user documentation** for those who will interact with the system. User documentation includes materials such as manuals, help screens, and tutorials.

During the **installation process**, an operational or production environment is created for the new system, separate from the testing environment. This environment contains live data and is accessible only to authorized users. All system changes must first be tested in the test environment before being applied to the operational environment.

**System changeover** refers to transitioning the new system into operation. There are four primary changeover methods: **direct cutover**, **parallel operation**, **pilot operation**, and **phased operation**. In **direct cutover**, the old system is shut down, and the new system starts immediately. This method is cost-effective but risky. **Parallel operation** involves running both the old and new systems simultaneously for a period, making it the safest but most expensive option. **Pilot operation** and **phased operation** are compromises, with **pilot operation** testing the new system in a specific group before a wider rollout, and **phased operation** introducing one module at a time until the full system is operational.

**Data conversion** is often required when installing a new system, especially when replacing an existing computerized system. If possible, the conversion should be automated, particularly if the old system can export data in a format the new system can use. However, if the data is coming from a manual system, it may require labor-intensive entry or scanning. Even with

automated data conversion, new systems may require additional data items, which might need to be entered manually. Throughout the conversion process, data integrity must be carefully monitored and maintained.

**Training** is crucial for everyone who will interact with the new system. The IT department typically handles training, although vendors or professional training organizations may also assist. Training programs should follow several key guidelines: train people in groups, use experienced users to train others, create separate programs for different employee groups, and offer various training methods like discussions, demonstrations, manuals, tutorials, webinars, and podcasts. Users learn best through interactive, self-paced training methods.

A **post-implementation evaluation** assesses and reports on the quality of the new system and the project team's performance. It is typically conducted by individuals who were not directly involved in the development. This evaluation should be done early while users still have a fresh memory of the development effort, but not too soon after system implementation to ensure users have enough experience with the system.

The **final report to management** includes the final versions of system documentation, any identified future system enhancements, and a recap of project costs. This report marks the end of the development phase and the start of the system's operational life.

## **EXERCISE**

### **Questions**

1. What is Quality Assurance (QA)?
2. What does application development involve?
3. How are structure charts utilized in application development?
4. Should classes be tightly coupled or loosely coupled in Object-Oriented Development (OOD)? Explain why.
5. What is pair programming?
6. What role do Integrated Development Environments (IDEs) play in coding?
7. Outline the three main types of testing and the sequence in which they are conducted.
8. What distinguishes program documentation, system documentation, operations documentation, and user documentation?
9. What is the role of online documentation?
10. What is the difference between an operational environment and a test environment?

### **Discussion Topics**

1. Discuss three techniques used to enhance Quality Assurance (QA).
2. What are the main differences between structured, Object-Oriented (OO), and agile development methods? What similarities do they share?
3. Experienced programmers sometimes opt for handcrafted tools instead of using sophisticated Integrated Development Environments (IDEs), often connecting them in a pipeline for complex workflows. What are the advantages and disadvantages of this approach compared to using sophisticated IDEs?
4. Your supervisor says, "Integration testing is a waste of time. If each program is tested adequately, integration testing isn't necessary. We'll move to system testing as soon as possible, and if modules don't interact properly, we'll address it then." Do you agree or disagree? Justify your answer.
5. If you were designing a tutorial to train someone in using specific software or hardware (e.g., a web browser), what specific information would you want to know about the learner? How would this information influence the design of the training materials?

### **Projects**

1. Based on the material in this chapter and additional research, create a presentation discussing the pros and cons of agile development methods.
2. After learning about the importance of testing in this chapter, design a generic test plan for a hypothetical system.
3. Which system changeover method would you suggest for upgrading an air traffic control system? Justify your choice.
4. Design a generic post-implementation evaluation form. The form should contain questions for evaluating any information system, including training received and any issues encountered with the system.

5. Develop an online training module on managing system implementations using platforms like Udemy or lynda.com.

## CHAPTER 6: Managing Systems Support and Security



### LEARNING OUTCOMES

After reading this Section of the guide, the learner should be able to:

1. Explain activities related to user support.
2. Define the four types of maintenance.
3. Discuss seven strategies and techniques for managing maintenance.
4. Outline techniques for managing system performance.
5. Describe concepts of system security and common system attacks.
6. Identify three tasks involved in risk management.
7. Evaluate system security across six levels: physical security, network security, application security, file security, user security, and procedural security.
8. Explain backup and disaster recovery procedures.
9. List six indicators that show a system has reached the end of its useful life.
10. Identify future challenges and opportunities for IT

### 6.1 USER SUPPORT

A systems analyst acts like an internal consultant, offering guidance, support, and training. Well-designed systems often require the most support, as users want to explore the features, capabilities, and discover how the system can improve their tasks. In many organizations, over half of the IT department's efforts are dedicated to supporting existing systems. Companies

provide user support in various ways, including user training and help desks that offer technical support. These services can be provided in-house or outsourced.

### **6.1.1 User Training**

Chapter 11 covered the initial training provided when a new system is introduced. In addition to this, new employees must also be trained on the company's information systems. When there are significant changes to the existing system or a new version is released, the IT department may develop a user training package. Depending on the changes, the training might involve online support via email, a special website, an update to the user guide, a supplement to the training manual, or formal training sessions. The goal is to demonstrate to users how the system can assist them in performing their job duties.

### **6.1.2 Help Desks**

As systems become more complex, users require ongoing support and guidance. To improve accessibility and empower users, many IT departments establish help desks. A help desk, or service desk, is a centralized resource staffed by IT professionals who provide the necessary support to users. A help desk has three main objectives: (1) show users how to utilize system resources more effectively, (2) answer technical or operational questions, and (3) increase user productivity by teaching them to meet their own information needs. It is the first point of contact for users when they need assistance.

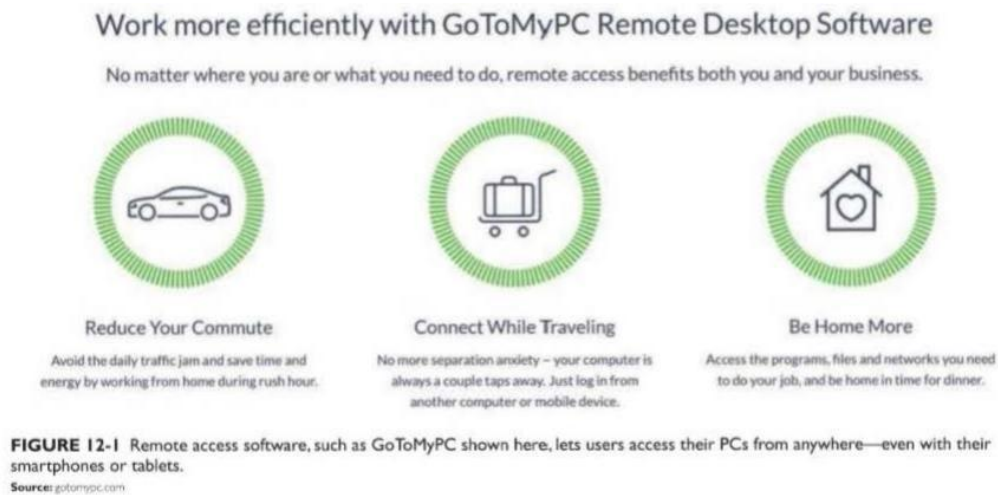
A help desk does not replace traditional IT maintenance and support but enhances productivity and optimizes the use of company resources. Help desk representatives need strong technical and interpersonal skills, as well as a solid understanding of the business, since they interact with users across various departments. It is essential for a help desk to document inquiries, tasks, and activity levels, as this information can help identify recurring issues and create a technical support knowledge base.

Help desks can improve efficiency by using remote control software, allowing IT staff to take control of a user's workstation for support and troubleshooting. An example of such software is GoToMyPC by Citrix.

A typical day for help desk staff may involve tasks like:

- Guiding a user on how to create a data query or report
- Resolving network or password issues
- Demonstrating advanced features of a system or commercial software
- Assisting with data recovery
- Offering tips for better system operation
- Explaining undocumented software features
- Showing how to use web conferencing tools
- Explaining how to access the company's intranet or internet
- Helping a user develop a database to track time spent on various projects
- Answering questions about software licensing and upgrades
- Providing information about system specifications and the cost of new hardware or software

- Suggesting system solutions to integrate data from different locations
- Supporting hardware installation or reconfiguration, including devices like scanners, printers, and wireless devices
- Demonstrating how to maintain data consistency across different devices (e.g., desktop, laptop, smartphone)
- Troubleshooting software issues using remote control tools.



(Tilley and Rosenblatt, 2024)

In addition to acting as an essential link between IT staff and users, the help desk serves as a central point of contact for all IT maintenance activities. It is where users report system issues, request maintenance, or submit new system requests. A help desk can integrate various forms of automated support, similar to what external vendors use, such as email responses, on-demand fax services, online knowledge bases, frequently asked questions (FAQs), discussion groups, bulletin boards, and automated voicemail systems. Many vendors also offer live chat features for online visitors.

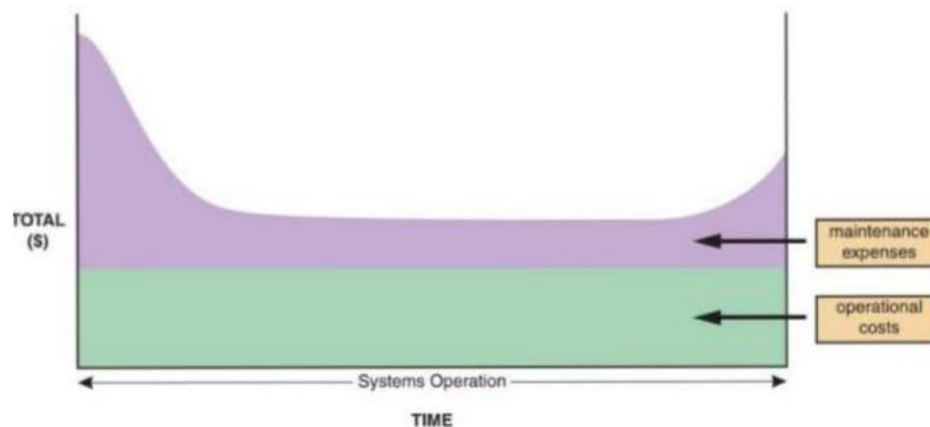
### 6.1.3 Outsourcing Issues

As mentioned in Chapter 7, many companies outsource different parts of application development, including IT support and help desks. Outsourcing has both advantages and disadvantages. The main reason for outsourcing is often cost reduction. Offshore call centers can lower expenses and free up valuable resources for product development.

However, companies have realized that if the quality of technical support declines, customers are likely to notice and may choose to take their business elsewhere. Key factors that contribute to this include long phone wait times, poor staff performance, and inadequate online support tools. The real challenge is whether a company can achieve the desired savings without compromising its reputation or losing customers. While the risks of outsourcing can be minimized, it requires active management and monitoring to ensure support quality and consistency are maintained.

## 6.2 MAINTENANCE TASKS

The systems support and security phase is a critical component of the total cost of ownership (TCO) because ongoing maintenance costs can impact the system's economic lifespan. Figure 12-2 illustrates the typical pattern of operational and maintenance expenses throughout the useful life of a system. Operational costs include things like supplies, equipment rental, and software leases. The lower section in Figure 12-2 represents fixed operational costs, while the upper section reflects maintenance expenses.



**FIGURE 12-2** The total cost of operating an information system includes operational and maintenance costs. Operational costs (green) are relatively constant, while maintenance costs (purple) vary over time.

(Tilley and Rosenblatt, 2024)

Maintenance expenses fluctuate significantly throughout the operational life of a system and encompass costs related to supporting maintenance activities. These activities involve modifying programs, procedures, or documentation to ensure the system continues to function correctly, adjusting the system to meet evolving requirements, and enhancing its efficiency. These needs are addressed through different types of maintenance.

### 6.2.1 Types of Maintenance

While there is some overlap, four primary types of maintenance tasks can be identified, as illustrated in Figure 12-3. These are:

1. **Corrective Maintenance:** This type is focused on fixing errors in the system.
2. **Adaptive Maintenance:** It involves adding new capabilities and enhancements to the system.
3. **Perfective Maintenance:** This aims to improve system efficiency.
4. **Preventive Maintenance:** This type reduces the likelihood of future system failures.

Some analysts use the term "maintenance" specifically to refer to corrective maintenance, which addresses problems. However, it's more useful to consider the concept of maintenance more broadly by recognizing the different types of tasks involved.

Maintenance costs are typically high at the start of a system's implementation, as issues need to be identified, investigated, and resolved through corrective maintenance. Once the system stabilizes, costs generally decrease, with only minor adaptive maintenance required. Over time, adaptive and perfective maintenance activities increase as the business environment evolves. Toward the end of a system's useful life, adaptive and corrective maintenance costs rise rapidly, while perfective maintenance usually decreases when it's clear that the system will be replaced. Figure 12-4 shows the typical patterns of these four types of maintenance throughout the system's lifespan.

#### **Examples of Maintenance Tasks**

##### **Corrective Maintenance**

- Diagnose and fix logic errors
- Replace defective network cabling
- Restore proper configuration settings
- Debug program code
- Update drivers
- Utilize remote control software for problem diagnosis and resolution

##### **Adaptive Maintenance**

- Add online capability
- Add support for mobile devices
- Add new data entry field to input screen
- Install links to website
- Create employee portal

##### **Perfective Maintenance**

- Upgrade or replace outdated hardware
- Write macros to handle repetitive tasks
- Compress system files
- Optimize user desktop settings
- Upgrade wireless network capability
- Install more powerful network server

##### **Preventive Maintenance**

- Install new antivirus software
- Develop standard backup schedule, including off-site and cloud-based strategies
- Implement regular defragmentation process
- Analyze problem report for patterns
- Tighten all cable connections
- Develop user guide covering confidentiality rules and unauthorized use of company IT resources

**FIGURE 12-3** Corrective maintenance fixes errors and problems. Adaptive maintenance provides enhancements to a system. Perfective maintenance improves a system's efficiency, reliability, or maintainability. Preventive maintenance avoids future problems.

(Tilley and Rosenblatt, 2024)

## **6.2.2 Corrective Maintenance**

Corrective maintenance is the process of diagnosing and fixing errors in an operational system. To avoid introducing additional problems, any maintenance work must undergo careful analysis before implementing changes. The most effective approach to maintenance is a scaled-down version of the SDLC, which involves investigation, analysis, design, and testing before implementing any solution. As noted in Chapter 11, the distinction between a test environment and an operational environment is crucial. Any maintenance work that

might affect the system must first be performed in the test environment, and only then migrated to the operational system.

IT support staff address errors in various ways, depending on the nature and severity of the issue. Most organizations have standard procedures for minor errors, such as incorrect report titles or improper data formats. Typically, a user submits a system request that is evaluated, prioritized, and scheduled by the system administrator or systems review committee. If approved, the maintenance team designs, tests, documents, and implements the solution.

As mentioned in Chapter 2, many organizations use a standard online form for submitting system requests, while smaller firms may handle it informally via email. For more significant issues, such as incorrect report totals or inconsistent data, users submit a system request with supporting evidence, which is given high priority, and the maintenance team begins work immediately.

|                                           | Immediately After Implementation | Early Operational Life | Middle Operational Life | Later Operational Life |
|-------------------------------------------|----------------------------------|------------------------|-------------------------|------------------------|
| Corrective Maintenance                    | High                             | Low                    | Low                     | High                   |
| Adaptive Maintenance (Minor Enhancements) | None                             | Medium                 | Medium                  | Medium                 |
| Adaptive Maintenance (Major Enhancements) | None                             | None                   | Medium to High          | Medium to High         |
| Perfective Maintenance                    | Low                              | Low to Medium          | Medium                  | Low                    |
| Preventive Maintenance                    | Low                              | Medium                 | Medium                  | Low                    |

**FIGURE 12-4** Information systems maintenance costs depend on the type of maintenance and the age of the system.

(Tilley and Rosenblatt, 2024)

In the case of a system failure, the emergency response is prioritized. The maintenance team bypasses the initial steps and works to fix the problem immediately. This often involves applying a patch, which is a temporary software module that provides a quick fix, allowing operations to resume. During this time, a system request is created either by the user or an IT department member and is added to the maintenance log. Once the system is up and running again, the maintenance team investigates the root cause, analyzes the issue, and designs a permanent solution. The IT response team updates the test data files, thoroughly tests the system, and prepares complete documentation.

Regardless of the prioritization method used, having a standard ranking framework can be beneficial. For instance, Figure 12-5 presents a three-level structure for assessing IT support's potential impact.

| PRIORITY | IMPACT                                                                                                                | TIME FRAME                                                         |
|----------|-----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| Level 1  | Significant impact on IT operations, security, or business activity that requires immediate attention.                | Implement patch as soon as possible.                               |
| Level 2  | Some impact on IT operations, security, or business activity. Requires prompt attention, but operations can continue. | Patch as necessary and begin implementation prior to next release. |
| Level 3  | Little or no impact on current IT operations, security, or business activity                                          | Implement in the next release.                                     |

**FIGURE 12-5** This three-level ranking framework for IT support considers potential impact and response urgency.

(Tilley and Rosenblatt, 2024)

### 6.2.3 Adaptive Maintenance

Adaptive maintenance involves enhancing an operational system by adding new features or capabilities to meet changing business needs. This can result from business environment shifts, such as the introduction of new products or services, advancements in manufacturing technology, or the need for new web-based operations. The process for minor adaptive maintenance follows a similar procedure to corrective maintenance, where a user submits a system request evaluated by the systems review committee. The maintenance team then analyzes, designs, tests, and implements the enhancement. For larger projects, adaptive maintenance can be more resource-intensive and resemble a small-scale SDLC project, as it must work within the constraints of the existing system.

### 6.2.4 Perfective Maintenance

Perfective maintenance focuses on improving an operational system to make it more efficient, reliable, and easier to maintain. While corrective and adaptive maintenance requests generally come from users, the IT department typically initiates perfective maintenance. This type of maintenance can be prompted by declining system efficiency due to changes in user activity or data patterns. Perfective maintenance helps improve system performance, reliability, and maintainability by addressing issues such as inefficient input processes or complex programs. However, it is often overlooked in favor of new projects or more urgent corrective and adaptive maintenance tasks. It is most cost-effective during the middle of a system's life cycle but may become costly later if the system is near the end of its operational life. Software reengineering is a key technique used in perfective maintenance, which involves using analytical techniques to identify improvements in performance and quality.

### 6.2.5 Preventive Maintenance

Preventive maintenance involves proactively identifying potential areas of trouble and taking steps to prevent issues before they occur. Like perfective maintenance, it is typically initiated by the IT department. Preventive maintenance aims to reduce downtime, improve user satisfaction, and lower total cost of ownership (TCO). However, it often competes with other projects for IT resources and may not always receive the attention it deserves.

Regardless of the type of maintenance, it is crucial that trained professionals provide the necessary support for computer systems to ensure optimal performance and avoid failures.



**FIGURE 12-6** Technicians use a motorized lift to get into position by the ATLAS particle detector at CERN. Regardless of the type of system, trained professionals must perform high-quality maintenance.

Source: Anna Pantelisi/CERN

(Tilley and Rosenblatt, 2024)

### CASE IN POINT 12.1: OUTBACK OUTSOURCING, INC.

You are a systems analyst at Outback Outsourcing, a firm that handles payroll processing for many large companies. Outback Outsourcing uses a combination of payroll package programs and in-house developed software to deliver custom-made payroll solutions for its clients. Lately, users have flooded you with requests for more new features and web-based capability to meet customer expectations. Your boss, the IT manager, comes to you with a question. She wants to know when to stop trying to enhance the old software and develop a totally new version better suited to the new marketplace. How would you answer her?

## 6.3 MAINTENANCE MANAGEMENT

Effective system maintenance involves strong management, quality assurance, and cost control. To meet these objectives, companies implement various strategies, including a maintenance team, a maintenance management program, a configuration management process, and a maintenance release procedure. Additionally, organizations use version control and baselines to monitor system releases and evaluate the system's life cycle. These concepts are explained in the sections that follow.

### 6.3.1 The Maintenance Team

A maintenance team typically consists of a system administrator and several systems analysts and programmers, all working collaboratively to ensure the smooth operation of IT systems. Each role in the maintenance team brings distinct skills and responsibilities:

- **System Administrator:** Responsible for the overall operation, configuration, and security of the system. A system administrator must possess solid technical expertise

and be capable of troubleshooting hardware and software issues. They often take proactive actions to prevent immediate emergencies, such as server crashes, network outages, or security breaches.

- **Systems Analysts:** These professionals investigate issues within the system, using analytical and synthesis skills to find solutions. Analysts should possess strong technical and business acumen, as well as effective communication skills. They often need to balance their knowledge of system operations with an understanding of the company's business functions.
- **Programmers:** Depending on the organization's size, programmers specialize in different areas—application programming, system software programming, or database management. In smaller firms, a single programmer might handle various tasks, while in larger organizations, the roles are more specialized. Many companies use the "programmer/analyst" title for individuals who combine systems analysis and programming skills.

## Organizational Issues

Some companies divide IT staff into separate groups: one focusing on new systems development, and the other on maintenance tasks. A more flexible approach integrates both functions, allowing staff to work on different projects as needed. This integration provides a sense of ownership for the maintenance team since they are familiar with the system from the outset. However, it can be challenging for analysts to maintain systems they did not originally design, especially if documentation is lacking.

In some cases, IT staff are rotated between development and maintenance teams. While this can improve versatility and skillsets, it may also increase overhead due to the time lost during transitions. Furthermore, rotating between projects might prevent individuals from becoming experts in a single area.

New IT staff are often assigned to maintenance tasks to help them understand existing systems. Though it can be a good training opportunity, it may be difficult for them, especially when troubleshooting poorly documented systems. It may be more effective to assign new hires to development teams for better training and mentorship.

### 6.3.2 Maintenance Requests

Maintenance requests typically follow a structured process, as illustrated in Figure 12-7. Here's how the process works:

**Request Submission:** A user submits a maintenance request, which could be related to corrective, adaptive, or perfective maintenance. Non-emergency requests are submitted in writing or via email.

**Initial Determination:** Upon receiving the request, the system administrator assesses whether the issue requires immediate attention. If the request involves a critical issue, the system administrator takes immediate action. For non-critical requests, the administrator checks whether it falls within the pre-approved cost limits.

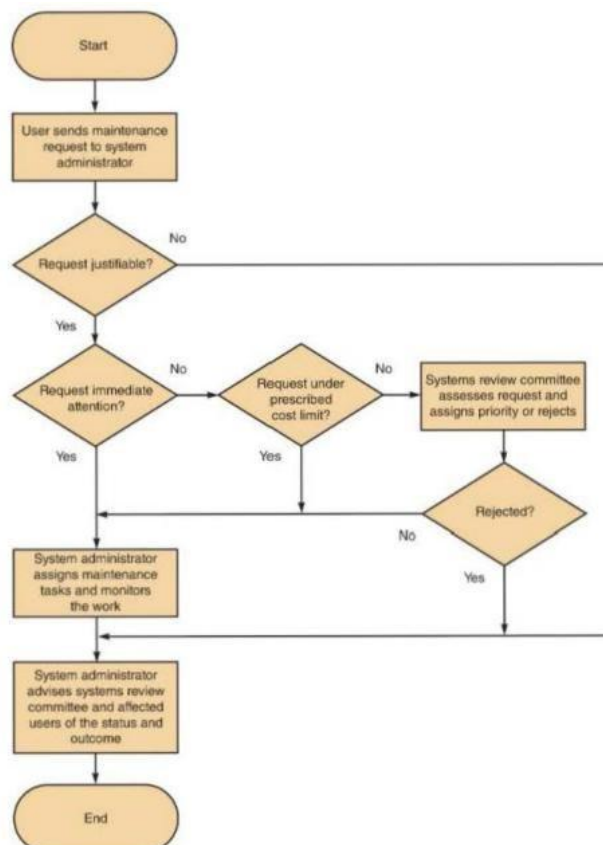
**System Review Committee:** If the request exceeds the cost limit or involves a major system change, the system review committee evaluates the request. They can approve it, assign a priority, or reject it.

**Task Completion:** The system administrator is responsible for assigning tasks to individuals or teams. Depending on company policies, the administrator may rotate maintenance tasks among IT staff or assign them to specific individuals or teams based on expertise.

In this structured process, every maintenance activity is documented through a formal request, helping to track and prioritize issues effectively while maintaining a maintenance log for future reference.

(Tilley and Rosenblatt, 2024)

**USER NOTIFICATION:** Users who initiate maintenance requests expect a quick response, particularly when the issue directly impacts their work. Even if immediate corrective action is not possible, users



**FIGURE 12-7** Although the procedure varies from company to company, the chart shows a typical process for handling maintenance requests.

value feedback from the system administrator and should be kept informed about any decisions or actions that could affect them.

### 6.3.3 Establishing Priorities

In many organizations, the system review committee separates maintenance and new development requests when setting priorities. In others, all requests are considered together, with the most important project given top priority, regardless of whether it's for maintenance or new development. Some IT managers believe that evaluating all projects together leads to better decision-making because both maintenance and new development require similar IT resources. However, in IT departments where maintenance and new development are not integrated, it might be better to assess requests separately. One advantage of this approach

is that maintenance is more likely to receive its fair share of IT resources. The main objective is to have a procedure that balances both new development and maintenance work to provide the best support for business needs and priorities.

### 6.3.4 Configuration Management

Configuration management (CM), also known as change control (CC), is a process for managing changes to system requirements during software development. CM is also essential for managing system changes and costs after the system is operational. Most companies establish a specific process outlining how system changes must be requested and documented. As enterprise-wide information systems become more complex, CM becomes even more critical. Industry standards, such as the IEEE's Standard 828-2012 for CM in systems and software, have been developed to help guide this process.



**FIGURE 12-8** IEEE Standard 828-2012 for configuration management in systems and software engineering.  
Source: IEEE.

(Tilley and Rosenblatt, 2024)

CM is especially crucial when a system has multiple versions running in different hardware and software environments. It also plays a key role in organizing and managing documentation. An operational system typically has extensive documentation covering development, modifications, and maintenance for all versions of the installed system. Most of this documentation, such as the initial system requests, project management data, end-of-phase reports, data dictionary, and IT operations and user manuals, is stored within the IT department.

Properly tracking and ensuring that documentation updates are properly distributed is an important aspect of CM.

### 6.3.5 Maintenance Releases

Tracking maintenance changes and updates can be challenging, especially in complex

systems. When a maintenance release methodology is used, all non-critical changes are accumulated until they can be implemented simultaneously. Each change is documented and released as a new version of the system, called a maintenance release.

For in-house developed systems, the time between releases typically depends on the level of maintenance activity. A release to fix a critical error, however, might be implemented immediately, rather than being postponed until the next scheduled release.

In systems using release methodologies, a version numbering pattern is adopted. For example, the initial version of a system is 1.0, and the release that includes the first set of maintenance changes would be version 1.1. A change from version 1.4 to 1.5 would indicate minor enhancements, while a change from version 1.0 to 2.0 or 3.4 to 4.0 suggests a significant upgrade.

The release methodology offers several advantages, especially when multiple teams perform maintenance on the same system. By using this approach, all changes can be tested together before a new version is released, leading to fewer versions, reduced costs, and minimal disruption to users. The methodology also reduces the documentation burden as all changes are coordinated and deployed at the same time.

However, there are some potential drawbacks. Users expect a quick response to their problems and requests, but with a release methodology, new features or upgrades may not be available as frequently. Even when changes could improve system efficiency or user productivity, these benefits may need to wait until the next release, which could result in increased operational costs.

Commercial software suppliers also provide maintenance releases, often called service packs. A service pack contains all the fixes and enhancements made since the last program version or service pack.

### **6.3.6 Version Control**

Version control is the process of tracking system releases or versions. When a new version of a system is installed, the prior release is archived or stored. If the new version causes issues or system failures, the company can reinstall the previous version to restore operations. Additionally, the IT staff must manage systems that consist of several modules at different release stages. For example, an accounting system may have an older accounts receivable module that needs to interface with a new payroll module.

Many companies use commercial applications for version control in complex systems. There are also several free and open-source alternatives. One of the most widely used version control systems is Git. Git is a free and open-source program designed for distributed systems, easy to use, available on most major platforms, and supported by the development community.



**FIGURE 12-9** A Microsoft service pack provides access to updated drivers, tools, security patches, and customer-requested product changes.  
Source: Microsoft

(Tilley and Rosenblatt, 2024)



**FIGURE 12-10** Git is a popular free version control system.  
Source: git-scm.com

(Tilley and Rosenblatt, 2024)

### 6.3.7 Baselines

A baseline serves as a formal reference point used to measure the characteristics of a system at a specific time. Systems analysts use baselines as benchmarks to document system features

and performance throughout the development process. There are three types of baselines:  
Functional Baseline: This baseline represents the system's configuration at the start of the project. It includes all the required system features and design constraints.

Allocated Baseline: This baseline is established at the end of the design phase and documents any changes made since the functional baseline. It includes the testing and verification of all system requirements and features.

Product Baseline: This baseline is created at the start of the system's operational phase. It incorporates any changes from the allocated baseline and includes the results of performance and acceptance tests for the operational system.

## **6.4 SYSTEM PERFORMANCE MANAGEMENT**

In the past, managing system performance was relatively simple when companies used central computers for data processing. Today, however, companies use complex networks, client/server systems, and cloud computing environments, making system management and performance measurement more challenging. A user at a workstation may interact with a system that relies on various other clients, servers, networks, and data distributed throughout the company. The overall system performance now depends on the integration of these components rather than a single computer.

To ensure the system effectively supports business operations, the IT department must manage system faults and interruptions, measure system performance and workload, and anticipate future needs. Many IT managers rely on automated software and CASE (Computer-Aided Software Engineering) tools to assist in these tasks. These tools can be particularly useful during the operation and support phases. Some common CASE tools include:

- Performance monitors to track program execution times
- Program analyzers that examine source code, provide cross-reference data, and assess the impact of program changes
- Interactive debugging analyzers to locate programming errors
- Reengineering tools
- Automated documentation generation
- Network activity monitors
- Workload forecasting tools

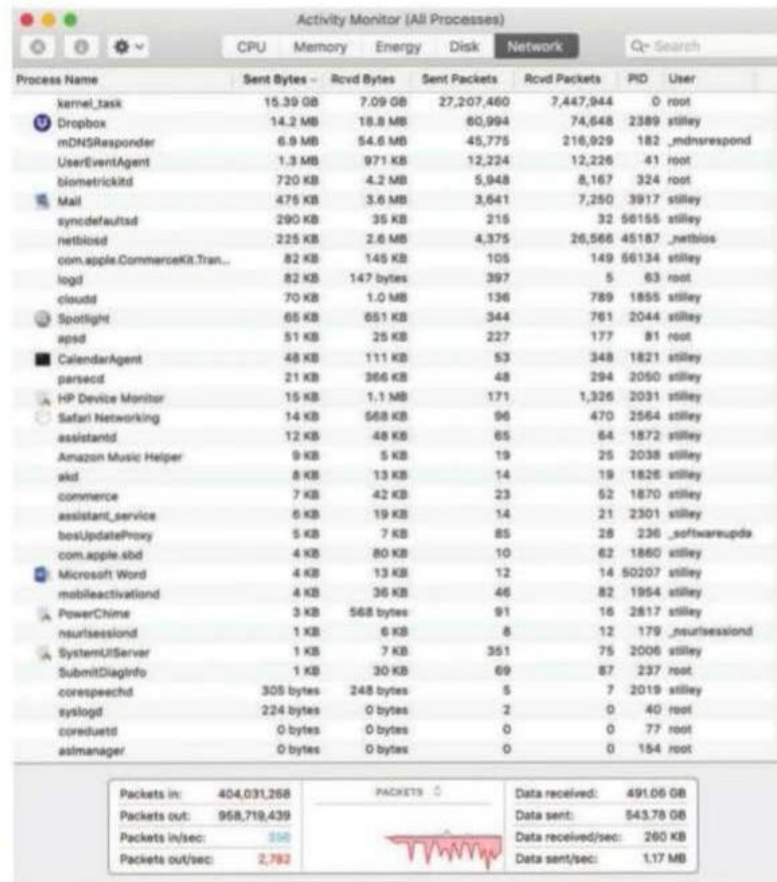
In addition to CASE tools, spreadsheet and presentation software can be used to calculate trends, perform what-if analyses, and create visual charts or graphs to present results. IT planning is a crucial part of business planning and often forms part of presentations made to management.

### **6.4.1 Fault Management**

Every system, regardless of how well it is designed, will experience some issues—such as hardware failures, software errors, user mistakes, or power outages. A system administrator must identify and resolve operational problems quickly. This task, known as fault

management, involves monitoring the system for potential issues, logging failures, diagnosing problems, and taking corrective actions.

As systems become more complex, diagnosing issues and identifying causes can become increasingly difficult. In addition to addressing immediate issues, it's essential to evaluate performance patterns and trends. For example, the Activity Monitor application on Apple's macOS displays real-time CPU, memory, energy, disk, and network activity for running applications, similar to programs like Resource Monitor on Windows. Fault management software helps identify root causes, speeds up response time, and reduces service outages.



**FIGURE 12-11** The Activity Monitor application on Apple's Mac OS X displays CPU, memory, energy, disk, and network activity of all running applications in real time.

Source: Apple

(Tilley and Rosenblatt, 2024)

## 6.4.2 Performance and Workload Measurement

In e-business, slow performance can be just as damaging as a complete system failure. Network delays and application bottlenecks negatively impact customer satisfaction, user productivity, and business outcomes. Many IT managers argue that network delays cause more harm than actual outages because they happen frequently and are harder to predict, detect, and prevent. Customers expect a quick, reliable response around the clock, and to meet this demand, companies use performance management software.

To evaluate system performance, many companies conduct benchmark testing using standard tests to assess the system's capacity. Alongside benchmark testing, performance

metrics, called "tricks," are used to monitor aspects like the number of transactions processed, records accessed, and online data volume. Network performance metrics include response time, bandwidth, throughput, and turnaround time.

**Response Time:** This measures the time between a request for system activity and the response. In online environments, it starts when the user presses "Enter" or clicks a mouse and ends when the screen displays or printed output is ready. Response time is influenced by system design, capabilities, processing methods, and communication factors when accessing the network or internet. Users expect near-instant responses, and delays can be frustrating. Response time is one of the most noticeable and complained-about performance metrics.

**Bandwidth and Throughput:** These two terms are often used interchangeably. Bandwidth refers to the amount of data a system can transfer in a fixed time, typically measured in bits per second (bps), kilobits per second (Kbps), megabits per second (Mbps), or gigabits per second (Gbps). Throughput measures actual system performance under specific conditions and is affected by network load and hardware efficiency. While bandwidth represents the potential capacity, throughput reflects the actual data transfer rate. Just as traffic jams slow down vehicles, throughput limitations can lead to slower system performance, especially in graphics-heavy and web-based systems.

In addition to these performance metrics, system administrators track other characteristics, although no standard set exists. Common ones include:

**Arrivals:** The number of items on a device within a given observation period.

**Busy:** The time a resource is unavailable.

**Completions:** The number of arrivals processed in a given period.

**Queue Length:** The number of pending requests.

**Service Time:** The time it takes to process a task once it reaches the front of the queue.

**Think Time:** The time a user takes to issue another request.

**Utilization:** The amount of a resource used to complete a task.

**Wait Time:** The time requests wait for a resource to be available.

**Turnaround Time:** This applies to batch processing operations like customer billing or credit card statement processing. It measures the time between a request for information and its fulfillment. Turnaround time can also be used to assess the quality of IT support, measuring the time from a user request for help to its resolution.

The IT department often measures response time, bandwidth, throughput, and turnaround time to assess system performance before and after system changes or shifts in business information needs. These performance metrics are also used for cost-benefit analyses of proposed maintenance and to evaluate systems nearing the end of their useful life. Lastly, management relies on performance and workload data for capacity planning.

### **6.4.3 Capacity Planning**

Capacity planning involves monitoring current system activity and performance, predicting future demand, and forecasting the resources necessary to meet desired service levels.

The first step is to create a current model based on the system's current workload and performance specifications. Then, future demand and user needs are projected over one to three years. The model is analyzed to determine what adjustments are necessary to maintain optimal performance and meet demands. A technique called "what-if analysis" is used to test various scenarios by modifying one or more elements in the model and observing how other elements are affected. For example, what-if analysis might answer questions like: How will

response time be impacted if additional client workstations are added to the network? Can the client/server system handle growth from a new website? What happens to server throughput if more memory is added?

Tools like Microsoft Excel's Goal Seek feature can assist in what-if analysis by determining the necessary changes to one value to achieve a specific result in another. In the example shown in the text, Excel automatically calculates the impact on processing time when the number of transactions increases to 9,000.

For effective capacity planning, detailed information about transaction volumes, transaction patterns (daily, weekly, or monthly), query numbers, and the size and types of reports generated is essential. If the system includes a local area network (LAN), network traffic levels must be estimated to assess whether the existing hardware and software can handle the load. For client/server systems, performance and connectivity specifications for each platform need to be evaluated.

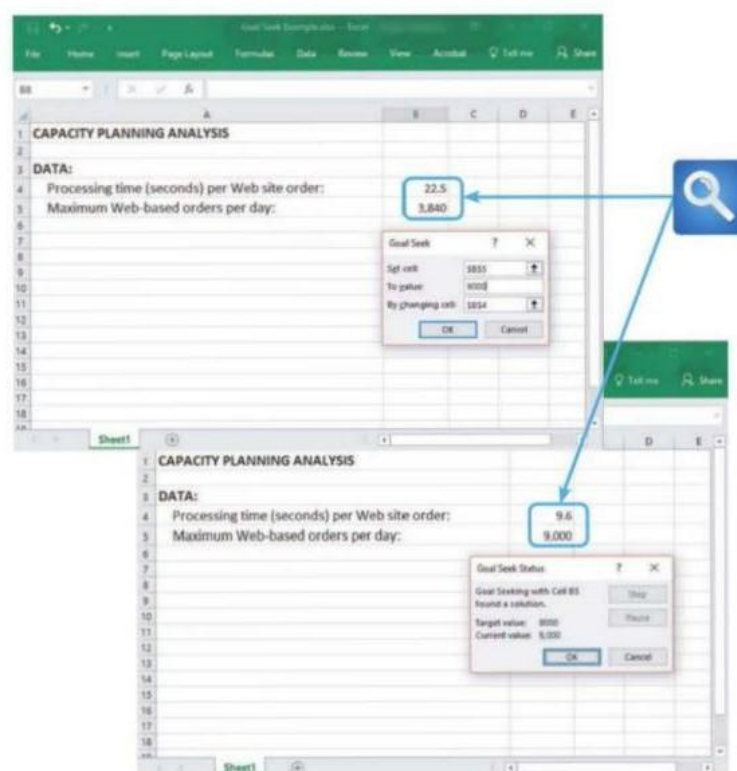


FIGURE 12-12 In this Goal Seek example, the user wants to know the effect on processing time if the number of daily transactions increases from 3,840 to 9,000.

(Tilley and Rosenblatt, 2024)

The most critical element is an accurate forecast of future business activities. If new business functions or requirements are anticipated, contingency plans should be developed with input from users and management. The goal is to ensure the system can meet future demands and continue providing support for business operations. Some companies handle their own capacity planning, while others purchase software like IDC's Uptime Infrastructure Monitor to assist in the process.



**FIGURE 12-13** Idera's Uptime Infrastructure Monitor is an example of capacity planning software and services.

Source: Uptime Software

(Tilley and Rosenblatt, 2024)

## 6.5 SYSTEM SECURITY

Security is a vital component of every information system. It ensures the system remains protected, secure, and dependable. In a global environment filled with numerous threats and attacks, the importance of security has never been greater. This section covers system security concepts, risk management strategies, and common attacks that systems may face.

### 6.5.1 System Security Concepts

Security is a critical aspect of every information system, ensuring the system remains safe, reliable, and protected from threats. In today's global environment, where various types of attacks are prevalent, security has become more crucial than ever. This section discusses system security concepts, risk management, and common attacks.



**FIGURE 12-14** System security must provide information confidentiality, integrity, and availability (CIA).

(Tilley and Rosenblatt, 2024)

The CIA Triangle (Confidentiality, Integrity, and Availability) outlines the three main elements of system security:

Confidentiality protects information from unauthorized access and ensures privacy.

Integrity prevents unauthorized modification, creation, or deletion of information.

Availability guarantees that authorized users have reliable and timely access to necessary information.

The first step in managing IT security is to establish a security policy based on these three elements.

#### 6.5.2 Risk Management

Achieving absolute security is unrealistic. Instead, managers need to balance the value of protected assets, potential risks to the organization, and the costs of security. For instance, investing in a high-end security system for an empty warehouse might not be justifiable. To achieve effective security, most organizations implement a risk management strategy focusing on three interconnected tasks: risk identification, risk assessment, and risk control.



**FIGURE 12-15** Risk management requires continuous risk identification, assessment, and control.

(Tilley and Rosenblatt, 2024)

Risk Identification involves listing and classifying business assets, such as hardware, software, data, networks, and people. The manager then evaluates potential threats to each asset and identifies vulnerabilities.

Risk Assessment evaluates the likelihood and potential impact of risks, helping prioritize those that require immediate attention.

Risk Control includes developing safeguards to mitigate risks, such as installing firewalls or assigning permissions to sensitive files.

### Threat Categories and Examples

| THREAT CATEGORY                            | EXAMPLE                                                                                                               |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Extortion                                  | Hacker steals trade secrets and threatens to release them if not paid.                                                |
| Hardware and software failures             | Router stops functioning, or software causes the application server to crash.                                         |
| Human error or failure                     | Employee accidentally deletes a file.                                                                                 |
| Natural disasters                          | Flood destroys company building and networked systems.                                                                |
| Service failure                            | Electricity is disrupted and brings the entire system down for hours.                                                 |
| Software attack                            | A group plants destructive software, a virus, or a worm into a company network.                                       |
| Technical obsolescence                     | Outdated software is slow, difficult to use, and vulnerable to attacks.                                               |
| Theft of physical or intellectual property | Physical server is stolen, intellectual property is stolen or used without permission; may be physical or electronic. |
| Trespass and espionage                     | Employee enters unlocked server room and views the payroll data on a forbidden system.                                |
| Vandalism                                  | Attacker defaces website logo, or destroys CEO's hard drive physically or electronically.                             |

**FIGURE 12-16** System threats can be grouped into several broad categories.

(Tilley and Rosenblatt, 2024)

Risk managers use one of four strategies for controlling risks:

Avoidance: Eliminating the risk through additional safeguards (e.g., installing a secure firewall to prevent unauthorized access).

Mitigation: Reducing the impact of the risk by planning for possible scenarios (e.g., preparing a disaster recovery plan for natural disasters).

Transference: Shifting the risk to another entity, such as an insurance company.

Acceptance: Choosing not to act when the cost of protection exceeds the risk, often after careful evaluation.

Risk management is an ongoing, iterative process that involves continuous identification, assessment, and control of risks. Effective risk managers require a mix of business expertise, IT knowledge, and experience with security tools and techniques.

### 6.5.3 Attacker Profiles and Attacks

An attack is a hostile act aimed at damaging or stealing from a system or organization. Attackers can range from disgruntled employees to distant hackers, and they may target systems to cause harm, steal information, or gain recognition. Attackers are categorized based on their motives and methods, as shown in Figure 12-17. Common types of attacks, described in Figure 12-18, highlight the variety of threats companies face. To address these risks, businesses adopt a multi-layered security strategy to protect against a broad range of potential attacks.

#### Attacker Characteristics

| ATTACKER       | DESCRIPTION                                                                                                                                 | SKILL SET |
|----------------|---------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Cyberterrorist | Attacks to advance political, social, or ideological goals.                                                                                 | High      |
| Employee       | Uses unauthorized information or privileges to break into computer systems, steal information, or cause damage.                             | Varies    |
| Hacker         | Uses advanced skills to attack computer systems with malicious intent (black hat) or to expose flaws and improve security (white hat).      | High      |
| Hacktivist     | Attacks to further a social or political cause; often involves shutting down or defacing websites.                                          | Varies    |
| Script kiddie  | Inexperienced or juvenile hacker who uses readily available malicious software to disrupt or damage computer systems, and gain recognition. | Low       |
| Spy            | Non-employee who breaks into computer systems to steal information and sell it.                                                             | High      |

**FIGURE 12-17** IT security professionals have names for various types of attackers.

(Tilley and Rosenblatt, 2024)

## Types of Attacks and Examples

| ATTACK                                             | EXAMPLES                                                                                                                                                                                                                                                   |
|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Back door                                          | Attacker finds vulnerability in software package and exploits it.                                                                                                                                                                                          |
| Denial of service or distributed denial of service | One or more computers send a stream of connection requests to disable a Web server.                                                                                                                                                                        |
| Dumpster diving                                    | Attacker scours the trash for valuable information that can be used to compromise the system.                                                                                                                                                              |
| Mail bombing                                       | Enormous volumes of email are sent to a target address.                                                                                                                                                                                                    |
| Malicious code                                     | Attacker sends infected email to the target system. Attackers may use viruses, worms, Trojan horses, keystroke loggers, spyware, or scripts to destroy data, bog down systems, spy on users, or assume control of infected systems.                        |
| Man in the middle                                  | The attacker intercepts traffic and poses as the recipient, sending the data to the legitimate recipient but only after reading the traffic or modifying it.                                                                                               |
| Password cracking                                  | Hacker attempts to discover a password to gain entry into a secured system. This can be a dictionary attack, where numerous words are tried, or a brute force attack, where every combination of characters is attempted.                                  |
| Phishing                                           | False DNS (Domain Name Server) information steers the user to the attacker's website. Attackers trick users into thinking they are visiting a legitimate site, such as a bank site, then attempt to obtain bank account numbers, usernames, and passwords. |
| Privilege escalation                               | Employee tricks a computer into raising his or her account to the administrator level.                                                                                                                                                                     |

(continues)

| ATTACK             | EXAMPLES                                                                                                                            |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Sniffing           | Network traffic is intercepted and scanned for valuable information.                                                                |
| Social engineering | An attacker calls the service desk posing as a legitimate user and requests that his or her password be changed.                    |
| Spam               | Unwanted, useless email is sent continuously to business email accounts, wasting time and decreasing productivity.                  |
| Spoofing           | IP address is forged to match a trusted host, and similar content may be displayed to simulate the real site for unlawful purposes. |

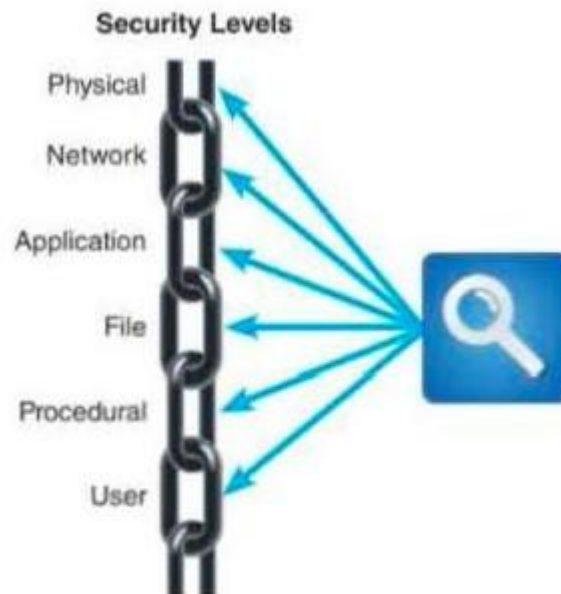
**FIGURE 12-18** Attacks can take many forms, as this table shows. IT security managers must be able to detect these attacks and respond with suitable countermeasures.

(Tilley and Rosenblatt, 2024)

## 6.6 SECURITY LEVELS

System security requires a comprehensive approach that encompasses six interconnected levels of protection. These levels include physical security, network security, application security, file security, user security, and procedural security, all of which must work together seamlessly. Similar to the links in a chain, the overall security of a system is determined by its

most vulnerable component. While top-level management typically has the final say in strategic decisions and budget allocations related to security measures, it's crucial for systems analysts to have a thorough understanding of all security aspects to provide well-informed recommendations regarding these interconnected security levels and their specific requirements. Like the chain shown in Figure 12-19, system security is only as strong as the weakest link.



**FIGURE 12-19** Each security link has a specific focus, and the overall chain is only as strong as the weakest link.

(Tilley and Rosenblatt, 2024)

### 6.6.1 Physical Security

The first level of system security focuses on the physical environment, which includes IT resources and personnel across the organization. It is crucial to pay special attention to critical equipment located in operations centers, where servers, network hardware, and other related devices are managed. Large companies typically have dedicated rooms specifically designed for IT operations, while smaller businesses may repurpose an office or storage space. Regardless of size or structure, an operations center must be safeguarded from unauthorized access. In addition to the centrally located equipment, all computers within the network must be secured, as each server or workstation could serve as a potential entry point. Physical access to computers represents a point of entry into the system, which needs to be carefully controlled and protected.

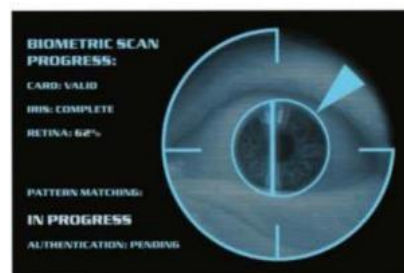
#### **OPERATIONS CENTER SECURITY:**

Perimeter security is crucial in any room or area where computer equipment is operated or maintained. Physical access should be strictly controlled, with each entrance fitted with a suitable security device. All access doors should feature internal hinges and electromagnetic

locks with battery backup systems to ensure power availability in case of an outage. When the battery runs out, the doors should fail in a closed position, but there must be an emergency release allowing those inside the room to open the door.

To improve security, many companies are incorporating biometric scanning systems that analyze an individual's facial features, fingerprints, handprints, or eye characteristics, as illustrated in Figure 12-20. These advanced authentication methods replace magnetic ID badges, which are susceptible to loss, theft, or tampering. For example, Apple's Face ID system, mentioned in Chapter 2, is a biometric security method used for smartphones and mobile devices, as discussed in the section on portable computers.

Video cameras and motion sensors can also be deployed to monitor the security of the computer room, documenting all physical activity. Motion sensors use infrared technology to detect movement and can be set to trigger audible or silent alarms and send email alerts when activated. Additionally, sensors can monitor the temperature and humidity within the computer room. Motion sensor alarms should be set to activate during times when no activity is expected, and authorized technicians should have codes to disable or enable the alarms as needed.



**FIGURE 12-20** Companies use biometric scanning to analyze the features of the eye's iris, which has more than 200 points that can be measured and used for comparison.

(Tilley and Rosenblatt, 2024)

## **SERVERS AND DESKTOP COMPUTERS:**

Whenever possible, server and desktop computer cases should be equipped with locks. This simple precaution helps prevent an intruder from altering the hardware configuration, damaging the equipment, or removing disk drives. Server racks should also be locked to prevent unauthorized insertion or removal of keystroke loggers. A keystroke logger is a device placed between a keyboard and computer, typically appearing as an ordinary cable plug, making it inconspicuous. It records everything typed, including passwords, while the system operates normally. While keystroke loggers can be used legitimately for system monitoring and restoration, they represent a serious security risk if used maliciously by an intruder.

In addition to hardware-based keystroke loggers, software-based versions exist. These programs, which can be disguised as legitimate software, are often downloaded from the internet or a company network. They operate invisibly, recording keystrokes and transmitting the information to the person who installed them. Such malicious software can be removed by antivirus and antispyware programs, discussed later in the Application Security section.

Tamper-evident cases are a valuable addition wherever possible. These cases are designed to indicate any attempt to open or unlock them. If a computer case is opened, an indicator LED remains lit until it is cleared with a password. While tamper-evident cases don't prevent

unauthorized access, they make it more likely that a security breach will be detected. Many modern servers are now offered with tamper-evident cases as standard.

For servers and workstations left unattended, monitor screen savers should be used that hide the display and require special passwords to be cleared. Locking the screen after a period of inactivity is another protective measure. A BIOS-level password (also known as a boot-level or power-on password) can be set up to prevent unauthorized individuals from booting the computer using secondary devices. This password needs to be entered before the computer can be started.

Lastly, companies must plan for power reliability. For mission-critical systems, large-scale backup power sources are necessary to ensure business continuity. In other cases, computer systems and network devices should be connected to an uninterruptible power supply (UPS) that includes battery backup with enough capacity. The UPS should be capable of supporting short-term operations to allow for proper system shutdown and backup processes.

## **PORTABLE COMPUTERS:**

When evaluating physical security, it's important to account for extra security measures for notebooks, laptops, and tablet computers. Due to their small size and high value, these devices are prime targets for theft and industrial espionage. While the following suggestions focus on notebook security, many also apply to desktop workstations.

Select a secure operating system: Choose an OS that supports secure logins, BIOS-level passwords, and strong firewall protection. Always use a user account with limited privileges rather than an administrator account. Additionally, obscure the administrator account name to make it harder for casual intruders to guess.

Mark the computer: Engrave the device with the company name and address or attach a tamper-proof asset ID tag. Many hardware vendors offer the option to add an asset ID tag to the BIOS, so a message like "Property of [Company Name] – Company Use Only" can appear upon startup. While these steps may not deter professional thieves, they can make the device less appealing to casual thieves or make it more difficult to resell. Security experts also recommend using a generic carrying case instead of a custom one that might attract attention.

Consider biometric devices: Opt for a computer with a built-in fingerprint reader or facial recognition system for an extra layer of security.

Universal Security Slot (USS): Many laptops are equipped with a USS that allows you to attach a cable lock or laptop alarm. While these precautions might not stop a professional thief, they can discourage casual theft.

Backup data: Always back up all critical data before using the notebook computer outside the office. For highly sensitive data, use removable media like a flash memory device instead of storing it on the laptop's hard drive.

Use tracking software: Consider installing software that enables the laptop to periodically contact a security tracking center. If the device is stolen, the tracking system can identify the computer and its location, helping law enforcement agencies recover it.

Location services and remote wiping: Services like Apple's Find My iPhone, Google's Find My Device, and Microsoft's similar offerings help locate lost devices. These services also allow remote data wiping and factory resets. For example, Apple's app, shown in Figure 12-21, can remotely erase data from a stolen iPhone.

Stay alert while traveling: Be cautious in high-risk situations where thieves might create a

distraction, like crowded airport baggage claim areas, rental car counters, and security checkpoints. When traveling by car, store the computer in a trunk or lockable compartment where it won't be visible.

Establish strong password protection policies: Require strong passwords with a minimum length and complexity, and set limits on how many invalid login attempts can be made before the system locks. In some cases, consider implementing file encryption policies to protect extremely sensitive data.



**FIGURE 12-21** Apple's Find My iPhone app helps customers locate lost devices and perform remote data wipes and factory resets if needed.  
Source: Scott Tilly/Apple

(Tilley and Rosenblatt, 2024)

## CASE IN POINT 12.2: OUTER BANKS COUNTY

Outer Banks County is a 200-square-mile area in coastal North Carolina, and you are the IT manager. The county has about a hundred office employees who perform clerical tasks in various departments. A recent budget crisis has resulted in a wage and hiring freeze, and morale has declined. The county manager has asked you to install some type of keystroke logger to monitor employees and determine whether they are fully productive. After your conversation, you wonder whether there might be some potential privacy and security issues involved.

For example, does an employer have a duty to notify its employees that it is monitoring them? Should the employer notify them even if not required to do so? From a human resources viewpoint, what would be the best way to approach this issue? Also, does a potential security issue exist? If an unauthorized person gained possession of the keystroke log, he or she might be able to uncover passwords and other sensitive data.

What are your conclusions? Are these issues important, and how would you respond to the county manager's recommendation? Before you answer, you should go on the Internet and learn more about keystroke loggers generally, and specific products that currently are available.

## 6.6.2 Network Security

A network consists of two or more devices connected to share data, known as network traffic. To connect to a network, a computer needs a network interface, which includes hardware and software that enables the device to interact with the network. Network traffic can be encrypted to protect it from unauthorized access, which involves encoding the data to ensure

its confidentiality.

## **ENCRYPTING NETWORK TRAFFIC**

Network traffic is vulnerable to interception and potential manipulation, redirection, or recording. For example, if an unencrypted password or credit card number is sent over a network, it could be stolen. Encryption masks the content and purpose of the traffic, making it unreadable to unauthorized users, though it remains visible.

There are two primary encryption techniques:

**Private Key Encryption (Symmetric Encryption):** This method uses a single key to both encrypt and decrypt data. While it is fast and straightforward, it presents a key challenge: both the sender and the receiver must share the same key. This introduces the risk of the key being intercepted if it's transmitted along with the message.

**Public Key Encryption (PKE) (Asymmetric Encryption):** Each user has a pair of keys: a public key and a private key. The public key encrypts messages, and it can be shared freely. Only the private key can decrypt these messages, which is kept secure. Public key encryption is commonly used in secure online transactions.

## **WIRELESS NETWORKS**

Wireless network security is especially critical because wireless transmissions are more vulnerable than those over wired networks. Encrypting wireless traffic makes intercepted data unusable to unauthorized parties. The evolution of wireless security includes:

**Wired Equivalent Privacy (WEP):** This early method required a pre-shared key for each wireless client but offered weak protection.

**Wi-Fi Protected Access (WPA):** WPA significantly improved security compared to WEP, using protocols from the Wi-Fi Alliance.

**WPA2:** This is the latest enhancement to wireless security and extends WPA with the full implementation of the IEEE 802.11i standard. WPA2 became mandatory for new Wi-Fi-certified devices after 2006 and is compatible with WPA, making it easier to upgrade security.

## **PRIVATE NETWORKS**

Encrypting all network traffic can significantly slow down the network, so it is not always practical. In some cases, such as between a web server and a database server, businesses may use a private network. This is a dedicated connection that doesn't interact with external networks, meaning unencrypted traffic is safe from external interception. Each device in a private network has a dedicated interface, and no interface connects to an outside network, allowing unencrypted data to remain secure.

## **VIRTUAL PRIVATE NETWORKS (VPNs)**

For larger groups of remote users, Virtual Private Networks (VPNs) offer a secure alternative. A VPN uses a public network, like the internet, to securely connect remote users. Instead of using a dedicated connection, a VPN establishes a secure "tunnel" between the user and the company's internal network. This encrypted tunnel ensures that all transmitted data is protected. VPNs provide a practical solution for secure remote work.

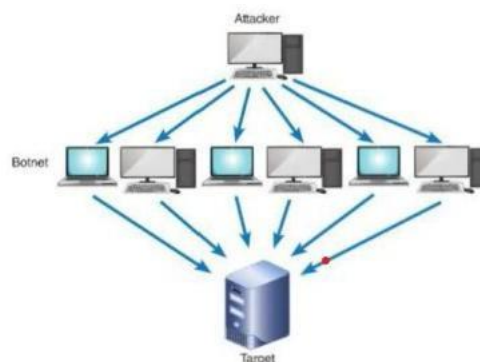
## PORTS AND SERVICES

A port is a number used to route incoming traffic to the correct application on a device. In TCP/IP networks, each packet of data contains a destination port, which tells the computer where to send it. Ports are crucial for computer security because open ports can be exploited by attackers.

**Port Scans:** Port scanning is a technique used by attackers to detect which services are running on a computer by attempting connections to different ports. A successful connection to port 25, for example, would indicate the presence of an email service. These scans can be used to map out a network and identify vulnerabilities.

**Denial of Service (DoS) Attacks:** A DoS attack occurs when an attacker floods a service with numerous requests, overwhelming the server and preventing it from responding to legitimate traffic. This can cause a server to crash or become unresponsive.

**Distributed Denial of Service (DDoS) Attacks:** A more severe form of DoS attack, a DDoS involves multiple attacking computers coordinating their efforts to overwhelm a server. This can cause significant disruptions to the target system, as illustrated in Figure 12-22. DDoS attacks represent a serious cybersecurity threat, and organizations like the U.S. Computer Emergency Readiness Team (US-CERT) are established to respond to such incidents (as shown in Figure 12-23).



**FIGURE 12-22** In a DoS attack, an attacker sends numerous authentication requests with false return addresses. The server tries unsuccessfully to send out authentication approval and is eventually disabled by the flood of requests. More sophisticated DoS attacks are distributed (DDoS), as shown in this figure. Instead of a single computer, the attacker uses an army of botnets (computers unknowingly infected with malware that are difficult to trace) to attack the target.



**FIGURE 12-23** The US-CERT is a key player in the battle against cybersecurity threats.  
Source: US-CERT

(Tilley and Rosenblatt, 2024)

**Firewalls:** A firewall serves as the primary barrier between a local network (or intranet) and the internet. It requires at least one network interface connected to the internet and another to the local network. Firewall software monitors all network traffic to and from each interface. Pre-configured rules define conditions for whether traffic will be allowed. When a rule matches, the firewall will either accept, reject, or drop the traffic. If the firewall rejects traffic,

it sends a response indicating that access is denied, but if traffic is dropped, there's no response. Firewalls can also be set up to detect and react to denial-of-service attacks, port scans, and other suspicious activities.

| RULE | INTERFACE | SOURCE | DESTINATION | PORT | ACTION |
|------|-----------|--------|-------------|------|--------|
| 1    | Public    | Any    | Web Server  | 80   | Accept |
| 2    | Public    | Any    | Web Server  | 443  | Accept |
| 3    | Public    | Any    | Web Server  | Any  | Reject |
| 4    | Public    | Any    | Mail Server | 25   | Accept |
| 5    | Public    | Any    | Mail Server | Any  | Reject |
| 6    | Public    | Any    | Any         | Any  | Drop   |
| 7    | Private   | LAN    | Any         | Any  | Accept |

FIGURE 12-24 Examples of rules that determine whether the firewall will allow traffic to pass.

(Tilley and Rosenblatt, 2024)

**Network Intrusion Detection:** If an intruder tries to breach a system, an alarm should go off when specific attack patterns or unusual activity are identified. A Network Intrusion Detection System (NIDS) works like a burglar alarm, alerting administrators to suspicious network traffic or configuration violations. NIDS needs fine-tuning to distinguish between legitimate traffic and attacks, and it should be installed on network devices like switches that can monitor all network traffic. While managing an NIDS can be time-consuming, it provides valuable insights into attacker behavior and network performance.

**Application Security:** Besides securing computer rooms and network traffic, server-based applications must also be safeguarded. This involves analyzing application functions, identifying security risks, and reviewing documentation. Key areas include managing services, hardening systems, setting application permissions, validating inputs, applying patches, and reviewing software logs.

**Services:** Services are applications listening on specific ports. Unnecessary services should be disabled to enhance security and performance. A service that's not needed might be vulnerable to exploitation, such as a poorly configured FTP service that allows hackers to upload harmful code.

**Hardening:** This process strengthens a system by removing unnecessary accounts, services, and features. Default configurations may introduce vulnerabilities, so hardening reduces risks by eliminating weaknesses, such as weak account permissions. Antivirus and antispyware software can also be part of the hardening process to protect against malware.

**Application Permissions:** Applications should restrict access based on user roles. For example, administrators have full access, while other users may have limited access. Restricting permissions prevents unauthorized changes to applications.

**Input Validation:** This involves checking user inputs to ensure they meet expected criteria. Proper validation prevents errors, improves data integrity, and reduces system maintenance. Failing to validate inputs can lead to incorrect outputs and system behavior.

**Patches and Updates:** Software vulnerabilities are discovered over time, and patches are issued to fix these issues. However, patches should be carefully tested before being applied, as they could introduce other problems. Automated update services can handle patches but must be used cautiously to avoid unintended consequences.

**Software Logs:** Logs track events, errors, and system activity, providing insights into potential attacks and helping administrators track unauthorized access. Logs should be reviewed regularly, and an NIDS can alert administrators to suspicious events.

**File Security:** Protecting files that contain sensitive data is crucial in any security program.

Encryption and file permissions help safeguard these files.

**Encryption:** This is the process of making files unreadable to unauthorized users. Sensitive information, such as financial or personnel records, should always be encrypted to protect privacy. Most modern operating systems offer built-in encryption features.

**Permissions:** File security relies on permissions that determine what actions users can perform on files. Permissions like read, write, and execute control access, ensuring that users only have the necessary rights for their tasks. Administrators should assign minimal permissions to limit risk.

**User Groups:** For easier management, user groups can be created with specific permissions. Instead of assigning permissions to individual users, it's more efficient to assign them to groups based on roles, which is particularly helpful when users change jobs within an organization.

**User Security:** Security also depends on users' actions and their adherence to best practices. Compromised user accounts are often the entry point for attackers, so it's critical to manage user identities, passwords, and be aware of social engineering tactics.

**Identity Management:** This refers to the controls and processes that identify legitimate users and system components. A balanced identity management strategy ensures secure and efficient access while maintaining privacy.

**Password Protection:** Passwords should be complex and changed regularly. IT managers should enforce policies that require passwords to meet specific criteria. However, users often bypass these measures by choosing weak passwords or writing them down in insecure places.

**Social Engineering:** Attackers may attempt to manipulate users into disclosing information through social interactions, such as pretending to be someone else. Common techniques include pretexting, where attackers gather personal information to commit fraud or identity theft. Awareness training can help users recognize and resist these tactics.

**User Resistance:** Some users may resist strict security measures, seeing them as inconvenient. It's essential to communicate the importance of security to users and create a culture of awareness. When users understand the impact of security on the company and its stakeholders, they are more likely to adhere to protocols.

**New Technologies:** Emerging technologies can enhance security but also introduce new risks. For example, security tokens can add an extra layer of authentication, but powerful search applications can inadvertently expose sensitive information. Proper safeguards are necessary to manage risks associated with these technologies.



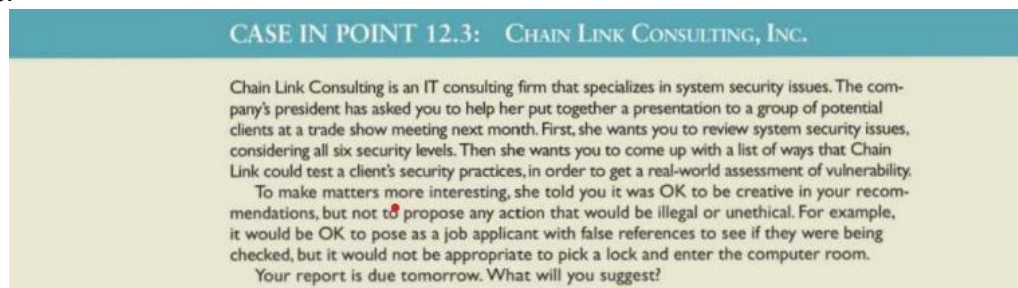
**FIGURE 12-25** Security tokens, which come in various forms, can provide an additional level of security.  
AP/Sputnik/Shutterstock.com

(Tilley and Rosenblatt, 2024)

**Procedural Security:** This focuses on the policies and procedures that ensure secure operations. Management must create a security-focused corporate culture, emphasizing that security is everyone's responsibility.

**Need-to-Know Concept:** This principle limits access to sensitive information to those who

need it for their tasks. Many organizations apply classification levels to restrict access to documents, ensuring that only authorized individuals can view highly sensitive information. Training and Awareness: Procedural security requires continuous support from management and regular employee training to ensure that security measures are followed. Employees should understand their responsibilities and be aware of security risks, both physical (e.g., securing documents) and digital (e.g., logging out of systems when not in use). By implementing these strategies and ensuring that all employees are educated on their roles in maintaining security, organizations can better protect their systems, data, and users from attacks.



(Tilley and Rosenblatt, 2024)

## 6. 7 BACKUP AND RECOVERY

Every system must have provisions for data backup and recovery. Backup refers to the process of copying data at specified intervals or continuously, ensuring that data is regularly stored in case of unexpected loss or system failure. Recovery involves restoring the backed-up data and restarting the system after an interruption or disaster to resume normal operations.

An overall backup and recovery plan that prepares for potential disasters is called a disaster recovery plan (DRP). A DRP ensures that a company can recover from catastrophic events, minimizing downtime and data loss, and maintaining business continuity.

Every system must have provisions for both data backup and recovery. Backup refers to copying data at specified intervals or continuously, while recovery involves restoring the data and restarting the system after an interruption. A comprehensive backup and recovery strategy, designed to prepare for potential disasters, is known as a disaster recovery plan.

### 6.7.1 Global Terrorism

The tragic events of September 11, 2001, and growing concerns about global terrorism have prompted many companies to improve their backup and disaster recovery plans. Increased attention to disaster recovery has led to the development of a whole new industry, offering new tools and strategies. Many IT professionals believe that the concern over terrorism has heightened security awareness across the corporate world. While backup and disaster recovery are distinct topics, they are often closely linked.

### 6.7.2 Backup Policies

A key element of business data protection is a backup policy, which outlines specific instructions and procedures. A robust backup policy can help a business continue operations

and survive a disaster. The backup policy should address backup media, backup types, and retention periods.

## **BACKUP MEDIA**

Backup media can include tapes, hard drives, optical storage, and online storage. Physical backups should be clearly labeled and stored securely. Offsiting is the practice of storing backup media in a location different from the main business site to protect against disasters like floods, fires, or earthquakes. While many operating systems include built-in backup utilities, many system administrators prefer using specialized third-party software for large-scale operations, offering more flexibility and better control.

In addition to offline storage, cloud-based storage is rapidly growing in popularity. Many companies rely on cloud backup and retrieval services from top providers. For small and medium-sized businesses, this option is often cost-effective and reliable.

## **BACKUP TYPES**

There are different types of backups: full, differential, incremental, and continuous.

**Full Backup:** A full backup copies all files on the system. While thorough, frequent full backups can be time-consuming and repetitive if most files haven't changed since the last backup.

**Differential Backup:** This type backs up only the files that have changed or been created since the last full backup. To restore the system, the last full backup is restored first, followed by the most recent differential backup. A combination of full and differential backups is often recommended, as it minimizes storage use while being relatively simple.

**Incremental Backup:** This method only backs up files that have not been backed up previously. Although it is faster and requires less storage, restoring the system takes longer, as each incremental backup must be applied sequentially.

**Continuous Backup:** Continuous backup is a real-time method that records all system activity as it happens. This method demands significant hardware, software, and network resources but allows for rapid and effective system restoration. Continuous backup often uses a redundant array of independent disks (RAID) system to mirror data, providing fault tolerance. This means that the failure of one disk does not impact the system. RAID systems offer better performance, higher capacity, and increased reliability compared to a single large disk. When set up on a server, a RAID array of multiple drives appears to the computer as a single logical drive.

Figure 12-26 compares the different backup methods.

| BACKUP TYPE  | CHARACTERISTICS                                                                | PROS AND CONS                                                                                                                                                                         | TYPICAL FREQUENCY                                           |
|--------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| Full         | Backs up all files.                                                            | Slowest backup time and requires the most storage space. Rapid recovery because all files are restored in a single step.                                                              | Monthly or weekly.                                          |
| Differential | Only backs up files that are new or changed since the last full backup.        | Faster than a full backup and requires less storage space. All data can be restored in just two steps by using the last full backup and the last differential backup.                 | Weekly or daily.                                            |
| Incremental  | Only backs up files that are new or changed since the last backup of any kind. | Fastest backup and requires the least storage space because it only saves files that have never been backed up. However, requires many restore steps—one for each incremental backup. | Daily or more often.                                        |
| Continuous   | Real-time, streaming method that records all system activity.                  | Very expensive hardware, software, and network capacity. Recovery is very fast because system can be restored to just before an interruption.                                         | Usually only used by large firms and network-based systems. |

FIGURE 12-26 Comparison of full, differential, incremental, and continuous backup methods.

(Tilley and Rosenblatt, 2024)

## 6.7.4 Retention Periods

Backups are stored for a specific retention period, after which they are either destroyed or the backup media is reused. These retention periods may be defined by legal requirements or company policy and can vary in length, ranging from a few months to several years. It is crucial that stored media be properly secured, protected, and periodically inventoried to ensure the integrity and safety of the backup data.

## 6.7.5 Business Continuity Issues

Global concerns about terrorism have heightened awareness and resulted in increased top-level management support for a business continuity strategy in the event of an emergency. A disaster recovery plan outlines the necessary actions to take, specifies key individuals and rescue authorities to notify, and defines the roles of employees in evacuation, mitigation, and recovery efforts. This plan should be accompanied by a test plan that simulates various levels of emergency scenarios, allowing responses to be recorded, analyzed, and improved as necessary.

Once personnel are safe, the next priority is to mitigate damage to company assets. The disaster recovery plan may include measures such as shutting down systems to prevent further data loss or relocating physical assets to secure areas. After these immediate actions, the focus shifts to resuming business operations, which could involve salvaging or replacing damaged equipment and recovering data from backups. The ultimate goal of a disaster recovery plan is to restore business operations to pre-disaster levels.

Disaster recovery plans are often a component of a broader business continuity plan (BCP), which goes beyond just recovery and defines how essential business functions will continue during a major disruption. Some BCPs specify the use of a hot site—an alternate IT location that can take over critical systems in the event of a power outage, system crash, or physical disaster. A hot site requires data replication, ensuring that any transaction on the primary system is mirrored in real time on the hot site. If the primary system becomes unavailable, the hot site will have the most up-to-date data and can continue operations with no downtime.

While hot sites offer a high level of reliability, they can be very expensive. Despite this cost, a hot site provides the most robust insurance against major business disruptions. Along with hot sites, business insurance can be vital in a worst-case scenario. Although costly, business insurance can help offset the financial losses caused by system failures and business interruptions.

## **6.8 SYSTEM RETIREMENT**

At some point, every system becomes obsolete and is ready for retirement. For example, in the 1960s, punched cards were the cutting edge of data management. Data was stored by punching holes in specific positions, and machines could sense the presence or absence of these holes. Each full-size card stored only 80 characters (or bytes), meaning more than 12,000 cards were needed to store a single megabyte. Punched cards were even used for checks and utility bills, but today, this technology is considered obsolete.

Constantly changing technology means that every system has a limited economic life span. Analysts and managers can predict system obsolescence in several ways, and it should never come as a complete surprise. A system becomes obsolete when it no longer supports user needs, or when the platform becomes outdated.

The most common reasons for discontinuing a system include:

**Increasing maintenance:** The system's maintenance history indicates that adaptive and corrective maintenance requirements are growing steadily.

**Operational costs:** Operational costs or execution times are increasing rapidly, and routine preventive maintenance does not reverse or slow down the trend.

**New software:** A software package is available that provides the same or additional services faster, better, and more affordably than the current system.

**Emerging technology:** New technology offers a way to perform the same or additional functions more efficiently.

**Expensive maintenance:** Maintenance changes or additions are difficult and expensive to perform.

**User demands:** Users request significant new features to support business requirements.

Once the system reaches its final stages, users are unlikely to submit new requests for adaptive maintenance because they are anticipating a new system release. Similarly, the IT staff typically does not perform much perfective or preventive maintenance because the system will not be around long enough to justify the cost. At this point, corrective maintenance is the only activity required to keep the system operational.

The user satisfaction factor usually determines the life span of a system. The critical success factor is whether the system helps users achieve their operational and business goals. Negative feedback should be investigated and documented, as it could signal the system is approaching obsolescence.

As the system reaches the end of its operational life, maintenance costs rise, users begin

asking for more features, new system requests are submitted, and the SDLC (Systems Development Life Cycle) begins again with a new system being planned and developed.

## **6.9 FUTURE CHALLENGES AND OPPORTUNITIES**

There is an old saying that the only constant in life is change, and the same is true for information technology (IT)—but the rate of change in IT seems to increase every year. Rapid technological advancements pose numerous challenges for organizations and individuals, but they also open up exciting new opportunities. The key to success is being ready for these changes and being proactive rather than reactive.

No prudent professional would embark on a complex journey without a map or plan, and similarly, companies require strategic plans (which were discussed in Chapter 2) to navigate the future of IT. Individuals also need to have a specific goal or destination in mind. To prepare for the challenges ahead, IT professionals must continuously plan and develop their knowledge, skills, and credentials.

### **6.9.1 Trends and Predictions**

Navigating an IT career can be compared to sailing a small ship in difficult seas. Even with a great map and plan, a good captain will face challenges beyond their control. The key is to understand the forces at play and try to prepare for them. Figure 12-27 (not provided) outlines some of the winds of change that may influence IT trends, including globalization, technology integration, service orientation, cloud computing, and the workplace of the future.

In addition to these trends, most organizations will also face economic, social, and political uncertainty. Many IT experts believe that, in this environment, the top priorities will be:

Safety and security of corporate operations.

Agility and the ability to quickly respond to changing market forces.

Bottom-line Total Cost of Ownership (TCO).

| TREND                         | DESCRIPTION                                                                                                                                                                                                                                                                                        | IMPACT ON IT GENERALLY                                                                                                                                                                                                                                                                                                         | IMPACT ON FUTURE SYSTEMS ANALYSTS                                                                                                                                                                                                                                                                            |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Globalization</b>          | New kinds of corporations will be globally integrated, driven by the Internet, and free of traditional trade barriers.                                                                                                                                                                             | Language skills will be extremely important. Technology will open new opportunities for developing countries to compete more effectively. Knowledge workers will communicate and collaborate across borders and boundaries. Work will be done 24/7 across multiple time zones.                                                 | Systems analysts will be directly affected by global trends. They will probably work for more firms in their careers, be exposed to more information, need more technical skills, and see greater change than at any time in history.                                                                        |
| <b>Technology Integration</b> | Powered by an enormous increase in computing power, new IT models will include networks with smart, interconnected devices such as communication systems, automobiles, entertainment, transportation infrastructure, and "smart" power grids.                                                      | New technology will drive major changes in how personal and business services are provided. Firms will compete in a global marketplace that will reward innovation, creativity, and positive societal outcomes.                                                                                                                | The systems analyst will need a mix of business savvy and technical skills. He or she will have a unique opportunity to work at the intersection of business operations and IT. Synergy between technology growth and globalization will create jobs and opportunities for people with the right skills.     |
| <b>Service Orientation</b>    | A business model of computing where processing capabilities and data storage are leased (rented) instead of bought (owned). Computing capabilities are seen as services used on-demand, like electricity or water, rather than as managed programs.                                                | Service orientation is a paradigm shift. It represents a maturing of the field, where end users no longer care about technical details as much as business functionality. IT systems will be built using SOA (service-oriented architecture) to enable current and new functionality to be delivered as services to customers. | Systems analysts will need to increase their business acumen to work effectively in a service-oriented IT environment. Technical skills will still be needed, but services are about fulfilling business requirements.                                                                                       |
| <b>Cloud Computing</b>        | Cloud computing is a new model for delivering business and IT services. It will have a profound impact on business strategies, including product innovation, marketing, customer relationships, and social media. Cloud computing provides the IT infrastructure that enables service orientation. | The full potential of cloud computing will be realized by firms that can integrate cloud-based infrastructure, services, servers, data access, and a secure environment. Creativity and innovation will be rewarded, and competition will be intense. Cloud computing will lower TCO by reducing capital expenditures.         | Like any new technology, systems development for cloud-based applications will be a challenge for some and an opportunity for others. Regardless of the development method or platform, an analyst will need business savvy and technical know-how to help an organization take full advantage of the cloud. |
| <b>The Workplace</b>          | In the face of worldwide competition and enormous technology change, firms will stress innovation, vision, and the ability to adapt rapidly. The ability to think critically will be vital.                                                                                                        | Successful IT workers must be able to innovate, analyze, communicate effectively, and "think outside the box." The winners will be those who can adapt to change and embrace new technology and new ways of doing business.                                                                                                    | Students preparing for the workplace of tomorrow will need a strong skill set. Systems analysts will be expected to bring communications, modeling, problem-solving, decision making, and critical thinking skills to the workplace — and to be aware of ethical issues that might affect them.              |

FIGURE 12-27 Major trends and their impact on IT in general and on future systems analysts.

(Tilley and Rosenblatt, 2024)

Here are some examples of possible trends and developments over the next few years:

Cybercrime will increase significantly, leading to negative financial, social, and national security implications.

Smartphones and tablets will become the dominant computing platform for most users, overtaking traditional PCs and laptops.

Software as a Service (SaaS) will become the norm, which will affect business models and consumer costs, as the industry moves from a purchase model to a leasing model for computer applications.

Cloud computing will become the principal computing infrastructure for enterprises, enabling SaaS and lowering TCO.

Insourcing (the movement of jobs back from offshore locations) will increase, driven by factors such as:

Higher wages in emerging markets.

Improved automation with sophisticated manufacturing robots.

Security concerns about outsourced components (hardware and software) developed overseas.

Political pressure to preserve jobs.

It's also possible that large enterprises will require suppliers to certify their security credentials and sourcing policies. Another potential issue is the growth of cloud computing

and large-scale data centers. These data centers can raise concerns related to access controls, international law, ownership, and surveillance of network activity across borders. These issues will become increasingly important as digital operations are transferred online. Another emerging concern is the environmental impact of massive data storage and server farms. These farms can use substantial amounts of electricity, both for regular operation and cooling purposes, which affects both the environment and the corporate bottom line. Some enterprises may require suppliers to certify their green credentials and sourcing policies as part of their sustainability efforts.



**FIGURE 12-28** The rapid growth of data centers and cloud computing has increased security and privacy concerns.  
Getty Images/istock.com

(Tilley and Rosenblatt, 2024)

As IT continues to evolve, staying ahead of trends and adapting to new challenges will be essential for success.

### **6.9.2 Strategic Planning for IT Professionals**

A systems analyst should approach their career like a small business entrepreneur, considering their assets, potential liabilities, and specific goals. Just like companies, individuals need a strategic plan for their careers. The starting point is to answer this question: Have career goals been set for the next year, the next three years, and the next five years? By working backward from these goals, individuals can develop intermediate milestones, much like an IT project is managed. The project management tools, such as Gantt charts or PERT/CPM charts, as discussed in Chapter 3, can be used to map out the timeline for achieving these milestones. Once the plan is developed, it should be regularly monitored to ensure that progress stays on track. Just like in an agile enterprise, adjustments can be made as needed to meet career goals.

### **6.9.3 IT Credentials and Certification**

In recent years, technical credentials and certification have become increasingly important for both employers and employees in the IT field. Credentials broadly include formal degrees, diplomas, or certificates issued by educational institutions to confirm that a certain level of education has been achieved. Certification, on the other hand, refers to specific hardware and software skills that can be tested and verified through exams. For example, someone might hold a two- or four-year degree in Information Systems and also have an ISTQB Foundation Level certification, which verifies expertise

in software testing. Other credentialing bodies such as ACM and IEEE, as well as major IT companies like Microsoft, Cisco, and Oracle, offer industry-recognized certifications.

#### **6.9.4 Critical Thinking Skills**

In addition to technical expertise, systems analysts must possess soft skills, including communication, interpersonal, and perceptive abilities. Employers often express concern that new hires may be technically proficient but lack these critical soft skills. For career success—especially in senior leadership positions—it's essential to master both technical and soft skills. Moreover, IT professionals need critical thinking skills to thrive in the workplace. The most important skill learned in school is how to learn, which enables individuals to adapt to dynamic environments throughout their career. For instance, developers may not need to know the latest programming language, but they must be capable of quickly learning a new one. The significance of critical thinking in IT has been acknowledged for years and is incorporated into higher levels of Bloom's Taxonomy of learning objectives. In today's digital society, where massive amounts of data are generated, data mining and advanced algorithms are crucial, but the most valuable asset is an employee capable of solving problems. Employers value critical thinkers who can:

- Find relevant data.
- Identify key facts.
- Apply knowledge to make real-world decisions.

To develop critical thinking, professionals can practice tasks that mirror actual workplace challenges. Studying systems analysis and design is one way to build a solid foundation in techniques for model development, data organization, and pattern recognition. While formal certification is valuable in the job market, the true value lies in learning critical thinking skills and applying them to achieve career goals. Exercises such as games, puzzles, brainstorming, creative problem-solving, decision tables, and working with tools like Pareto charts, X-Y diagrams, fishbone diagrams, and Boolean logic can help strengthen critical thinking.

#### **6.9.5 Cyberethics**

As computers increasingly permeate our lives, the decisions made by IT professionals can have profound implications. Ethical dilemmas may arise, and IT professionals should be prepared to address the ethical, social, and legal aspects of their work. In the next "Question of Ethics" section, ask yourself:

- What would you do?
- Where would you draw the line?
- How much would you risk doing what you thought was the right thing?

The decisions you make could affect your job and future employment (as in Scenario 1), but other situations might have even more serious consequences (as in Scenario 2). Understanding the implications of ethical decisions in IT is crucial, as it may shape the direction of your career and professional reputation.

As the IT field evolves, it's essential to develop a combination of technical expertise, critical thinking, and ethical awareness to remain competitive and responsible in the workplace.

## A QUESTION OF ETHICS



Stock.com/5076755\_5

**SCENARIO 1:** You have just completed a routine security audit on the company's information systems, and you found several areas of vulnerability. For example, file permissions have not been updated in some time, no comprehensive password policy exists, and network traffic is not fully encrypted. You noted these areas, among others, in a report to your supervisor. The report included specific recommendations to fix the problems.

Your supervisor responded by saying that budgets are tight right now, and she could not approve your requests to resolve these issues. As an IT professional, you are very uncomfortable with the risk level, but you have been unable to sway your supervisor. When you discussed the situation with a colleague, he said, "Why worry about it? If it's good enough for her, it should be good enough for you."

What do you think of your colleague's advice, and why? Is this an ethical question? If you are still is uncomfortable, what are your options?

**SCENARIO 2:** You work for a large IT company. The company is presented with a legal directive from the federal government to divulge personal data regarding some of the company's customers who are suspected of wrongdoing.

You feel that this is a violation of the customers' privacy and you are reluctant to comply with the request. Your boss tells you the company has no choice; it must follow the law. You are told you can always resign from the company if you feel so strongly about the situation.

Instead of complying or resigning, you go public. You use the media to advocate your point of view for freedom and privacy, even while knowing that your actions will have far-reaching consequences. Instead of just losing your job, you risk losing your freedom and your future.

Did you do the right thing? Are you a hero of free speech or a criminal (and possibly even a traitor)? What other options were available to you? What should the company have done when you went public? What should the government do?

(Tilley and Rosenblatt, 2024)

## SUMMARY

### Systems Support and Security

Systems support and security cover the period from the implementation of an information system until the system is no longer used. A systems analyst's primary role in this phase is to manage and solve user support requests. Corrective maintenance involves making changes to correct errors, while adaptive maintenance addresses new system requirements, and perfective maintenance enhances system efficiency. Adaptive and perfective maintenance are often referred to as enhancements. Preventive maintenance aims to avoid future issues.

The typical maintenance process is a mini-version of the systems development life cycle. A system request for maintenance is submitted and evaluated. If accepted, the request is prioritized and scheduled for the IT group. The maintenance team follows a logical progression of investigation, analysis, design, development, testing, and implementation.

**Corrective Maintenance:** Occurs when users or IT staff report problems. Standard procedures are followed for minor errors, but immediate work is often initiated for significant issues.

**Adaptive Maintenance:** Initiated in response to user requests for system improvements or to meet changes in the business or operating environment.

**Perfective Maintenance:** Typically initiated by IT staff to improve system performance or maintainability, including program restructuring and reengineering.

**Preventive Maintenance:** Performed to avoid future issues by analyzing potential trouble areas.

A maintenance team usually consists of systems analysts and programmers. Systems analysts need the same skills for maintenance work as they do for new system development. In many IT departments, new development and maintenance teams are separate, with staff sometimes transitioning between the two groups.

### Maintenance Methodology

Change management (CM) is essential for handling maintenance requests, managing different system versions, and distributing documentation changes. Maintenance changes can be implemented immediately or through a release methodology, where non-critical changes are implemented simultaneously, which benefits users by reducing the frequency of system changes. Systems analysts use functional, allocated, and product baselines as formal reference points to measure system

characteristics at specific times.

System performance is measured using response time, bandwidth, throughput, and turnaround time. Capacity management uses these measurements to forecast future needs for service and support. CASE tools with system evaluation and maintenance features can be used during the system's operational phase.

#### Security in Information Systems

System security is a critical component of any information system, dependent on a comprehensive security policy that defines how organizational assets are protected and how attacks are managed. Risk management creates a functional security policy by identifying, analyzing, anticipating, and reducing risks to an acceptable level. Security involves six interrelated levels:

**Physical Security:** Protecting the physical environment, including critical equipment in computer rooms and safeguards for servers and desktops across the organization.

**Network Security:** Utilizing encryption techniques, private networks, and protective measures, particularly for wireless transmissions.

**Application Security:** Includes understanding services, hardening, application permissions, input validation, software patches, and logs.

**File Security:** Involves encryption and permission management for files, which can be assigned to users or user groups.

**User Security:** Managing identity, enforcing password policies, raising awareness about social engineering risks, and overcoming user resistance.

**Procedural Security:** Ensuring secure operations through managerial controls and policies.

#### Data Backup and Recovery

Data backup and recovery issues include choosing backup media, establishing backup schedules, setting retention periods, and designing backup systems such as RAID and cloud-based backups. Over time, all information systems become obsolete. A system's economic life ends when maintenance or operating costs increase rapidly, new software or hardware becomes available, or the system can no longer meet new requirements. When this occurs, the information system must be replaced, and the systems development life cycle begins again.

#### Challenges and Career Planning for IT Professionals

Many IT experts predict significant competition in the future, with economic, political, and social uncertainty. As a result, key priorities for IT professionals will include ensuring the safety and security of operations, addressing environmental concerns, and managing the bottom-line total cost of ownership (TCO).

An IT professional should have a strategic career plan that includes long-term goals and intermediate milestones. A critical aspect of this plan is obtaining IT credentials and certifications to demonstrate specific knowledge and skills. Many industry leaders offer certifications, and in addition to technical ability, critical thinking skills are increasingly valuable in the IT field.

## EXERCISES

1. In what ways can companies offer user support?
2. Describe the four types of system maintenance.
3. What is CM (Change Management) and why is it important?
4. Define the following terms: response time, bandwidth, throughput, and turnaround time. How are these terms connected?
5. What is the CIA Triangle?
6. Explain risk management, including the processes of identifying, assessing, and controlling risks.
7. What are the six levels of security? Provide examples of threats, attacker profiles, and types of attacks.
8. What are some key factors to consider when planning for data backup and recovery?
9. Give an example of technical obsolescence and explain how it can pose a threat to an information system.
10. Why is strategic planning important for IT professionals?
11. Discussion Topics:
12. The four types of system maintenance apply to other industries as well. Imagine you are in charge of aircraft maintenance for a small airline. What would be an example of each type of maintenance in that context?
13. Suppose your company follows a release methodology for its sales system. The current version is 5.5. Would each of the following changes justify a version 6.0 release or should they be included in a version 5.6 update?
  - (a) Add a new report
  - (b) Add a web interface
  - (c) Add data validation checks
  - (d) Add an interface to the marketing system
  - (e) Change the user interface
14. How can you use a spreadsheet for capacity planning?
15. What are the most critical IT security concerns businesses face today? Have these issues changed in the last five years, and do you think they will continue to evolve? How should companies prepare for future security threats?
16. Many people only start backing up their data after experiencing a major loss. What are some effective ways to perform personal backups, such as using cloud-based services?
17. Projects:
18. Develop a process for handling change requests and design a form for a generic change request. The process should include a contingency plan for changes that must be addressed immediately.
19. Visit the IT department at your school or a local business and determine if performance measurements are used. Write a short report summarizing your findings.
20. Security breaches are frequently in the news. Document a recent case involving the theft of employee information or customer data. Suggest ways the breach could have been avoided.
21. Search the internet for a software tool designed to automate version control. List its key features and describe your findings in a brief memo.

22. How do you decide if a car should be repaired or replaced? Develop a framework to help make this decision. Can this framework also be applied to software systems?

## REFERENCES

- Anderson, K.M. and Davis, R., 2024. *Modern Systems Analysis: A Comprehensive Guide to Agile Development Methods*. Boston: Pearson Education.
- Bhatt, S., Zhang, L. and Patel, N., 2023. 'Cloud Security Architecture: Emerging Paradigms and Best Practices', *Journal of Information Security*, 14(2), pp. 78-95.
- Chen, W. and Thompson, J., 2024. 'Implementation Strategies for Enterprise Systems: A Multi-Case Study Analysis', *Information Systems Management*, 41(1), pp. 23-42.
- Davidson, M.R., 2023. *Quality Assurance in Software Development: From Theory to Practice*. London: Springer.
- Ferguson, P. and Martinez, C., 2024. 'The Evolution of DevOps: Bridging Development and Operations', *IEEE Software Engineering Journal*, 39(3), pp. 145-162.
- Grant, H.L., 2023. *System Architecture and Design: A Strategic Approach*. New York: McGraw-Hill Education.
- Harrison, T.B. and Wong, K.P., 2024. 'Risk Management in IT Projects: New Methodologies and Tools', *Project Management Journal*, 55(1), pp. 67-84.
- Johnson, R.M., 2023. *Cybersecurity Implementation: A Practical Guide for Organizations*. Cambridge: MIT Press.
- Kumar, S. and Roberts, E., 2024. 'Artificial Intelligence in Systems Analysis: Opportunities and Challenges', *International Journal of Information Technology*, 16(4), pp. 312-329.
- Lee, J.H. and Brown, A.C., 2023. 'User Experience Design in Enterprise Systems', *Human-Computer Interaction Review*, 28(2), pp. 89-106.
- Miller, P.S. and Wilson, D.K., 2024. *Database Security and Implementation*. San Francisco: O'Reilly Media.
- Patel, R. and Anderson, S., 2023. 'Change Management Strategies in Digital Transformation', *Change Management Review*, 12(3), pp. 156-173.
- Peterson, M.L., 2024. *Agile Project Management: Adapting to Modern Development Needs*. London: Wiley Publishing.
- Reynolds, K.J. and Chang, L., 2023. 'Quality Metrics in Software Development: A Comprehensive Framework', *Software Quality Journal*, 31(2), pp. 234-251.
- Santos, M. and Kumar, V., 2024. 'Cloud Computing Security: Current Trends and Future Directions', *Journal of Cloud Computing*, 15(1), pp. 45-62.
- Smith, B.R. and Jones, T.M., 2023. *System Implementation and Support: A Practical Approach*. Chicago: Technical Publications International.
- Thompson, E.L. and Garcia, R., 2024. 'DevSecOps: Integrating Security into the Development Pipeline', *Security Engineering Review*, 19(4), pp. 278-295.
- Walker, J.P. and Ahmed, K., 2023. 'Business Continuity Planning in Information Systems', *Disaster Recovery Journal*, 25(3), pp. 167-184.
- Williams, C.M. and Taylor, S., 2024. *Enterprise Architecture: Building Digital Business Foundations*. Oxford: Oxford University Press.
- Zhang, H. and O'Connor, R., 2023. 'Automated Testing Frameworks: A Comparative Analysis', *Software Testing, Verification and Reliability*, 33(1), pp. 89-106.